



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more about us.

Understanding EKS

EKS – The Basics



Amazon EKS

In this section, we're just going to start with the basics. What is EKS? What does it Mean? and how is it different than a normal Kubernetes cluster? Like we said in the intro, we do expect you to have some Kubernetes basics and some AWS basics because you're running a Kubernetes cluster in an AWS environment. So what does that Mean for EKS? Well, EKS is the hosted Service for Kubernetes and to explain that a little better. Let's just diagram out what a normal Kubernetes cluster looks like.

EKS – The Basics



Kubernetes Cluster



Amazon EKS

In this section, we're just going to start with the basics. What is EKS? What does it Mean? and how is it different than a normal Kubernetes cluster? Like we said in the intro, we do expect you to have some Kubernetes basics and some AWS basics because you're running a Kubernetes cluster in an AWS environment. So what does that Mean for EKS? Well, EKS is the hosted Service for Kubernetes and to explain that a little better. Let's just diagram out what a normal Kubernetes cluster looks like.

Requirements



Kubernetes Basics



AWS Basics

In this section, we're just going to start with the basics. What is EKS? What does it Mean? and how is it different than a normal Kubernetes cluster? Like we said in the intro, we do expect you to have some Kubernetes basics and some AWS basics because you're running a Kubernetes cluster in an AWS environment. So what does that Mean for EKS? Well, EKS is the hosted Service for Kubernetes and to explain that a little better. Let's just diagram out what a normal Kubernetes cluster looks like.

EKS – The Basics

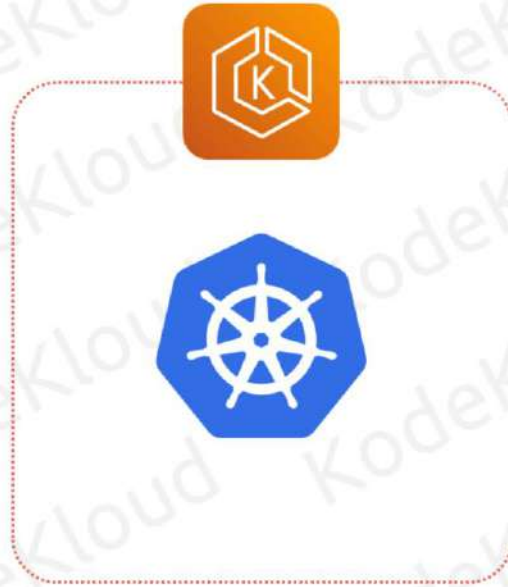


Amazon EKS

© Copyright KodeKloud

In this section, we're just going to start with the basics. What is EKS? What does it Mean? and how is it different than a normal Kubernetes cluster? Like we said in the intro, we do expect you to have some Kubernetes basics and some AWS basics because you're running a Kubernetes cluster in an AWS environment. So what does that Mean for EKS? Well, EKS is the hosted Service for Kubernetes and to explain that a little better. Let's just diagram out what a normal Kubernetes cluster looks like.

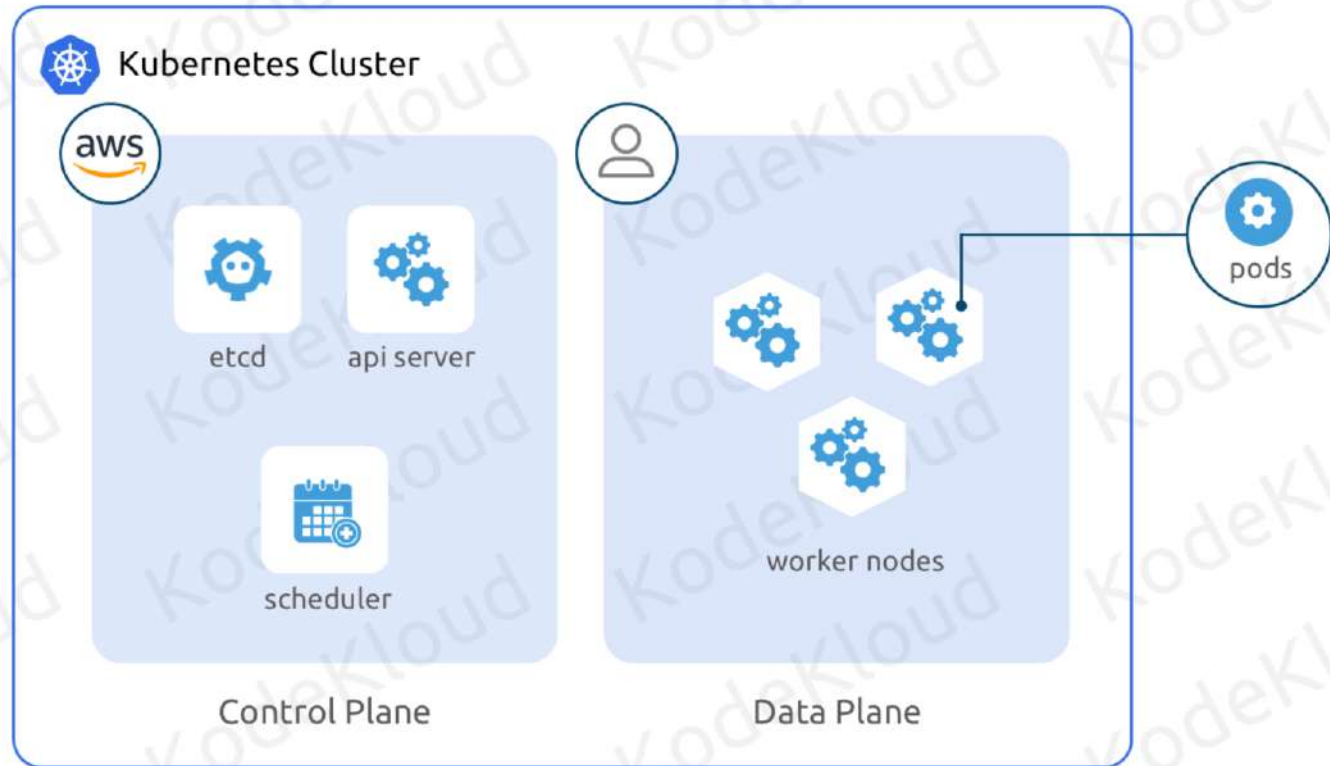
EKS – The Basics



© Copyright KodeKloud

In this section, we're just going to start with the basics. What is EKS? What does it Mean? and how is it different than a normal Kubernetes cluster? Like we said in the intro, we do expect you to have some Kubernetes basics and some AWS basics because you're running a Kubernetes cluster in an AWS environment. So what does that Mean for EKS? Well, EKS is the hosted Service for Kubernetes and to explain that a little better. Let's just diagram out what a normal Kubernetes cluster looks like.

Kubernetes Cluster Components



© Copyright KodeKloud

Abstractly in Kubernetes, we have these two components of a cluster. We have the control plane, which is your etcd, your API server, your scheduler, All the things you need to Run workloads in a cluster and then you have the Data plane, which is all of those worker nodes and things that actually run the pods that you should be caring about for an EKS cluster. we split that in Half and we say this part over here, the control plane runs in AWS accounts, and this Part over here runs in your user account. It's a very clean line between who's responsible for what? But there are some things that go across that border. So we'll go ahead.

EKS Cluster



Amazon EKS



Kubernetes Cluster



etcd



api server



scheduler

Control Plane



worker nodes

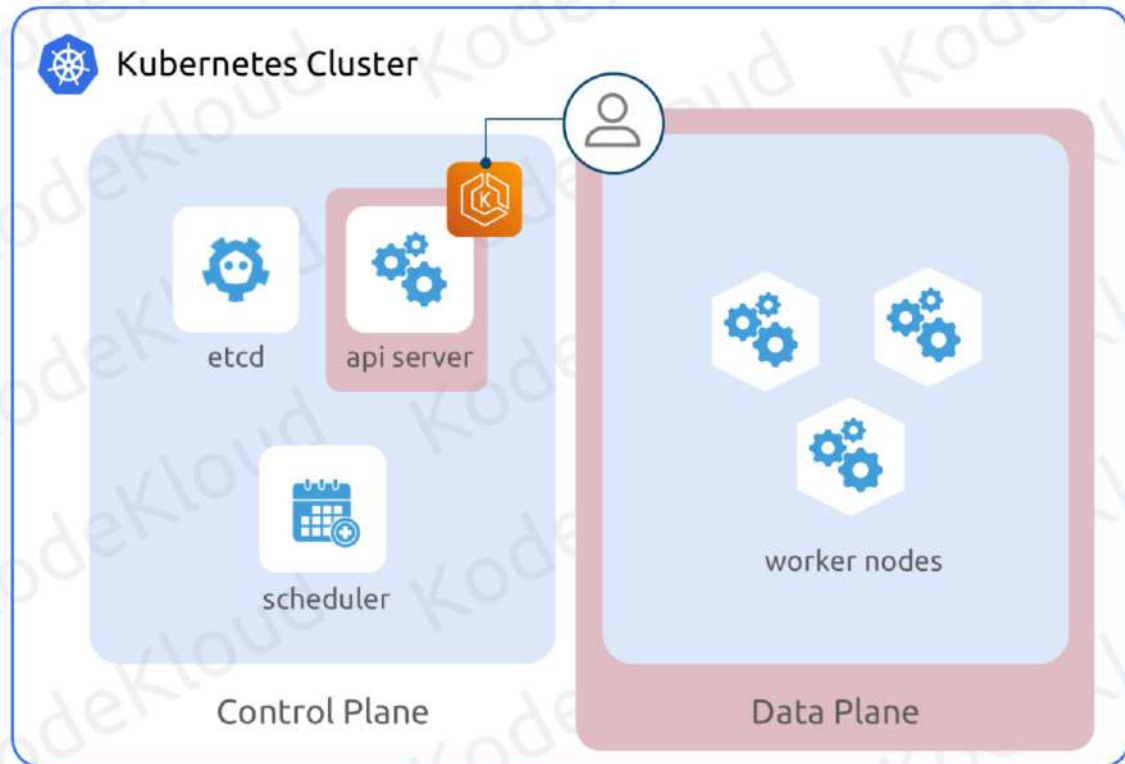
Data Plane

We'll get into that in a little bit and when we're talking about EKS specifically, We're talking about the API that creates this section over here the control plane, but also can be used to affect some things over here on the data plane in your account. The main difference being the things that are running in your account you'll actually see if you log into the console, the control plane side of things, you'll just see a cluster, an EKS cluster. You won't necessarily see all of the components that are part of that control plane.

EKS Cluster



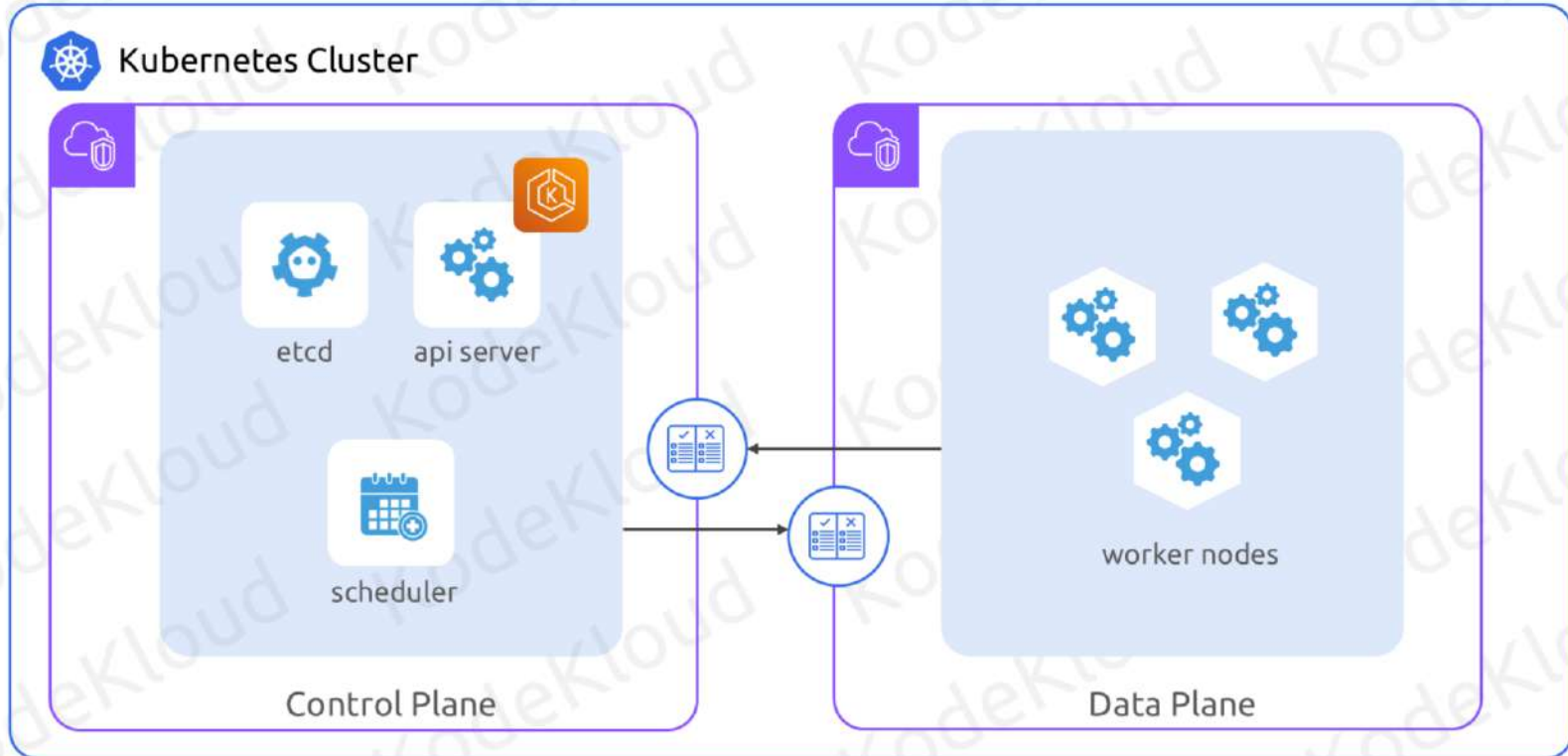
Amazon EKS



© Copyright KodeKloud

We'll get into that in a little bit and when we're talking about EKS specifically, We're talking about the API that creates this section over here the control plane, but also can be used to affect some things over here on the data plane in your account. The main difference being the things that are running in your account you'll actually see if you log into the console, the control plane side of things, you'll just see a cluster, an EKS cluster. You won't necessarily see all of the components that are part of that control plane.

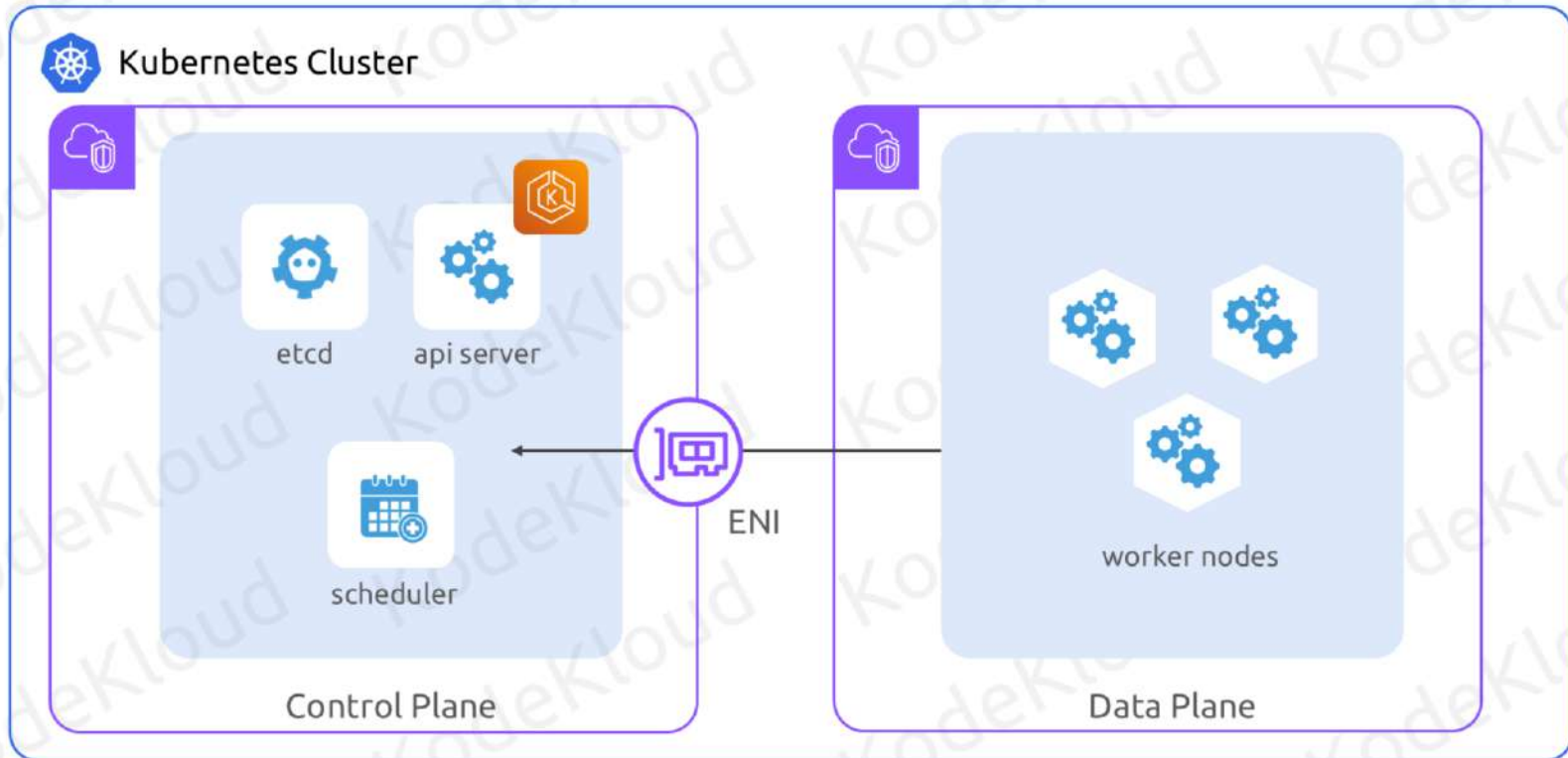
EKS Cluster



© Copyright KodeKloud

One thing that's a little bit tricky sometimes with an EKS cluster is just the fact that people think it's magic and the EKS control plane side is really just the same as what you would have on your side, it's just managed by someone else so even on the AWS side, on the control plane, there's still a VPC just like you have on your cluster side so your networking is still bound by the same rules on the control plane as it is for the user and we can bridge these networks with ENIs that are cross account that go from the control plane into the user account for access from the nodes in the data plane over to the control plane. This all tends to sound really abstract when we talk about it that way, so I like to break it down as just.

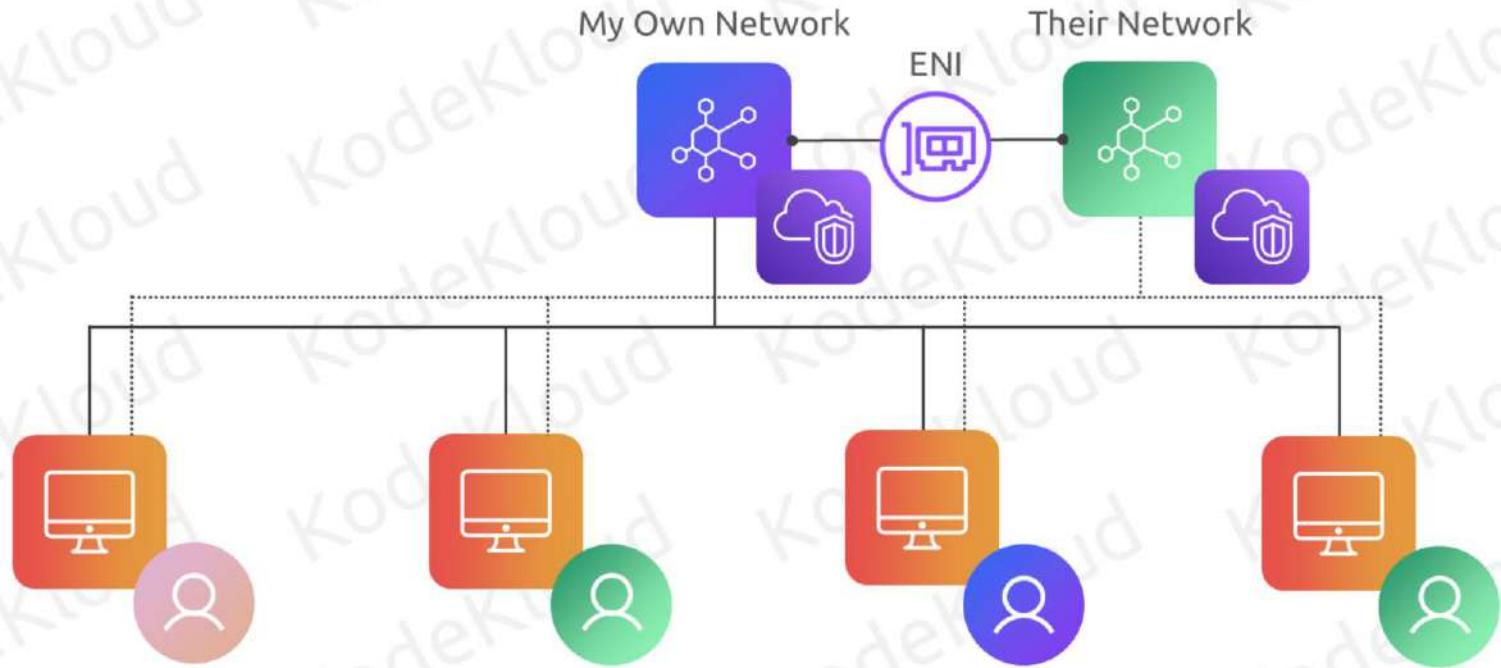
EKS Cluster



© Copyright KodeKloud

One thing that's a little bit tricky sometimes with an EKS cluster is just the fact that people think it's magic and the EKS control plane side is really just the same as what you would have on your side, it's just managed by someone else so even on the AWS side, on the control plane, there's still a VPC just like you have on your cluster side so your networking is still bound by the same rules on the control plane as it is for the user and we can bridge these networks with ENIs that are cross account that go from the control plane into the user account for access from the nodes in the data plane over to the control plane. This all tends to sound really abstract when we talk about it that way, so I like to break it down as just.

Visualizing ENIs



© Copyright KodeKloud

Let's say I have a network and I plug in all of my computers on the back of this network and I need that network to run my machines but someone else is running some machines for me and they have their own network. They plug their computers into the back of their network in order to make these two things talk together, we have to plug something in together they have to talk to each other. This is 1 VPC on my account. This is a VPC on their accounts and all we are doing is connecting a cable between the two. It makes a lot of sense when I physically have them in front of me and I can plug them together but when I start talking about ENIs and VPCS and all that stuff seems. To be abstract to me, so I just like to break things down by just saying, these are two networks that have different sets of computers that connect to each other with essentially a

cable.

Common Use Cases

When and Why to Use EKS



© Copyright KodeKloud

So, when and why would you want to use an EKS cluster, and as we were just talking about, EKS is just a way to run Kubernetes in AWS, and this is no different than any other way you might run Kubernetes in AWS except for the fact that half of it is managed by AWS.

Kubernetes Cluster in AWS With EC2



Amazon EC2

And one way you might want to run a Kubernetes Cluster in AWS with EC2.

Kubernetes Cluster in AWS With EC2



Virtual Machine (VM)

And one way you might want to run a Kubernetes Cluster in AWS with EC2.

Kubernetes Cluster in AWS With EC2



© Copyright KodeKloud

You could absolutely run your own VMS inside of AWS and carve them up. To say, some of them are control plane and some of them are the data plane and this has been done for a very long time

VMS Tools



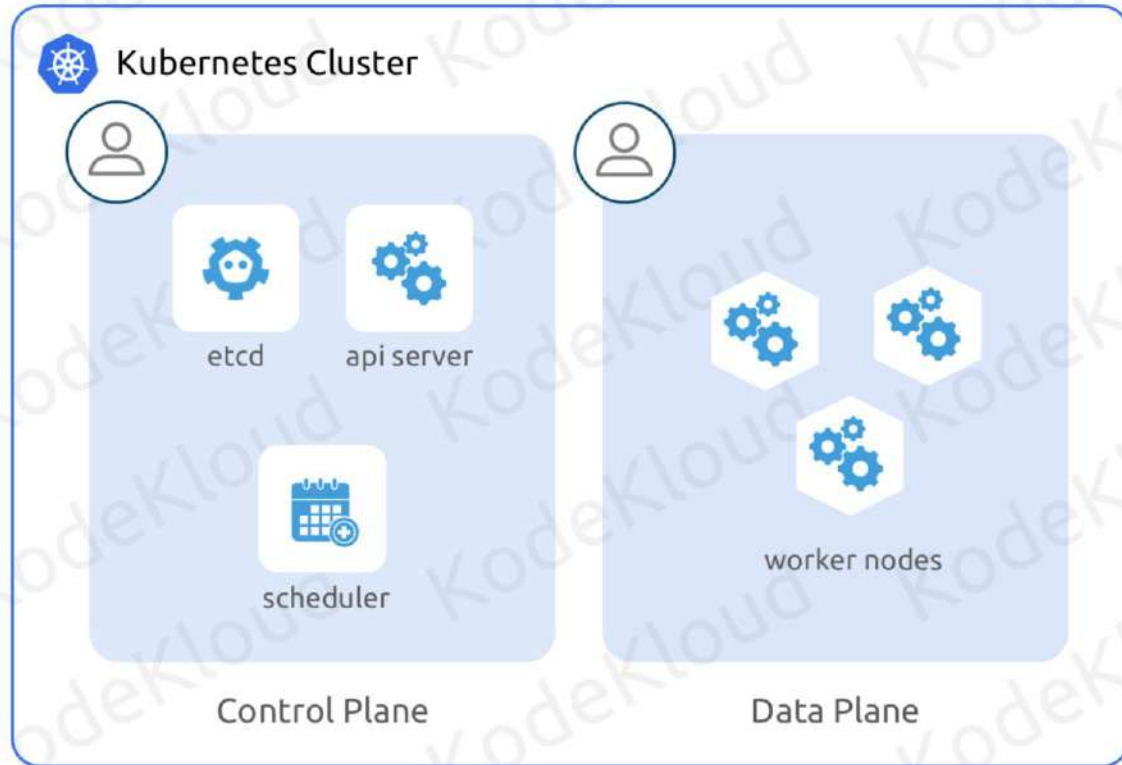
Kubernetes
Operations (KOPS)



Kubespray

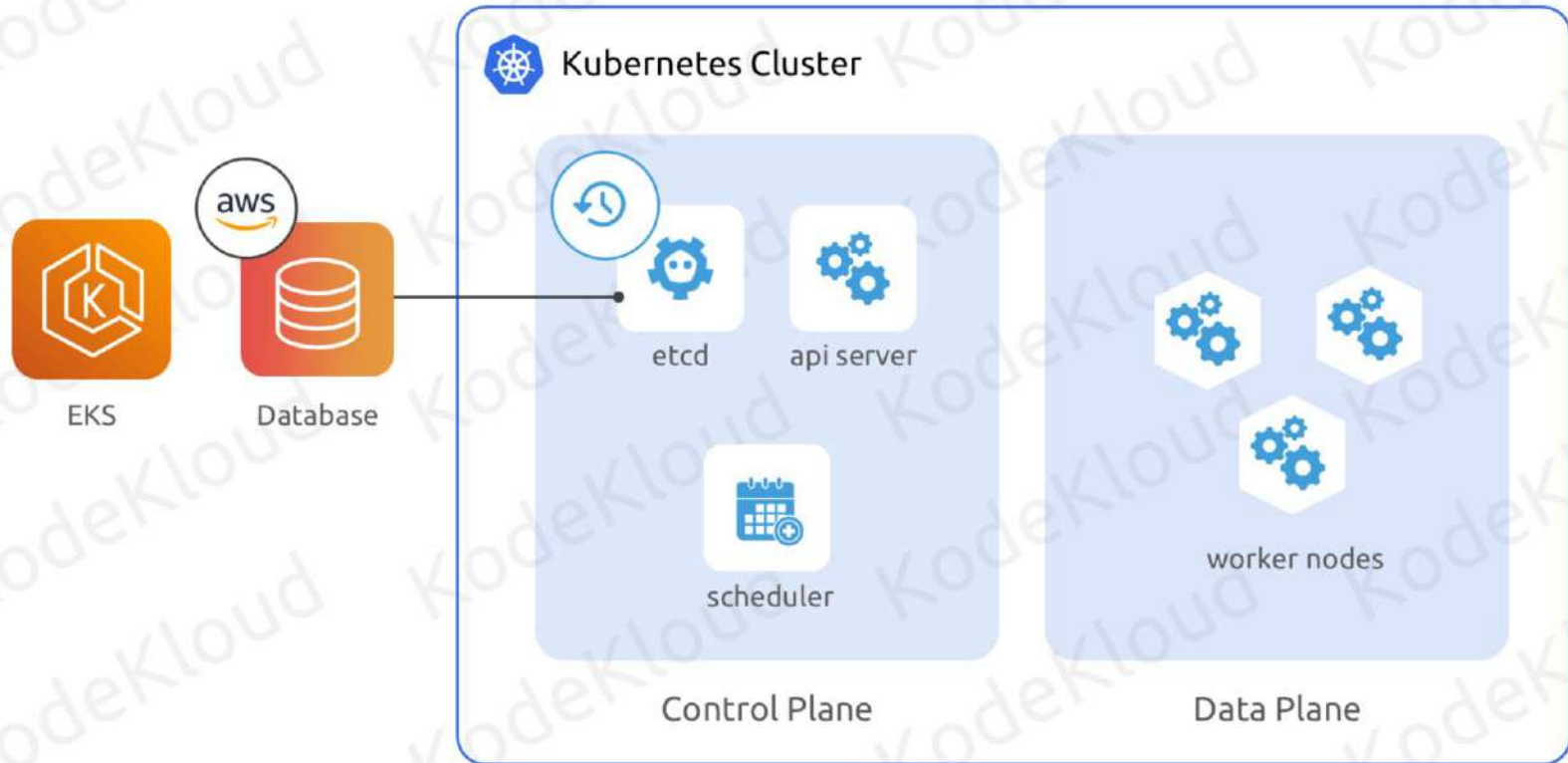
think tools like cops that came before and even tools like cube spray will help you manage a bunch of VMS.

VMS Tools



As your own cluster and you are responsible for both sides of the Kubernetes cluster the control plane. The data and etcd. As well as the data plane and the nodes that you run your workloads on. The benefits of doing something like this is because you have full control over the control plane and the data plane you can pick. What type of flags you set on the scheduler. You can pick what versions you install and when you upgrade them. You can even say how big of a control plane you want or how small. So if you need to scale really large or you want this for small temporary clusters, that's up to you and you have full control over that. The downside of course is that you have to own all of that. That is now your responsibility especially the data behind the Kubernetes cluster.

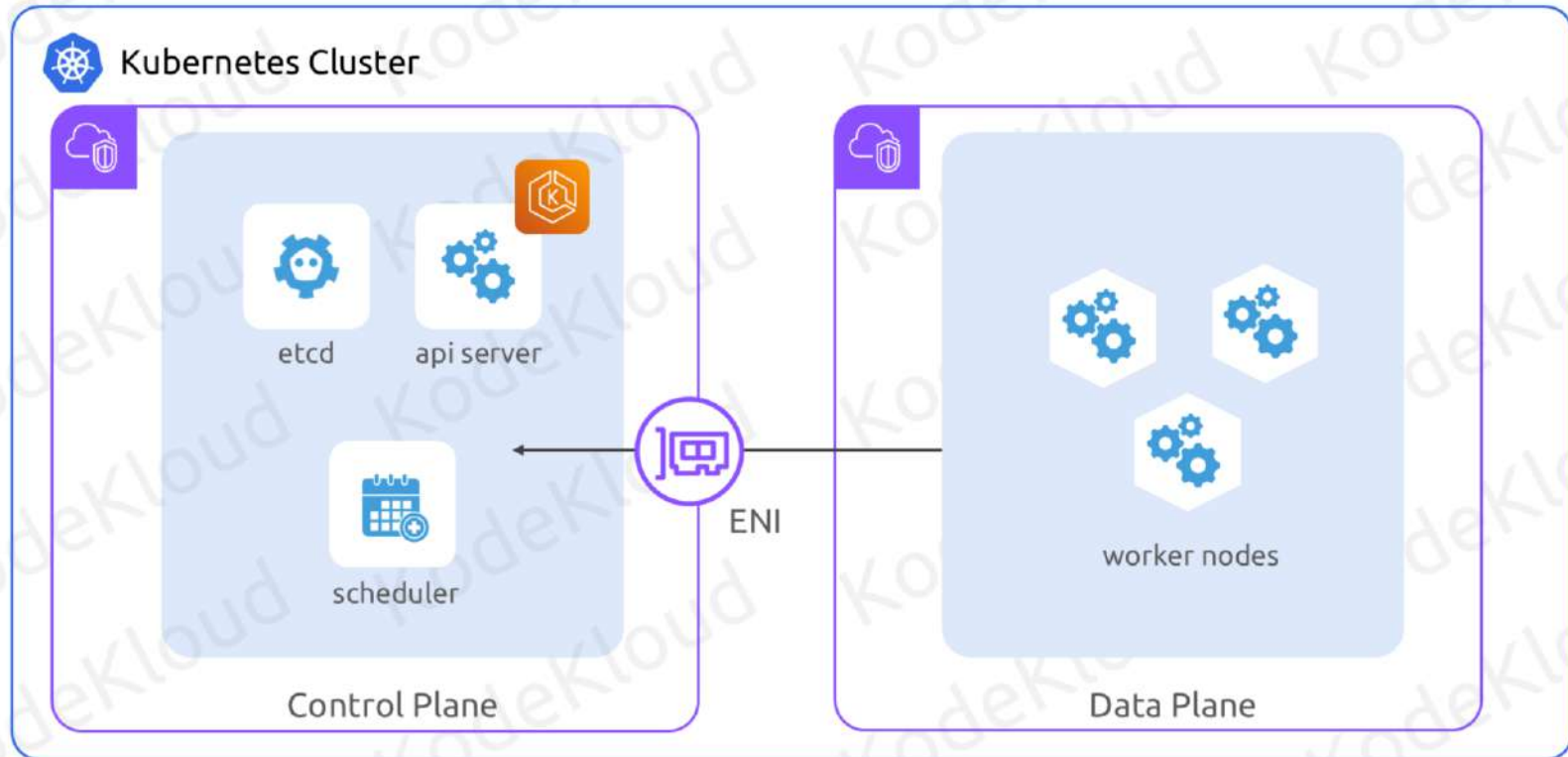
Use Cases



© Copyright KodeKloud

Many people that I know. The first time they want to use a hosted service for Kubernetes is when they have to restore a backup for Etcd. Etcd is a distributed database that backs every Kubernetes cluster and that is one of the things that most and that is one of the most critical pieces of the cluster because if that data goes away, your cluster basically doesn't exist. Everything else relies on that data, and you are very much responsible for that. EKS puts that management of managing the data on AWS. AWS is responsible for your data and they are the ones that would keep it running and available and if you don't want that responsibility you should be running EKS.

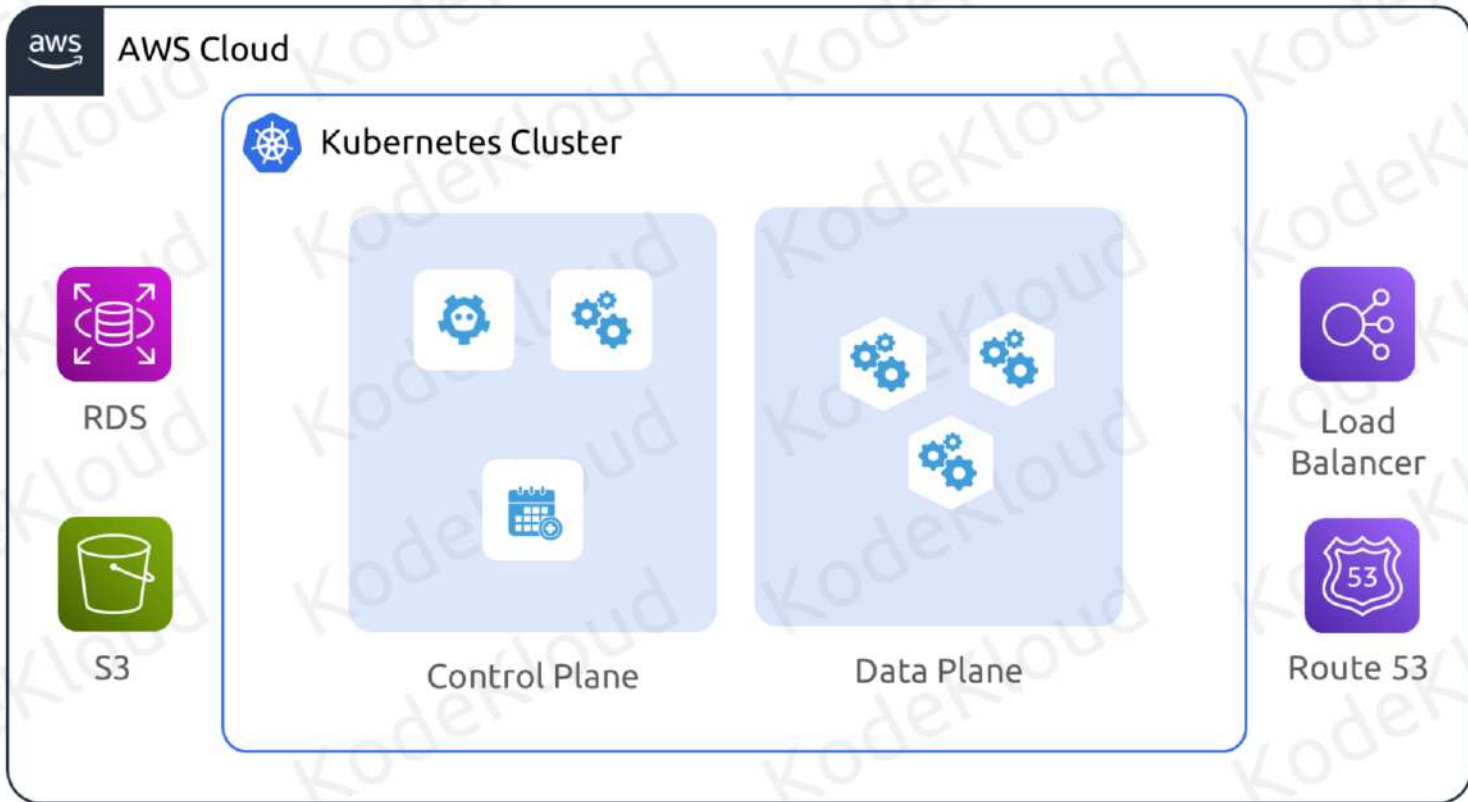
Use Cases



© Copyright KodeKloud

But if you can adhere to having your own EKS cluster. Such as you need to keep that cluster updated, you need to make sure that the VPC Cross account still exist. That some permissions exist that allow AWS to manage some resources in your account all of those things are your responsibility to make sure that you have a healthy available EKS cluster that AWS can manage their part and you can manage yours. So you can choose if you want to run Kubernetes with an EKS Cluster or run the Cluster yourself, but either case.

Cloud Services



© Copyright KodeKloud

You're running something inside of an AWS accounts which gives you access to the rest of your AWS resources and the main use case for running Kubernetes in AWS is because you have other things there. Most customers are not using AWS just for Kubernetes. If they are just using it for Kubernetes, they're probably are fine managing it themselves. Everything else that they need to run is Inside the cluster, but a lot of people will want to use things like RDS or S3 or global load balancers or Route 53 all of those other services that AWS provides to integrate them into their cluster and Kubernetes is just one part of a larger whole of AWS services that they use and as we see in future segments, the easiest way to integrate a Kubernetes cluster with the broader AWS services is by using EKS.

EKS as an Integrator



So you can choose if you want to run Kubernetes with an EKS Cluster or run the Cluster yourself, but either case. You're running something inside of an AWS accounts which gives you access to the rest of your AWS resources and the main use case for running Kubernetes in AWS is because you have other things there. Most customers are not using AWS just for Kubernetes. If they are just using it for Kubernetes, they're probably are fine managing it themselves. Everything else that they need to run is Inside the cluster, but a lot of people will want to use things like RDS or S3 or global load balancers or Route 53 all of those other services that AWS provides to integrate them into their cluster and Kubernetes is just one part of a larger whole of AWS services that they use and as we see in future segments, the easiest way to integrate a Kubernetes

cluster with the broader AWS services is by using EKS.

Architecture

Introduction



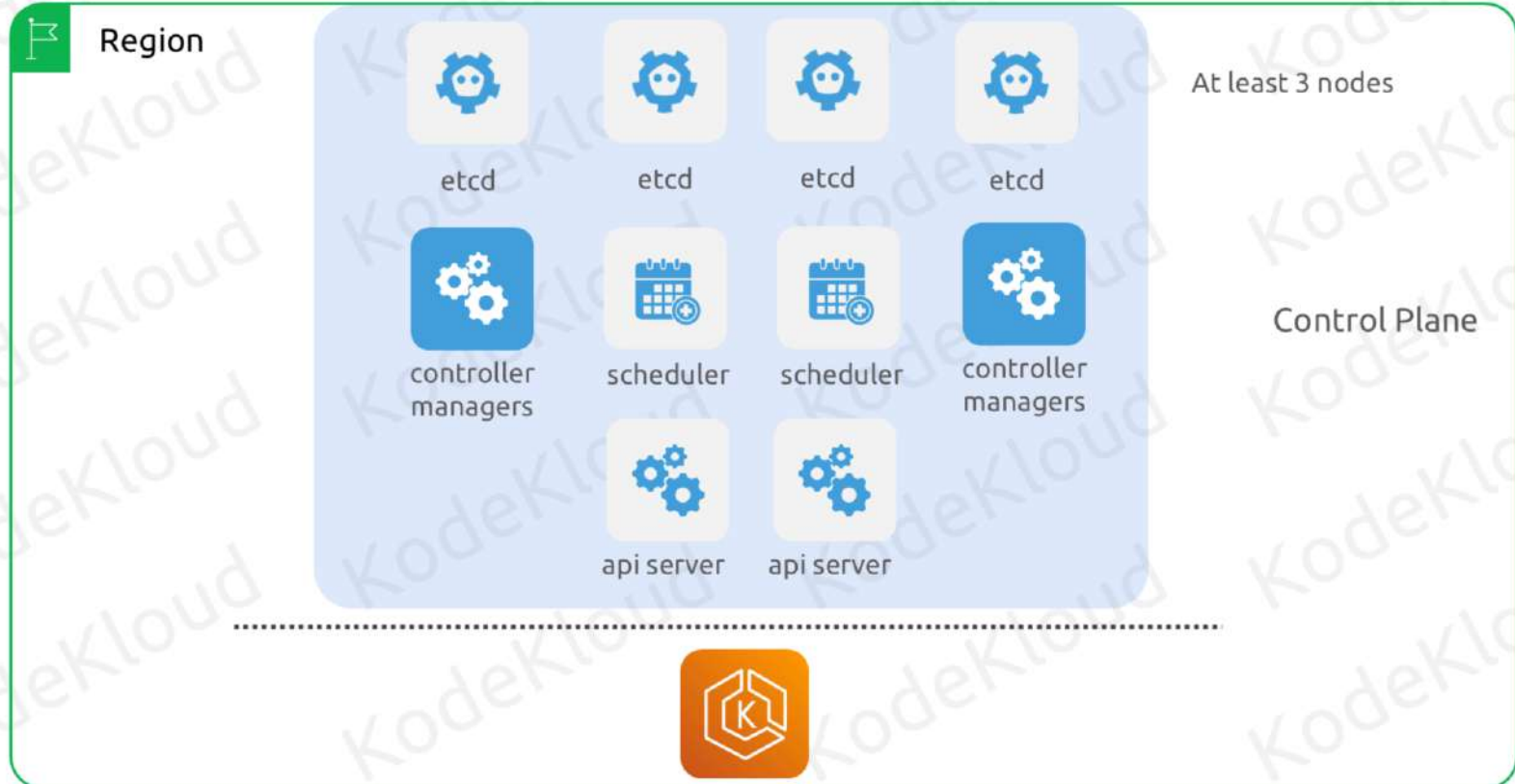
In the intro we already showed you a little bit of the high level architecture of an EKS cluster and how that's a shared responsibility between AWS and you as a user but let's go a little bit deeper into just the control plane of EKS and what you get when you're creating and been using a control plane in an EKS cluster.

Introduction



In the intro we already showed you a little bit of the high level architecture of an EKS cluster and how that's a shared responsibility between AWS and you as a user but let's go a little bit deeper into just the control plane of EKS and what you get when you're creating and been using a control plane in an EKS cluster.

EKS and Control Plane Components



© Copyright KodeKloud

So, here's our border between us and EKS as far as like an API that just says we want a cluster and what's going to happen behind the scenes and for us, we're going to get things like etcd as a database and so we get a few of those nodes, which again, if you're running at CD, it's a distributed database, it needs quorum. So you need at least three of those nodes, It might scale up to maybe 5, but you have to do things that at CD needs to make sure that you always have a leader election and all that stuff to keep your data safe. So at CD is the first part. The second thing we're going to get is a couple API servers. This is what normally we might think of as the Kubernetes cluster. That's the API that kube control is going to call and say create a pod or deployment or something like that. We're talking to the API server. We're also going to need a

couple schedulers and a couple controller managers. These are just the basics that you need for a cluster. All of these things are part of the control plane, and they're all components that hopefully you're familiar with. If this is new to you, please check out the other earlier kind of training on what makes the Kubernetes cluster. We're not going to dive deep into what these things do, but just that they're required for the API server and the control plane to run. Something we haven't really mentioned yet is EKS is a regional service and so within your region, you have all of these components. But a region isn't just one data center. A region is made-up of multiple Availability Zones, which are other data centers. And so the way this gets carved up is some of these are going to run in one AZ, some of them in another, and a third over here, you're going to have a minimum of three AZs just to spread your SCD databases across all three and these are just like normal data data centers. These data centers exist within a region. They're geographically separate. So that's hopefully outages won't affect one another, but close enough so that latencies are not really a big issue between the data centers. If you were designing this yourself, you would have literally built 3 datacenters and spread your SCD database across all three and had two of those four API servers and controller managers and schedulers. This does mean that if one of those availability zones go down, you might be in a state where HD is read only or your API server needs to fail over to one or the other nodes that had the API server running. This is a normal part of running a distributed system and something that Etcd and something that EKS handles for you and this is one of those nice things again that if you want to do this yourself and run Kubernetes in AWS you absolutely can do this. But you have to consider these things and EKS is meant to be highly available in account for an easy failing. So at this point our control plane as we normally would expect for a Kubernetes cluster is up. This is, these are the pieces that we would need, and if we wanted to manually do this, these are the components we would be running, whether that's through automation or running containers. Whatever it is, AWS handles this side of it. Now the other parts that are part of an EKS cluster that aren't necessarily parts of a Kubernetes cluster are things like an OIDC endpoint. This gives you authentication for things that want to call into the cluster and integrate into AWS. This is a way that you can map yourself to IAM roles that also go into the cluster. We're going to go into authentication a lot in another section, so don't worry about it too much right now, but it's one piece of the larger component that you usually get with every cluster. You're going to get things like logging that go into Cloudwatch, and so if you want to send all of these logs from these components somewhere that's centrally stored, you're going to send them over to Cloudwatch. Typically you would have to have another service that gathers all of those logs so you can debug things like an API server

having problems or a controller manager not making replicas. In an EKS cluster that by default is Cloud Watch. You do have to enable it for it to send logs, but it's there already available for you and Cloud Watch has all of the retention and authentication and access things that you would normally expect for any AWS service. There's also some things like the ways you can authenticate into the cluster. This might be through the auth config map, which is kind of the older way that's EKS authenticates users into the cluster, or the new authentication APIs that are provided. So you can say who in your AWS account, which I am principals have access to the cluster itself and this is the way to tie something that's outside the cluster that can do things in the cluster and the other way around And so these auth, this auth API is part of the EKSAPI and again we'll dive into that a little bit in a future section. There are a couple components of this EKS cluster that also touch into your account. Things like node groups, these directly affect the data plane. This is how pods get scheduled and the data plane components of the cluster. We're gonna go deep into what all of the options are and how they work in a future section, but Node Groups is part of an EKS API that affect your account. Add-ons is another section that looks like it's kind of part of the Kubernetes API. It's part of the EKSAPI, but it actually runs workloads inside the cluster. And so those add-ons, even though you schedule them through the API, they deploy workloads into your cluster. And so it's something that is part of your responsibilities when they're running in your account. And this paints a much more complete picture of what an EKS cluster actually is. And if you want to again, run a Kubernetes cluster yourself, you're more than welcome to you are responsible for these extra components if you need them. If you don't need them, then sometimes you can ignore them, sometimes you can just configure yourself around them for an EKS cluster. But in general, these are some of the defaults that you'll get when you create an EKS cluster in AWS.

The diagram illustrates a multi-region EKS architecture. It shows four regions, each containing a set of EKS control plane components: etcd, controller managers, scheduler, and api server. The first region has red 'X' marks over its etcd and api server components. The second region has a red 'X' over its api server component. The third and fourth regions have all components. A dashed line separates the control plane from the data plane, which consists of Node Groups (represented by gear icons) distributed across the regions. The architecture includes an OIDC Endpoint and Add-ons on the left. On the right, AWS CloudWatch is used for logging, and a Config map is used for authentication. The text 'At least 3 nodes' is present near the top right.

So, here's our border between us and EKS as far as like an API that just says we want a cluster and what's going to happen behind the scenes and for us, we're going to get things like etcd as a database and so we get a few of those nodes, which again, if you're running at CD, it's a distributed database, it needs quorum. So you need at least three of those nodes, It might scale up to maybe 5, but you have to do things that at CD needs to make sure that you always have a leader election and all that stuff to keep your data safe. So at CD is the first part. The second thing we're going to get is a couple API servers. This is what normally we might think of as the Kubernetes cluster. That's the API that kube control is going to call and say create a pod or deployment or something like that. We're talking to the API server. We're also going to need a

couple schedulers and a couple controller managers. These are just the basics that you need for a cluster. All of these things are part of the control plane, and they're all components that hopefully you're familiar with. If this is new to you, please check out the other earlier kind of training on what makes the Kubernetes cluster. We're not going to dive deep into what these things do, but just that they're required for the API server and the control plane to run. Something we haven't really mentioned yet is EKS is a regional service and so within your region, you have all of these components. But a region isn't just one data center. A region is made-up of multiple Availability Zones, which are other data centers. And so the way this gets carved up is some of these are going to run in one AZ, some of them in another, and a third over here, you're going to have a minimum of three AZs just to spread your SCD databases across all three and these are just like normal data data centers. These data centers exist within a region. They're geographically separate. So that's hopefully outages won't affect one another, but close enough so that latencies are not really a big issue between the data centers. If you were designing this yourself, you would have literally built 3 datacenters and spread your SCD database across all three and had two of those four API servers and controller managers and schedulers. This does mean that if one of those availability zones go down, you might be in a state where HD is read only or your API server needs to fail over to one or the other nodes that had the API server running. This is a normal part of running a distributed system and something that Etcd and something that EKS handles for you and this is one of those nice things again that if you want to do this yourself and run Kubernetes in AWS you absolutely can do this. But you have to consider these things and EKS is meant to be highly available in account for an easy failing. So at this point our control plane as we normally would expect for a Kubernetes cluster is up. This is, these are the pieces that we would need, and if we wanted to manually do this, these are the components we would be running, whether that's through automation or running containers. Whatever it is, AWS handles this side of it. Now the other parts that are part of an EKS cluster that aren't necessarily parts of a Kubernetes cluster are things like an OIDC endpoint. This gives you authentication for things that want to call into the cluster and integrate into AWS. This is a way that you can map yourself to IAM roles that also go into the cluster. We're going to go into authentication a lot in another section, so don't worry about it too much right now, but it's one piece of the larger component that you usually get with every cluster. You're going to get things like logging that go into Cloudwatch, and so if you want to send all of these logs from these components somewhere that's centrally stored, you're going to send them over to Cloudwatch. Typically you would have to have another service that gathers all of those logs so you can debug things like an API server

having problems or a controller manager not making replicas. In an EKS cluster that by default is Cloud Watch. You do have to enable it for it to send logs, but it's there already available for you and Cloud Watch has all of the retention and authentication and access things that you would normally expect for any AWS service. There's also some things like the ways you can authenticate into the cluster. This might be through the auth config map, which is kind of the older way that's EKS authenticates users into the cluster, or the new authentication APIs that are provided. So you can say who in your AWS account, which I am principals have access to the cluster itself and this is the way to tie something that's outside the cluster that can do things in the cluster and the other way around And so these auth, this auth API is part of the EKSAPI and again we'll dive into that a little bit in a future section. There are a couple components of this EKS cluster that also touch into your account. Things like node groups, these directly affect the data plane. This is how pods get scheduled and the data plane components of the cluster. We're gonna go deep into what all of the options are and how they work in a future section, but Node Groups is part of an EKS API that affect your account. Add-ons is another section that looks like it's kind of part of the Kubernetes API. It's part of the EKSAPI, but it actually runs workloads inside the cluster. And so those add-ons, even though you schedule them through the API, they deploy workloads into your cluster. And so it's something that is part of your responsibilities when they're running in your account. And this paints a much more complete picture of what an EKS cluster actually is. And if you want to again, run a Kubernetes cluster yourself, you're more than welcome to you are responsible for these extra components if you need them. If you don't need them, then sometimes you can ignore them, sometimes you can just configure yourself around them for an EKS cluster. But in general, these are some of the defaults that you'll get when you create an EKS cluster in AWS.

Deployment Options

Creating an EKS Cluster



© Copyright KodeKloud

Now let's talk a little bit about how you can create an EKS cluster, and there's a variety of ways to do it and some tooling. Some of it exists as part of AWS services, some of it is external tools and services that create this for you. I'm going to start with the most obvious, which is you can do it through the console. You can log into the AWS console, the web interface, and click enough buttons to make a cluster happen. It's kind of rough. When I was working on EKS, it was always one of the biggest complaints that the console experience was one of the worst and hardest things to reason about. Because you have to. EKS is such an integral part to multiple AWS services that it was hard to bounce between the different tabs and services to make sure the right IAM was set up over here and these options were set up over here. So if you're doing it through the

console, that is probably one of the harder ways to do it.

Creating an EKS Cluster

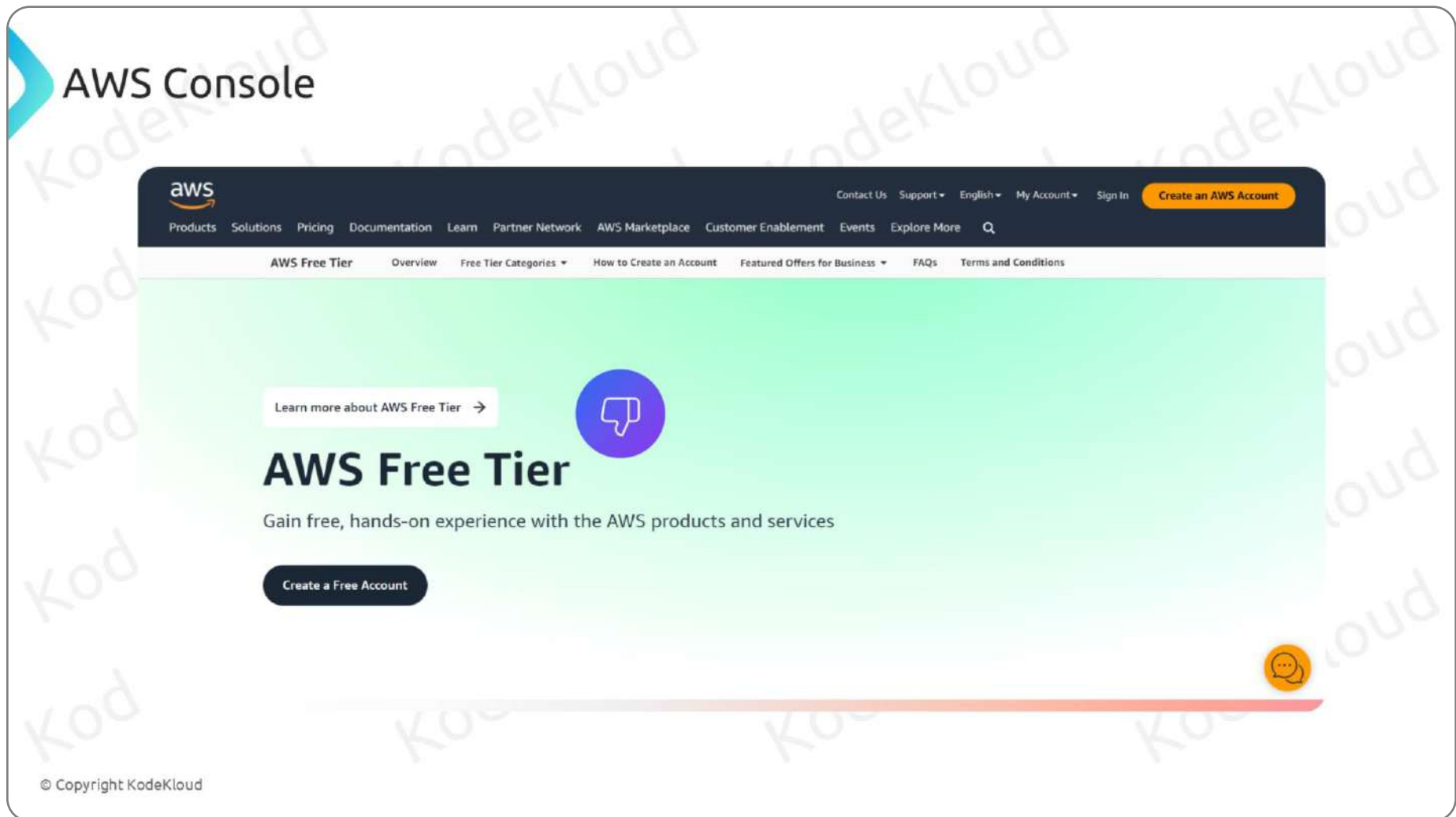


AWS EKS



Now let's talk a little bit about how you can create an EKS cluster, and there's a variety of ways to do it and some tooling. Some of it exists as part of AWS services, some of it is external tools and services that create this for you. I'm going to start with the most obvious, which is you can do it through the console. You can log into the AWS console, the web interface, and click enough buttons to make a cluster happen. It's kind of rough. When I was working on EKS, it was always one of the biggest complaints that the console experience was one of the worst and hardest things to reason about. Because you have to. EKS is such an integral part to multiple AWS services that it was hard to bounce between the different tabs and services to make sure the right IAM was set up over here and these options were set up over here. So if you're doing it through the

console, that is probably one of the harder ways to do it.



Now let's talk a little bit about how you can create an EKS cluster, and there's a variety of ways to do it and some tooling. Some of it exists as part of AWS services, some of it is external tools and services that create this for you. I'm going to start with the most obvious, which is you can do it through the console. You can log into the AWS console, the web interface, and click enough buttons to make a cluster happen. It's kind of rough. When I was working on EKS, it was always one of the biggest complaints that the console experience was one of the worst and hardest things to reason about. Because you have to. EKS is such an integral part to multiple AWS services that it was hard to bounce between the different tabs and services to make sure the right IAM was set up over here and these options were set up over here. So if you're doing it through the

console, that is probably one of the harder ways to do it.

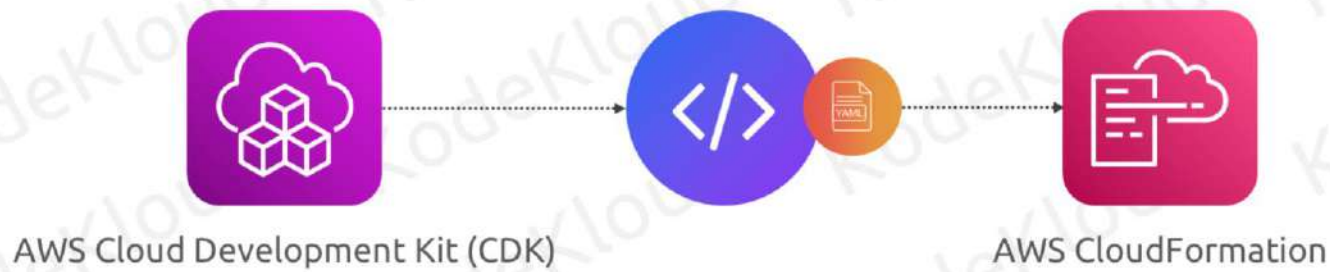
(Infrastructure as code)



AWS CloudFormation

the 2nd way you can do it is with Cloud Formation. Cloud Formation is the AWS service that creates infrastructure as code. Cloud Formation can be generated with something like the CDK and the CDK will use a traditional or general purpose programming language to output the YAML needed to push into the Cloud Formation API. Or you can just author the YAML it's yourself. You could go through and write all of the YAML needed. In both those cases, you're kind of doing it from scratch. You're figuring out all of the pieces you need and then putting them together into a complete cloud formation templates that you are going to deploy. One of the benefits of course of doing one of the benefits of course of creating an EKS cluster with a template or a infrastructure as code tool is you can create multiple of them.

CloudFormation



the 2nd way you can do it is with Cloud Formation. Cloud Formation is the AWS service that creates infrastructure as code. Cloud Formation can be generated with something like the CDK and the CDK will use a traditional or general purpose programming language to output the YAML needed to push into the Cloud Formation API. Or you can just author the YAML it's yourself. You could go through and write all of the YAML needed. In both those cases, you're kind of doing it from scratch. You're figuring out all of the pieces you need and then putting them together into a complete cloud formation templates that you are going to deploy. One of the benefits of course of doing one of the benefits of course of creating an EKS cluster with a template or a infrastructure as code tool is you can create multiple of them.

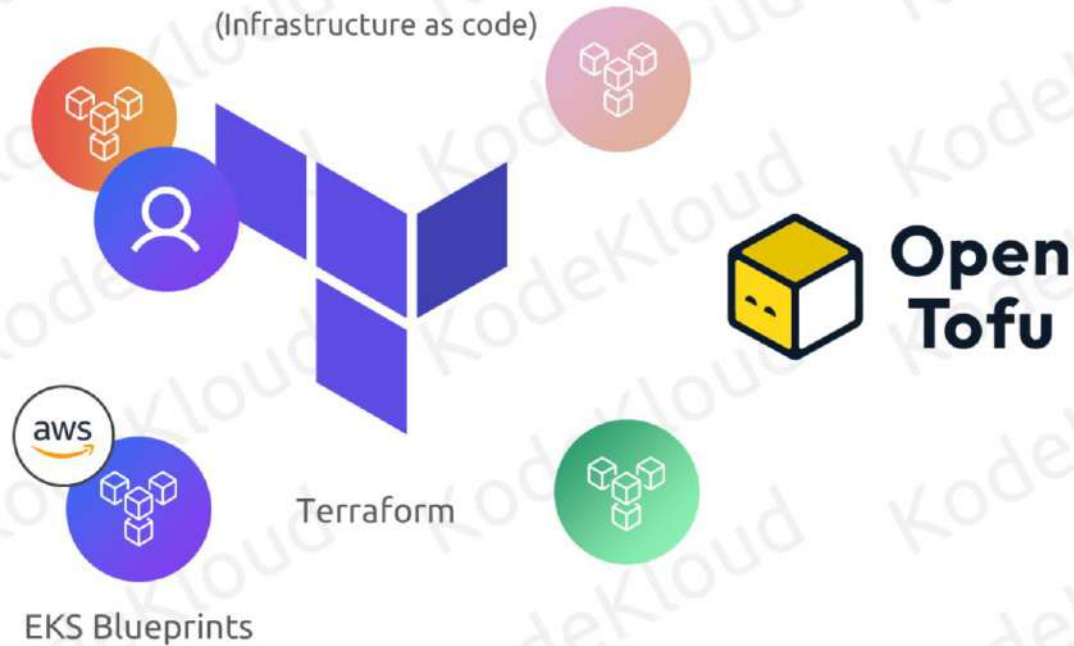
CloudFormation



the 2nd way you can do it is with Cloud Formation. Cloud Formation is the AWS service that creates infrastructure as code. Cloud Formation can be generated with something like the CDK and the CDK will use a traditional or general purpose programming language to output the YAML needed to push into the Cloud Formation API. Or you can just author the YAML it's yourself. You could go through and write all of the YAML needed. In both those cases, you're kind of doing it from scratch. You're figuring out all of the pieces you need and then putting them together into a complete cloud formation templates that you are going to deploy. One of the benefits of course of doing one of the benefits of course of creating an EKS cluster with a template or a infrastructure as code tool is you can create multiple of them. If you're using cloud

formation, it's really easy to change some variables and create another cluster that looks just like the one you created before. I will show you a couple tools that make that side of it easier so that you don't have to do everything from scratch in the next section.

Terraform



© Copyright KodeKloud

The other tool which is probably the most popular option is to use Terraform. Terraform has a lot of helper modules that already exist for creating EKS clusters and you can get pretty far with a lot of the defaults that Terraform modules are going to give you. Amazon also maintains a set of Terraform modules called EKS Blueprints. If you go into the GitHub repo you can find EKS Blueprints which have a lot of templated out information that'll get you started quickly. And Terraform is another infrastructure as code tool, but it's not a managed service, so you do have to manage things like your access into the AWS resources and APIs, the state for the Terraform on how it's deploying the cluster and when you want to destroy it. All of those things are owned by you if you're doing it through Terraform, but it is a very flexible and popular option.

Other Services



Pulumi



Cluster API

Last, I will just point out that there are probably a dozen other services and startups and tools that'll create a EKS cluster for you. Things like Palumi or Cluster API or services that exist. That'll say, hey, you can run this little components and we will create the cluster for you. All of those are up to you. They all essentially do the same thing with maybe even the same languages. Some of them will template out CDK or cloud formation, others will use Terraform. Some will directly call the CLI, some will directly call the API by scripting out the CLI so you can do AWCLI commands and just go step by step and watch while things are being created in the cluster. And again, because EKS, IT kind of touches a lot of components inside of AWS, you do need to have some pretty broad permissions in order to create and manage EKS clusters because you're doing

things to EC2.

EKS Cluster – Components



© Copyright KodeKloud

VPCS, security groups, load balancers, I am roles. All of those things are just little components that go together to make a complete EKS cluster, so whichever tool you're going to use long term is up to you. It's probably the one that you're familiar with today that's going to be the one that's most easy for you to get on boarded and and running. In the next section, I'm going to point out a couple tools that I've used a lot in the past that make it easier to learn things and to create clusters quickly and then destroy them quickly just so that I can test new test new options and settings to see how they work inside of a cluster and how they work with the rest of my infrastructure.

Tools for EKS

What's Next?



Node Groups



Scaling



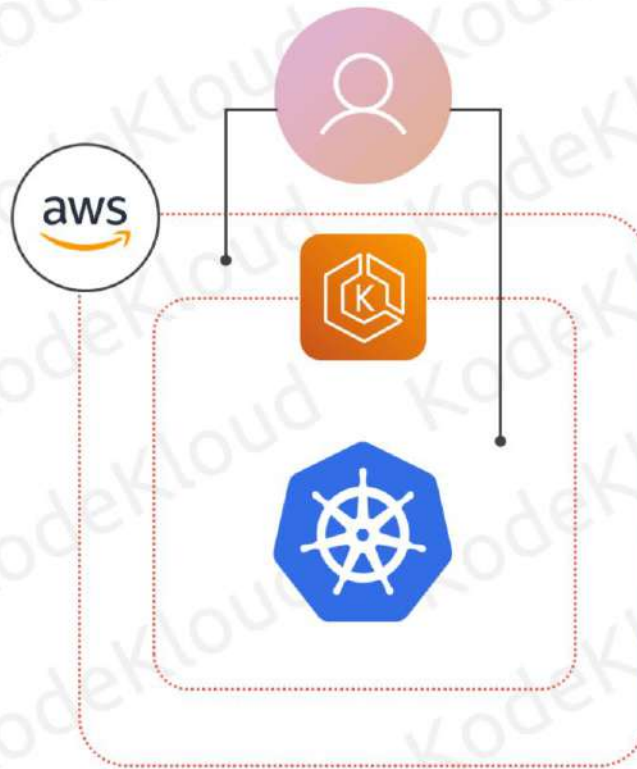
VPC

If you want to do that manually, the last tool I'm going to say is a must for managing EKS clusters is the AWS. I am Authenticator for Kubernetes and this is a plugin for kube control that allows you to authenticate your user to the AWS APIs with your IAM credentials to call an EKS cluster. The EKS API server is behind a load balancer which requires you to have AWS credentials. So in order to call the EKS or the Kubernetes API behind the load balancer, you need to authenticate first as an IAM user and say, hey this is me, I'm at AWS accounts, please let me access the cluster and this plugin makes that seamless for kube control. So we'll get to it later as we actually mess with the so we'll get to it later as we kind of deploy workloads and set up a cluster. But for now, you should just know that this is one of the required components of using an

EKS cluster and you've made it through the high and now you've made it through the high level segment of what EKS is and how it kind of works in some of the components at an abstract level, what they're doing. Next, we're going to dive into a little more details in how specific things work, especially around node groups and scaling and VPCS. So get ready to learn a little bit more about the lower levels of what is required inside of AWS and how that integrates into a Kubernetes cluster.

Authorization

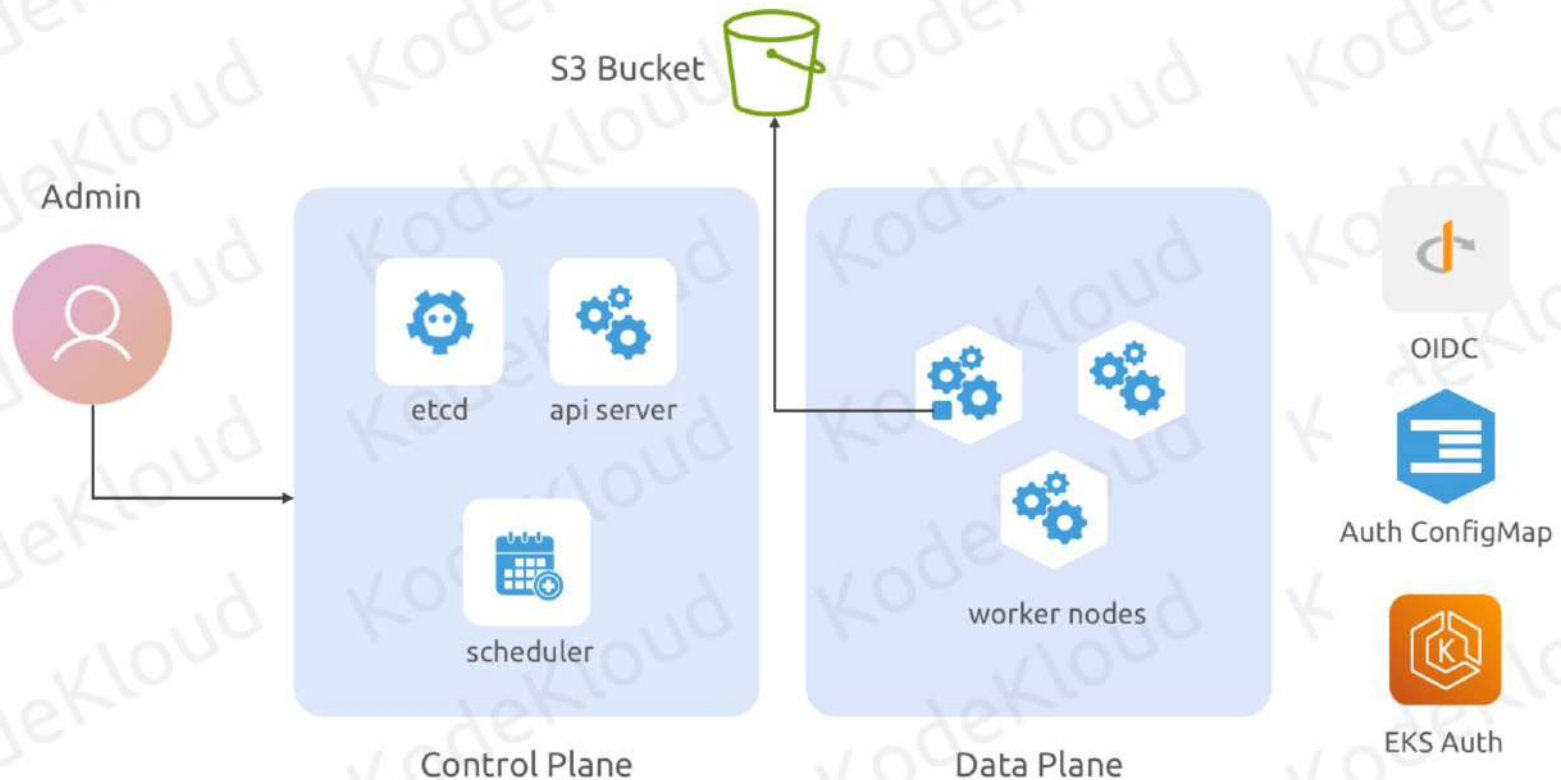
Introduction



© Copyright KodeKloud

The final thing we want to talk about with an EKS cluster is how you access things in the cluster and things inside AWS. These are all components that work together to figure out who you are and if you have access to reach the API server, if a pod can call S3. All of those things are kind of difficult when you have something that runs a little separate from AWS within an AWS environment.

Authentication



© Copyright KodeKloud

We're going to dive into all of these details in a future segment. But just as a general overview, again, you have your control plane and your data plane, and let's say somewhere out here you have an S3 bucket and a pod inside your data plane somewhere that needs to call out to the S3 bucket and get something. How do you do that? How do you say that this process has access to find the S3 bucket and even? And also how would I say that me as an admin out here can access the control plane? I need to be able to authenticate and say who I am and route myself into that control plane to say yes, I have access to do these things. There's a couple different components inside of an EKS cluster. One is OIDC which is Openid Connect and that's one way that we can identify how things are authenticated within a DKS cluster and within an AWS

accounts. There's an off config map that's lives within your cluster and does this mapping around who has access inside the cluster and how those relate to AWS. I am credentials. And finally, there's the EKSAPIS itself that can allow access into clusters by handling some of the permissions on the control plane side as part of the service. These are three different ways that things can get mapped and authenticated within the cluster and within the accounts, and we'll dive into those in the future.



What's Next?

Networking

© Copyright KodeKloud

If you're running a Kubernetes cluster on Prem or on your local laptop, you don't really worry about those permissions because everything around you you might already have permission to. But inside of an AWS account, everything is tightly with IAM permissions and identity that you need to be able to interact with and prove that your process has access to do something or your user has access to do something. So mapping all those pieces together does get a little bit complicated, but hopefully in that segment we can show you why these segments, why these, But in that section, we'll show you how these pieces are different and how they work together or when you might use one or the other so you can verify if this is something that you actually want to do. So that's it For the introduction and overview sections. We're going to dive right

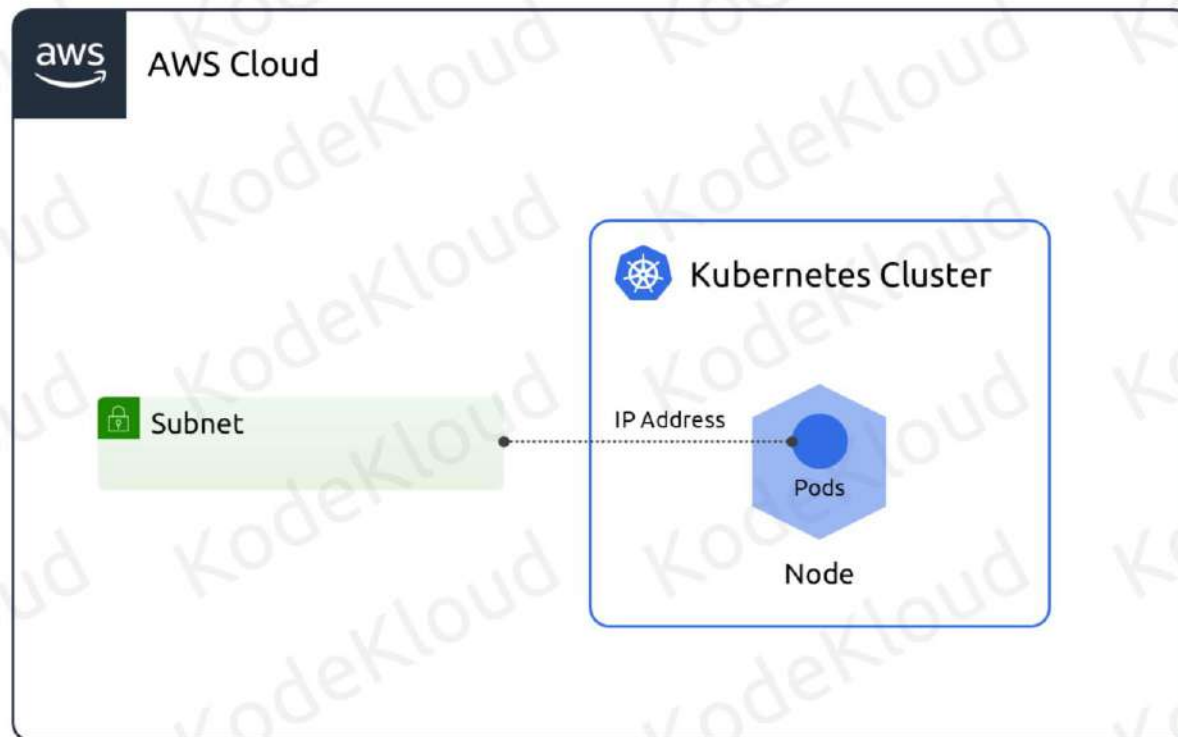
into the networking components because that's really where you want to start any of your Kubernetes planning inside of an AWS account with EKS. Some of the key key components around that are the EKS APIs themselves and the OIDC endpoints, and so we're going to dive deep and show you how those work in a future segments. But I just wanted to touch on them first because this might be different than what you're used to with like a vanilla Kubernetes cluster.

How CNI Works

© Copyright KodeKloud

In the introduction we showed you a little bit of how the VPC CNI networking works.

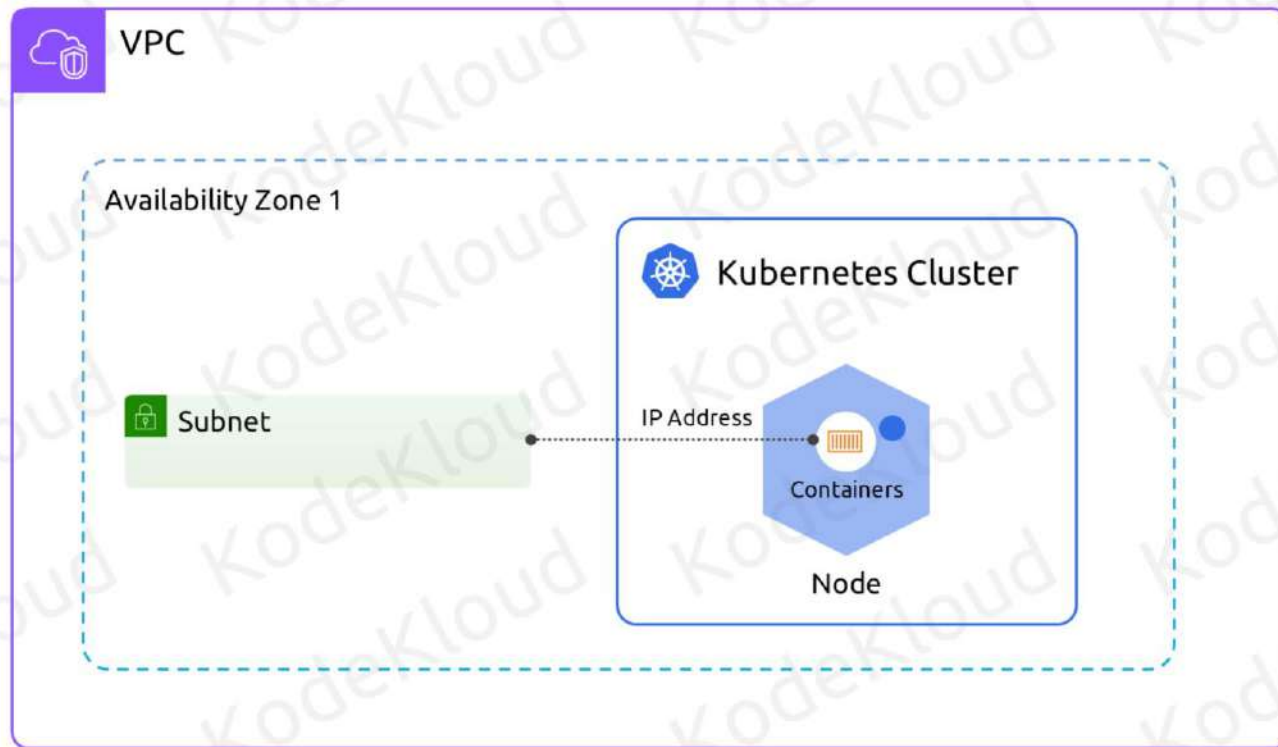
How CNI Works



© Copyright KodeKloud

Kubernetes cluster that runs in an AWS environment needs some things and particularly it usually gets IP addresses from the subnets that your nodes are running in and the pods.

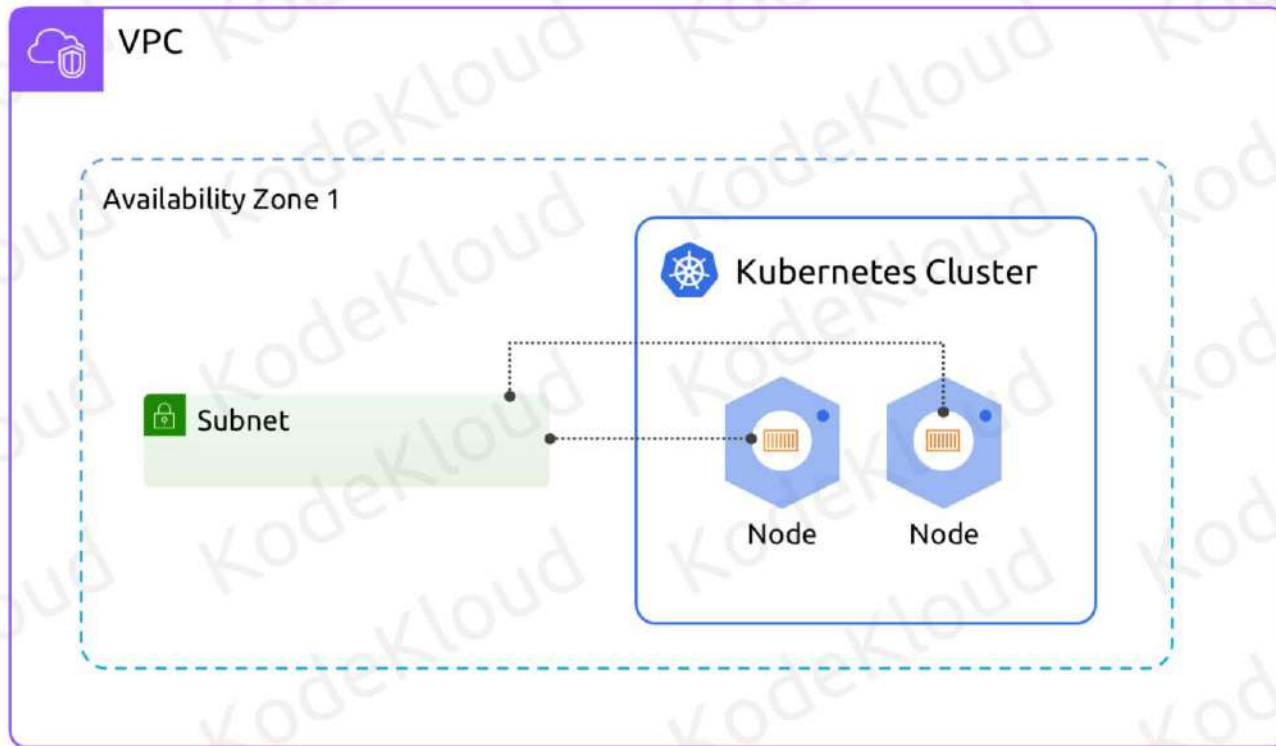
How CNI Works



© Copyright KodeKloud

The containers on the nodes themselves also consume IP addresses from the subnets within the VPC. And if you know how subnets work inside of VPC, subnets are bound to the availability zone where they are contained.

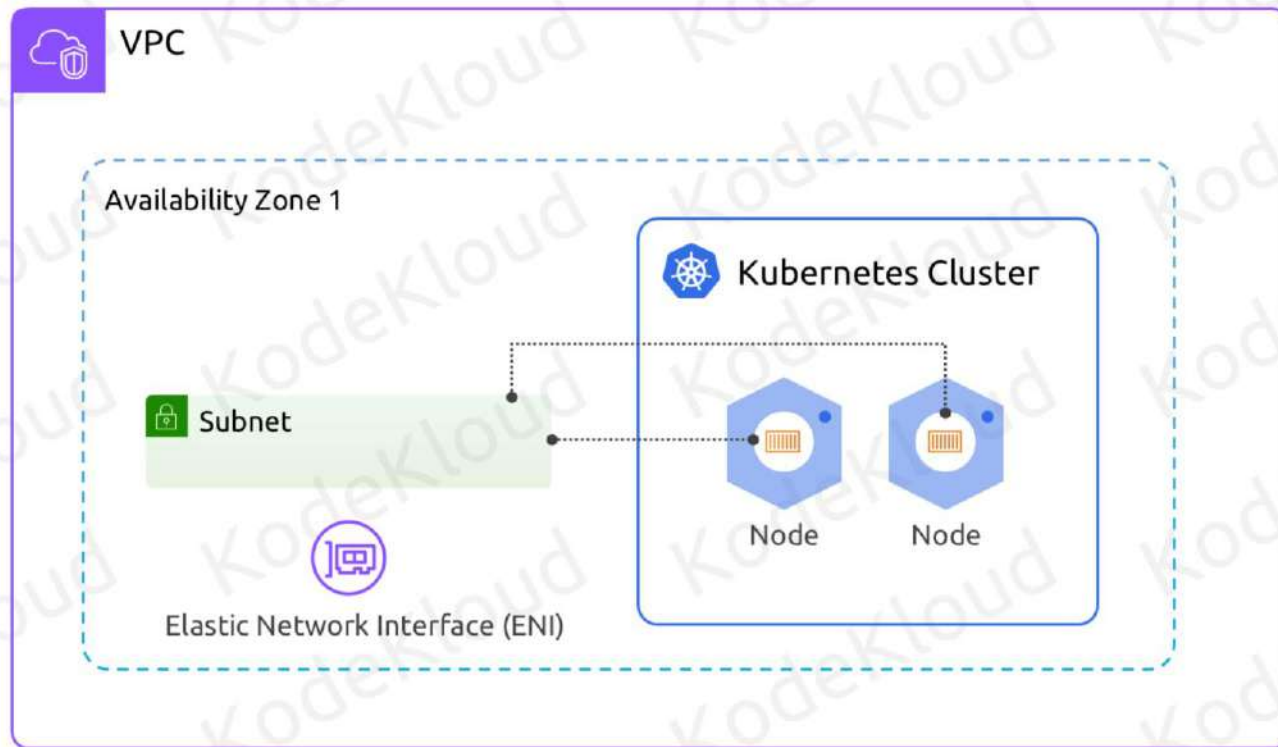
How CNI Works



© Copyright KodeKloud

So every availability zone where your nodes are running, each one has its own subnets, and each node is going to pull IP addresses from the subnet in that availability zone. Again, those are part of a VPC and that's how it's getting IP addresses for the nodes and then also the pods.

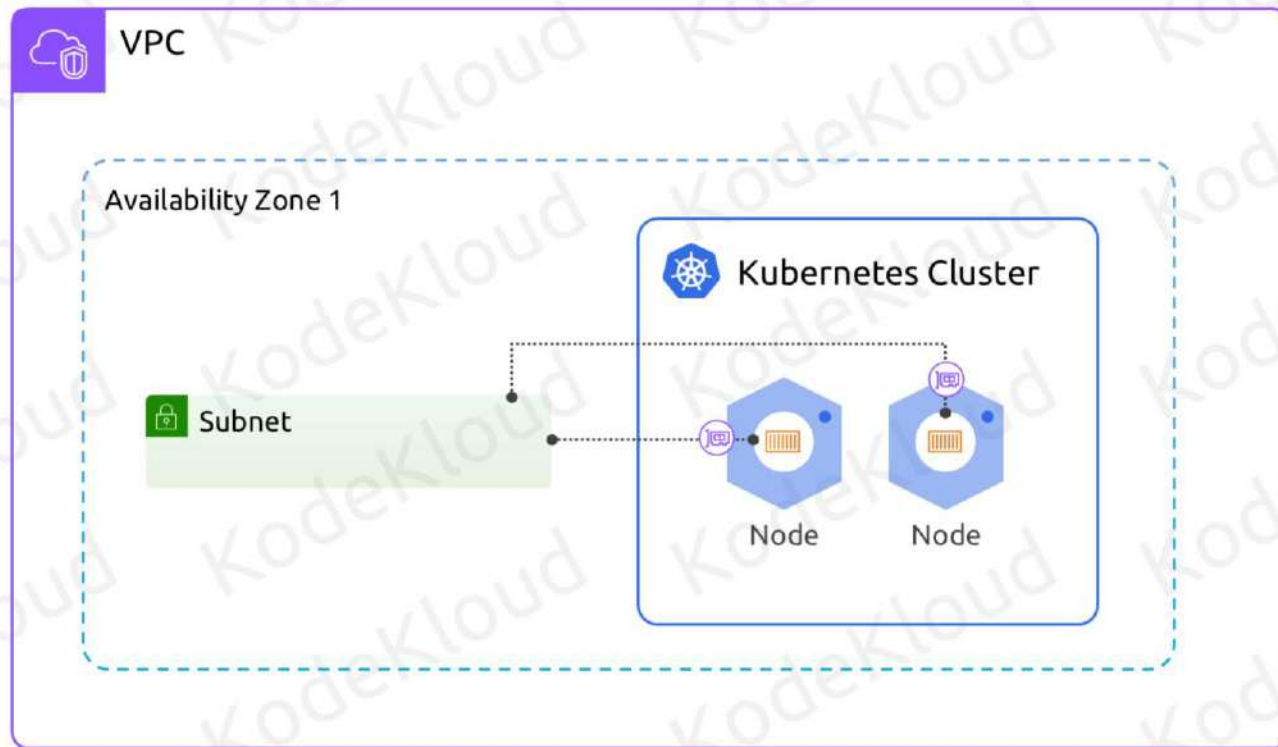
How CNI Works



© Copyright KodeKloud

And you may also remember that each of those nodes and pods are using Elastic Network Interfaces (ENIs) to be able to access and plug into the network where they're where they're running in the subnet. But let's actually just show you how it works instead of showing you diagrams and theoretical. I'm I have a cluster, let's look at it.

How CNI Works



© Copyright KodeKloud

And you may also remember that each of those nodes and pods are using Elastic Network Interfaces (ENIs) to be able to access and plug into the network where they're where they're running in the subnet. But let's actually just show you how it works instead of showing you diagrams and theoretical. I'm I have a cluster, let's look at it.

CNI Prefix Delegation

Introduction

01

How and why of configuring a CNI

02

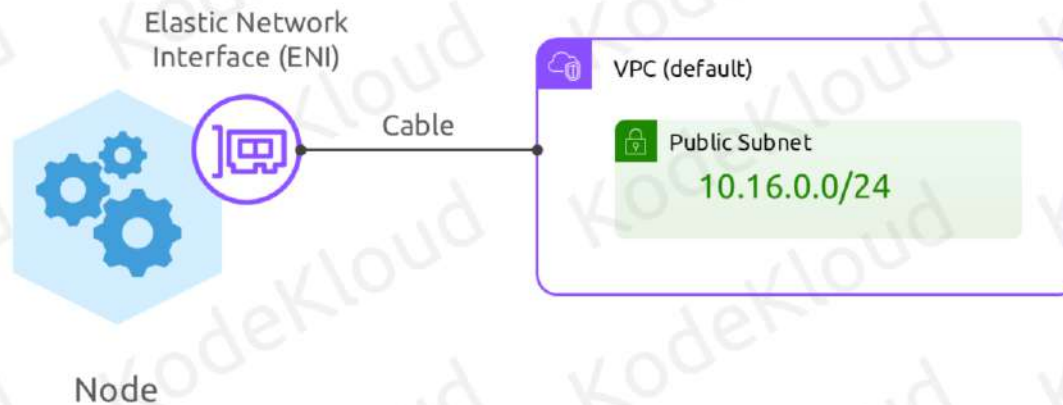
Working of Network Cards, ENIs, and IP Addresses inside a VPC

03

Details of the actual working system

Okay, so now that we know some basics on how the VPC CNI works, let's go into how it's configured in a little more details in why you might want to configure it one way or another. And this part might be a little bit complicated because this isn't something that you normally would see on an actual physical server. But the way that network cards, Enis and IP addresses work inside of a VPC is sometimes a little bit different. So let's just try to show you how it works first, and then we'll dive into showing you the details on an actual working system.

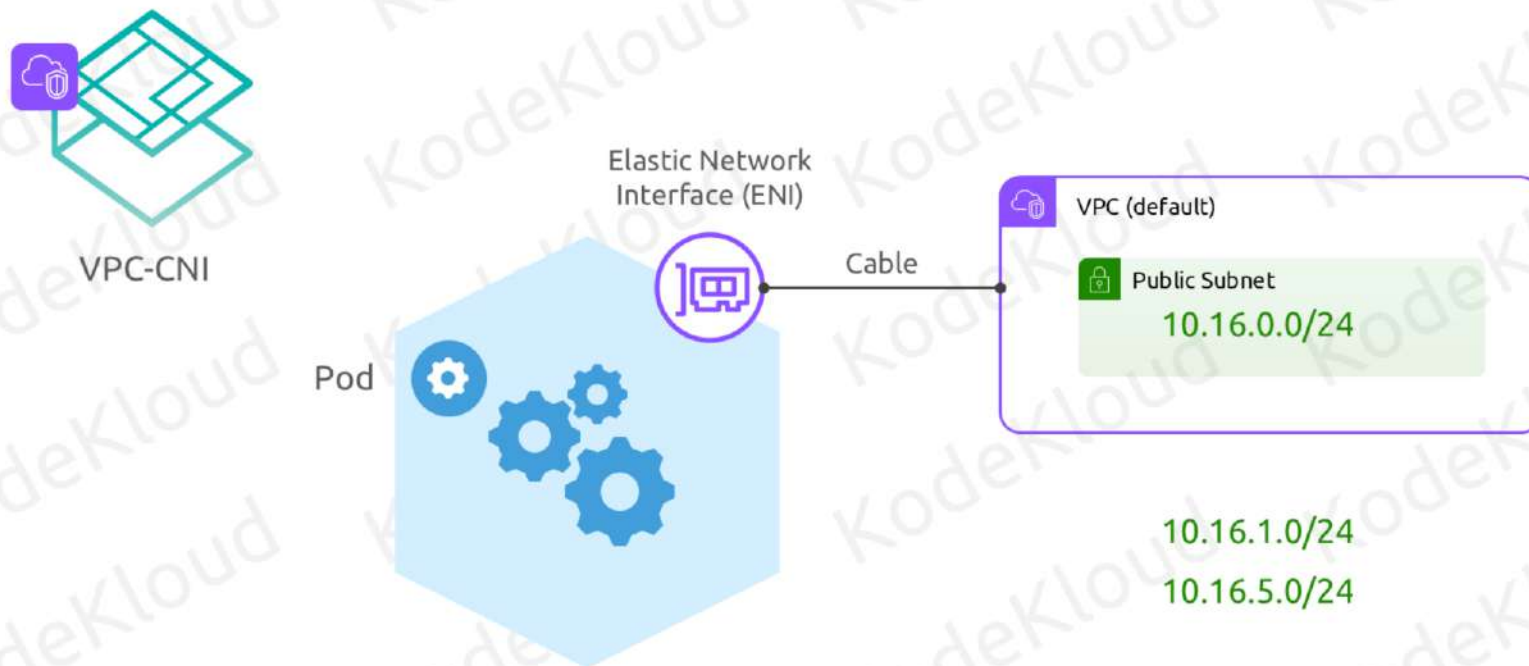
Attaching Node to Cluster



© Copyright KodeKloud

All right, so let's just say that this is one of my nodes inside of my cluster. And this node, in order to just come up and join the cluster at all, needs to have an ENI attached to it. And again, this is a network interface that, that gets attached to a system just like a normal network card would be on a laptop or a desktop or a physical server. So if you think of it that way, all we do is we plug in a cable here and then this connects out to the VPC to get an IP address, because all IP addresses are still handled by the VPC or actually by the subnet which, which is within the VPC. So we're getting our IP addresses from the subnets and those are just being assigned to our node on this ENI. And we're going to have a primary IP address for this node, which is how it actually communicates.

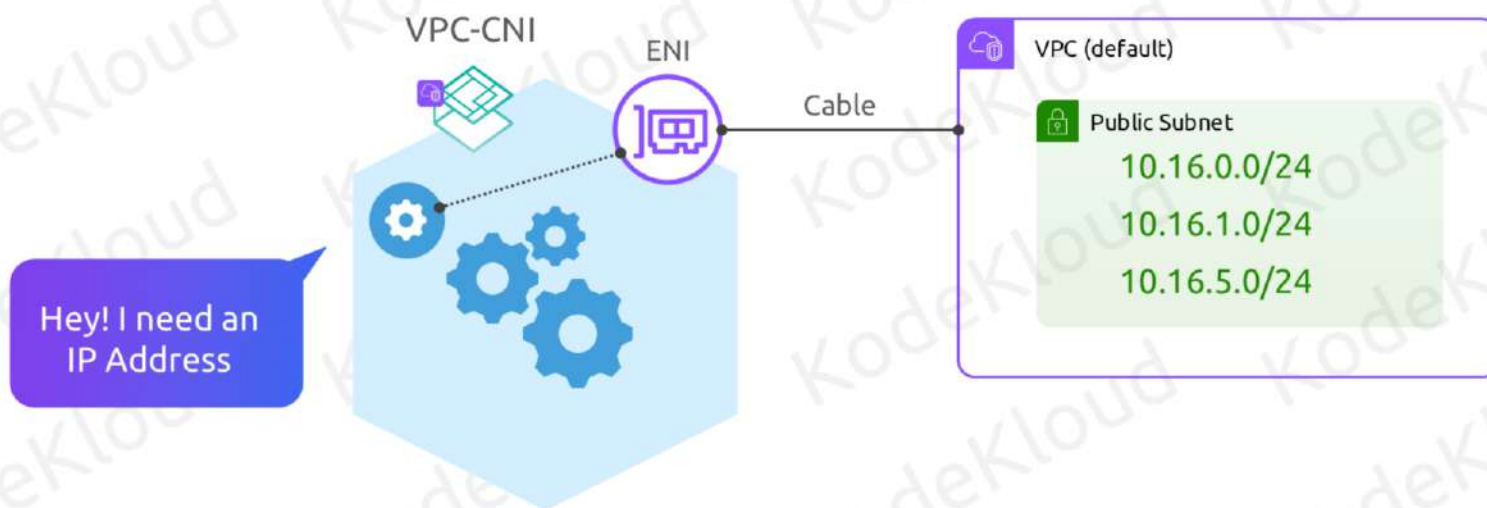
Attaching Node to Cluster



© Copyright KodeKloud

The Kubelet itself communicates back into the rest of the Kubernetes API services. But remember, we have this VPC CNI which handles pod networking. So when we have a pod that is created on the server itself, it gets a different pool of IP addresses. Typically it would get some sort of either overlay or something else that's managed it, but it's a VPC CNI, so it's still actually going to consume IP addresses from the VPC. So the CNI is running on each one of your nodes and it's handling how IP addresses get assigned. Pod comes up, it starts up and it says, hey, I need an IP address.

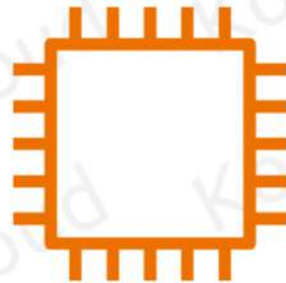
Attaching Node to Cluster



The Kubelet itself communicates back into the rest of the Kubernetes API services. But remember, we have this VPC CNI which handles pod networking. So when we have a pod that is created on the server itself, it gets a different pool of IP addresses. Typically it would get some sort of either overlay or something else that's managed it, but it's a VPC CNI, so it's still actually going to consume IP addresses from the VPC. So the CNI is running on each one of your nodes and it's handling how IP addresses get assigned. Pod comes up, it starts up and it says, hey, I need an IP address. And thankfully we can actually get an IP address from the existing ENI on the node. And the ENI actually can, is capable of handling multiple IP addresses at the same time. And thankfully we can actually get an IP address from the existing ENI on the node. And the ENI

actually can, is capable of handling multiple IP addresses at the same time. So one, what you could think of as physical network port can have multiple IP addresses assigned to it. And that's great because then we don't need to worry about another cable or another Eni. In this case, we don't need to plug anything else into the server, but different instance types.

EC2 Instances

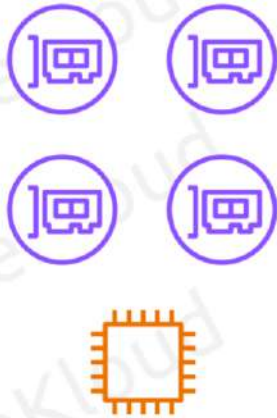


Instances

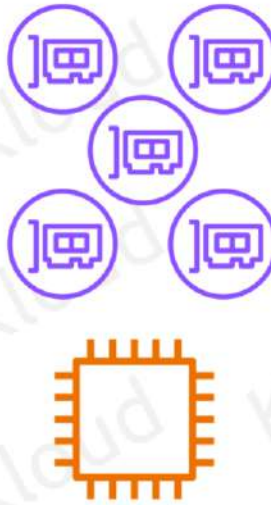
EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five ips per eni. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that Eni can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. Eni's actually need to be registered within the VPC and the subnet and attached to a

node and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one.

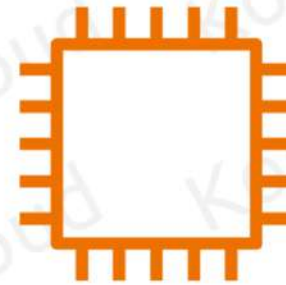
EC2 Instances



Small Instances



Medium Instances

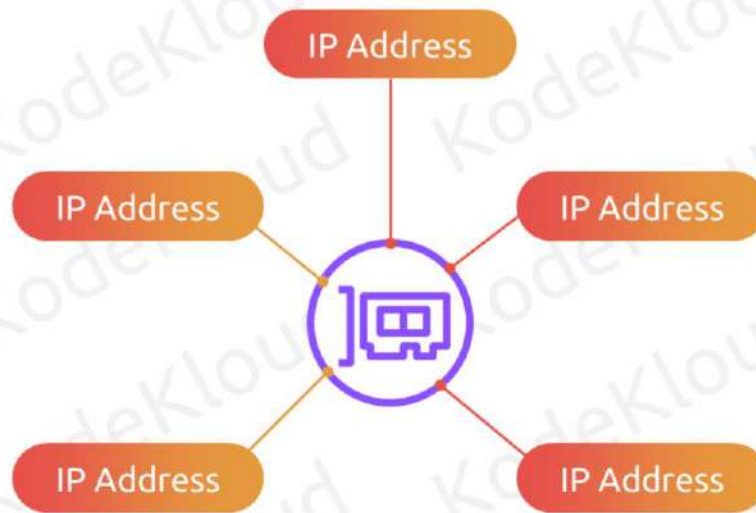


Large Instances

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five ips per eni. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that Eni can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. En eyes actually need to be registered within the VPC and the subnet and attached to a

node and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one.

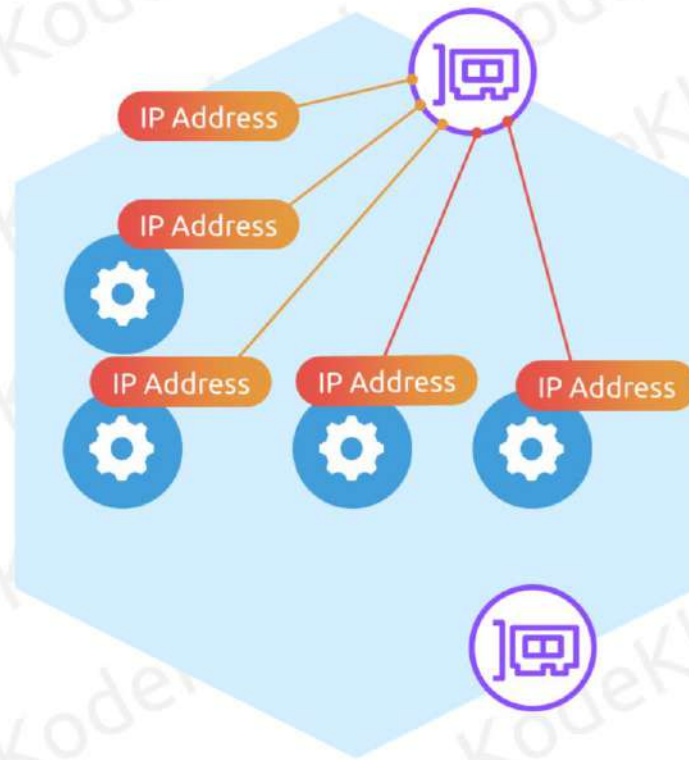
EC2 Instances



EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that Eni can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. Eni's actually need to be registered within the VPC and the subnet and attached to a

node and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one.

EC2 Instances

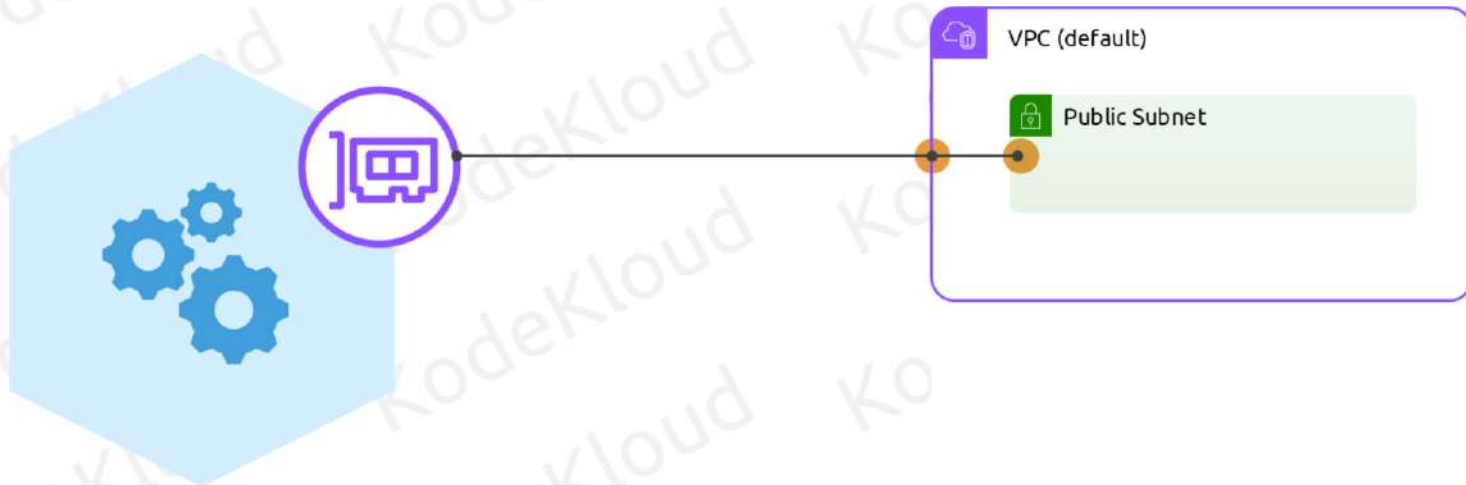


© Copyright KodeKloud

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one.

EC2 Instances

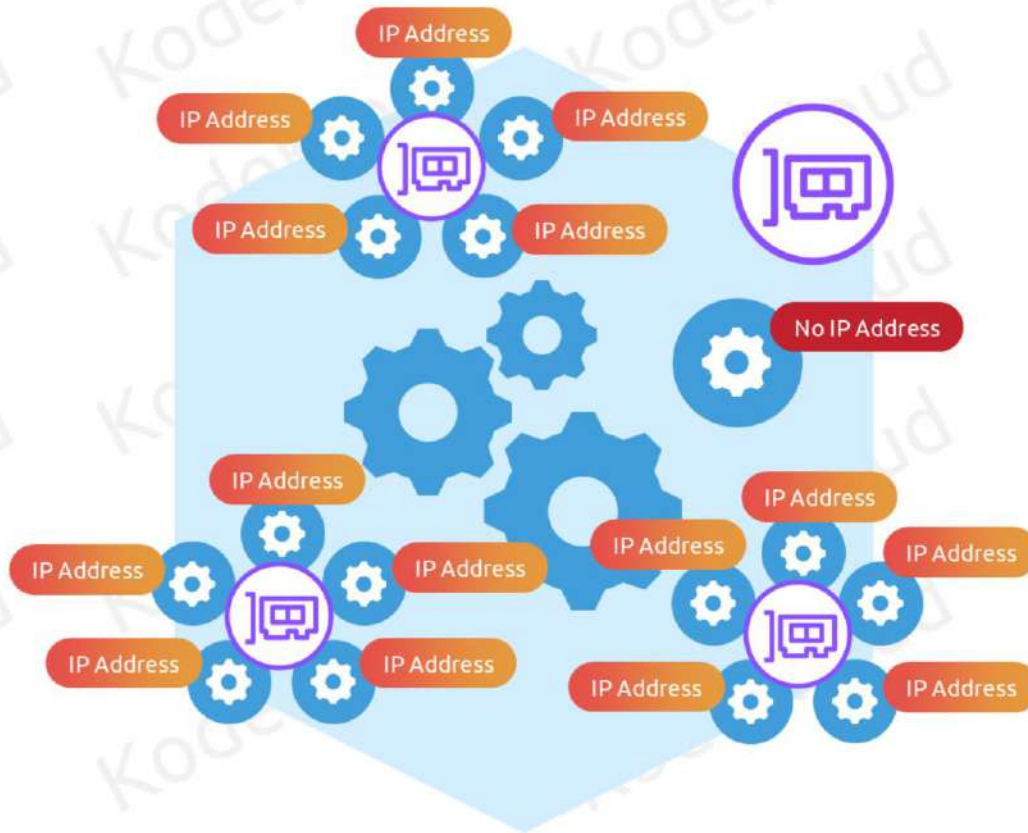


© Copyright KodeKloud

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one.

EC2 Instances

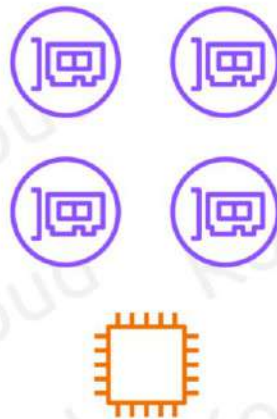


© Copyright KodeKloud

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one. So you need the first one plus three more just to be able to handle all IP addresses for those pods. But if we schedule a 16th pod, and let's say this instance type only can have four ENIs, this 16th pod is not going to get an IP address. You have consumed all of the IP addresses available to this node. So, while it might try to schedule and try to start up, it'll never connect to the network. There's a couple different ways we can fix that. One is by create using larger instance types that can have more Enis and more IP addresses per ENI. Or we can actually have what's called prefix delegation, which gives us more pod IP addresses per eni. And we'll show you how that works in just a second. Just leave it.

EC2 Instances

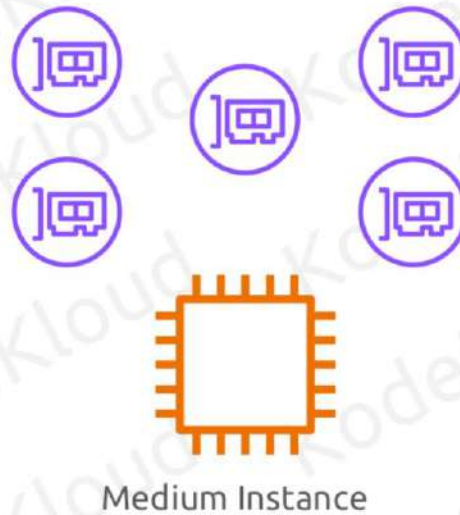


Small Instance

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one. So you need the first one plus three more just to be able to handle all IP addresses for those pods. But if we schedule a 16th pod, and let's say this instance type only can have four ENIs, this 16th pod is not going to get an IP address. You have consumed all of the IP addresses available to this node. So, while it might try to schedule and try to start up, it'll never connect to the network. There's a couple different ways we can fix that. One is by create using larger instance types that can have more Enis and more IP addresses per ENI. Or we can actually have what's called prefix delegation, which gives us more pod IP addresses per eni. And we'll show you how that works in just a second. Just leave it.

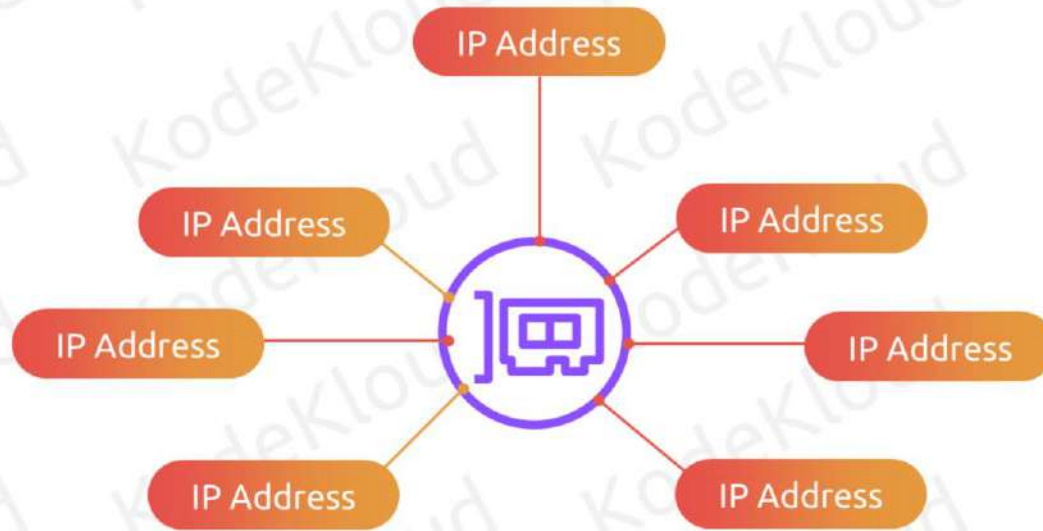
EC2 Instances



EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one. So you need the first one plus three more just to be able to handle all IP addresses for those pods. But if we schedule a 16th pod, and let's say this instance type only can have four ENIs, this 16th pod is not going to get an IP address. You have consumed all of the IP addresses available to this node. So, while it might try to schedule and try to start up, it'll never connect to the network. There's a couple different ways we can fix that. One is by create using larger instance types that can have more ENIs and more IP addresses per ENI. Or we can actually have what's called prefix delegation, which gives us more pod IP addresses per eni. And we'll show you how that works in just a second. Just leave it.

EC2 Instances

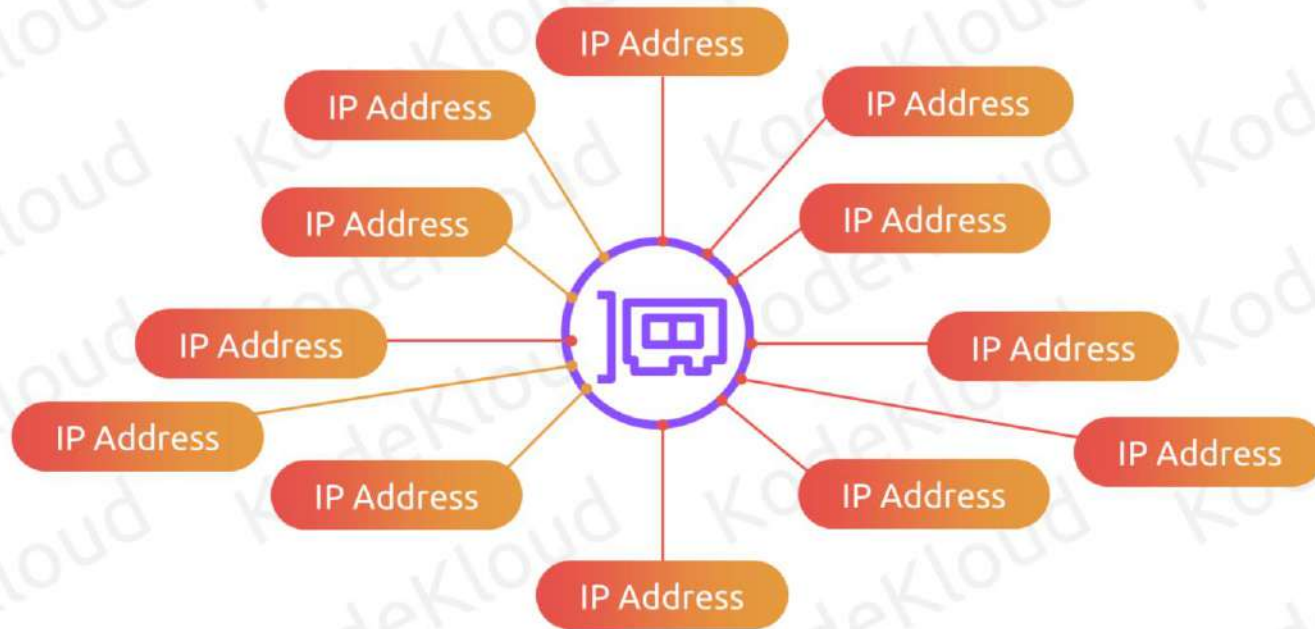


© Copyright KodeKloud

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one. So you need the first one plus three more just to be able to handle all IP addresses for those pods. But if we schedule a 16th pod, and let's say this instance type only can have four ENIs, this 16th pod is not going to get an IP address. You have consumed all of the IP addresses available to this node. So, while it might try to schedule and try to start up, it'll never connect to the network. There's a couple different ways we can fix that. One is by create using larger instance types that can have more ENIs and more IP addresses per ENI. Or we can actually have what's called prefix delegation, which gives us more pod IP addresses per eni. And we'll show you how that works in just a second. Just leave it.

Prefix Delegation



© Copyright KodeKloud

EC two instance types have different limits on how many IP addresses and how many ENIs they can attach to them. So, a small or medium instance may only be able to have four or five ENIs, and each of those ENIs might only be able to have four or five IP addresses. So once this server, let's say it's a small instance, and, and just for the sake of argument, we have an IP limit of five IPs per ENI. So once we have four pods, we are requesting four different IP addresses for the workloads and there's one for the node. So now we have consumed all five IP addresses that that ENI can handle. So what do we need to do? We need to get another Eni attached to this node, and that's usually fine. It does take a little bit longer than just getting an IP address from the subnets. ENIs actually need to be registered within the VPC and the subnet and attached to a node

and then brought up and get an IP. All this stuff has to happen, so it's not instantaneous. And if you all of a sudden want to schedule, say, 15 more pods on this node, you are going to need 15 more IP addresses, which means you're going to need three enis from that first one. So you need the first one plus three more just to be able to handle all IP addresses for those pods. But if we schedule a 16th pod, and let's say this instance type only can have four ENIs, this 16th pod is not going to get an IP address. You have consumed all of the IP addresses available to this node. So, while it might try to schedule and try to start up, it'll never connect to the network. There's a couple different ways we can fix that. One is by create using larger instance types that can have more ENIs and more IP addresses per ENI. Or we can actually have what's called prefix delegation, which gives us more pod IP addresses per eni. And we'll show you how that works in just a second. Just leave it.

ENI Attachment



I also want to talk to you real quick on just that Eni attachment, because that's actually a bottleneck for a lot of nodes when you're trying to create a lot of pods across the cluster. These ENIs need to be attached, and like I said, they're kind of slow, they're not instantaneous. It's not as immediate as just, hey, I just need more IP addresses. Give them to me. Now, you will be waiting maybe 1015 or seconds or even a minute for some of these ENIs and IP addresses to be available in the subnet, especially depending on how, how many IP addresses you have available in the subnets. Because let's say you're running low on the amount of IP addresses you need to wait for leases to expire before you can actually get an IP address. So leases might be set for a couple hours or a day. So if pods are churning really fast, let's say you have a thousand IP addresses. Let's

say you have a thousand.What the. It's not working.

ENI Attachment



I also want to talk to you real quick on just that Eni attachment, because that's actually a bottleneck for a lot of nodes when you're trying to create a lot of pods across the cluster. These ENIs need to be attached, and like I said, they're kind of slow, they're not instantaneous. It's not as immediate as just, hey, I just need more IP addresses. Give them to me. Now, you will be waiting maybe 1015 or seconds or even a minute for some of these ENIs and IP addresses to be available in the subnet, especially depending on how, how many IP addresses you have available in the subnets. Because let's say you're running low on the amount of IP addresses you need to wait for leases to expire before you can actually get an IP address. So leases might be set for a couple hours or a day. So if pods are churning really fast, let's say you have a thousand IP addresses. Let's

say you have a thousand.What the. It's not working.

ENI Attachment



I also want to talk to you real quick on just that Eni attachment, because that's actually a bottleneck for a lot of nodes when you're trying to create a lot of pods across the cluster. These ENIs need to be attached, and like I said, they're kind of slow, they're not instantaneous. It's not as immediate as just, hey, I just need more IP addresses. Give them to me. Now, you will be waiting maybe 1015 or seconds or even a minute for some of these ENIs and IP addresses to be available in the subnet, especially depending on how, how many IP addresses you have available in the subnets. Because let's say you're running low on the amount of IP addresses you need to wait for leases to expire before you can actually get an IP address. So leases might be set for a couple hours or a day. So if pods are churning really fast, let's say you have a thousand IP addresses.

ENI Attachment



950 Pods



I also want to talk to you real quick on just that Eni attachment, because that's actually a bottleneck for a lot of nodes when you're trying to create a lot of pods across the cluster. These ENIs need to be attached, and like I said, they're kind of slow, they're not instantaneous. It's not as immediate as just, hey, I just need more IP addresses. Give them to me. Now, you will be waiting maybe 1015 or seconds or even a minute for some of these ENIs and IP addresses to be available in the subnet, especially depending on how, how many IP addresses you have available in the subnets. Because let's say you're running low on the amount of IP addresses you need to wait for leases to expire before you can actually get an IP address. So leases might be set for a couple hours or a day. So if pods are churning really fast, let's say you have a thousand IP addresses.

ENI Attachment



Lease



950 IP Addresses

950 Pods



Only 50 unleased IPs

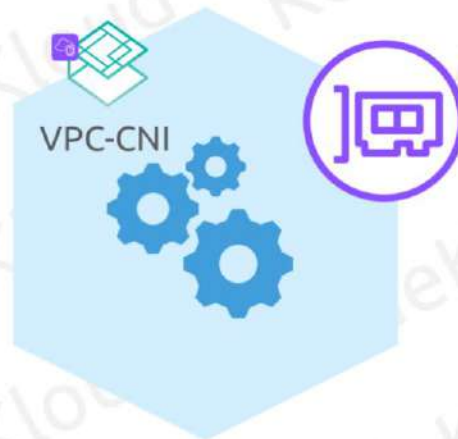
51 Pods



I also want to talk to you real quick on just that Eni attachment, because that's actually a bottleneck for a lot of nodes when you're trying to create a lot of pods across the cluster. These ENIs need to be attached, and like I said, they're kind of slow, they're not instantaneous. It's not as immediate as just, hey, I just need more IP addresses. Give them to me. Now, you will be waiting maybe 1015 or seconds or even a minute for some of these ENIs and IP addresses to be available in the subnet, especially depending on how, how many IP addresses you have available in the subnets. Because let's say you're running low on the amount of IP addresses you need to wait for leases to expire before you can actually get an IP address. So leases might be set for a couple hours or a day. So if pods are churning really fast, let's say you have 1000 IP addresses available in

your subnets and you create 950 pods that day. You can create and delete 950 pods. Well, if you have a day lease on those IP addresses, they're not going to necessarily disappear right away. They're still leased to those pods. And so you might run out of IP addresses if you try to schedule 51 more pods within that, whatever time the lease expires for those IP addresses. So you really have to be careful on how many IP addresses you're using in the subnet, what the lease times are. All of those things kind of matter when IP addresses are coming and going quickly in an ephemeral cluster like Kubernetes.

WARM_ENI_TARGET



So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC two instances have different numbers of Enis that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC. So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these Enis and the VPCC. And I can figure out how many ENis it needs to meet that requirement.

WARM_ENI_TARGET



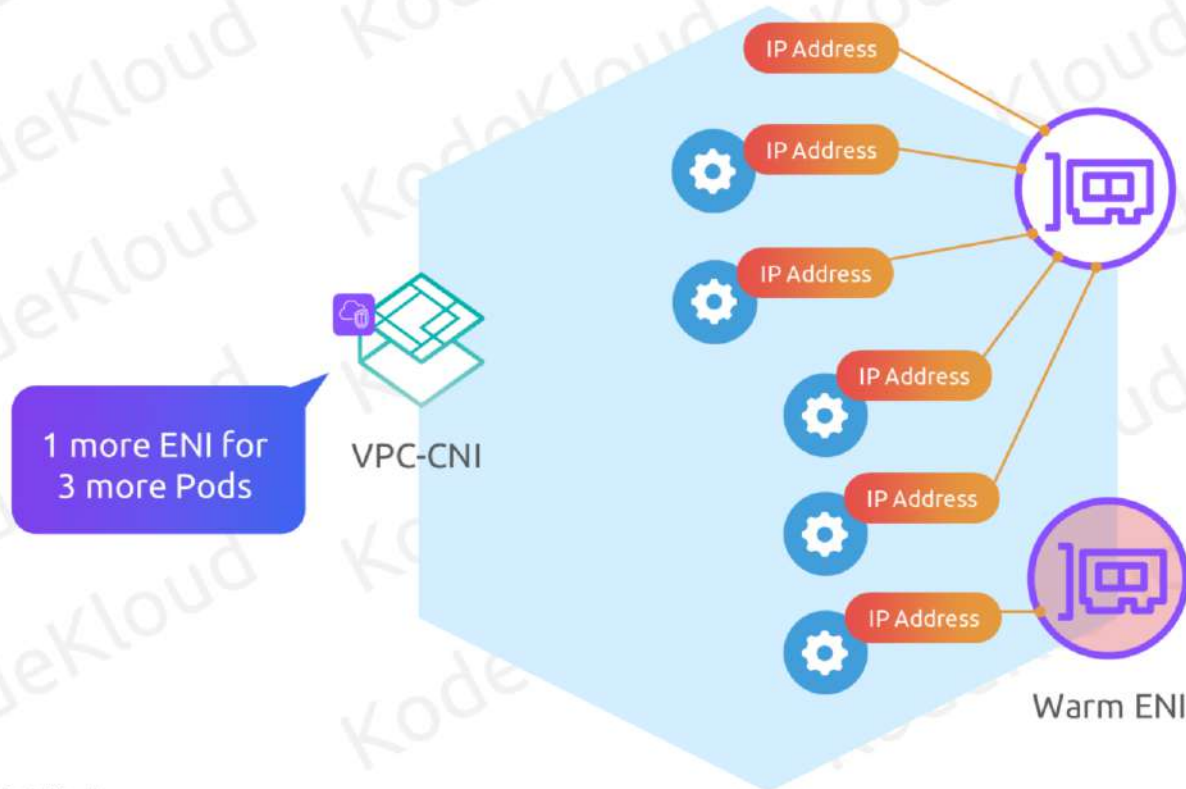
WARM_ENI_TARGET

An environment variable that keeps a certain number of ENIs available above what you currently have

So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one ENI and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC two instances have different numbers of Enis that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC. So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these Enis and the VPCC. And I can figure out how many ENis it needs to meet that requirement.

WARM_ENI_TARGET



© Copyright KodeKloud

So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC2 instances have different numbers of ENIs that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC.

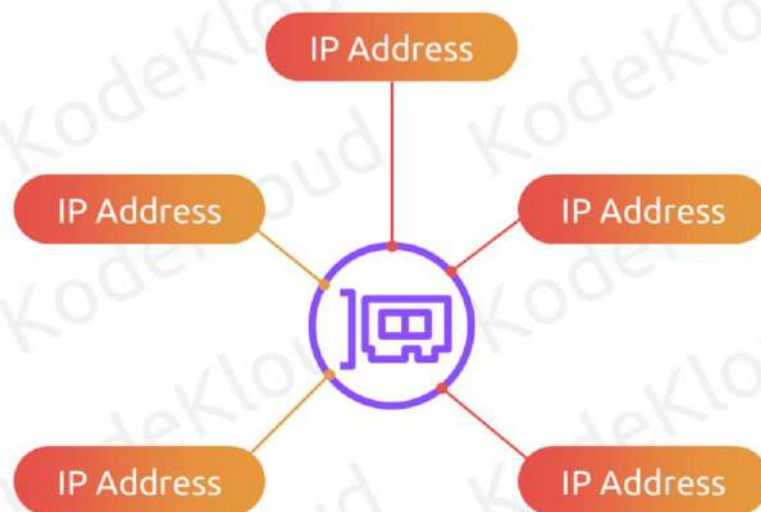
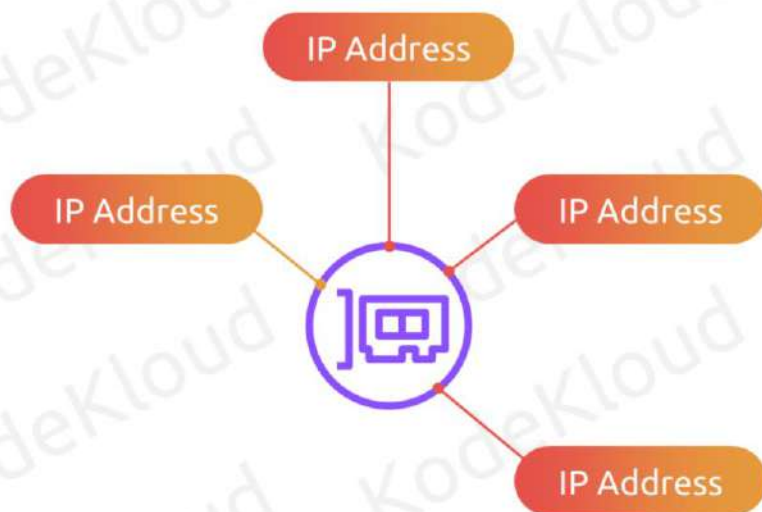
WARM_ENI_TARGET



So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC2 instances have different numbers of ENIs that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC.

WARM_ENI_TARGET



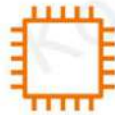
So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC2 instances have different numbers of ENIs that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC.

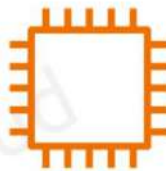
WARM_ENI_TARGET



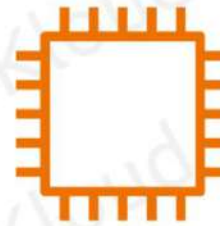
Kubernetes Cluster



Small Instances



Medium Instances

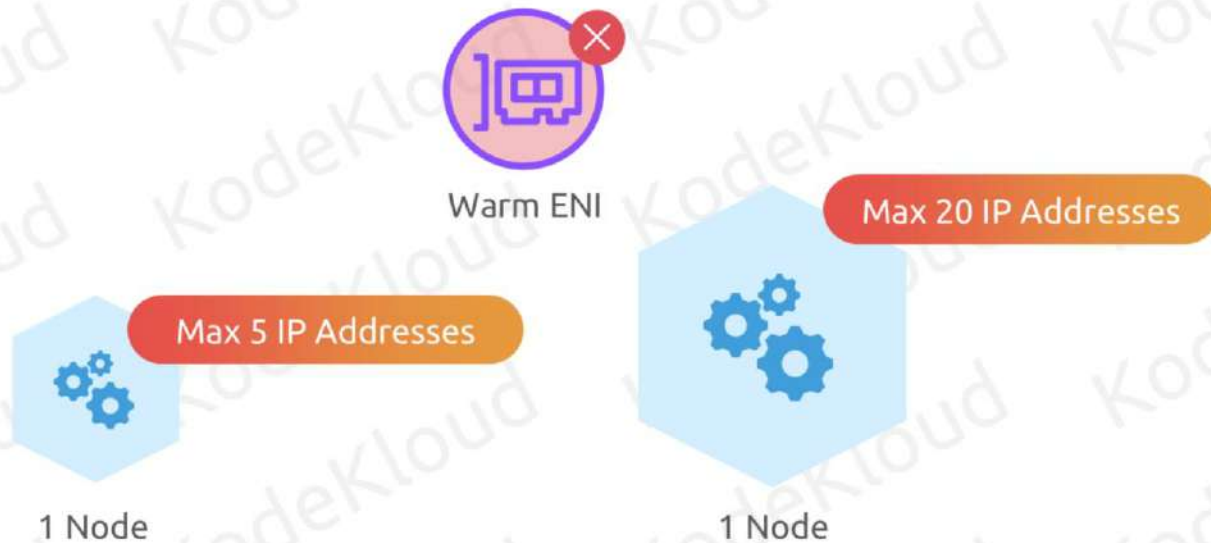


Large Instances

So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC2 instances have different numbers of ENIs that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC.

WARM_ENI_TARGET

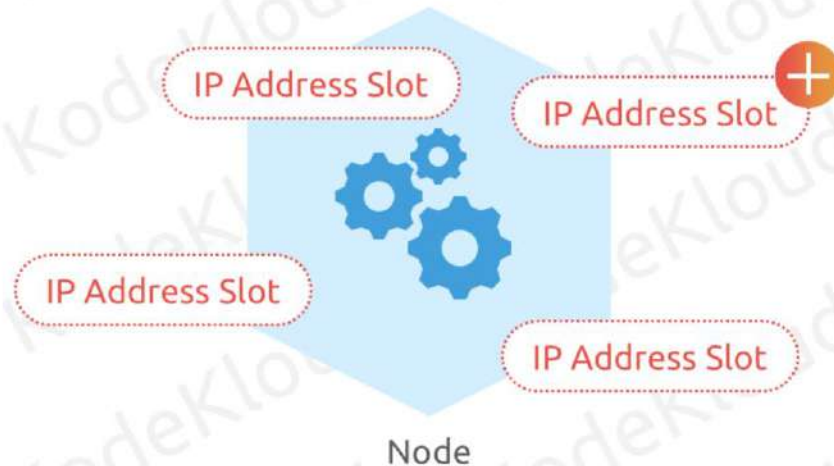


© Copyright KodeKloud

So these ENIs come in and they get attached and detached from nodes pretty regularly. And the VPC CNI has a couple variables that you can set to try to make this spin up and spin down happen a little faster. So, there's a warm, there's a warm ENI target which is an environment variable that you can set for your VPC-CNI, which is going to try to keep a certain number of ENIs available above what you currently have. So if you have one Eni and you're using three IP addresses from it, and let's again say it can have a total of five, the VPC CNI will see that and say, hey, I'm actually running low on these physical connections into my VPC, so I'm going to provision one more Eni just so that if I schedule three more pods, I can take two IP addresses from that first Eni and then I can take the third IP address from this new Eni that's just warm, it's

ready to go. It's not being used yet, but like I said, some EC2 instances have different numbers of ENIs that can attach to them and different number of IP addresses per Eni. So it's really difficult to figure out what your warm Eni target should be because if you have a cluster with mixed instance sizes, you might have one node that could only have five IP addresses per Eni and another can have 20. In that case, you don't necessarily want a warm Eni on both of them because you're just going to be wasting Eni connections inside your VPC.

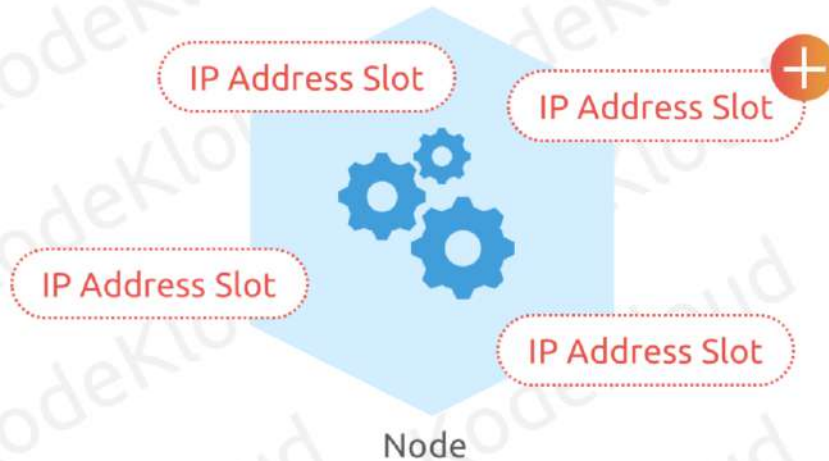
WARM_IP_TARGET



So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, if it allows, say 20 per VPC-CNI, that one will only have one extra Eni because it has a minimum set there of ten. But the smallest number you can actually allocate to the node is 20. So it's just going to give us one Eni to meet that requirement. No,

actually I misspoke on that. Erase that last part. So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the out of the existing Eni. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET



So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, if it allows, say 20 per VPC-CNI, that one will only have one extra Eni because it has a minimum set there of ten. But the smallest number you can actually allocate to the node is 20. So it's just going to give us one Eni to meet that requirement. No,

actually I misspoke on that. Erase that last part. So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the out of the existing Eni. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET

Warm_IP_Target: 10

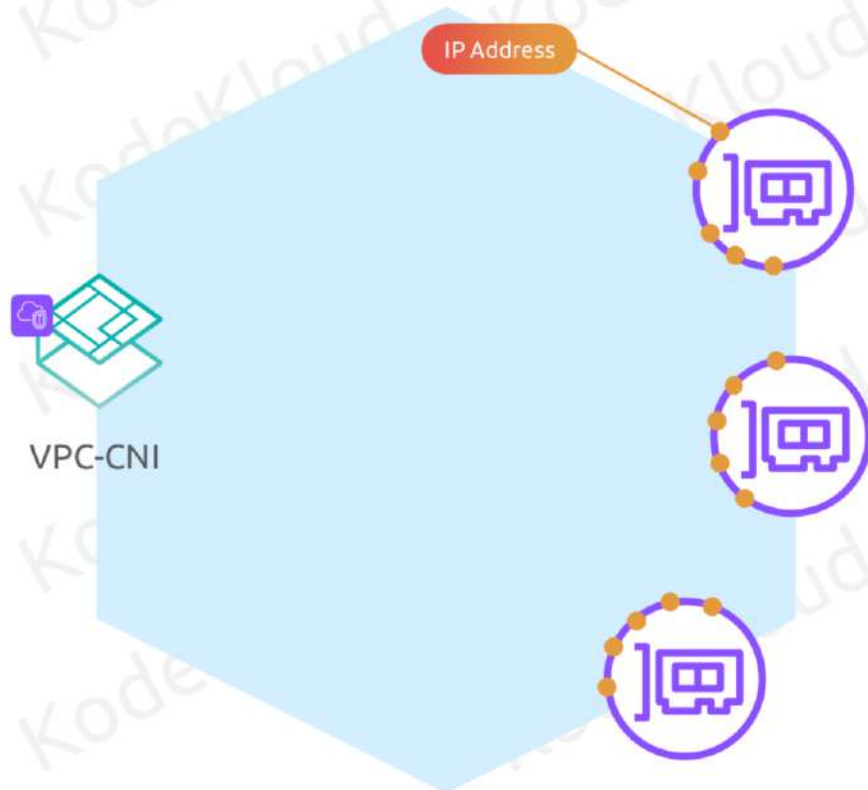


So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC-CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, if it allows, say 20 per VPC-CNI, that one will only have one extra Eni because it has a minimum set there of ten. But the smallest number you can actually allocate to the node is 20. So it's just going to give us one Eni to meet that requirement. No,

actually I misspoke on that. Erase that last part. So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the out of the existing Eni. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET

Warm_IP_Target: 10



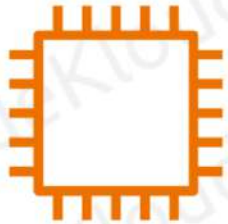
© Copyright KodeKloud

So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC-CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, if it allows, say 20 per VPC-CNI, that one will only have one extra Eni because it has a minimum set there of ten. But the smallest number you can actually allocate to the node is 20. So it's just going to give us one Eni to meet that requirement. No,

actually I misspoke on that. Erase that last part. So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the out of the existing Eni. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET

Warm_IP_Target: 10



Large Instances



VPC-CNI

20 IPs per ENI

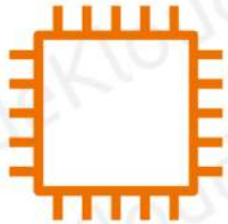


So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC-CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the

out of the existing ENI. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET

Warm_IP_Target: 10



Large Instances



VPC-CNI

20 IPs per ENI

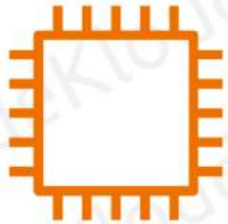


So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC-CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the

out of the existing ENI. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each eni per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way Enis have limits on these instance types. But let's talk about the other setting that I usually recommend everyone just assign on their eks clusters. And when we were writing the eks best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable.

WARM_IP_TARGET

Warm_IP_Target: 10



Large Instances



VPC-CNI

11 IPs per ENI



So there's also a warm IP target, and in this case you're going to say how many IP addresses you want available to the node based on these ENIs and the VPC-CNI can figure out how many ENIs it needs to meet that requirement. So, if you set the warm IP target to, let's say ten, and you have an ENI on an instance that can only have five total IP addresses, in this case the VPC-CNI will have you. That will give you that default one first because we're already using one IP addresses for the node itself, and it will create two new ENIs. Each of them can have five IP addresses, and then on a really large instance again, So if you have a larger node and it can have say 20 IAP addresses per Eni, and you have your IP target set to ten, in this case, it's not actually going to provision a new VPC-CNI because it can already match. It already can take ten IP addresses after the

out of the existing ENI. So until your existing Eni goes to eleven allocated IP addresses, it's not going to create a second Eni on the node to get more IP addresses. So in this case, a warm IP target will allow you to mix and match sizes a little bit better without needing to worry about the limits of each ENI per instance type. So I'm sorry if that is confusing and it just kind of is confusing. It's not something that you normally would see for a physical machine because of the way IP addresses are assigned within the VPC and subnets, and the way ENIs have limits on these instance types.

ENABLE_PREFIX_DELEGATION



But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get an eye on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached but with prefix delegation. Instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPCC and I it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this cidr. So everything under this dot, this slash 28 is going to be assigned. Jeez.

ENABLE_PREFIX_DELEGATION



But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get an eye on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached but with prefix delegation. Instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPCC and I it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this cidr. So everything under this dot, this slash 28 is going to be assigned. Jeez.

ENABLE_PREFIX_DELEGATION

Prefix Delegation



But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC and I with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get an eye on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached but with prefix delegation. Instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPCC and I it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this cidr. So everything under this dot, this slash 28 is going to be assigned. Jeez.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation



VPC-CNI

Boolean

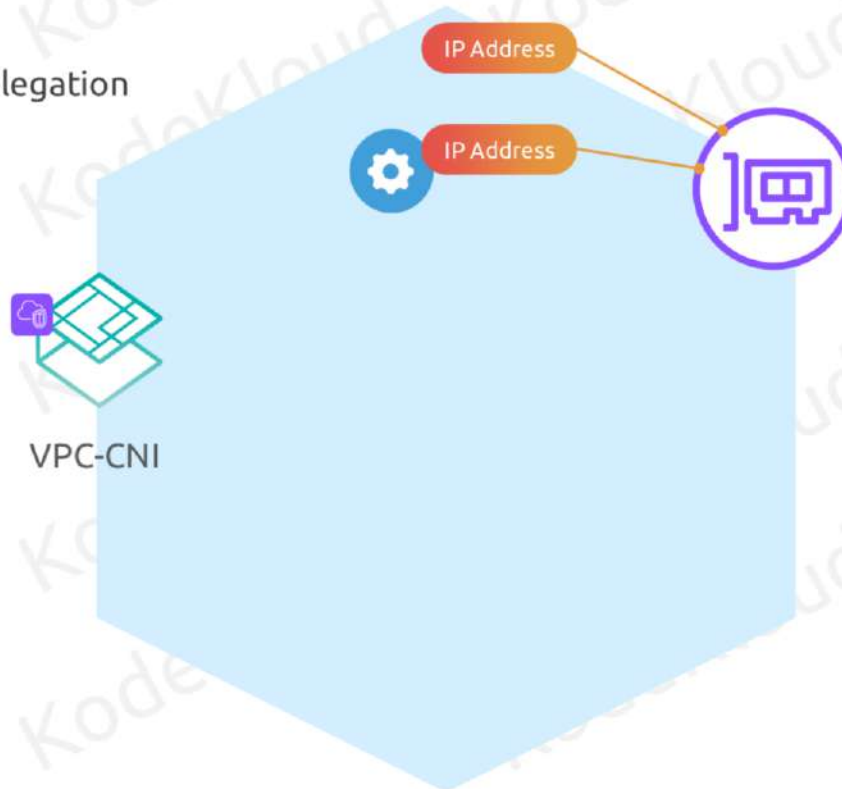
True	False
Yes	No

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached but with prefix delegation. Instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPCC and I it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this cidr. So everything under this dot, this slash 28 is going to be assigned. Jeez.

ENABLE_PREFIX_DELEGATION

☐ Prefix Delegation

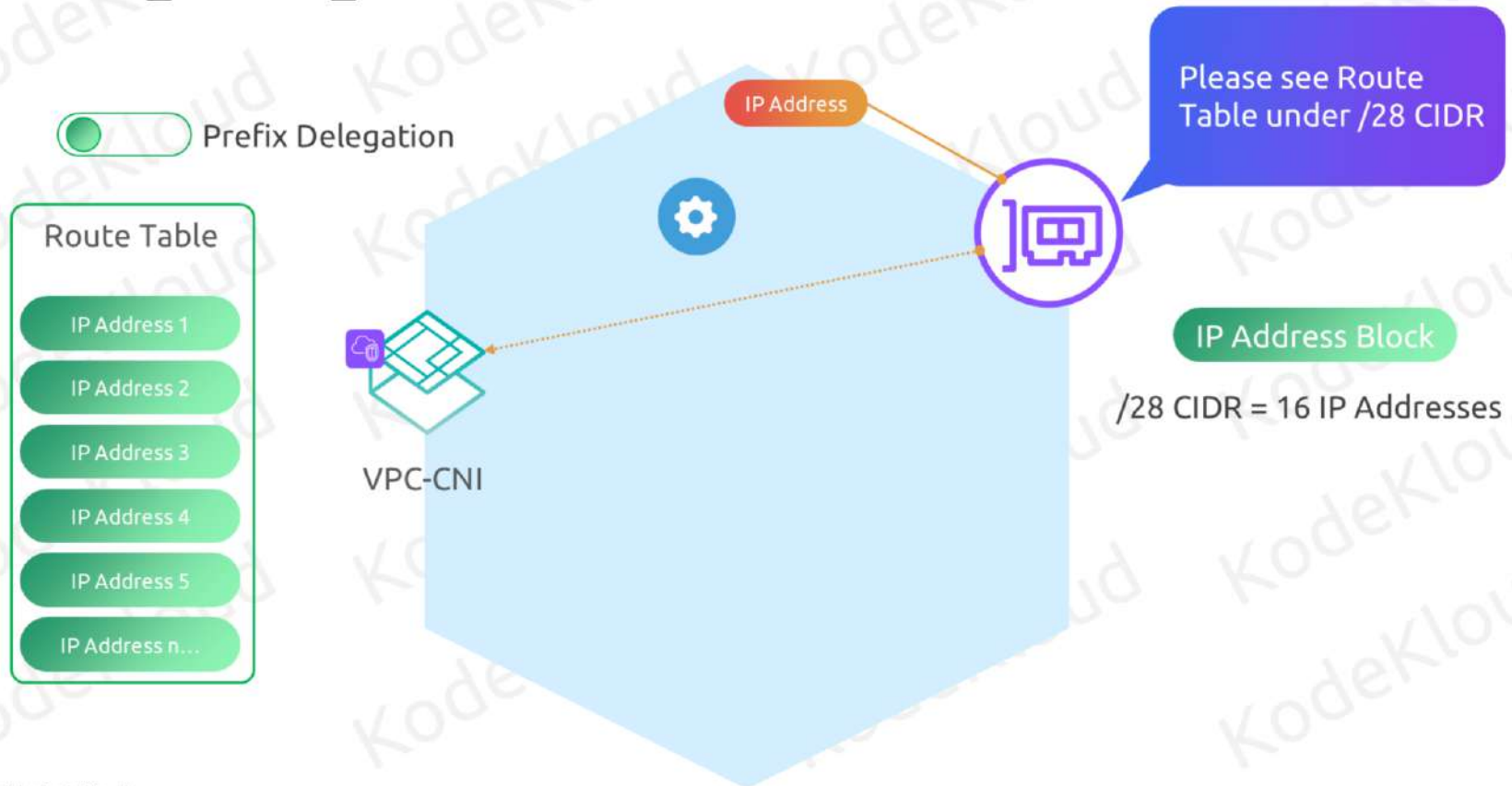


© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this cidr. So everything under this dot, this slash 28 is going to be assigned. So everything under this slash 28 is going to be assigned to this one ip address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an Eni and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION



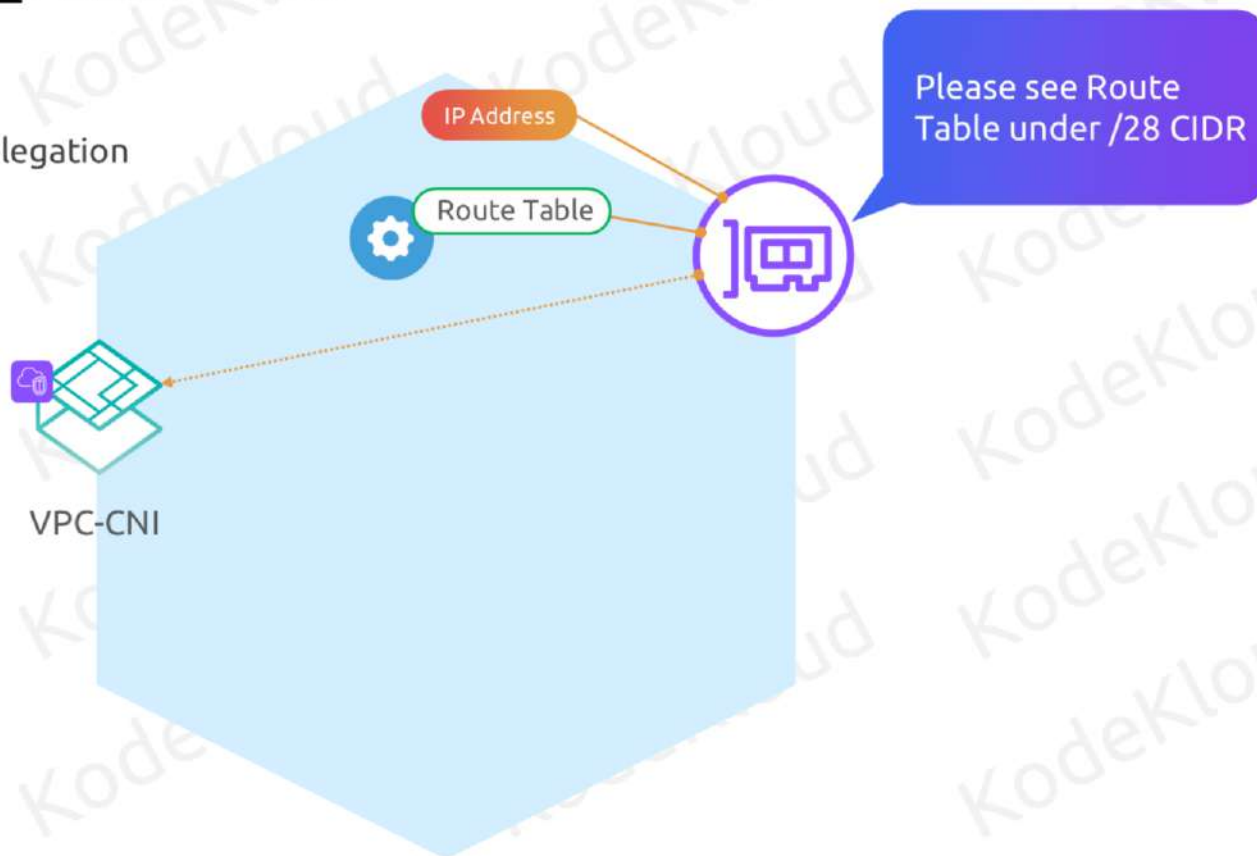
© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an Eni and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation



© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an Eni and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION



Route Table

IP Address 1

IP Address 2

IP Address 3

IP Address 4

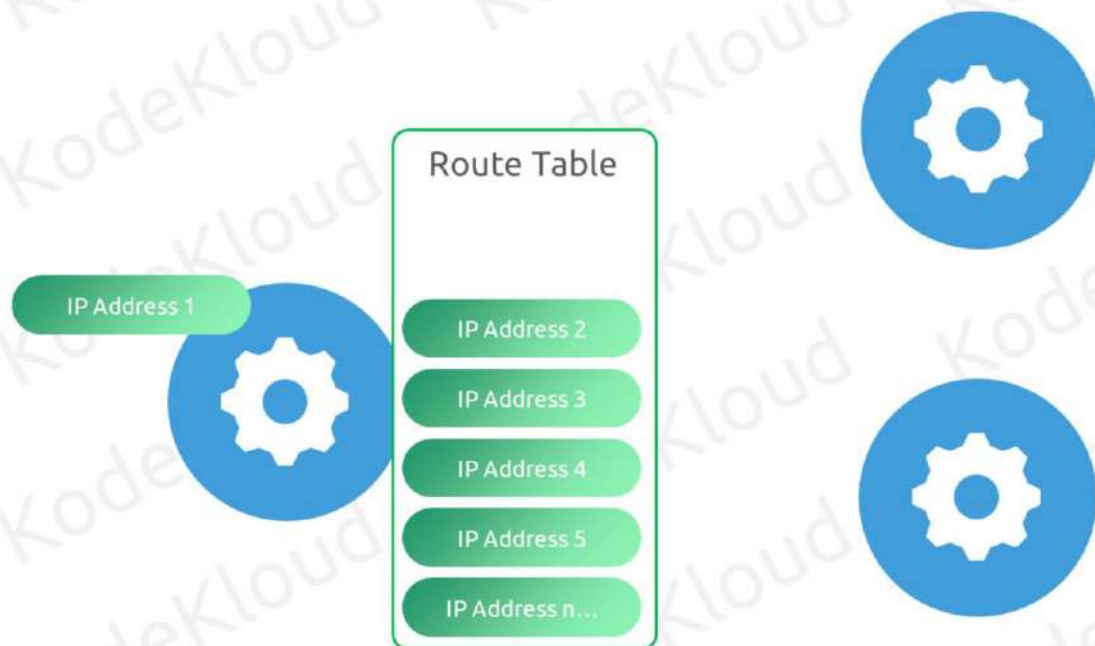
IP Address 5

IP Address n...

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the enable prefix delegation environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an Eni and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION



But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an Eni and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION



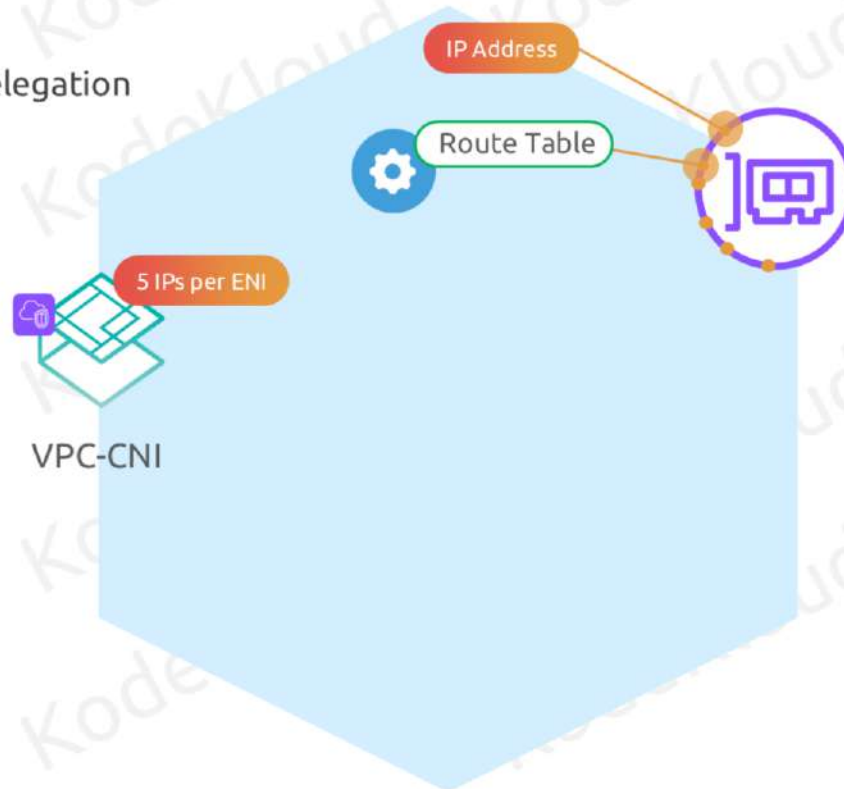
Pod = Router

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the ENI to get 16 IP addresses for pods. So what that means is if we have a node that again can have an ENI and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default Eni at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation



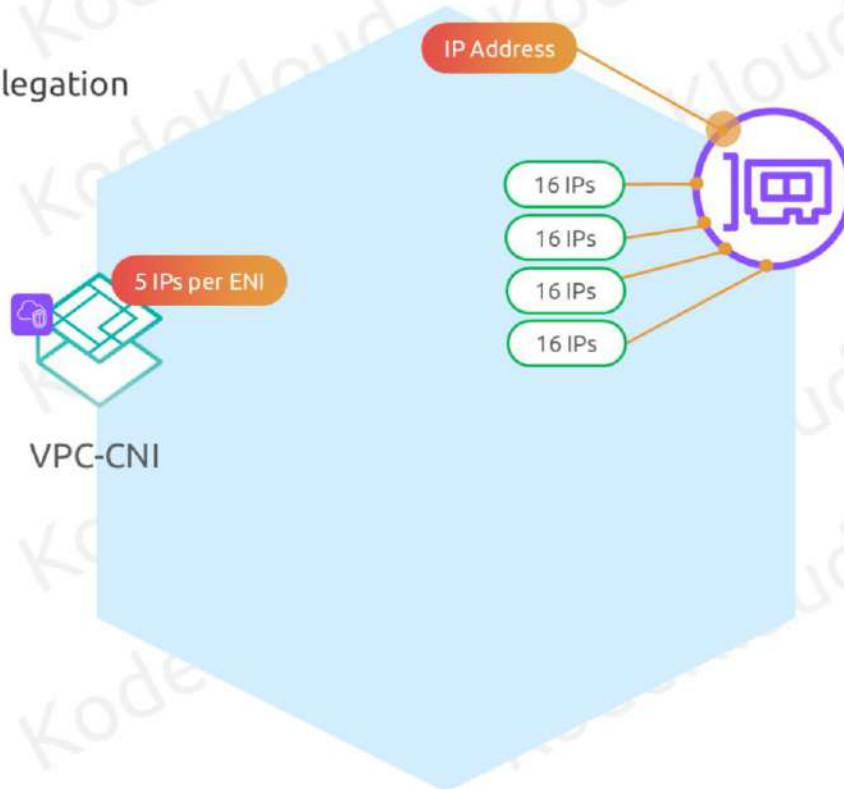
© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an ENI and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default ENI at least. So we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation



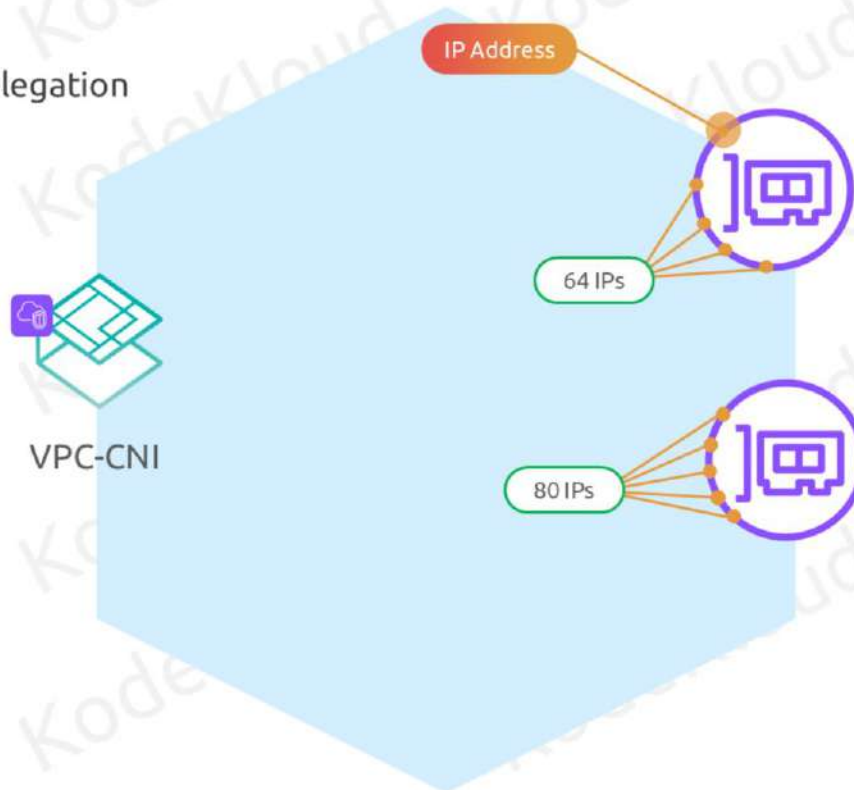
© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an ENI and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default ENI at least. So, we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation



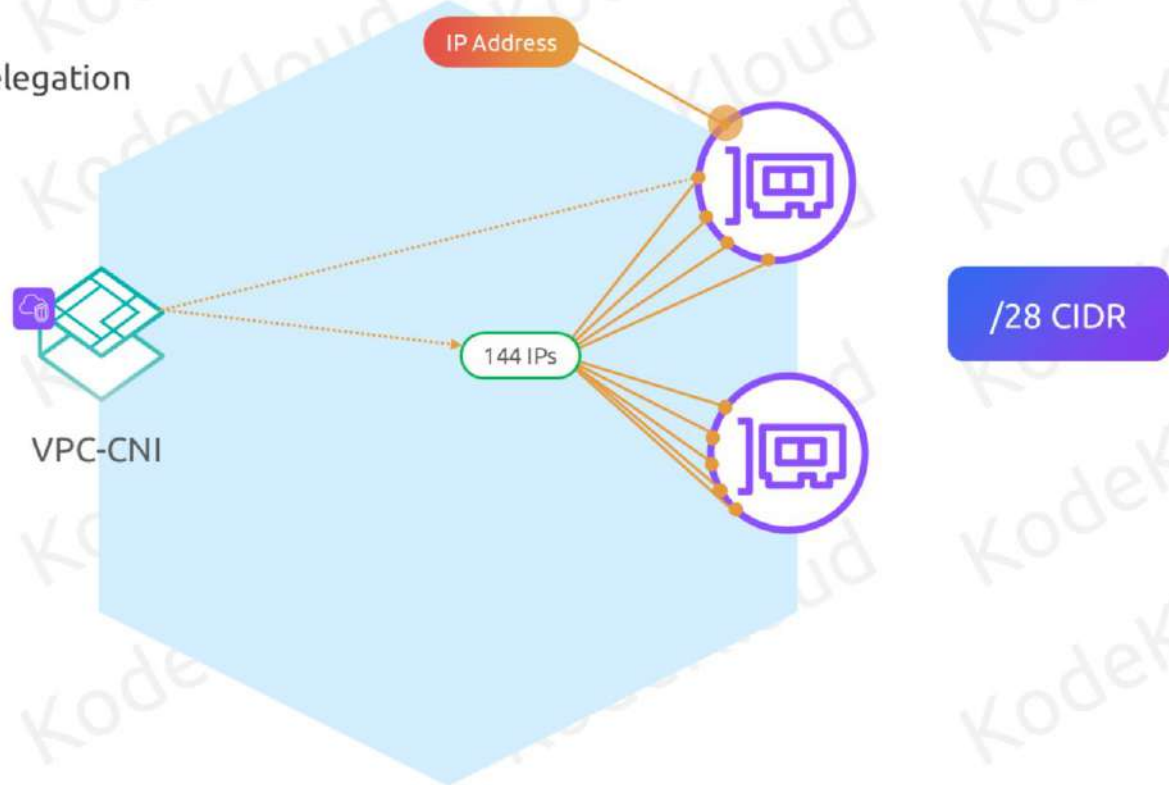
© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an ENI and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default ENI at least. So, we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation. and so not only does this mean you have more IP addresses available to the node automatically by just taking this prefix delegation, but also by the time you need a new ENI attached to it, you get now you'll have five for the second Eni because you're not taking one for the node and you'll get 80 IP addresses. So which is two Enis, we now have 144 IP addresses for pods.

ENABLE_PREFIX_DELEGATION

☒ Prefix Delegation

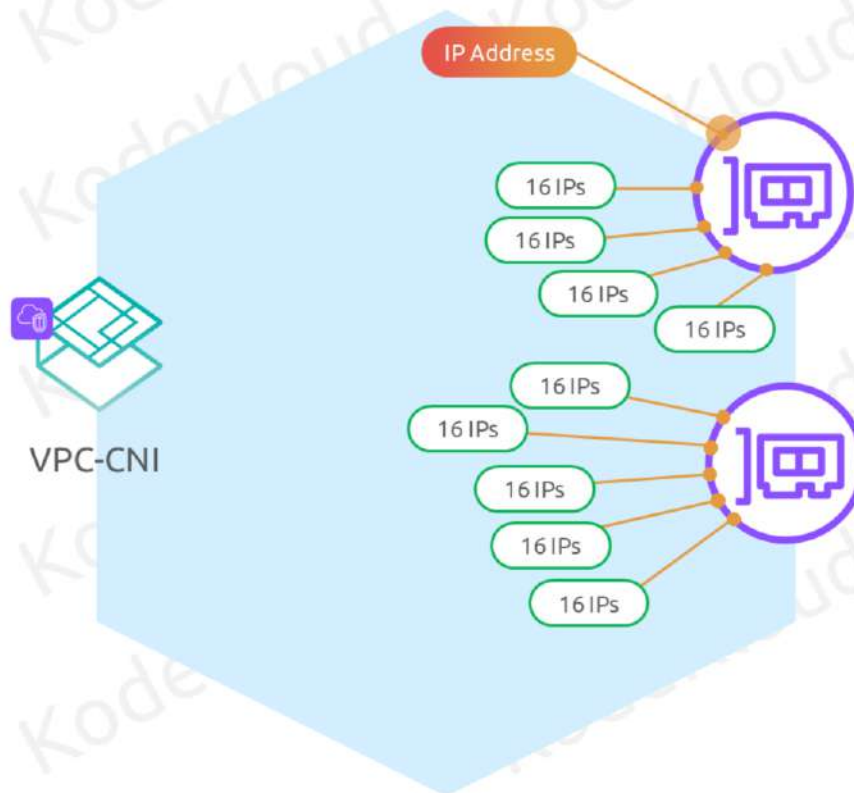


© Copyright KodeKloud

But let's talk about the other setting that I usually recommend everyone just assign on their EKS clusters. And when we were writing the EKS best practices for scalability, this was one of the settings that we absolutely recommended everyone used and that's called prefix delegation. You can enable it in the VPC-CNI with the `enable prefix delegation` environment variable. And this is just a boolean, you just say yes or no, true or false. And in this case we're still going to get ENIs on our nodes and those are still going to have one IP address assigned for the node at the, at the beginning. So we get one IP address that's already taken away. And now when we schedule a pod on this node, it's still going to ask for an IP address from the ENI that's attached. But with prefix delegation, instead of assigning one IP address, we actually assign a block of IP

addresses, a slash 28 cidr or 16 IP addresses. So what this does is it actually makes a route inside the VPC-CNI it says hey, this IP address that I'm putting in here as an IP on this node is actually just a route table for all of these other IP addresses underneath this CIDR. So everything under this slash 28 is going to be assigned to this one IP address going to this one node. And everything inside the VPC will automatically get that route. That route just happens to work because that's how route tables inside of a VPC and a subnet work. So this pod can just take one IP address from that CIDR and now the route table just says, oh, I don't know actually where that's assigned, just go to this next hop, this next box, which is kind of like a router or. Yeah, which is kind of like a router in the network, that node now becomes a router for this 28 address. And again we only took one IP address from the Eni to get 16 IP addresses for pods. So what that means is if we have a node that again can have an ENI and take five IP addresses, let's just keep going with that number. It can take five total IP addresses. And remember one of those is used for the node already for the default ENI at least. So, we have four available slash 28s available. And so if we take four times 16, what do we get? What do we get? Four, two. So if we take four times 16 we get 64 IP addresses off of one Eni. And that's a lot better than five, I guess that's a lot better. And that's a lot better than four available that we had before without prefix delegation. and so not only does this mean you have more IP addresses available to the node automatically by just taking this prefix delegation, but also by the time you need a new ENI attached to it, you get now you'll have five for the second Eni because you're not taking one for the node and you'll get 80 IP addresses. So which is two Enis, we now have 144 IP addresses for pods. And that's great because we don't have to wait for ENIs to be assigned to the node through the API calls and attached, which often is slow and so scaling ends up being slow. This prefix delegation just works by adding those routes and giving us blocks of usable IP addresses before giving us blocks of usable pod IP addresses for each ENI IP address that we consume. The one downside of prefix delegation is because it assigns blocks in 28s, you actually need to have consistent 28 cidrs available. So sometimes if your subnets are really small in your VPC, you may not have enough blocks available. And so you'll have to check that to make sure that your subnet is large enough to have multiple blocks of 28 available.

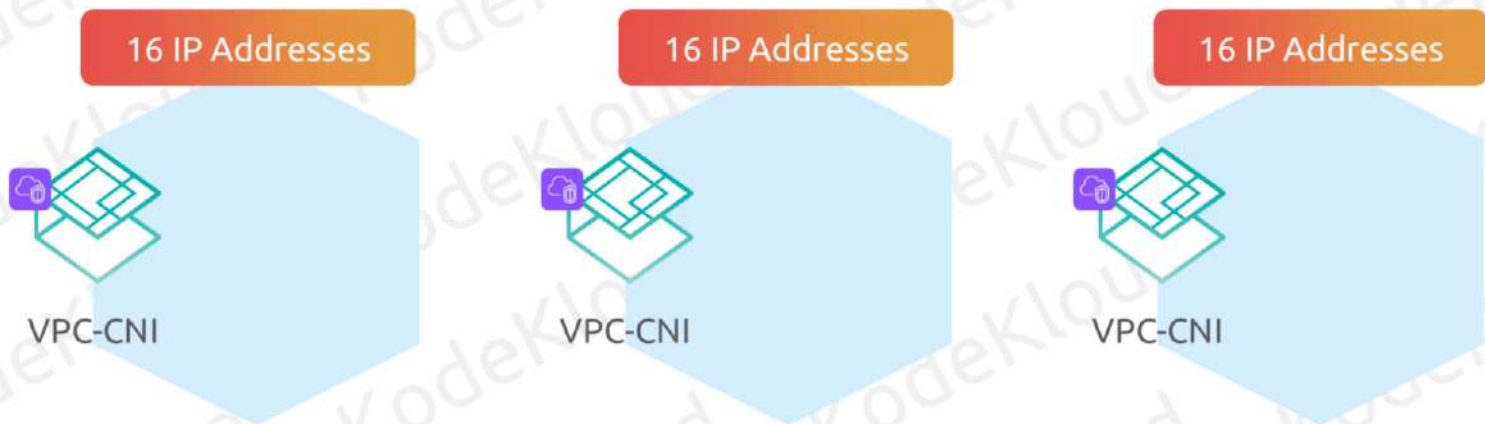
ENABLE_PREFIX_DELEGATION



© Copyright KodeKloud

And these are just again 16 IP addresses at a time. And as you scale out the amount of nodes you have, each one of them is going to need their own blocks of 16 IP addresses. Those blocks don't have to be consistent like the second time I add a slash 28 to this node, I don't actually need it to be a consistent 32. Two IP addresses we can take multiple blocks at a time and those, like I said, just become route tables within the VPC CNI and those become route tables within the VPC.

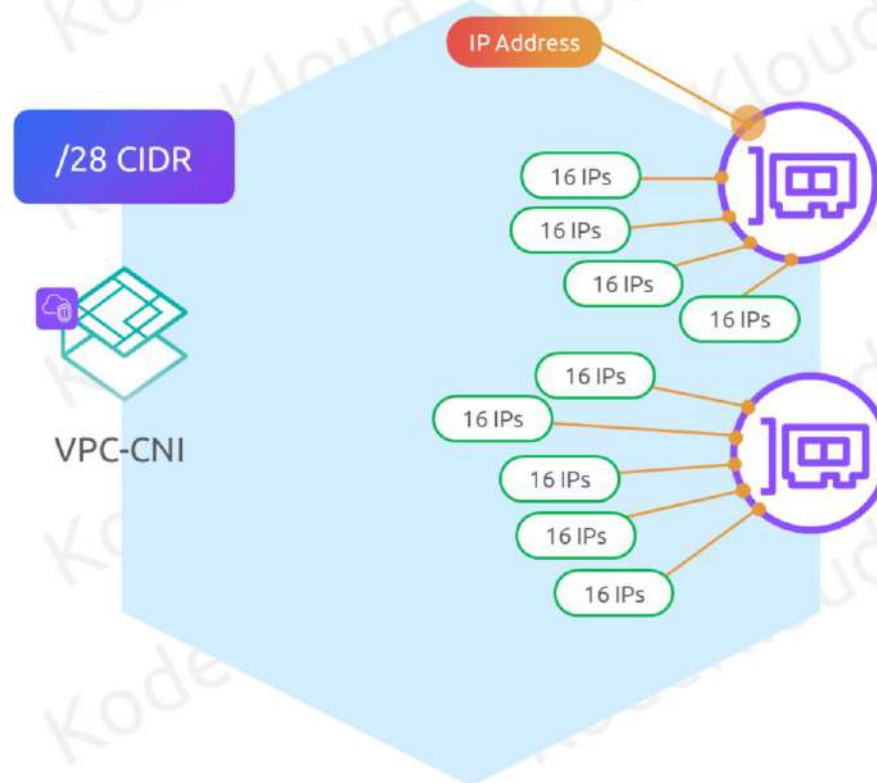
ENABLE_PREFIX_DELEGATION



© Copyright KodeKloud

And these are just again 16 IP addresses at a time. And as you scale out the amount of nodes you have, each one of them is going to need their own blocks of 16 IP addresses. Those blocks don't have to be consistent like the second time I add a slash 28 to this node, I don't actually need it to be a consistent 32. Two IP addresses we can take multiple blocks at a time and those, like I said, just become route tables within the VPC.

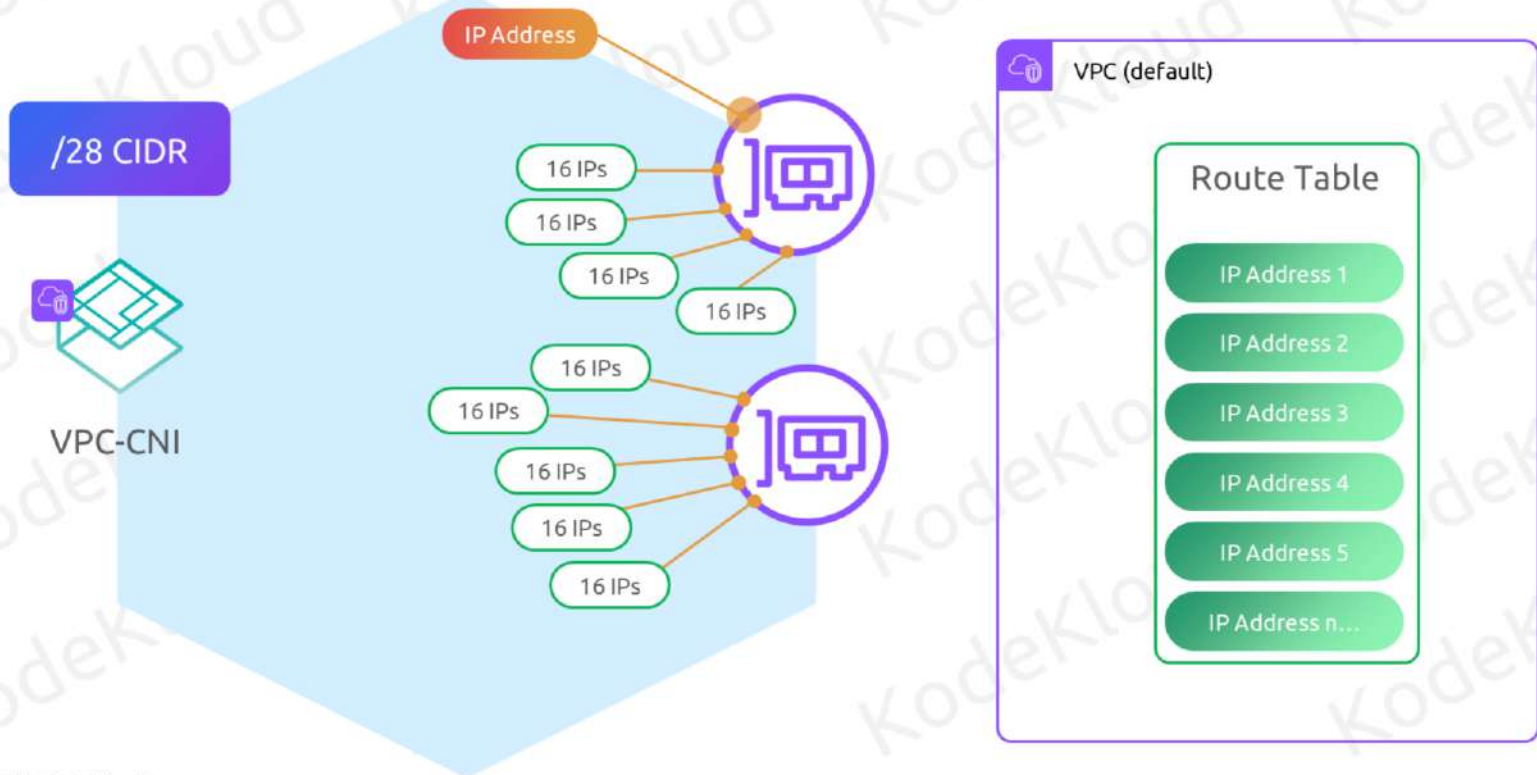
ENABLE_PREFIX_DELEGATION



© Copyright KodeKloud

And these are just again 16 IP addresses at a time. And as you scale out the amount of nodes you have, each one of them is going to need their own blocks of 16 IP addresses. Those blocks don't have to be consistent like the second time I add a slash 28 to this node, I don't actually need it to be a consistent 32. Two IP addresses we can take multiple blocks at a time and those, like I said, just become route tables within the VPC CNI and those become route tables within the VPC.

ENABLE_PREFIX_DELEGATION



© Copyright KodeKloud

And these are just again 16 IP addresses at a time. And as you scale out the amount of nodes you have, each one of them is going to need their own blocks of 16 IP addresses. Those blocks don't have to be consistent like the second time I add a slash 28 to this node, I don't actually need it to be a consistent 32. Two IP addresses we can take multiple blocks at a time and those, like I said, just become route tables within the VPC CNI and those become route tables within the VPC.

CNI IPv6

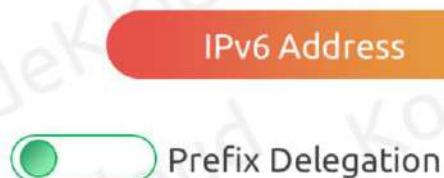


IPv6 Address

IPv6 Address

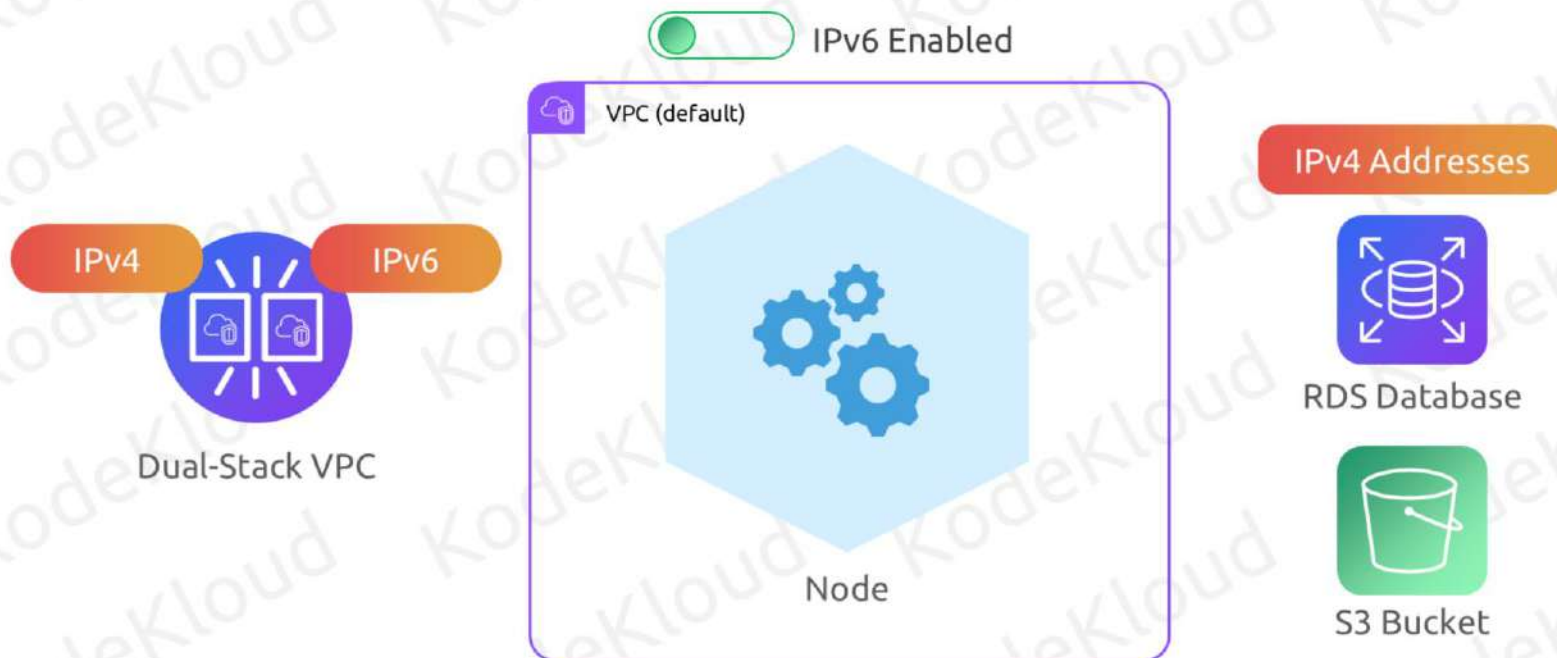
Okay, now that we know how prefix assignment works with the VPC CNI, let's talk about IPv6. Now this gets a little more complicated than what we have already set up with prefix delegation and the VPCC CNI about pods getting IP addresses. This works very similarly to the prefix delegation that we had in the last section. But now we have much larger IP addresses that we have IPv6. So let's show how some of that works.

IPv6 Address



Okay, now that we know how prefix assignment works with the VPC CNI, let's talk about IPv6. Now this gets a little more complicated than what we have already set up with prefix delegation and the VPCC CNI about pods getting IP addresses. This works very similarly to the prefix delegation that we had in the last section. But now we have much larger IP addresses that we have IPv6. So let's show how some of that works.

Visualizing IPv6 Address



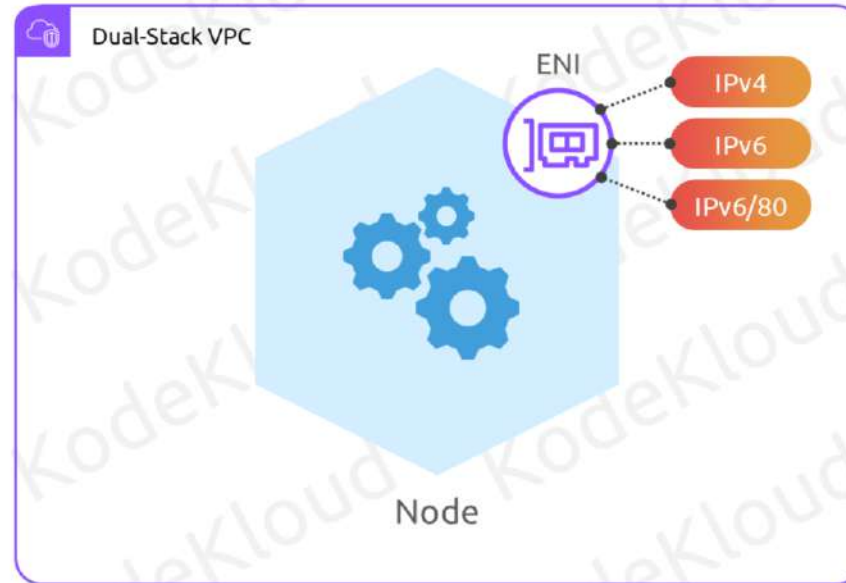
© Copyright KodeKloud

Okay, so here is our node. It exists within our VPC and just remember as you're setting this up, if you want to use IPv6, you have to do it from the start because the VPC itself needs to have IPv6 enabled. And this is what's called a Dual-Stack VPC because it'll have IPv4 and IPv6. We can't make IPv6 only VPCs currently because there are a lot of things within AWS that don't support IPv6 jets. Maybe you have something like an external RDS database or an S3 bucket. You can call those things with IPv4 addresses still today and some of them might support IPv6 by the time you're watching this but there's a mixed mode here of we need both IPs within the VPC.

Visualizing IPv6 Address



EKS Cluster



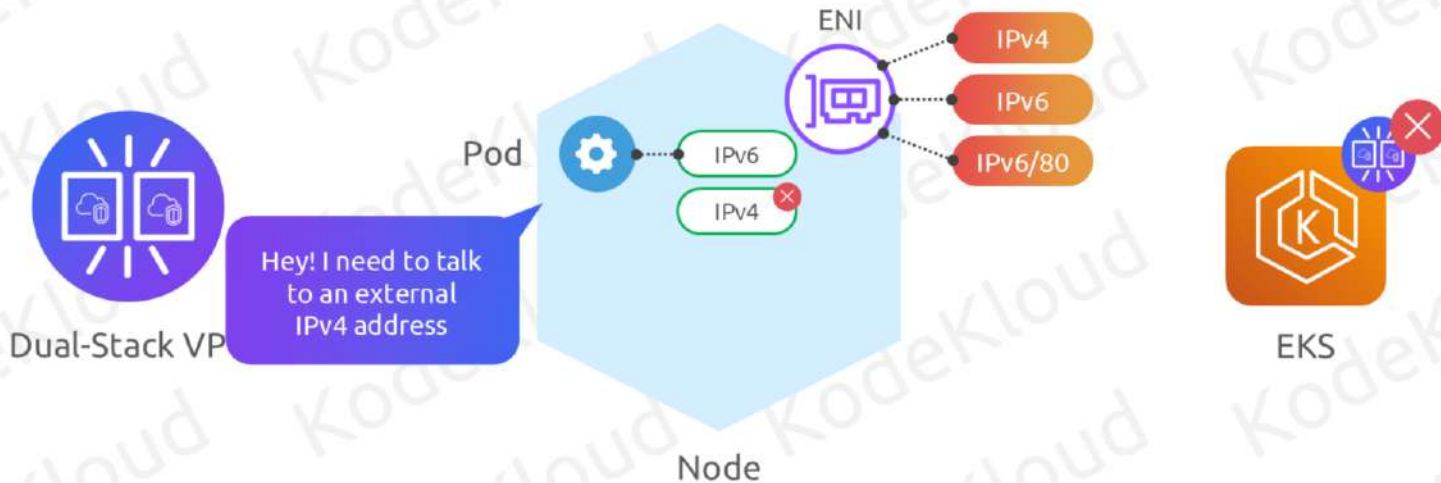
4 Billion IP addresses

100 Trillion IP addresses

So when you enable this on a VPC during creation, you can also enable it on your EKS cluster. And what that means is your node is still going to get that ENI. Like we were talking about before. Basically, a network plug that attaches to the node and we're going to get one IPv4 address. This is the main IPV4 address that the node can use to talk to IPv4 endpoints and there's also one IPv6 address and this is going to be how we can talk to IPv6 addresses. Because you can't necessarily route a six or four without having some gateway or something in between within a node, inside of Kubernetes. We're also going to get an IPV6 route. We're going to get an 80 route here and this is just a default assigned. Anytime you're using IPV6 inside of an EKS cluster, you're always going to be using this prefix delegation, which again gives you a whole bunch of IP

addresses automatically assigned and just remember, IPv6 is huge. It's like bigger than you can even imagine. This 80 that we get per node has 100 trillion IP addresses. 100 trillion IP addresses. The entire Internet of IPv4 only has 4 billion IP addresses. So, we have more way, way, way more IP addresses per node than all of the Internet. So, you don't really need to worry about running out of IP addresses to assign to pods. You're going to worry about running out of other resources on the node like say your CPU and memory or disk space or something like that. So, we're just going to eliminate the problem of we have not enough IP addresses because that is often a problem with IPv4 and now with IPv6, it is no longer a problem and everything within your VPC gets extremely large.

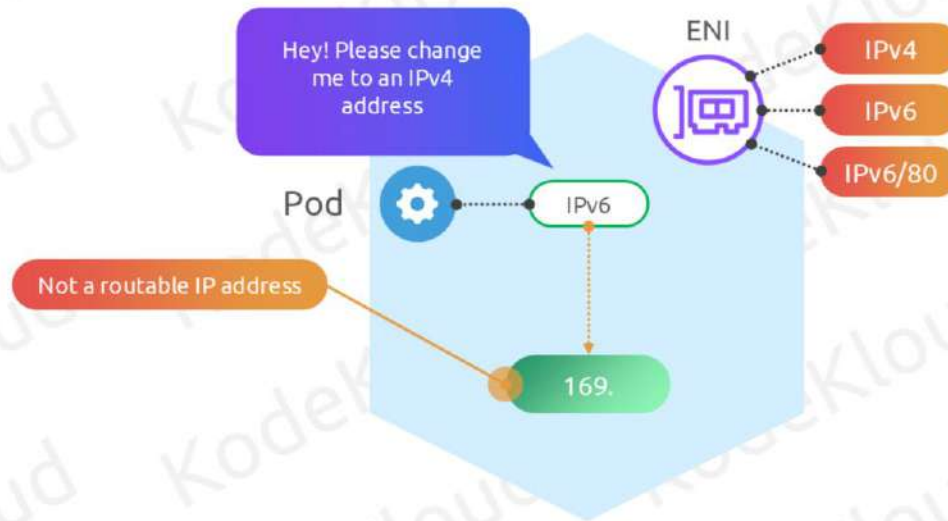
Visualizing IPv6 Address



© Copyright KodeKloud

Extremely large cidr blocks that you don't really have to worry about how many IPs you're going to have available when you switch over to IPv6. There are some other things you still do need to be concerned about when a pod starts up and it's inside your cluster again, it's going to get an IPv6 address. It will not get an IPv4 address. This is something that a lot of other providers, they will move their dual stack implementation down to the pod level. In EKS, there is no pod dual stack. So, I know there are other providers that will say a pod will get an IPv4 and an IPv6, and so that you can talk to IPv4 endpoints and in this case we only have the IPv6 address. The pod doesn't get IPv4. So how would your pod talk to an external IPv4 address?

Visualizing IPv6 Address

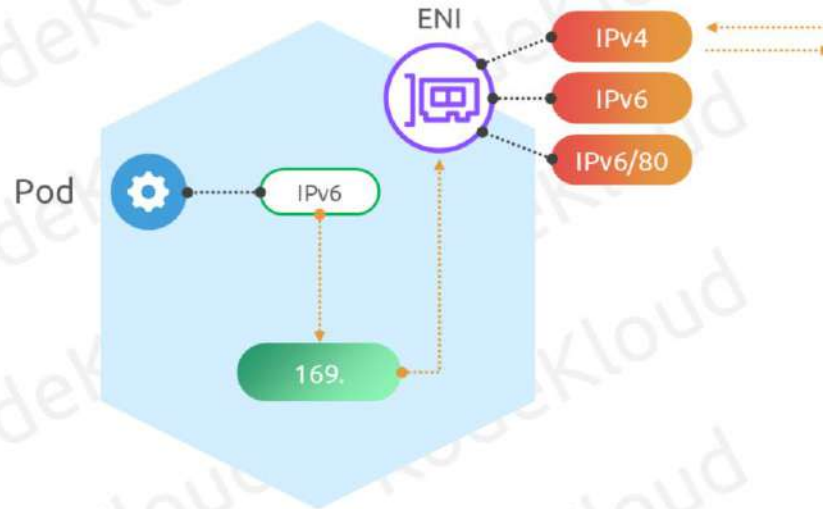


© Copyright KodeKloud

Well, on the node itself we set up a new network, just a local network interface that has a 169.169 address that is not typically a routable IP address. This is something that is only available locally on the machine and this is where you get things like your metadata endpoint. These are all local node IP addresses that don't route anywhere. They're just classes that they stay here. But what this actually does on the node with IPv6 is it will take an IPv6 call and if you get it, say you want to get a DNS endpoint that ends up being IPv4. Your IPv6 is going to route through here on a source NATted address. So, your IPv6 address is going to say, hey, please change me to an IPv4 address and then go out my IPv4 address to the endpoint I'm calling and when the traffic comes back, it comes back on IPv4 and it goes routed through this IPv6 address or

IPv4 address and changed into the IPv6 address locally on the machine. So, all of your translation for this dual stackness of talking between IPv6 and IPv4 happens through this, either IPv4 address, or if you have a Standard external IPv6 address, the pod will just go through the IPv6 network interface.

Visualizing IPv6 Address

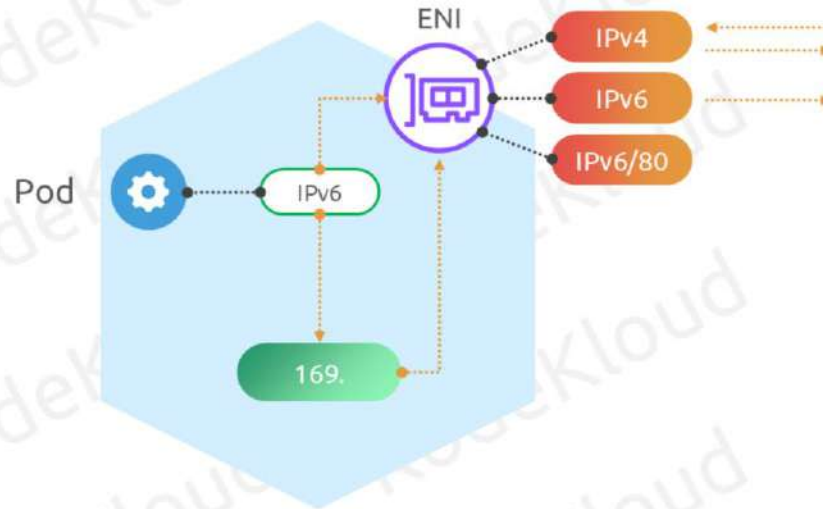


© Copyright KodeKloud

Well, on the node itself we set up a new network, just a local network interface that has a 169.169 address that is not typically a routable IP address. This is something that is only available locally on the machine and this is where you get things like your metadata endpoint. These are all local node IP addresses that don't route anywhere. They're just classes that they stay here. But what this actually does on the node with IPv6 is it will take an IPv6 call and if you get it, say you want to get a DNS endpoint that ends up being IPv4. Your IPv6 is going to route through here on a source NATted address. So, your IPv6 address is going to say, hey, please change me to an IPV four address and then go out my IPv4 address to the endpoint I'm calling and when the traffic comes back, it comes back on IPv4 and it goes routed through this IPv6 address or

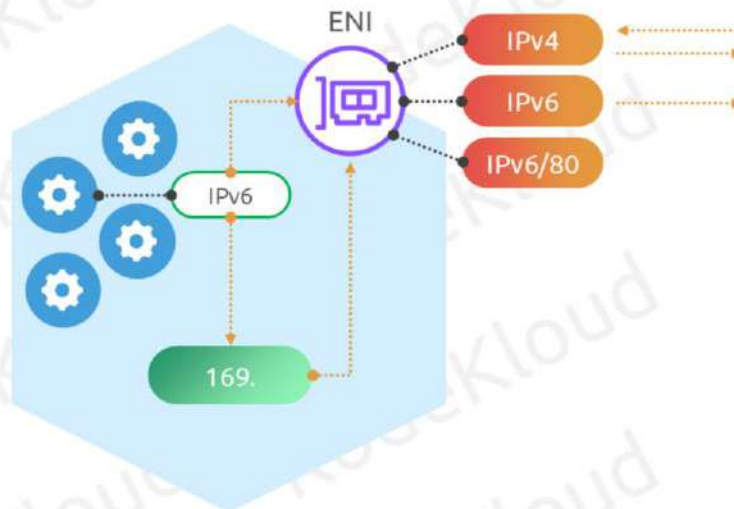
IPV 4169 address and changed into the IPv6 address locally on the machine.

Visualizing IPv6 Address



So, all of your translation for this dual stackness of talking between IPv6 and IPv4 happens through this, either .169 address, or if you have a Standard external IPv6 address, the pod will just go through the IPv6 network interface.

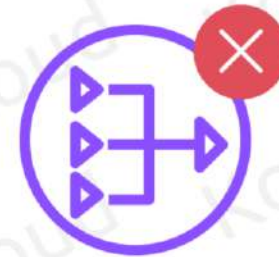
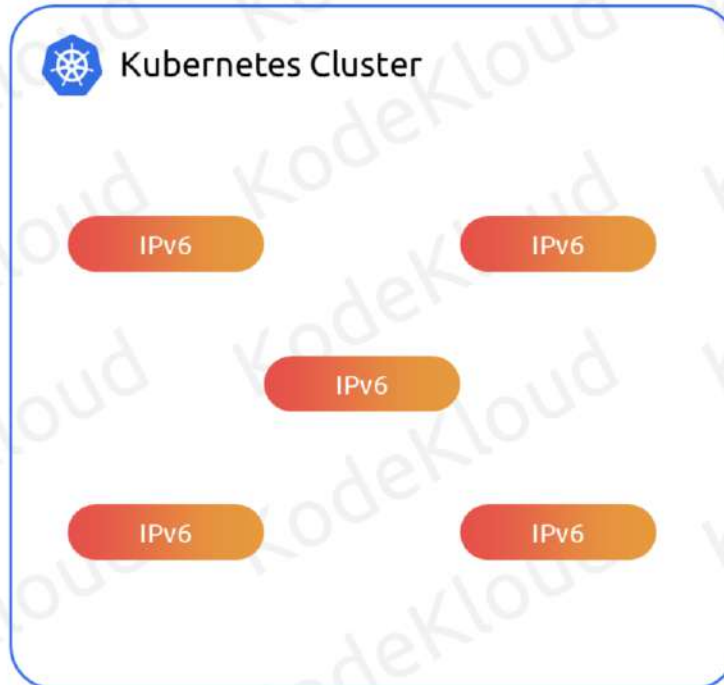
Visualizing IPv6 Address



That's kind of confusing. 100% agree. If you're, if you're a little confused right now, I understand that this is a bit confusing, but this is how. It works and we're going to show it in just a minute here. One thing to note with all of this is if I have a lot of pods that are all talking to IPv4 addresses externally, they're all going to talk out of the same IPv4 outbound connection on my node. So, this IPv4 address, depending on how much of your supporting infrastructure is IPv6 already might become a bottleneck. For the amount of pods that are running on this node. Because again, like I said, we're not going to run out of IP addresses. We could have 100 or 200 pods all on this node and if they all need to talk to something externally, that one network connection might be a limitation. So be aware of those sorts of things when you're ready to switch to IPv6, you'll

want to make sure that a lot of your supporting infrastructure is also ready for IPv6 traffic.

IPv6 – Interesting Facts



Network Address Translation (NAT)

There are some interesting things with IPv6 within the Kubernetes cluster itself. All of your services will get IPv6 addresses. I can't spell IPv6 addresses, so that all of the traffic between the pods should go over IPv6 by default. We want to encourage as many things within the cluster to use IPv6 by default. Your VPC also, let's say that way. If you know much about IPV six, most of it is public addressable. These aren't NAT it anymore. The addresses in IPV six, there's so many of them, we don't need to put things behind a gateway, we don't need to put things behind a Nat. And so, when you're setting up your clusters in your VPC, again, this is a little bit outside the scope for this workshop, but it's worth noting that someone that is setting up your VPC also needs to know how to make.

IPv6 – Interesting Facts

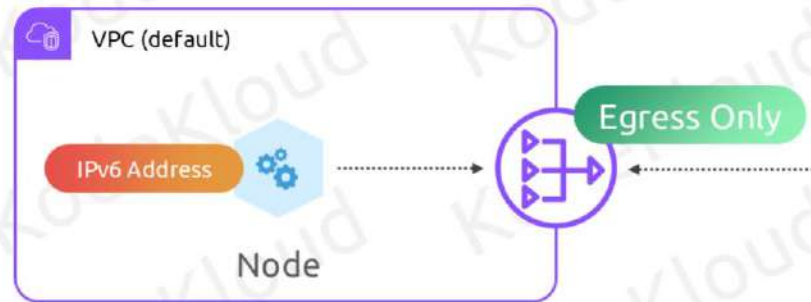
IPv6 Address

(Private)

Your IPv6 addresses are private. And the way you do that is we still can have a gateway on the VPC and we will have an Egress only gateway so that only traffic going out through the egress will work. So, let's say your node is going to contact some other service, but doesn't need any traffic to come back, you would want to go through this egress only gateway. You could have an ingress only gateway where traffic comes in but you really want to make sure that if these IPv6 addresses are publicly visible on the Internet, if someone knows that the IP address exists and you don't have an egress only gateway, they could theoretically contact that node or that pod, whatever that IPv6 address is. Because again these are, globally unique IP addresses. This is not like running your house, which has a gateway and a NAT and someone can't contact my 192

address because IPv6 is just the same namespace everywhere.

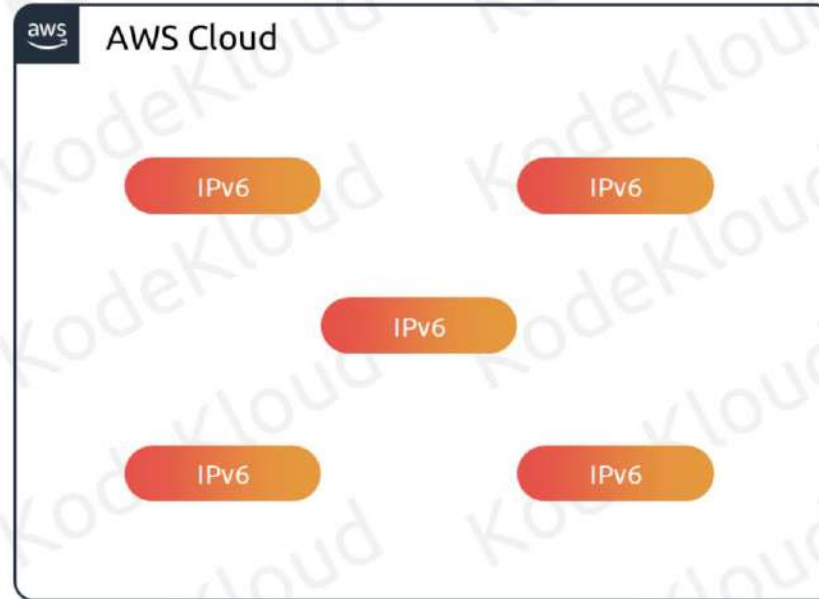
IPv6 – Interesting Facts



Your IPv6 addresses are private. And the way you do that is we still can have a gateway on the VPC and we will have an Egress only gateway so that only traffic going out through the egress will work. So, let's say you're your node is going to contact some other service, but doesn't need any traffic to come back, you would want to go through this egress only gateway. You could have an ingress only gateway where traffic comes in but you really want to make sure that if these IPv6 addresses are publicly visible on the Internet, if someone knows that the IP address exists and you don't have an egress only gateway, they could theoretically contact that node or that pod, whatever that IPv6 address is. Because again these are, globally unique IP addresses. This is not like running your house, which has a gateway and a NAT and someone can't contact my 192

address because IPv6 is just the same namespace everywhere.

Conclusion



So while IPV six is a little bit confusing and actually will change a little over time, probably lots of things within AWS are still supporting IPV six. It is a great way just to eliminate the problem of not having enough IP addresses. A lot of times when I was working at EKS, the number one problem we would see customers have is they'd run out of IP addresses in their subnets. They'd get more pods than they ever expected, and you can add secondary subnets, but you never add enough. You could never go through and add more and more IP addresses that you're like, actually I need a larger chunk of subnets. And at some point, it's just easier, easier to do IPV6. So, if you can start today with a brand-new cluster and use IPV6, that would be the best way to go for scalability. If you don't think you actually need humongous scale, and you don't need, you

know, tens of thousands of pods or even 100,000 pods, like, you may not need it, but if it was part of over and over again, some of the things that we saw with customers having problems was constantly running out of IP addresses and doing a dual stack VPC and pods that only got IPv6 addresses completely eliminated that as an issue.

Conclusion



Public Subnet

10.16.0.0/24

10.16.1.0/24

10.16.5.0/24



Amazon EKS



Pod



Pod



Pod



Pod

Conclusion



Public Subnet



Public Subnet



Amazon EKS



Pod



Pod



Pod



Pod

Conclusion



Public Subnet



Public Subnet



Amazon EKS



Pod



Pod



Pod



Pod

Conclusion



Public Subnet



Public Subnet

IPv6 Address



Pod



Pod



Pod



Pod

CNI Network Policy

Network Policy



Network Policy

Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource.

Network Policy



© Copyright KodeKloud

Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource.

Network Policy



Network Policy



Network Policy



Network Policy

Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource.

Network Policy



CNI Plugins



CNI Plugins



CNI Plugins



Network Policy



Network Policy



Network Policy

Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource.

Network Policy

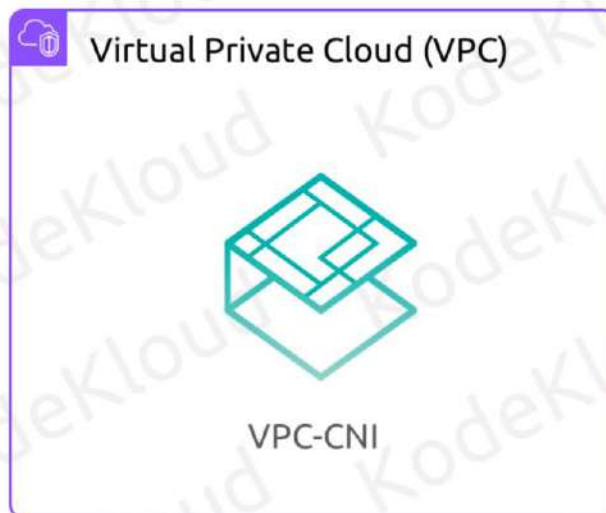


Calico

© Copyright KodeKloud

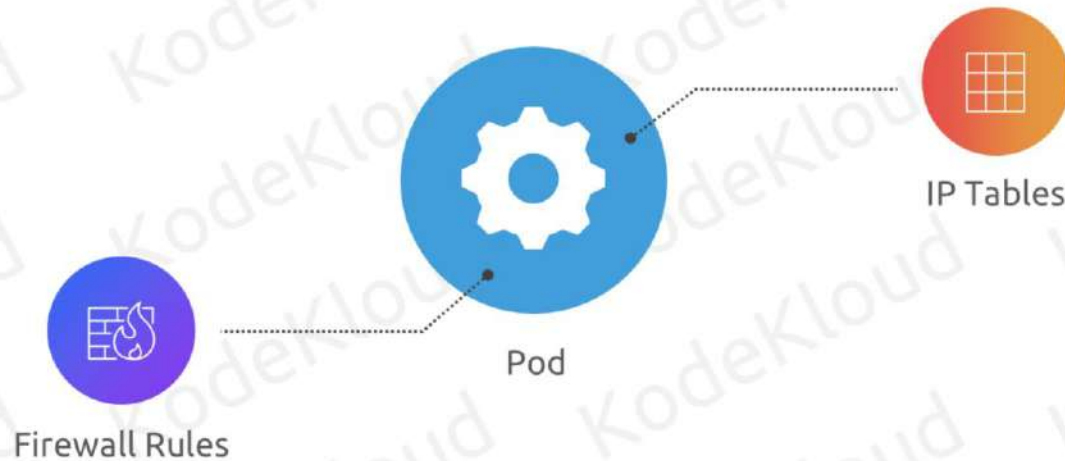
Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource. So, if you're using the VP CNI, it might be different than what you're used to with a different CNI provider.

Network Policy



Alright, to close out the network section we want to talk about one last thing, and that's network policies. Network policies are a standard piece of Kubernetes that allow you to manage how network is able to be accessed from a pod to another pod. And this is both ways for ingress and egress. So how much, how is traffic going out from the pod and how is traffic coming into the pod? You can do this a lot of different ways. This isn't a one size fits all solution, and varying CNI plugins have different degrees of what they do or don't support. Some of the CNI plugins that have been around for a while, like Calico, have been doing network policies even before Kubernetes provided them as a default resource. So, if you're using the VP CNI, it might be different than what you're used to with a different CNI provider.

Network Policy



© Copyright KodeKloud

And like I said, you can do this in a lot of different ways. Everything on the pod could be done from IP tables down to firewall rules. Inside of AWS there's security groups that could also be applied and can be applied in other places. There's a lot of places you can control how network goes from one place to another. One of the benefits of doing network policy is it can be closely aligned with what the pod or the application actually needs. So, let's say you have a team that usually does the firewall rules. Usually, you would have to go email them and say hey, I need a new port open or I want to cut off traffic from this other IP address. Something along those lines. With network policies you can possibly control that within the cluster and not have to worry about what someone else is responsible for. And so, it does make you more responsible for

more things, but also allows you to move a lot faster and to more closely scope what your network traffic needs are for the application as the application changes over time. Because maybe you only have one database endpoints when you start the app, but by the time it's evolved over time you have an s three endpoint, you have RDS, you have some other DNS entry. All of those things you'll need to evolve with the application and it's hard to coordinate those things as you deploy new versions of the app. And you need to tell the network policy team hey, at this time I need you to open this port or allow this traffic. Those sorts of things get difficult to handle within Kubernetes. You can more closely align those things that you apply them all at the same time because they're managed by the same system which with the same manifests for the application deployment.

Network Policy



And like I said, you can do this in a lot of different ways. Everything on the pod could be done from IP tables down to firewall rules. Inside of AWS there's security groups that could also be applied and can be applied in other places. There's a lot of places you can control how network goes from one place to another.

Network Policy



© Copyright KodeKloud

One of the benefits of doing network policy is it can be closely aligned with what the pod or the application actually needs. So, let's say you have a team that usually does the firewall rules. Usually, you would have to go email them and say hey, I need a new port open or I want to cut off traffic from this other IP address. Something along those lines. With network policies you can possibly control that within the cluster and not have to worry about what someone else is responsible for. And so, it does make you more responsible for more things, but also allows you to move a lot faster and to more closely scope what your network traffic needs are for the application as the application changes over time. Because maybe you only have one database endpoints when you start the app, but by the time it's evolved over time you have an s three

endpoint, you have RDS, you have some other DNS entry. All of those things you'll need to evolve with the application and it's hard to coordinate those things as you deploy new versions of the app. And you need to tell the network policy team hey, at this time I need you to open this port or allow this traffic. Those sorts of things get difficult to handle within Kubernetes. You can more closely align those things that you apply them all at the same time because they're managed by the same system which with the same manifests for the application deployment.

Network Policy



You



I want to cut off
traffic from this
other IP address

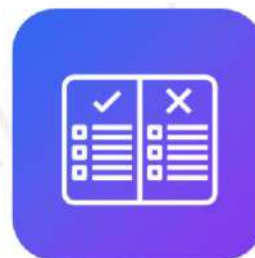


Firewall Team

Network Policy



You



Network Policy



Firewall Team

Network Policy



Kubernetes Cluster



etcd



api server



scheduler

Control Plane



worker nodes

Data Plane

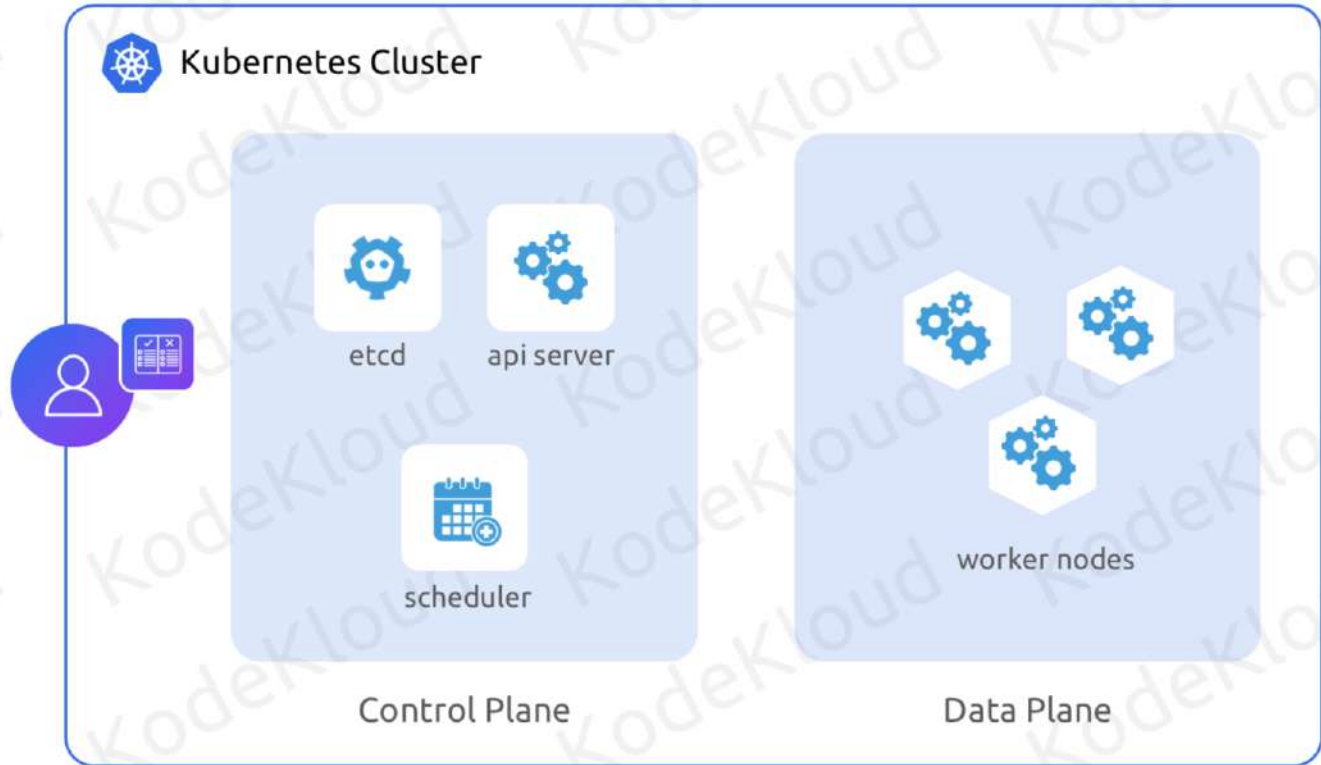
Network Policy



Faster



Closer Scope



Network Policy



AWS Cloud



S3



RDS



Route 53



Kubernetes Cluster



etcd



api server



scheduler

Control Plane



worker nodes

Data Plane

Network Traffic Blocking

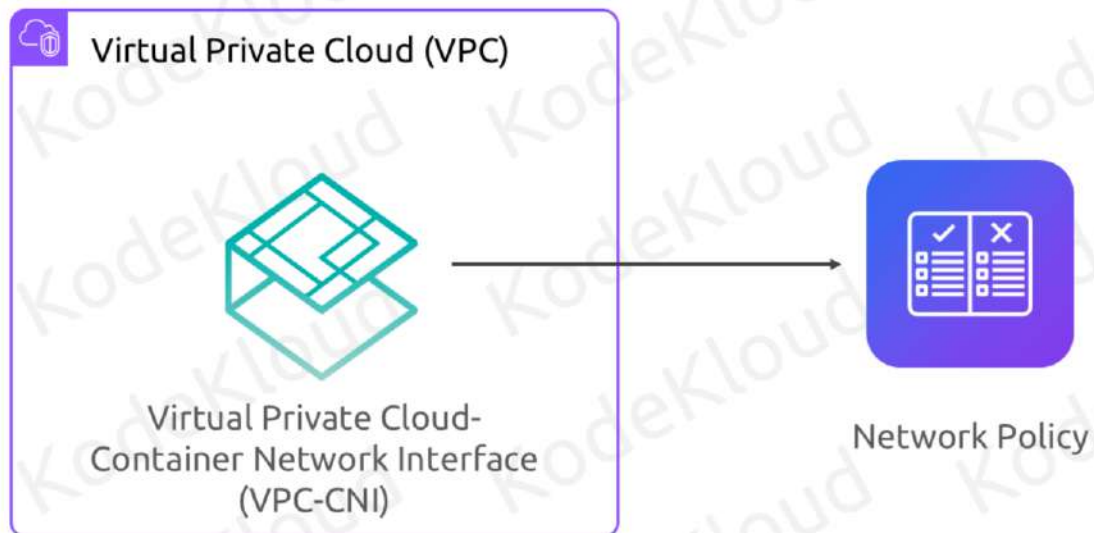


Network Policy

Network traffic blocking isn't something that's really new because like I said, it's been around in a lot of different forms and network policy and VPC CNIs doing network policy just does it however we want it to happen. And so different CNIs will do it different ways with the AWS VPC CNI, it does it with EBPF extended Berkeley packet filtering. You don't need to know that name, you just know that this is an application that runs in the Linux kernel of the node itself. So, the node has basically a little application that's running inside of it that manages all the network traffic for the pods. And so, this isn't something that you necessarily see as a sidecar, this isn't another process that's running. This is something running in the kernel that the CNI talks to. And whenever a pod comes up we can say oh, we need to attach some new policy to this pod because we

don't want, or we do want to allow some sort of traffic, like I said, either egress or ingress going out or coming in.

Network Traffic Blocking



Network Traffic Blocking



Flannel



Calico



VPC-CNI

Network Traffic Blocking



VPC-CNI

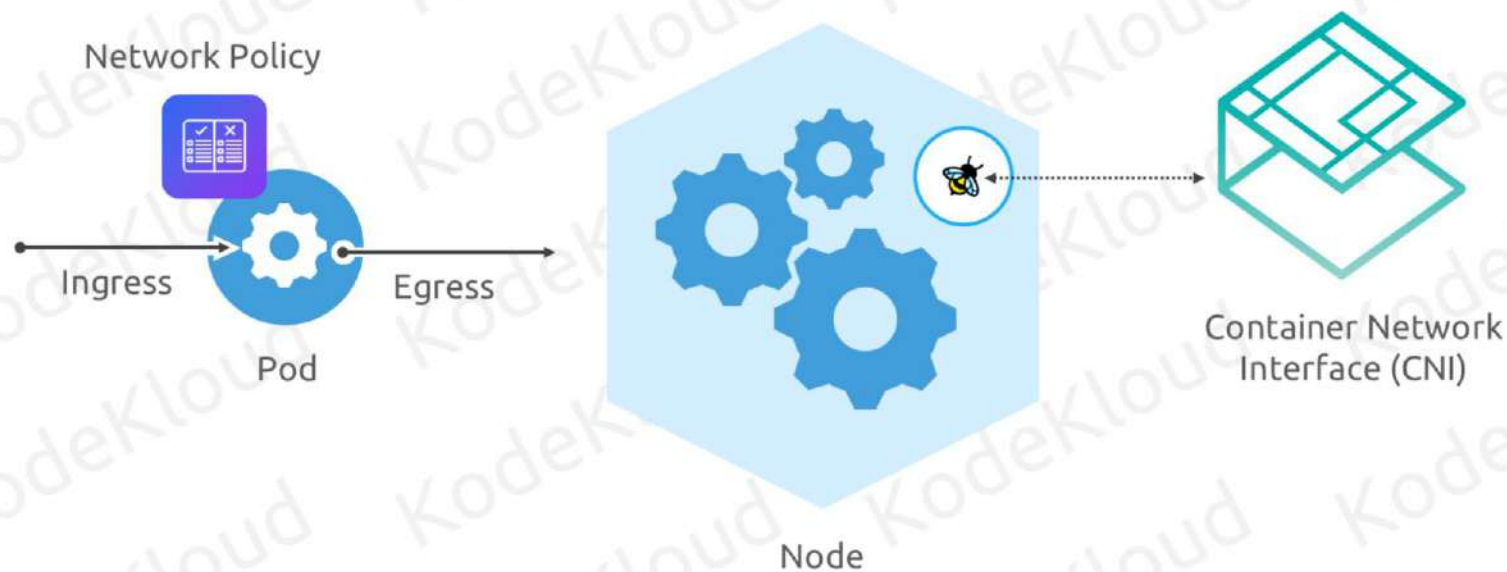


Extended Berkeley Packet Filter

Network Traffic Blocking



Network Traffic Blocking



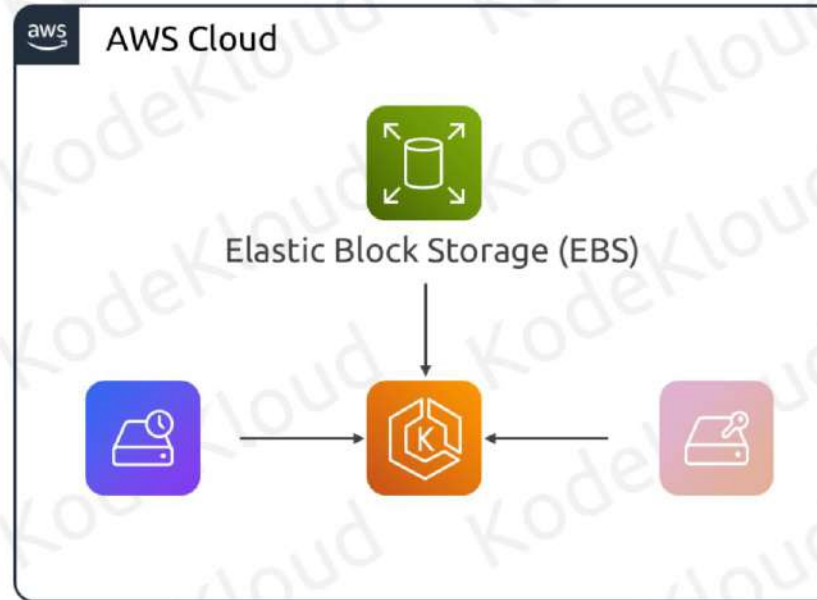
Summary

- 01 A standard piece of Kubernetes but new feature of VPC-CNI
- 02 Enabled through EBPf
- 03 Brings network closer to your application
- 04 More of an application-native firewall that allows adding and blocking traffic
- 05 Does not require talking to teams

And like I said, this is a standard piece of Kubernetes, but it's a fairly new feature of the VPC CNI, and it's one that enables this feature with EBPf, which isn't how the rest of the CNI works today. So, it gets a little confusing when you're trying to debug these things. What layer is this happening in? Where am I actually getting traffic blocked? Is this at IP tables? Is this at my security groups? It's another place you might want to look and be aware of, but it also brings that network control closer to your application. It's more of an application native firewall that allows you to add or block traffic. And you don't necessarily have to talk to a systems team or networking team or someone else that manages the node or the VPC or the AWS account. You can apply that to your cluster and to your application itself.

Storage EBS

Storage for EKS Cluster



Okay, in this section, we're going to talk about storage for your EKS cluster. And the storage that you're going to use inside of EKS is going to be from usually some other storage source within your AWS region or account.

Storage for EKS Cluster



How it looks



Pieces needed



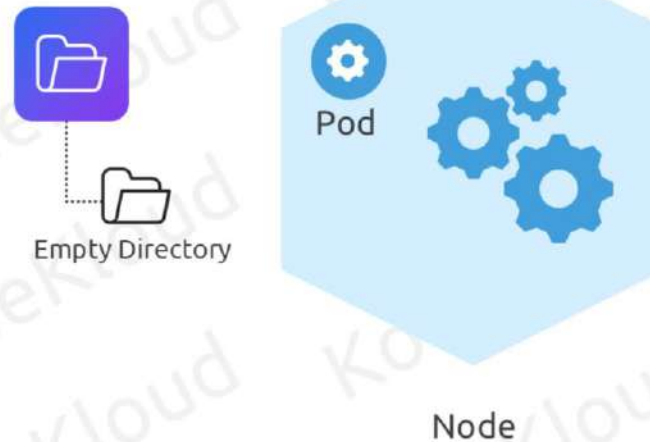
How it works



Elastic Block Storage (EBS)

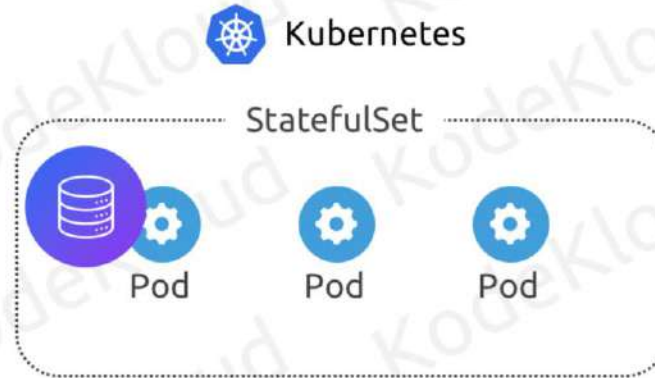
And we're going to start with the most common, which is the EBS storage as mounting that into pod. And we're going to start conceptually again how that looks, what pieces that need to run, and then show you actually how it works in real life.

Storage for EKS Cluster



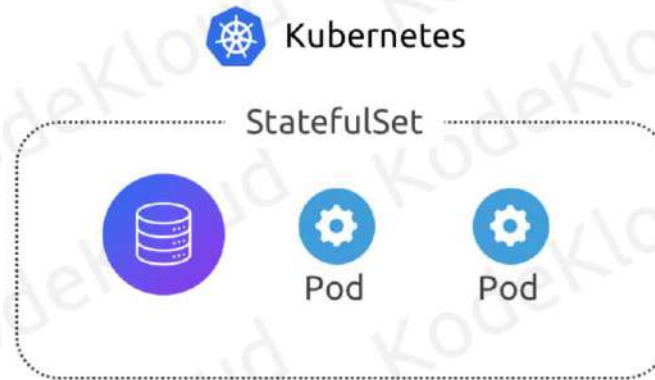
So here we have this node that exists inside of our cluster, and we could do all different things to get storage into a pod. So we have a pod that runs, and you can do things like empty Dir, which is a Kubernetes storage construct, which just mounts a directory from the host somewhere. And that sometimes works, but it doesn't live longer than the pods. So as soon as the pod goes away, that directory is cleaned up as well.

Storage for EKS Cluster



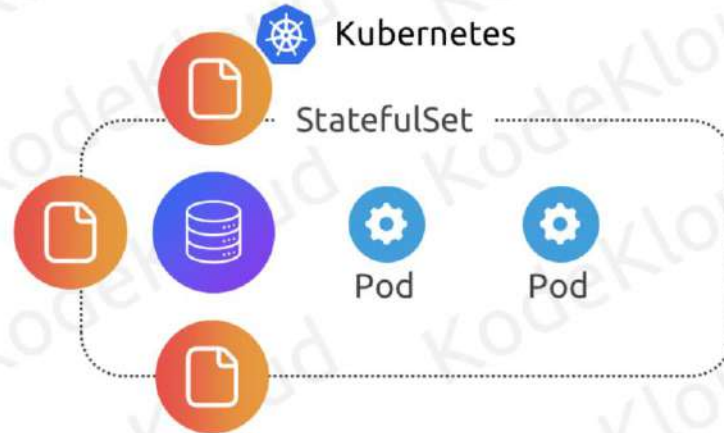
That might be fine for scratch space or something like that, but we want something that is more durable. Let's say we want to run a stateful set, and stateful sets in Kubernetes allow us to run some workload, and we could write data to some volume.

Storage for EKS Cluster



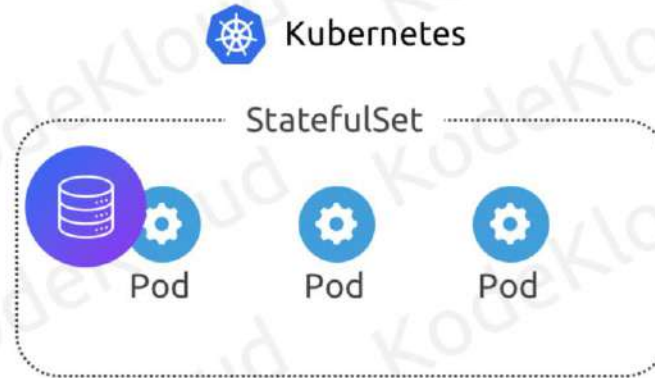
And when, if the pod goes away, that volume can still exist. And if the pod goes away, that volume, the data itself is safe. It will be remounted if a new pod that replaces it comes up. And that's what we actually want. We want something that is more durable and exists outside of the Kubernetes cluster, or at least outside of the pod lifecycle. So one of the very common ways to do that is with an EBS volume.

Storage for EKS Cluster



And when, if the pod goes away, that volume can still exist. And if the pod goes away, that volume, the data itself is safe.

Storage for EKS Cluster



It will be remounted if a new pod that replaces it comes up. And that's what we actually want. We want something that is more durable and exists outside of the Kubernetes cluster, or at least outside of the pod lifecycle. So one of the very common ways to do that is with an EBS volume.

Storage for EKS Cluster



We want something that is more durable and exists outside of the Kubernetes cluster, or at least outside of the pod lifecycle.

Storage for EKS Cluster



We want something that is more durable and exists outside of the Kubernetes cluster, or at least outside of the pod lifecycle.

Storage for EKS Cluster



Elastic Block Storage (EBS) Volume

So one of the very common ways to do that is with an EBS volume. And EBS stands for elastic block storage.

Storage for EKS Cluster



Elastic Block Storage (EBS) Volume



Block Storage

Node

And so this is going to present block storage to your node.

Block Storage vs File Storage

Plugging a Hard Drive



Block Storage

Elastic File System (EFS)

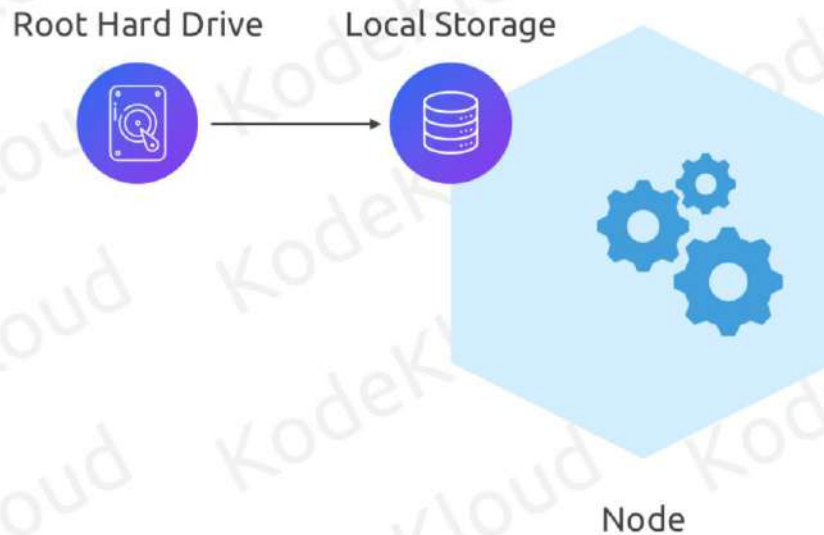


File Storage



And if you're not familiar with the difference between block storage and file storage is essentially a block storage is like plugging in a new hard drive that gives you low level system stuff you can do to it, that allow you to format the drive and really do whatever you would normally do to a hard drive. File storage like EFS, which we'll look at later, is giving you a file system already existing, and we just want to read and write files. Something like NFs is the most common one. That's what EFS is going to be using, where we're just like, hey, I don't actually care about what the format is, where you're writing this. Just, I want to write a file somewhere. And they both have some strengths and weaknesses, and we'll talk about that as we go through it.

Elastic Block Storage (EBS)



© Copyright KodeKloud

So first up with EBS storage. So first up, let's still remember that we do have local storage already attached to our node. This local storage is used for the actual root hard drive, the system that gets booted. Whatever your operating system is, it needs hard drive space by itself.

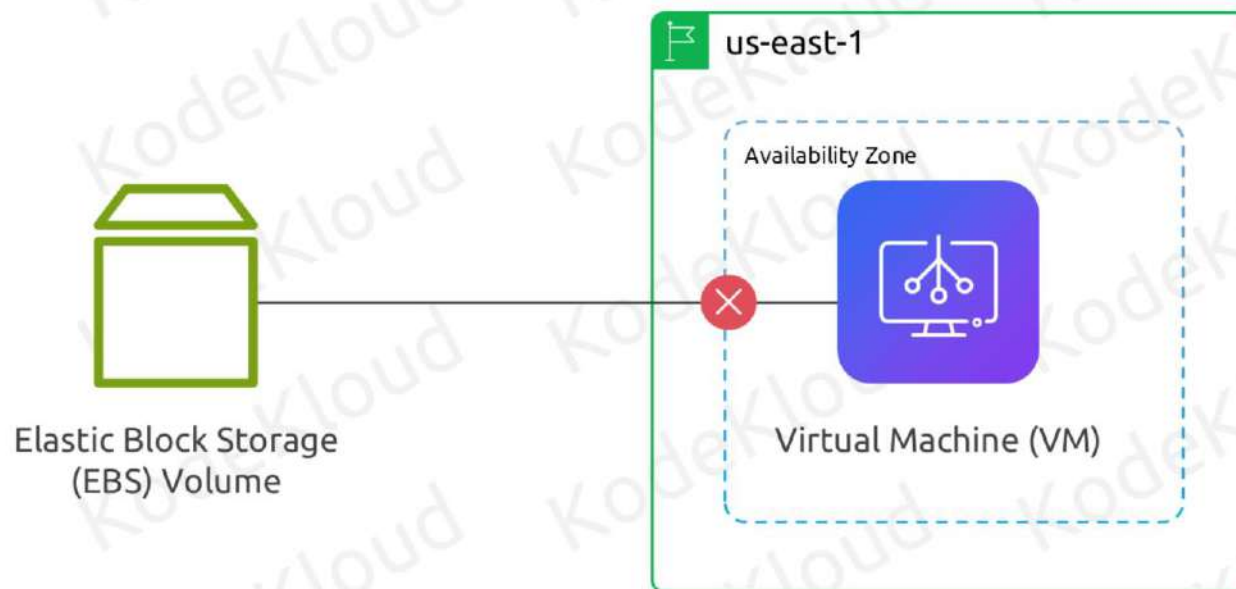
Elastic Block Storage (EBS)



Elastic Block Storage
(EBS) Volume

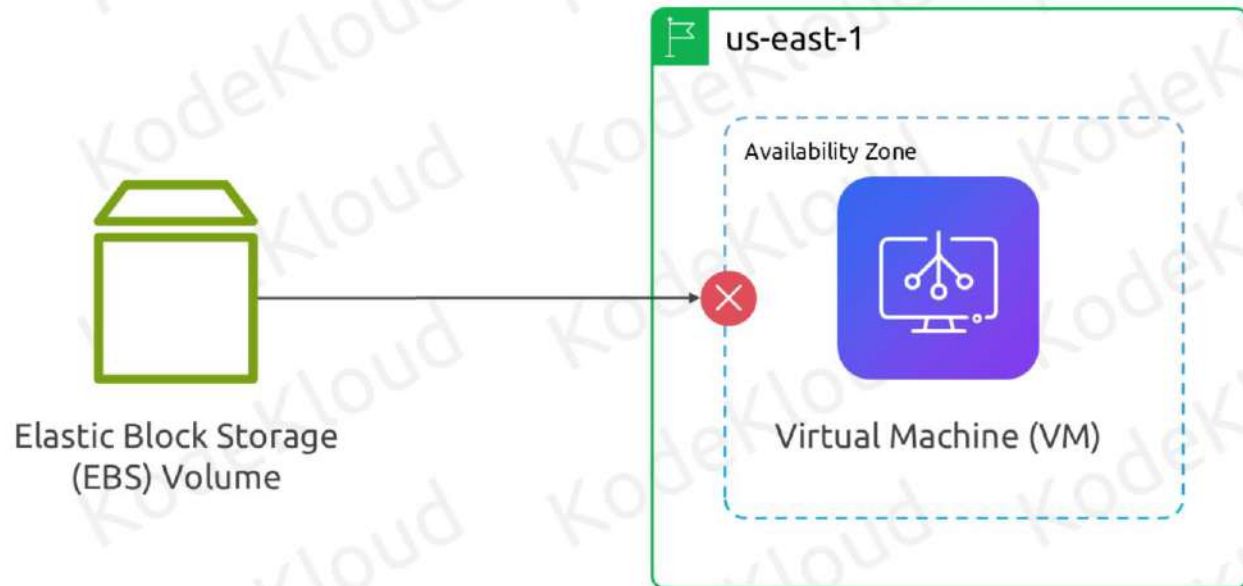
And so you usually do that through an EBS volume already

Elastic Block Storage (EBS)



even though EBS isn't locally like physically attached to the VM, because these VMs exist inside of regions and availability zones,

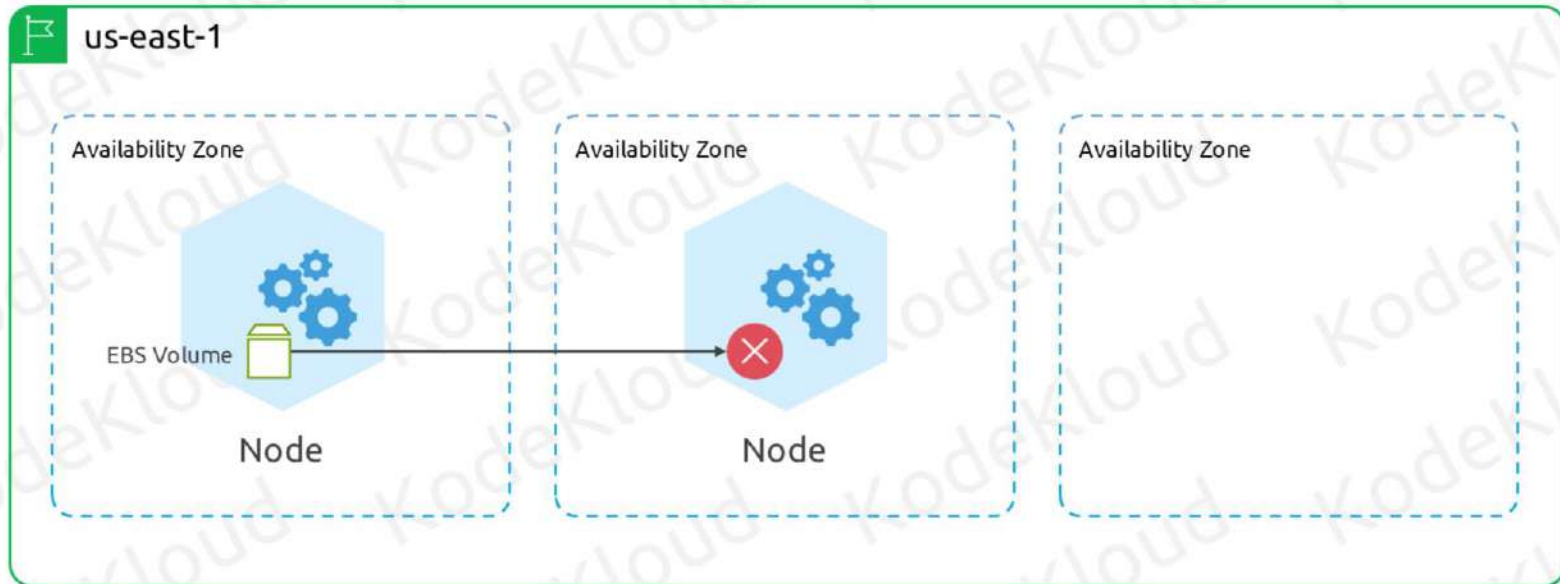
Elastic Block Storage (EBS)



© Copyright KodeKloud

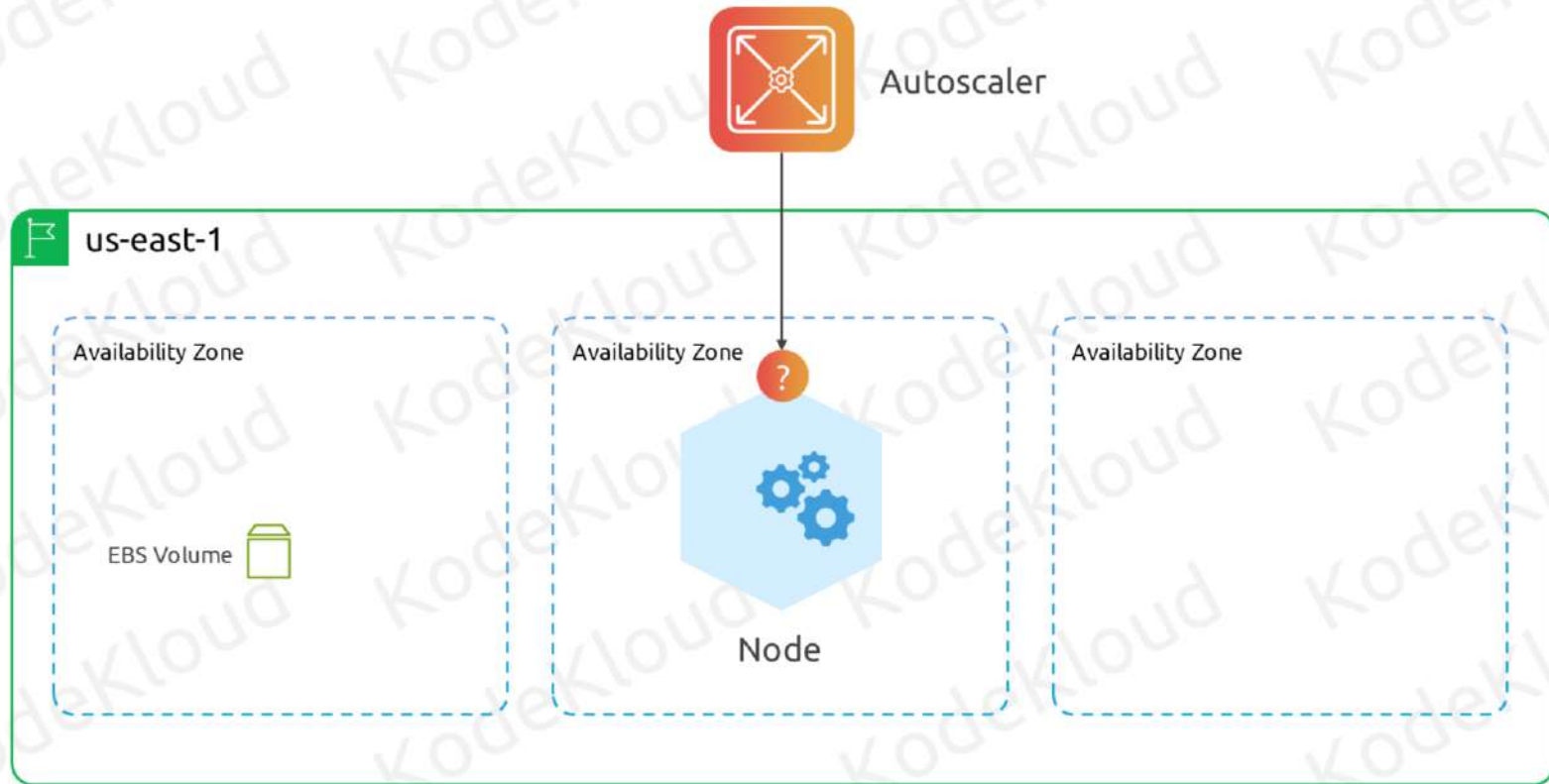
EBS is a availability zone bounded service.

Elastic Block Storage (EBS)



So that means if you have a region that let's say has three azs, which is a very common setup for high availability, and I have an EBS volume in one of those azs, if my node mounts that volume inside that AZ and then my node goes away and a new one is replaced in a new availability zone, it cannot mount the EBS volume from the old node because the EBS volume stays within the availability zone. And this can be something that causes some problems,

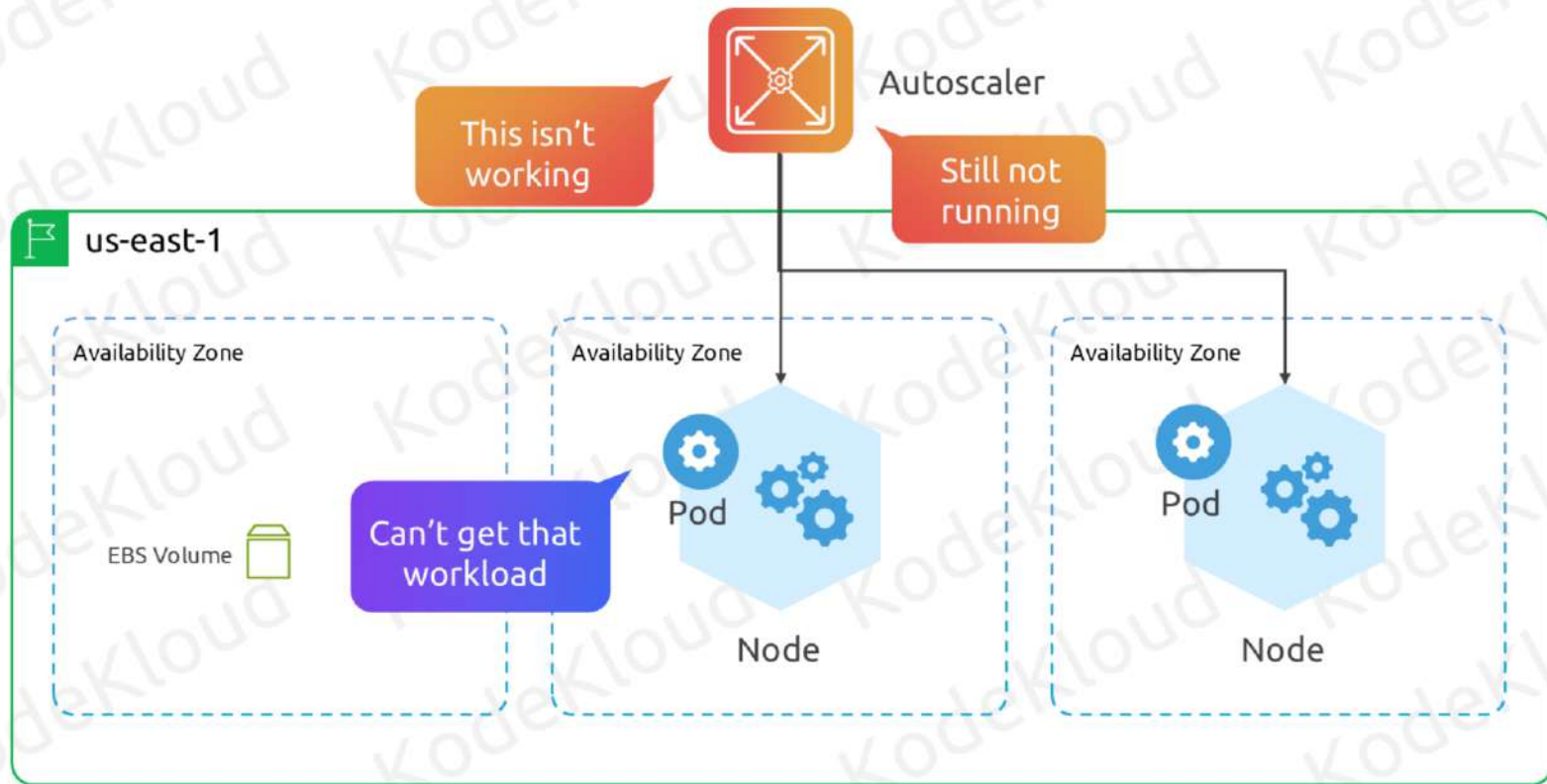
Elastic Block Storage (EBS)



© Copyright KodeKloud

because if your auto scaler is not aware of where the nodes need to come up and what volumes they might need to mount, then you can get yourself into trouble where something like the cluster autoscaler, using auto scaling groups will just create whatever new node in the auto scaling group.

Elastic Block Storage (EBS)



© Copyright KodeKloud

So let's say we have that pod, it's trying to get that EBS volume, and we created a node in the wrong AZ. Well it's just going to sit there and timeouts until it realizes hey, I can't actually get that workload. The cluster auto scaler will say, actually this isn't working. I still have a pod that isn't running for some reason. So I'm going to create a new node, and if you have three azs, it's going to go to the next availability zone. And this availability zone might still be the wrong availability zone. So it cannot mount that EBS volume. So the cluster auto scalar is going to wait, it's going to timeout and say hey, actually I still have a pod and it's still not running.

Elastic Block Storage (EBS)



Autoscaler



us-east-1

Availability Zone



Node

Availability Zone



Node

Availability Zone

So let's create another node. And maybe this is going to go on the right AZ, maybe it's not.

Elastic Block Storage (EBS)



Autoscaler



us-east-1

Availability Zone

Availability Zone

Availability Zone

Availability Zone

Availability Zone

AWS regions have a minimum of three availability zones per region, but they have upwards to five or six.

Elastic Block Storage (EBS)



Autoscaler



us-east-1

Availability Zone

Availability Zone

Availability Zone

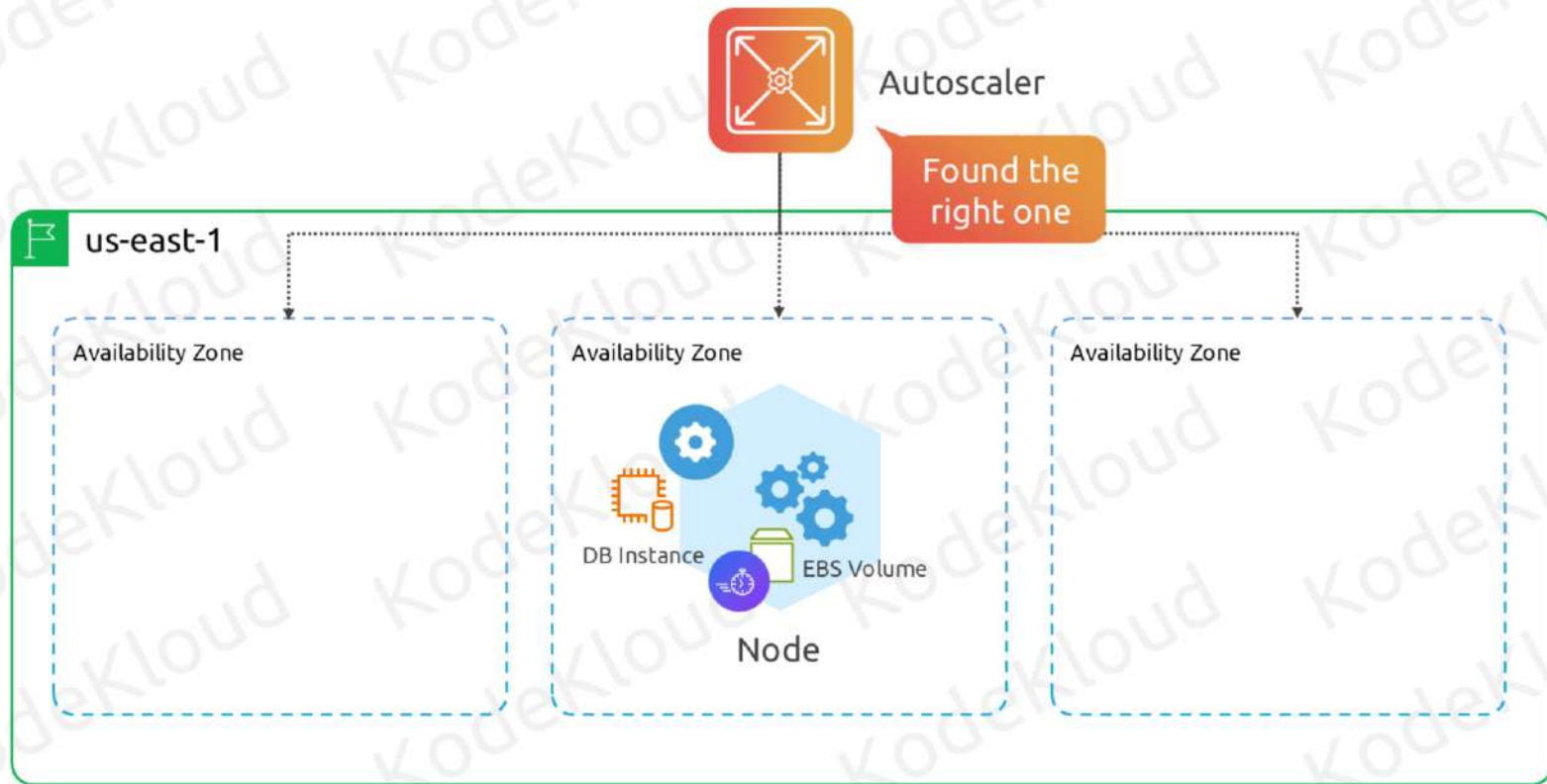
Availability Zone

Availability Zone

Availability Zone

AWS regions have a minimum of three availability zones per region, but they have upwards to five or six.

Elastic Block Storage (EBS)



© Copyright KodeKloud

So depending on how many you're using in your cluster, this might take you a little while. If that pod is down and it was say a database instance, you probably needed that and you don't want to wait for the cluster autoscaler to cycle through all of these availability zones until it figures out. Oh, I found the right one. We can mount that EBS volume now and the pod can run. So that is something you really want to be aware of when you use something like EBS volumes that are zonally constricted and where they can be attached to nodes. The benefit of an EBS volume is it's actually pretty fast because it is within the same availability zone, which in AWS terms that's, you know, some failure domain, but that's like just data center. It's within the data center and a volume can get mounted into the vm within the data center will have really low latencies.

Types of EBS Volumes

Terabytes of
information



Terabytes of
information



(The Common Ones)

Terabytes of
information



(Faster Throughputs)

And there are different types of EBS volumes. There's GP two, GP three are the common ones. There's also IO volumes which will have even faster throughputs. So if you need a lot of storage, that's fairly quick. Great way to get it. If you need a really large storage, you can attach terabytes of information with any of these volumes and not even worry about it. It just provisions gets mounted in and you have a lot of storage data available for you really quickly.

Faster Volumes



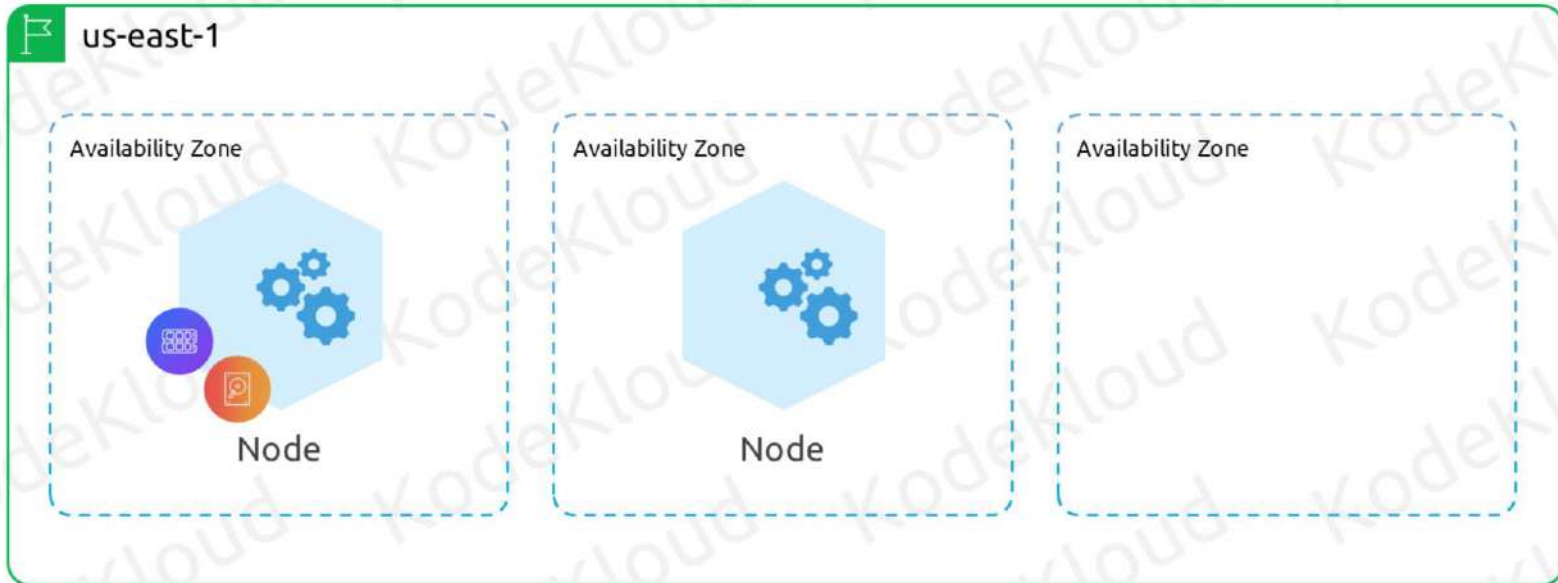
NVMe Drives



Local Volume

There are of course faster volumes available, such as local NVMe drives and just a local volume in general.

Faster Volumes



But like I said, those do not persist outside of the node. And if the node goes away for any reason, that data is also gone.

Supporting Tools for EBS



Snapshot



EBS Volume



Container Storage
Interface (CSI) Driver
(Volume Controller)

And lastly, I'll point out that EBS volumes have a lot of other supporting tools that allow you to work with that data. Things like snapshotting just works where you can take a snapshot of a volume over time. And if for some reason the data is corrupted or something goes away, you can restore from a snapshot. Those are really valuable things to have if you're putting important information on these volumes. And so just be aware that although EV's volumes have the region or AZ availability restrictions, they tend to have a lot of support with third party tools and how you're going to use the data, which makes them pretty common for a lot of people to use. So of course AWS has an EBS volume controller. It's a CSI driver. So the CS standard interface. Storage interface. Container storage interface, that's it. So AWS has the EBS CSI driver, and the

CSI stands for container storage interface.

Container Storage Interface (CSI)



And so this is the way that the Kubelet talks to some storage driver. And that storage driver is usually external.

Container Storage Interface (CSI)



© Copyright KodeKloud

But like I said, it can also be something that is internal to the node

Container Storage Interface (CSI)



or even just abstracted away that it doesn't know if it's where the storage comes from.

Container Storage Interface (CSI)



© Copyright KodeKloud

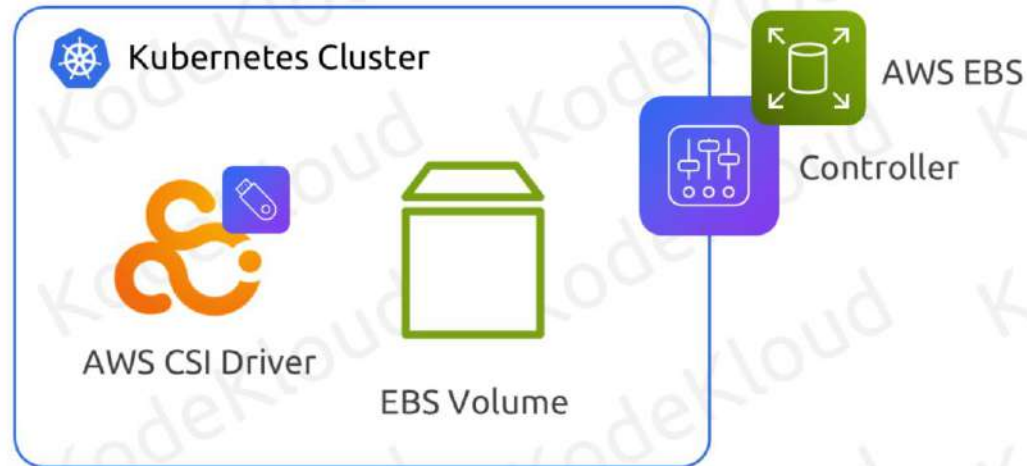
it's just one of them sitting out there inside your cluster that can do this coordination and know how things are being mapped. The other component is a daemon set which will run one on every one of your nodes.

Container Storage Interface (CSI)



And inside your Kubernetes cluster, when you install the AWS EBS CSI driver, you're going to get two components, you're going to get one, which is the controller which is managing full lifecycle of these volumes.

Container Storage Interface (CSI)



So when you're requesting a volume, if you need to delete a volume, it's going to be creating these on the AWS side so that it can know, oh, like what volumes exist.

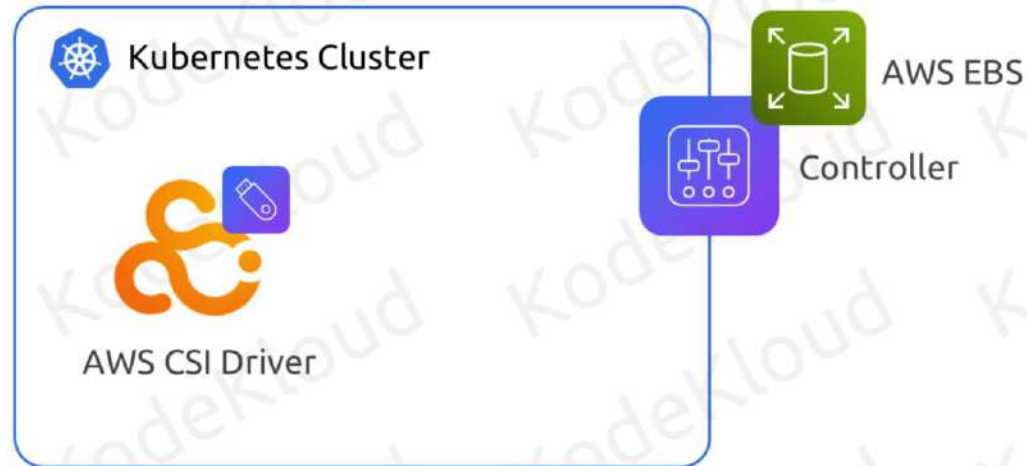
Container Storage Interface (CSI)



© Copyright KodeKloud

If you have a large cluster and you have 1000 volumes, you need something that kind of coordinates that for you. And the EBS controller is doing that as a deployment and it's just one of them sitting out there inside your cluster that can do this coordination and know how things are being mapped.

Container Storage Interface (CSI)



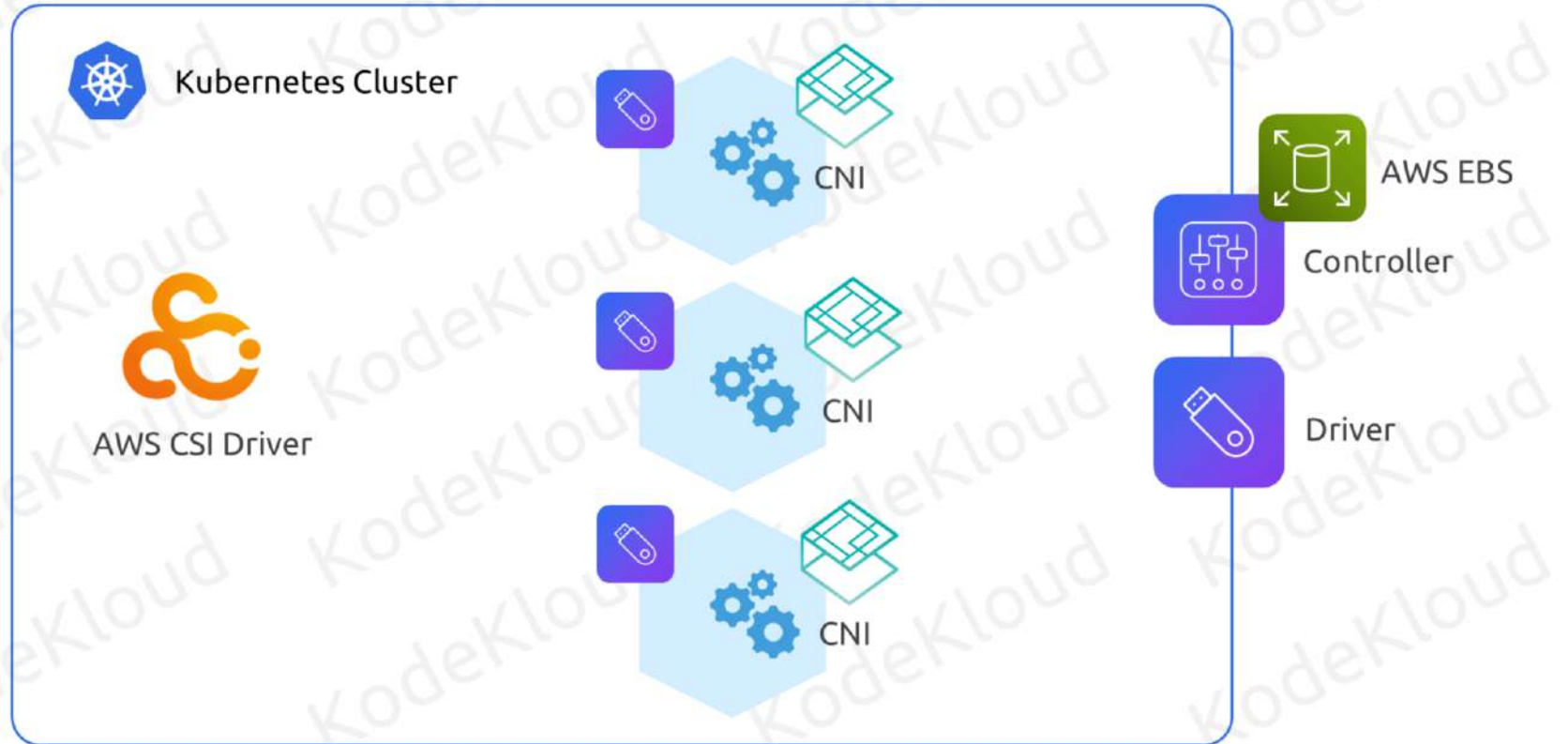
And the EBS controller is doing that as a deployment and it's just one of them sitting out there inside your cluster that can do this coordination and know how things are being mapped.

Container Storage Interface (CSI)



The other component is a daemon set which will run one on every one of your nodes. And this is the driver itself.

Container Storage Interface (CSI)



© Copyright KodeKloud

This actually will mount in a bunch of file systems from the host to drop files in place just like the CNI,

Container Storage Interface (CSI)



the VPC CNI driver does where it initializes drops a bunch of files so the Kubelet can use them and then the Kubelet can execute those files as needed to get IP addresses.

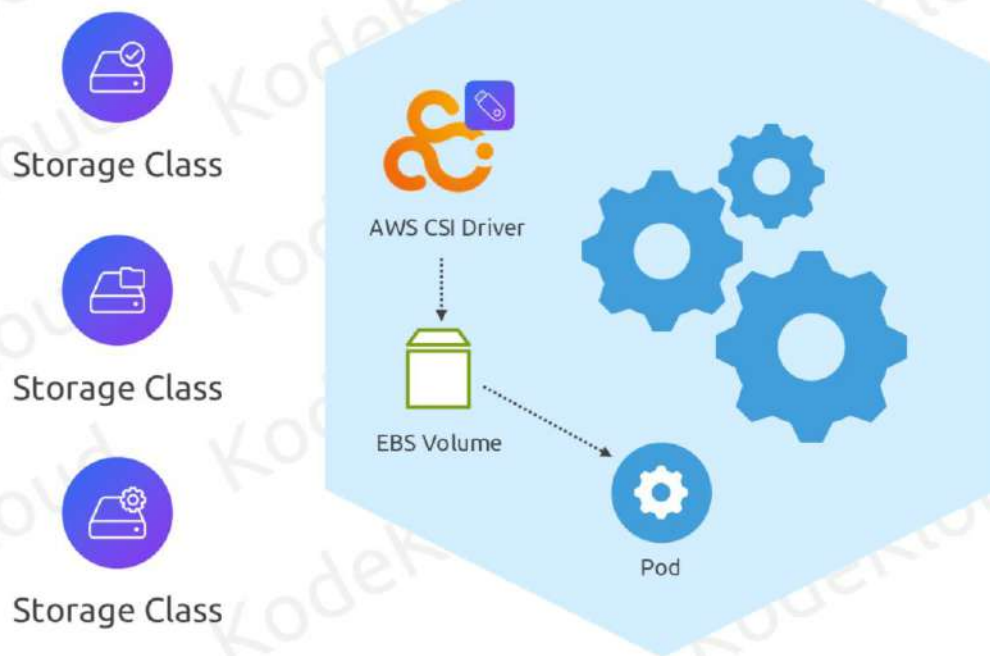
Container Storage Interface (CSI)



AWS CSI Driver

The CSI driver is doing basically the same thing.

Container Storage Interface (CSI)



© Copyright KodeKloud

The node portion of it, where it runs on the node mounts these volumes in, throws some files in place and you're off to the races with being able to request these storage endpoints for your pods. There are also a couple resources within the cluster, things like a storage class or storage classes. There's multiple of them that get installed which will allow you to provision persistent volumes depending on what your workloads are needing. So, some of that is going to come by default when you install the controller. And we'll show you exactly how that works in just a second.

Conclusion



If you've ever used any sort of stateful set or external storage driver inside of Kubernetes.

Conclusion



The EBS CSI driver should work just like you would expect. If there are any surprises, it's probably because of something that is available in EBS as a service, or how it's constructed that maybe your other service wasn't running that way whatever you were using before. And so that's just a quick overview on how the storage driver works, when you might want to use it, and some things to be aware of as you're trying to run more and more workloads in your Kubernetes cluster, especially stateful workloads that might have some requirements that you don't normally have with a stateless workload.

Elastic File System (EFS)

© Copyright KodeKloud

Okay, now that we've talked all about EBS and how to get volumes put into your containers using Amazon's EBS for block storage, let's talk about the other main storage system that a lot of people use with Kubernetes, and that's EFS elastic file system.

Introduction to EFS



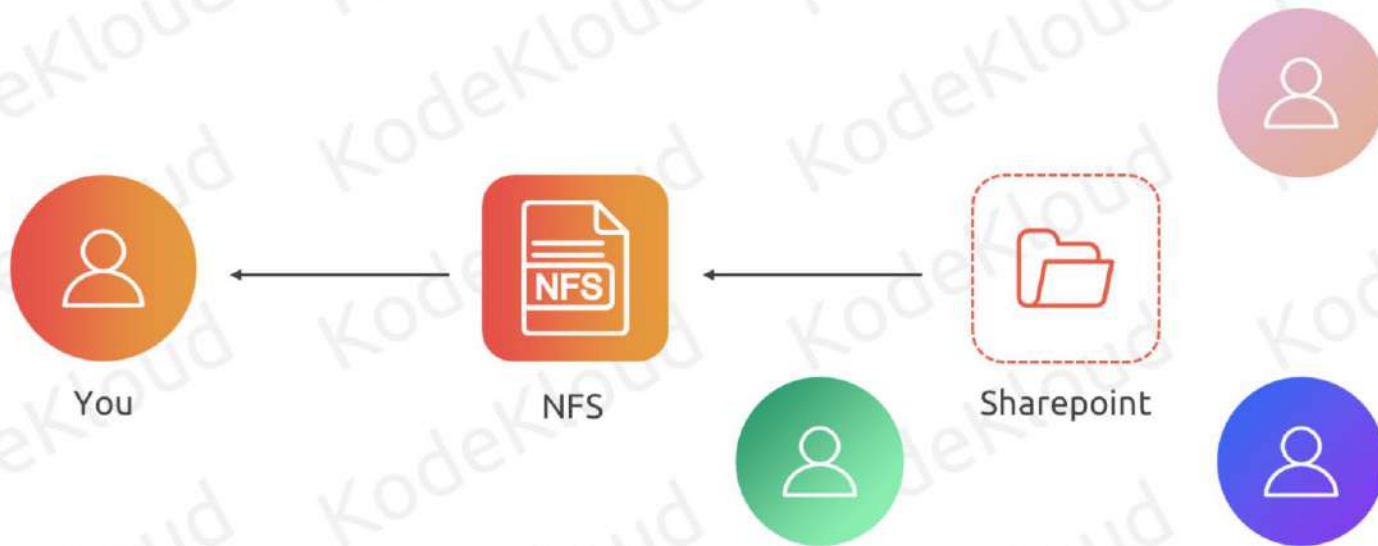
Elastic File System (EFS)



Network File System (NFS)

EFS is just NFS with AWS complexity. If you've ever used NFS in the past, it allows you to mount a sharepoint from something externally and give you files. You don't need to worry about what format it's in or necessarily how big it is when you mount it. That's all handled externally from you. You say, I just need some files at this path, please give them to me. And the benefits that people use EFS for is the fact that first of all, it is a regional service, which means that if you have a pod in a zone and another pod in a different zone, they can mount the same EFS volume. And so they can take that mount point from both AZs and put them in the pod and the pods can actually share the volume.

Introduction to EFS



© Copyright KodeKloud

EFS is just NFS with AWS complexity. If you've ever used NFS in the past, it allows you to mount a sharepoint from something externally and give you files. You don't need to worry about what format it's in or necessarily how big it is when you mount it. That's all handled externally from you. You say, I just need some files at this path, please give them to me. And the benefits that people use EFS for is the fact that first of all, it is a regional service, which means that if you have a pod in a zone and another pod in a different zone, they can mount the same EFS volume. And so they can take that mount point from both AZs and put them in the pod and the pods can actually share the volume.

Benefits of EFS



EFS is just NFS with AWS complexity. If you've ever used NFS in the past, it allows you to mount a sharepoint from something externally and give you files. You don't need to worry about what format it's in or necessarily how big it is when you mount it. That's all handled externally from you. You say, I just need some files at this path, please give them to me. And the benefits that people use EFS for is the fact that first of all, it is a regional service, which means that if you have a pod in a zone and another pod in a different zone, they can mount the same EFS volume. And so they can take that mount point from both AZs and put them in the pod and the pods can actually share the volume.

Benefits of EFS

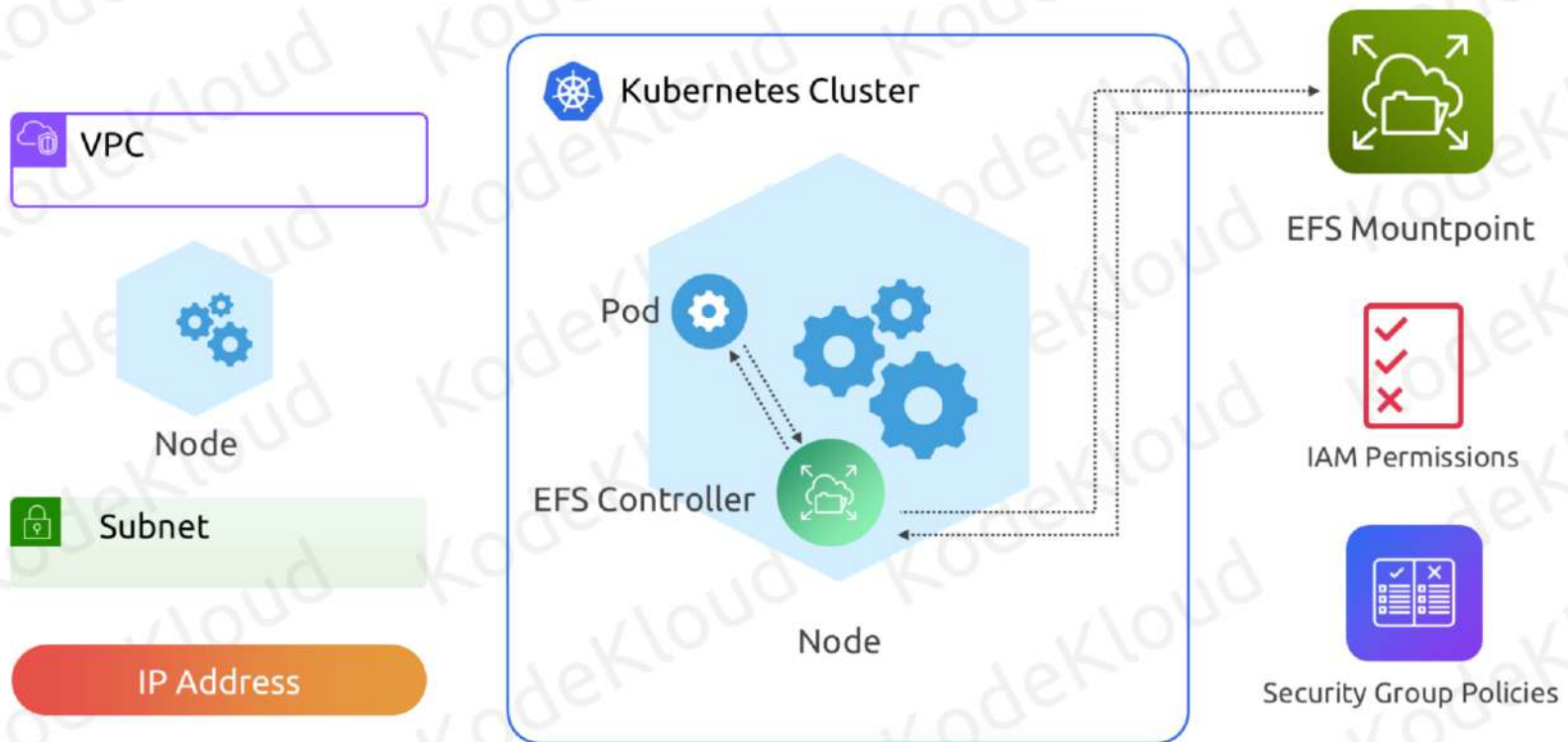


© Copyright KodeKloud

EFS is just NFS with AWS complexity. If you've ever used NFS in the past, it allows you to mount a sharepoint from something externally and give you files. You don't need to worry about what format it's in or necessarily how big it is when you mount it. That's all handled externally from you. You say, I just need some files at this path, please give them to me. And the benefits that people use EFS for is the fact that first of all, it is a regional service, which means that if you have a pod in a zone and another pod in a different zone, they can mount the same EFS volume. And so they can take that mount point from both AZs and put them in the pod and the pods can actually share the volume. This is different than EBS. If you remember, EBS volumes are locked to a zone, so you can't move those around without snapshotting and putting them

somewhere else. And they only get mounted one to one for a pod to a volume. With EFS, you actually get to again share those and you get to do them regionally. So those are huge benefits, because if a node goes down or pod goes down in one AZ, you can spin up the pod in a different AZ and still mount the same sharepoint without or the same, you can still mount the same volume from that pod and not worry about, oh, I'm not in the right AZ, I don't have access to that volume from over here. Definitely a big benefit of EFs. But let's go ahead and diagram it out and show you a little bit more of how it works and when you may want to or not want to use it.

How EFS Works



© Copyright KodeKloud

So let's again say that I have a node here and this is part of my Kubernetes cluster. And out here I have this EFS mount point and EFS volumes and shares out. EFS volumes are created completely outside of the Kubernetes cluster. This is something that is not created dynamically for you. From the controller that runs inside of Kubernetes, there is an EFS controller that runs inside the mount the volumes. And make sure that it's available for pods and do some dynamic provisioning. But it's not actually creating the full mount point and access to EFS like you might be doing from EBS to get volumes. So for EFS to work, we have to first create our mount point. So to, so to get an EMS volume mounted into a pod, we first have to go externally and create our EFS mount point. Again, this is something that you have to do as part of just normal cluster

creation. And this would be similar to, if you have NFS on Prem, you have to have someone export a mount point for you to go ahead and mount the things with EFS volumes is because you're running in AWS, you need to add things like IAM permissions of who can mount them. And you also need to add security group policies to allow the pods and the nodes from the EKs cluster to be able to mount this at all. EFS volumes are not actually that trivial to make. It does take a little bit of work to make sure that everything is set up, because again, wherever your cluster is running, you need to know the VPC for it, you need to know which nodes, and you need to know the subnets that nodes are going to be coming from so that you can make sure you can allow those IP addresses to mount this EFS volume. Now the controller is going to take whatever that EFS volume is allowing for the pod and then mount it as a normal path that any pod should see. This is how persistent volumes always work in Kubernetes. Where that file system comes from is really transparent to the pod itself. And then the pod will be able to, if it has the IAM access, write files into that mount point.

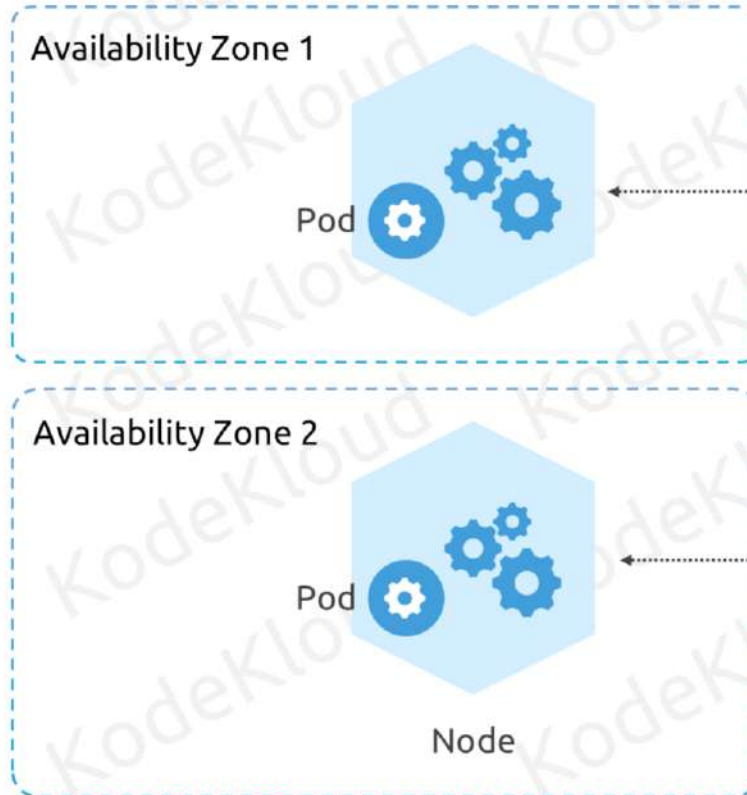
Pods and EFS Volume



Fargate



Windows



And one of the biggest values that people use EFS for is the fact that I can have multiple pods all mounting the same EFS volume. So if I have a different node with a different pod, even in a different AZ, I can also get that volume mounted and both of those pods will be able to write files to this EFS volume. There are some concerns that if you've ever used NFS before, things like file locking. So if you have say a database in EFS and one of the, one of the pods is going to update the database, it's going to lock the file that exists in EFS and that locking may cause the other pod to hang. Those are all things that will happen inside of an NFS environments, just in general, and EFS has no exception to that rule. So if you're running things that need high concurrent writing, or things that are going to be writing a lot of little files to disk, these are all

concerns that maybe a block storage has better trade offs for what you're trying to do and better performance for that characteristic. But like I said, the main thing that people use EFS for is just so they can get multi zonal mount points into their pods. There are some other considerations with EFS depending on what workload you're running, where things like dynamic provisioning for EFS aren't available if you're using fargates, and they're also not available if you're using Windows. Those are both, those are both node run types that do not support dynamic provisioning for the controller. So depending on what you're doing, you do want to make sure that EFS will even work for your use cases.

EFS vs NFS



Elastic File System (EFS)



Network File System (NFS)

There is also a provisioner, a storage provisioner for Kubernetes that is a generic NFS provisioner. And a lot of times people ask, hey, if EFS is just NFS, can I use that one to make these mount points? And depending on how much work you do upfront, the answer is maybe because EFS does require IAM and security groups and allow listing the IP addresses for the nodes. At some point it looks like an external NFS with all of the authorization transparent to the nodes and the workloads. But some of the benefits you usually get with NFS are but the benefits you get with the EFS controller is the dynamic provisioning for paths, because sometimes you want to share one mount point with a lot of different workloads and give them a different sub path where they can actually write their information. So app 1 may be at a top level and app b has a

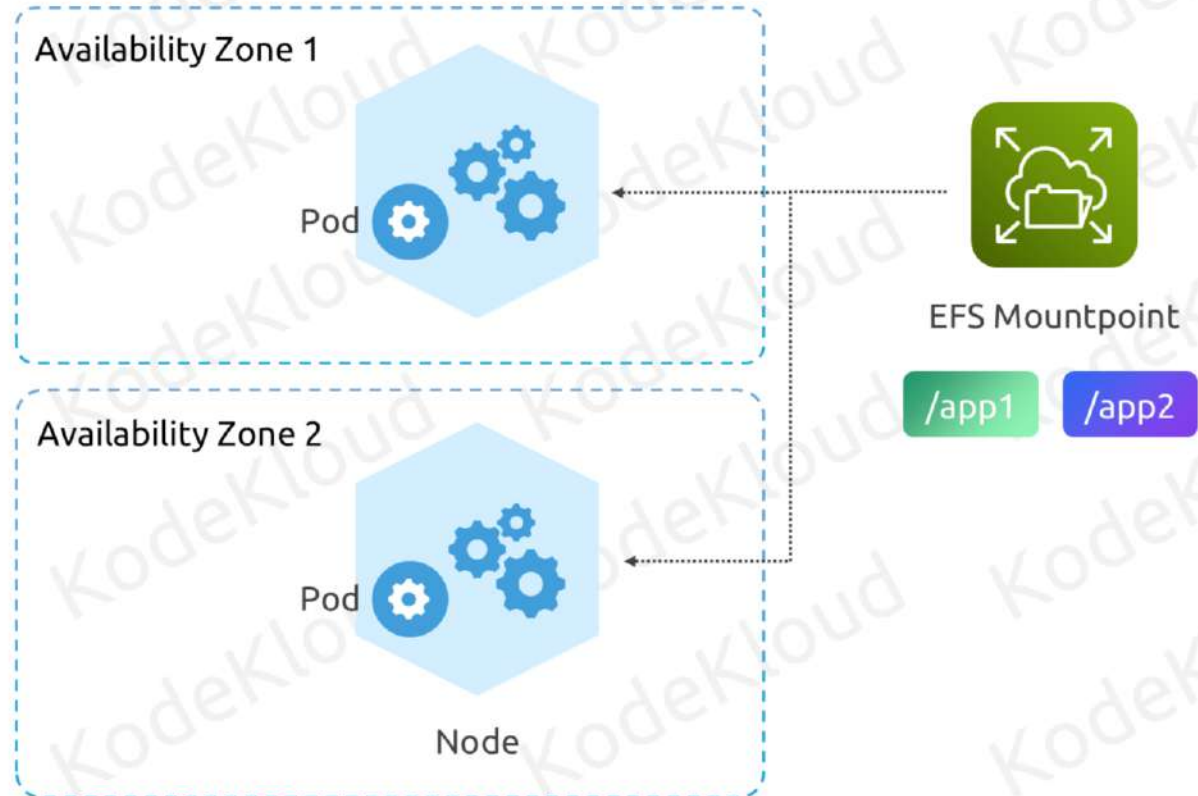
different top level that it writes to. So you're not overlapping, you're using the same top level mount points, but each app has its own paths underneath that, similar to maybe an s three buckets, where the s three bucket has a bunch of different things in one bucket that are apps writing in different sub paths. Some other things to consider with something like EFS is maybe the amount of things you're storing or how big the file system needs to be. EBS volumes used to have smaller limitations on how big the volume could be, and now they don't really have that concern as much.

EFS vs NFS



There is also a provisioner, a storage provisioner for Kubernetes that is a generic NFS provisioner. And a lot of times people ask, hey, if EFS is just NFS, can I use that one to make these mount points? And depending on how much work you do upfront, the answer is maybe because EFS does require IM and security groups and allow listing the IP addresses for the nodes. At some point it looks like an external NFS with all of the authorization transparent to the nodes and the workloads.

Dynamic Provisioning



© Copyright KodeKloud

But the benefits you get with the EFS controller is the dynamic provisioning for paths, because sometimes you want to share one mount point with a lot of different workloads and give them a different sub path where they can actually write their information. So app 1 may be at a top level and app b has a different top level that it writes to. So you're not overlapping, you're using the same top level mount points, but each app has its own paths underneath that, similar to maybe an s3 bucket, where the s3 bucket has a bunch of different things in one bucket that are apps writing in different sub paths. Some other things to consider with something like EFS is maybe the amount of things you're storing or how big the file system needs to be. EBS volumes used to have smaller limitations on how big the volume could be, and now

they don't really have that concern as much.

EFS vs EBS



1 In EFS, File System Size matters



2 In EBS, volume is not a concern



3 EBS and S3 are cheaper options

Some other things to consider with something like EFS is maybe the amount of things you're storing or how big the file system needs to be. EBS volumes used to have smaller limitations on how big the volume could be, and now they don't really have that concern as much. You can get into terabytes of size for a single EBS volume. EFS used to be the way to store a lot of data, but now you can also do that in EBS volumes or even s three, which is probably a cheaper option than EFS. So knowing if you want EBS or EFS for storage for your pods really depends on your workload and kind of what you're familiar with. If you're familiar with NFS, EFS might be the right path for you because it's the thing that you can kind of debug. And you know how some of it works. If you're not used to NFS, then you may want to lean towards EBS or in store or, or

alternatives like s three. And we'll talk about that in the next section.

Other Storage

Other Storage Options



FSX



Amazon FSX for Lustre



Amazon S3



Local Storage

Before we wrap up the storage section, we didn't talk about the other options. There are things like FSX, which is lustre AWS managed lustre. There's s three and there's local storage that are all just available inside of AWS that can be used from a Kubernetes cluster,

Other Storage Options



Kubernetes Cluster

Before we wrap up the storage section, we didn't talk about the other options. There are things like FSX, which is lustre AWS managed luster. There's s three and there's local storage that are all just available inside of AWS that can be used from a Kubernetes cluster,

Other Storage Options



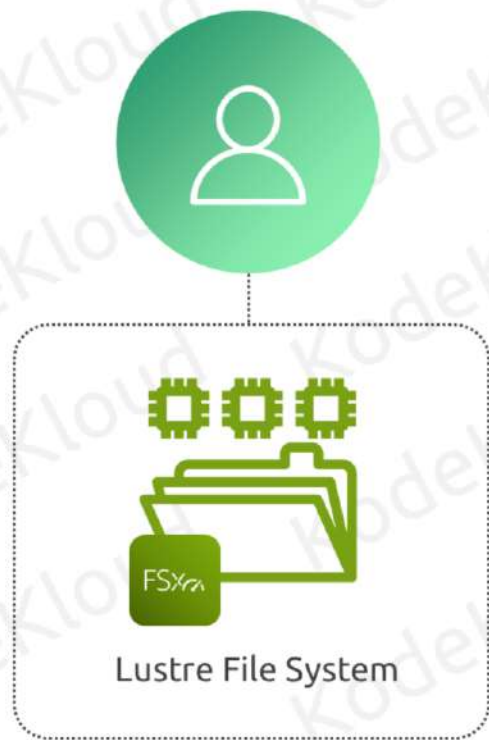
but don't necessarily have the automatic volume mounting that you might get from a container storage interface driver. And that's what we really wanted to focus on, something that makes the experience a little more native inside the Kubernetes cluster. Because while you can mount these other file systems from anywhere, really it's up to you to be able to do that. And so that usually requires some manual provisioning of storage and saying, oh, I need to create a volume, I need to export that into Kubernetes, and then I need to mount that to my workload.

Other Storage Options



So if you know, you need something like a luster file system for high performance throughputs and file based access,

Other Storage Options



then you might be doing some of that on your own.

Other Storage Options



Amazon S3



IAM Credentials

S three is an interesting one, because a lot of applications use s three natively to put objects, and so they automatically build in s three capabilities to read and write to an s three endpoints. And there's nothing you have to do in your infrastructure to allow them to do that, except for possibly add IAM credentials.

Other Storage Options



Sometimes they bake that into the app, sometimes they do it other ways.

Other Storage Options



© Copyright KodeKloud

Sometimes they bake that into the app, sometimes they do it other ways.

Other Storage Options

Mount Point for S3



S3 Buckets



S3 Buckets



S3 Buckets

But if they want to transparently handle hey, I need to read and write from this buckets, you might need to provide them the ability to do that from the infrastructure layer. S three also recently announced a mount point for S three, which basically takes an S three buckets and presents it as a file system similar to EFS, where you don't have to worry about volumes, where you don't have to worry about the disk formatting. You just get files from an S three buckets, but it tightly adheres to what the s three API provides. An s three API is not a posix, a Posix compliance API. Things that you normally would do on a file system in Linux maybe don't work with the s three mount points.

Other Storage Options

Mount Point for S3



S3 Buckets



S3 Buckets



S3 Buckets

But if they want to transparently handle hey, I need to read and write from this buckets, you might need to provide them the ability to do that from the infrastructure layer. S three also recently announced a mount point for S three, which basically takes an S three buckets

Other Storage Options



and presents it as a file system similar to EFS, where you don't have to worry about volumes, where you don't have to worry about the disk formatting.

Other Storage Options

Files



S3 Buckets

You just get files from an S three buckets, but it tightly adheres to what the s three API provides. An s three API is not a posix, a Posix compliance API.

Other Storage Options



Linux

Things that you normally would do on a file system in Linux maybe don't work with the s three mount points. So this might be a way to get a simplified I can see this globally or regionally into my application, but I don't know how to actually use it.

Container Storage Interface (CSI) Driver



CSI

Container Storage Interface Driver

The s three mount points CSI driver has a whole document about how this works.

S3 Does Not Support Renames

Update a Name

Delete and Object

Upload a New One

Multiple API Calls

You would have to update a name, or you have to delete an object and then upload a new one. And to get a renaming, and so you need multiple API calls.



POSIX-Compliant File System

POSIX-Compliant File System

There's no just way to rename something like there is in a POSIX compliant file system.

POSIX-Compliant File System

POSIX-Compliant File System



Amazon S3



Directory

There's no just way to rename something like there is in a POSIX compliant file system. There are also things like directories don't actually exist in s three. These are, these are interpreted by forward slash characters. But when you create, if you go mount an s three mount points and say make directory, make dir, and you put something there, it doesn't go anywhere inside of s three because there isn't a directory. So these sorts of things kind of have different behavior that you have to be aware of if you want to use s three as a file system replacement. So that might be a good option to get started. But again, the behavior and how your application is going to use that storage will really determine on what type of storage you need.



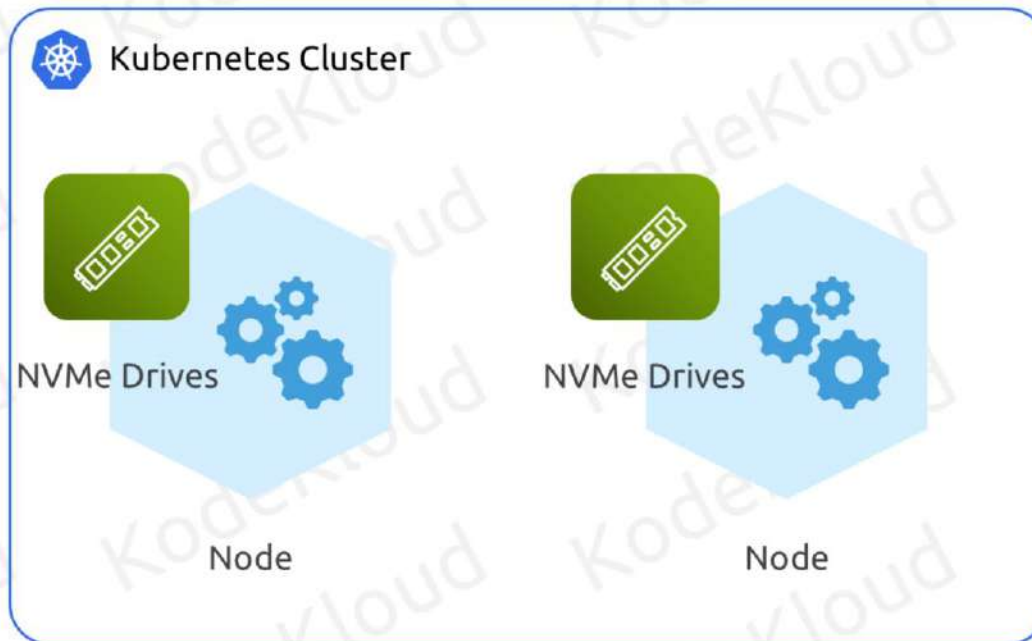
Local Volumes



Local Volumes

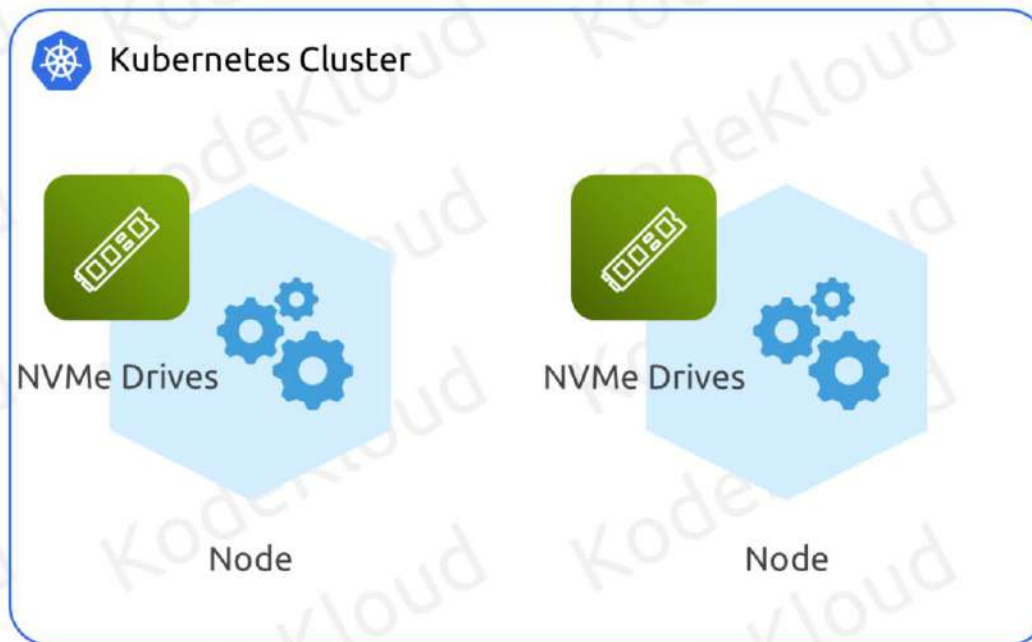
We should also talk about local volumes,

Local Volumes



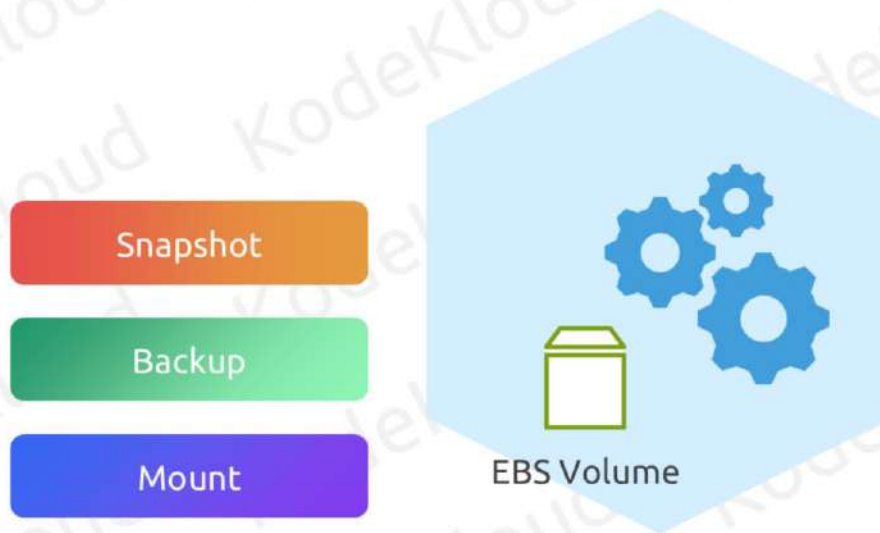
because all of these nodes that you're running in your Kubernetes cluster have hard drives, and some of them may even have local NVMe drives, which are basically the fastest storage you can get in AWS. And if you want to mount those and use them for reading and writing data, that's really fast locally, that's a valid option.

Storage and Nodes



Of course you need to be aware of that. Storage could disappear when the node disappears. That is ephemeral storage, and so that local volume isn't going to save the data externally unless you're snapshotting it.

Storage and Nodes



© Copyright KodeKloud

If you have an EBS volume mounted to the node, you can snapshot it, you can backup some of that, you can mount it later, you can mount it as a second drive. These are all options, but they add some complexity in how you're managing it,

Local Volume and CSI Driver



Local Volumes



CSI

Container Storage Interface Driver

because a local volume doesn't have external CSI driver that's going to do that mapping, snapshotting and remounting for you on new nodes.



Cluster Options

Cluster Options

The final option I'll talk about is in cluster options. Kubernetes has a lot of different ways you can run storage, and there are plenty of things that run inside the cluster that do use the node storage.

Cluster Options



CEPH



Longhorn



OpenEBS



Rook

There's things like Ceph, which will allow you to have replicated data inside the cluster. There's Longhorn, there's, there's longhorn which is from rancher which gives you a similar distributed file stored inside the cluster using local volumes on the nodes. Open EBS rook the list goes on and on. There's a lot of these storage providers that'll run inside the cluster, and these get handled in a more Kubernetes native way because you're not calling out to external APIs in requiring IAM access or VPC routing or subnet or security group access. These are all things that handle inside the cluster and so the workloads can spin up and manage everything in a Kubernetes native way.

Cluster Options



IAM Credentials



VPC



Subnet

Security group



Kubernetes Cluster



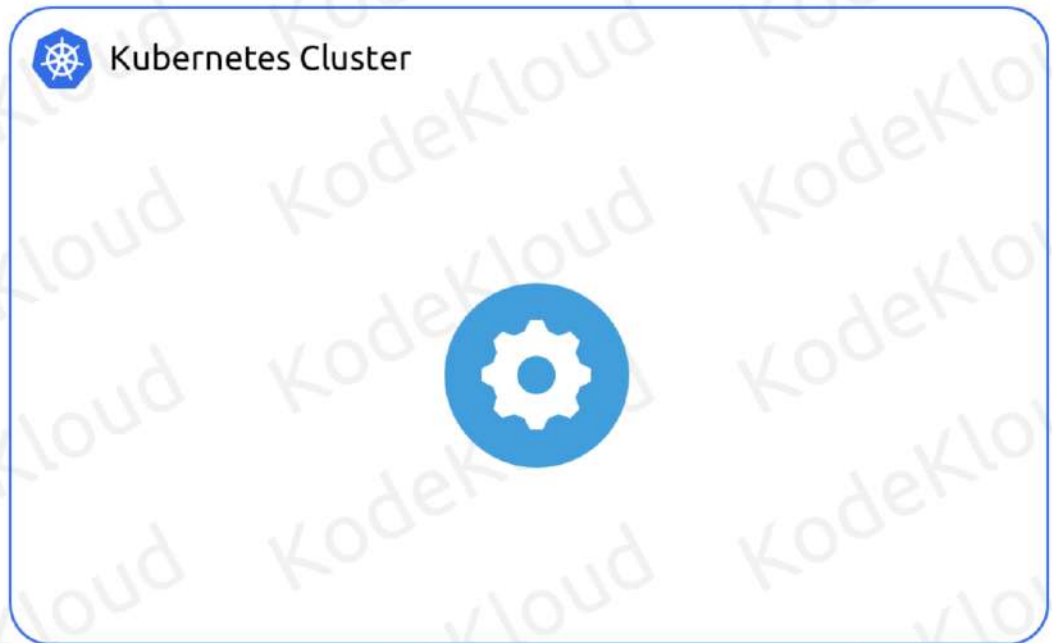
There's things like Ceph, which will allow you to have replicated data inside the cluster. There's Longhorn, there's, there's longhorn which is from rancher which gives you a similar distributed file stored inside the cluster using local volumes on the nodes. Open EBS rook the list goes on and on. There's a lot of these storage providers that'll run inside the cluster, and these get handled in a more Kubernetes native way because you're not calling out to external APIs in requiring IAM access or VPC routing or subnet or security group access. These are all things that handle inside the cluster and so the workloads can spin up and manage everything in a Kubernetes native way.

Cluster Options



You're usually managing this with a controller, maybe an operator and a CRD and some RBAC, and that's how the storage inside the cluster gets spun up and attached to other volumes

Cluster Options



You're usually managing this with a controller, maybe an operator and a CRD and some RBAC, and that's how the storage inside the cluster gets spun up and attached to other volumes.

Cluster Options



while there would be more ongoing maintenance for you and you are responsible for that storage, there might be a lower barrier to entry to get started because you don't have, maybe you don't have access to EFS management or EBS volumes, or those things are too expensive,

Cluster Options



EFS Management



EBS Volume

or you don't know how to manage that,

Cluster Options



EFS Management



EBS Volume

or you're running in multiple clouds or locations.

Cluster Options



Google Cloud Platform (GCP)



Azure

If you're running on Prem in GCP or Azure, you might need to do this in cluster so that you have something that's a little more portable and, and can be deployed completely within a Kubernetes cluster and not only specific to an EKS cluster. State management and storage inside of a cluster is tricky.

Conclusion



FSX



Amazon FSX for Lustre



Amazon S3



Local Storage

This isn't an easy thing to do, and depending on how you're going to manage it and how important that data is, you need to make the decisions for which one you're going to use. Data is probably data is one of the most important things that any application deals with. And so you need to make sure that wherever you're storing that data is and how you're managing it is suitable for how your business runs. Because if you lose this data, could the business go away? How would you back it up? How would you do these other things that might be required of you as an administrator of a cluster and at some point responsible for this information? So make sure you either know how you're going to handle that and back it up, or you're giving that over to AWS to help manage that and maybe have APIs that other teams can back it up and integrate it into

different workflows.

EKS Secrets

Config Map



Kubernetes Secret



Config Map



Okay, so let's talk about a Kubernetes secret. A Kubernetes secret is an aspect of the API that allows us to store sensitive information. And if you're familiar with a config map, a config map is in the cluster and this is just data that we're sending into the API that we're going to use later for a workload.

Config Map



© Copyright KodeKloud

We want to send some sort of either file or configuration or environment variables, whatever it is, we want to send that data so that when our pod starts in our cluster, we can then consume that config map in the workload and that could be through like a volume mounts or through the environment variables.

Config Map



However it is that we want to consume that data, we just want to separate it out. Because if every time we don't, because every time we change configuration for our application, we don't want to have to like rebuild the entire application, we don't want to have to redeploy just to change some values. So we separate out those pipelines to say configuration goes in a config map and application goes in the deployment. But sometimes our configuration that we're using for our application is sensitive information.

Kubernetes Secret



Kubernetes Secret

It's a secret, it's something that we don't want people to see by default. And so Kubernetes made this secrets API that is essentially the same thing as a config map. The only difference is before we send it into the API, we base 64, encode it. And base 64 encoding is not encryption, it's not actually secret. You can very easily reverse base 64 encoding with decoding and you can just take that string and send it through base 64 and you get the original values out of it. So there's no encryption keys, there's nothing going on that's actually hiding this information. It's more obscuring the information. That's the difference here of when we're actually using a secret versus a config map.

Kubernetes Secret



Base 64 Encoded Data



Plain Text Data

Again, they're both very similar in the type of data that we're sending, but the secret is going to be encoded before it gets sent into the API server. So that has some limitations because base 64 encoded data is larger than plain text data that we're sending from a config map.

Kubernetes Secret



Namespaces

So we might have limitations on how big our secrets could be and we might have some actual sensitive information that maybe we can't easily separate out in our Kubernetes cluster because ideally all of your applications are going to be deployed into namespaces and you have RBAC that separates out these namespaces. So that's one person using namespace number one can't read information in namespace number two, and that's a good practice, just to be able to separate those things out. And you could also put a secret in one namespace and not the other one, so that those can't be seen across namespaces. But again, it's not actually secret, it's not actually encoded or encrypted. And so how do we solve that problem? How do we make sure that our data is actually encrypted? And that's what this section is about. We'll show you a

couple different options and show you why this is a different implementation than what we use if we're using native kubernetes secrets.

Kubernetes Secret

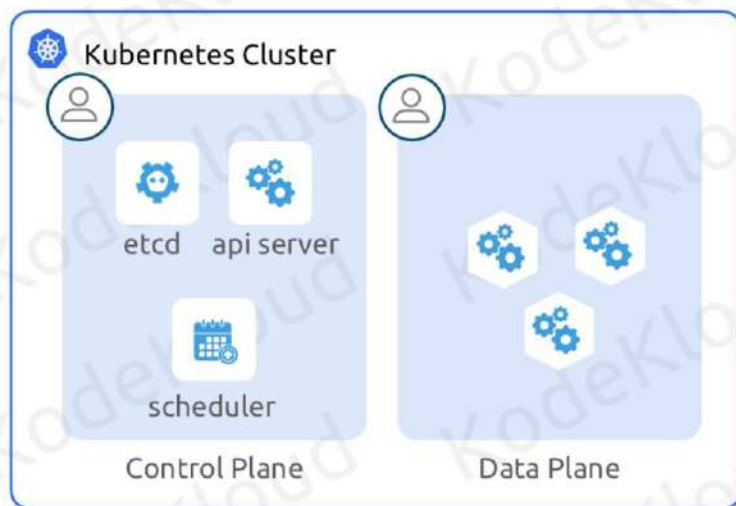


Kubernetes Secret Options

© Copyright KodeKloud

So if a Kubernetes secret isn't actually a secret, what do we do? How do we, how do we make sure that this information is going to be stored securely? And the way that Kubernetes has solved that problem is they said, go use something else.

Secret Storage



Secret Storage

And it's a great option, actually, if you're running kubernetes in environments, if you have some sort of secure secret storage or outside of kubernetes, somewhere where your secrets are maintained and rotated and audited, and all of the things that you might need for secrets,

Secret Storage



Secure Secrets



API Keys



Root Passwords



Secret Storage

especially like really secure secrets, API keys, root passwords, anything of that nature, you probably want some secure place to store that.

On-premise or own environment



Hashicorp Vault



Secret Storage

And if you're running this on prem or in your own environment, and you need some control over that, you're probably looking at something like Hashicorp vaults.

AWS Environment



Secret Storage



AWS Secret Store

And if you're running this inside of an AWS environment, like using eks, you're probably looking at AWS secret store.

Options for storing secrets



Hashicorp Vault



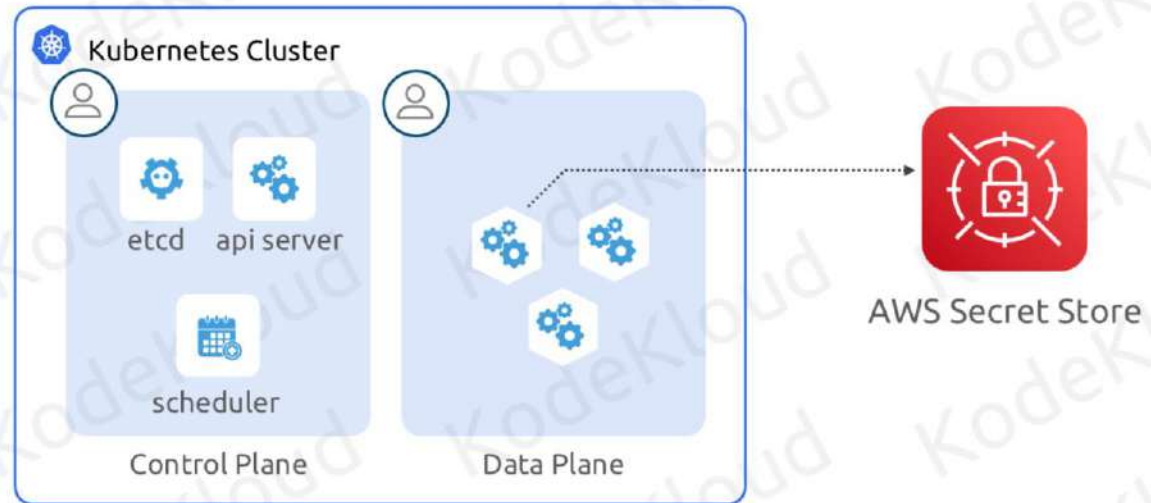
Secret Storage



AWS Secret Store

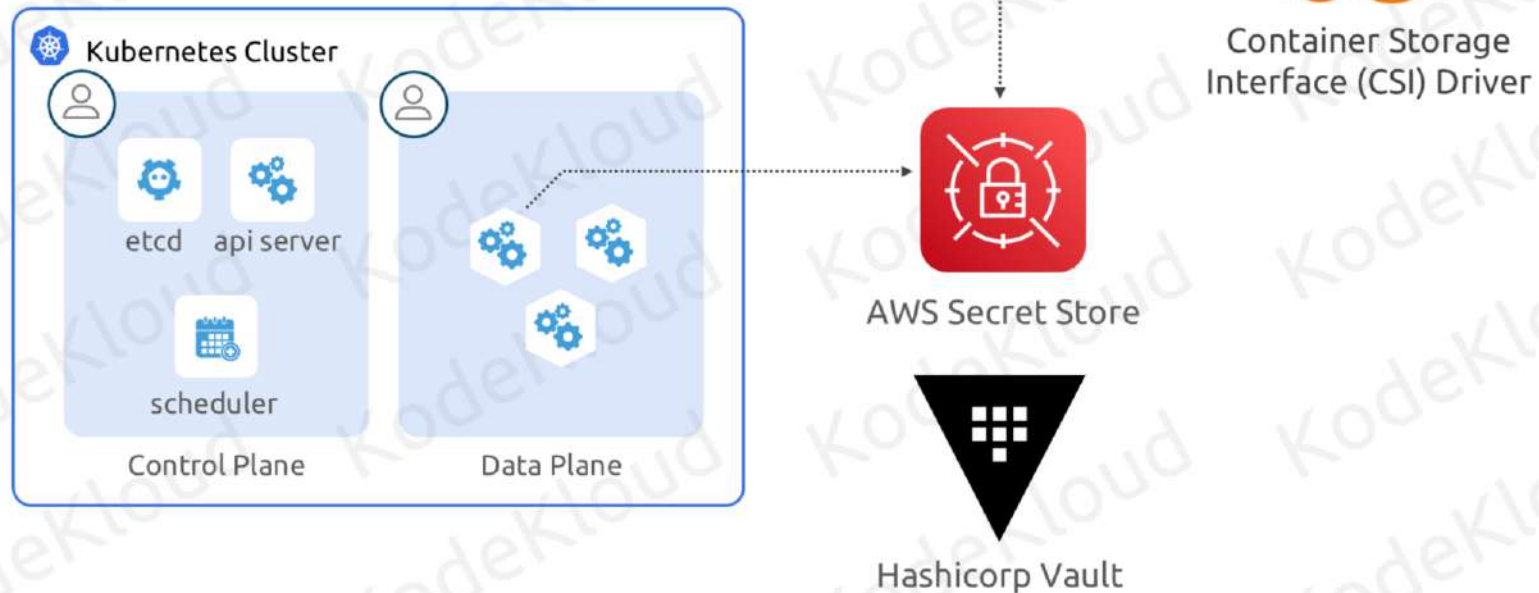
And these are both options for storing your secrets, where you're going to put your secrets and maintain them and have them actually encrypted on disk and not available via just some base 64 decoding. But how do we integrate that into a Kubernetes cluster?

Integrating into a Kubernetes Cluster



How do we make sure that we can read those secrets? So when I run a workload here, I need to fetch a secret from some external service, and I don't want to build that in my application because my application would need the AWS SDK and a lot of things for permissions and the stuff that maybe I don't have access to the code base or I can't extend my application to be able to do that.

Integrating into a Kubernetes Cluster



© Copyright KodeKloud

So what the Kubernetes project has done is they've made a generic CSI driver, a container storage interface CSI for a secret store. So it can actually mount secrets from a multiple of different secret stores backends depending on where you're running kubernetes, and there's a variety of them that are available. Pretty much every major cloud provider that has a secret storage implementation is supported as a backend, as well as things like Hashicorp vaults that you can run yourself.

Integrating into a Kubernetes Cluster



Container Storage
Interface (CSI) Driver



Kubernetes Secret



AWS Secret Store



Parameter Store



Config Map

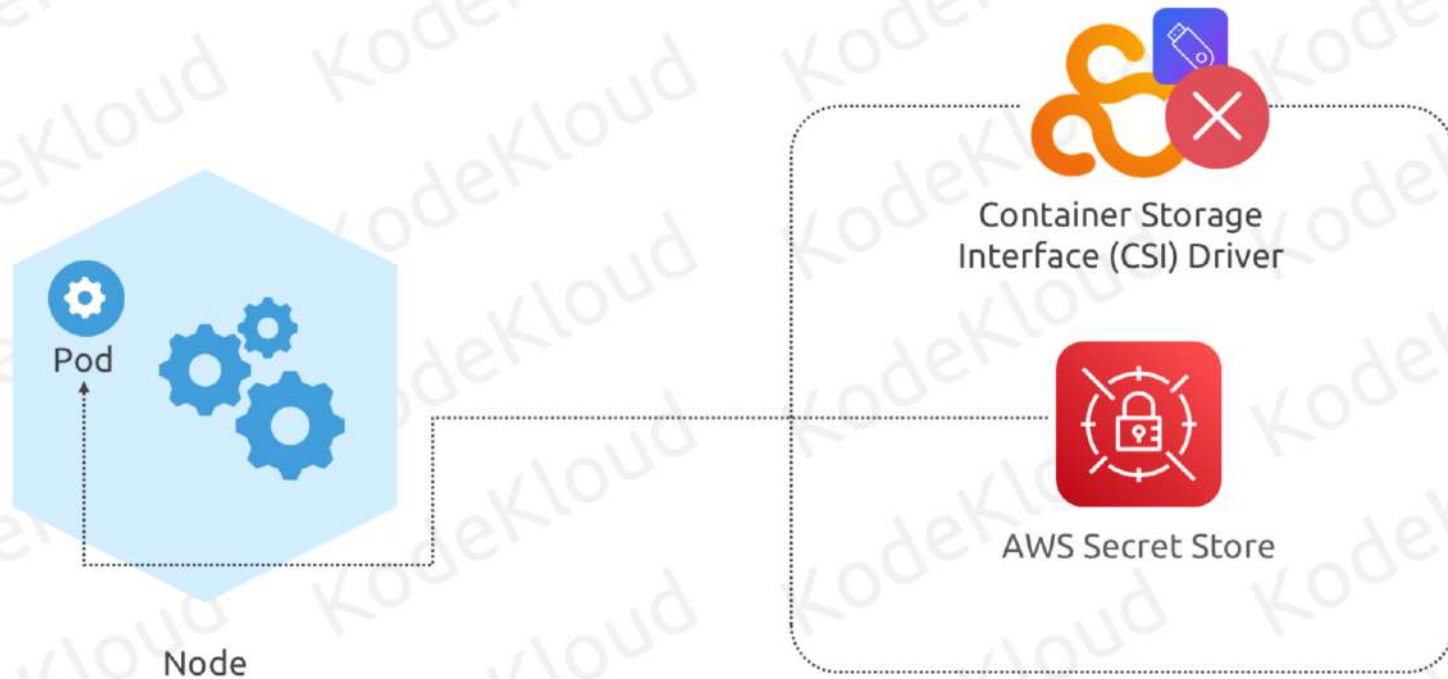
And so what this CSI driver does, oh, and if you watched our storage section already, you know that we kind of went over this already once we talked about the other storage options that you have inside of AWS is actually mounting volumes from AWS Secret store as well as parameter store. And those are both handled by the AWS CSI driver, CSI secret store. And that is basically a way to replace your secrets and config maps. So instead of using the Kubernetes native secrets and config maps, you could store them externally in AWS. And the AWS secret store driver will be able to mount those in.

For Kubernetes Project



for the Kubernetes project. It actually doesn't do the parameter store. It's not doing generic config maps, it's only looking for secrets. So this CSI driver will so the CSI driver will go out and fetch those secrets for from the AWS secret storage

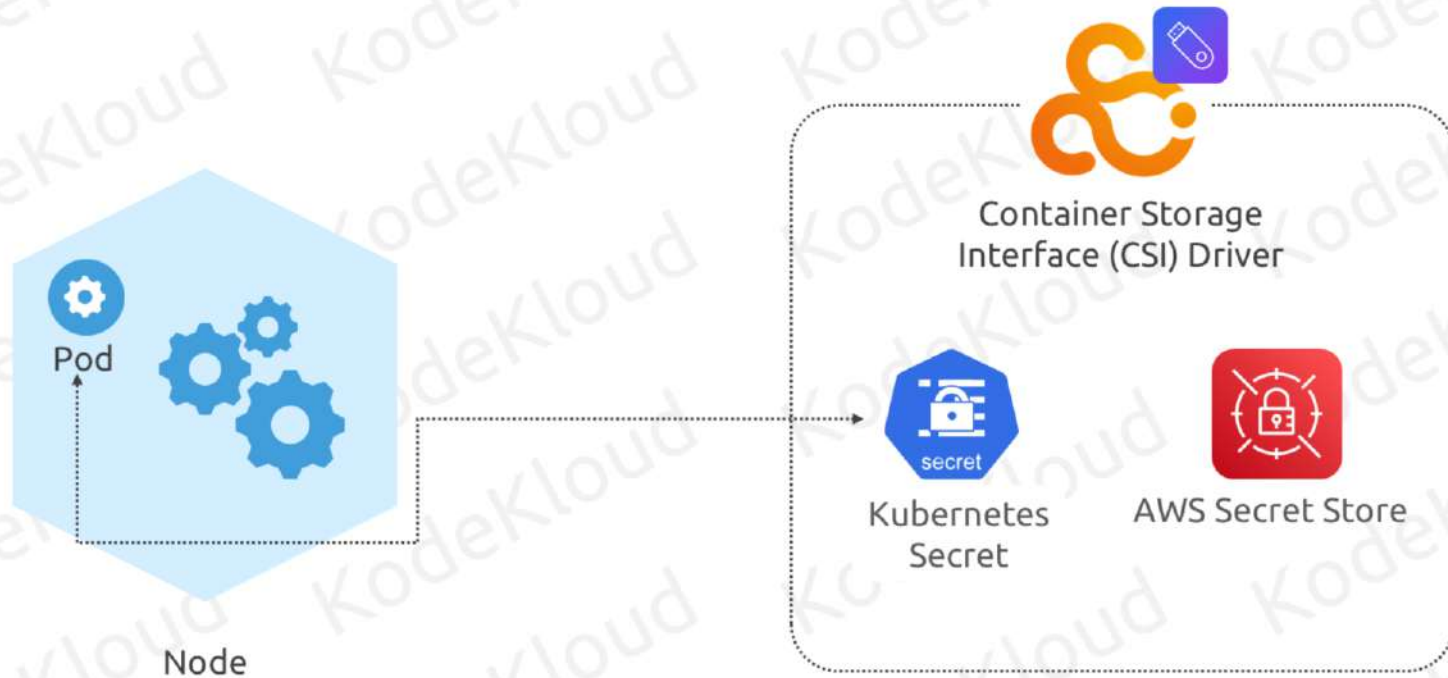
For Kubernetes Project



© Copyright KodeKloud

and then mount them into your pod as a volume. It also can do some secret syncing. So let's say I have a pod that runs and it is going to request a secret, but it wants some environment variables instead of a volume. A CSI driver typically can't do that.

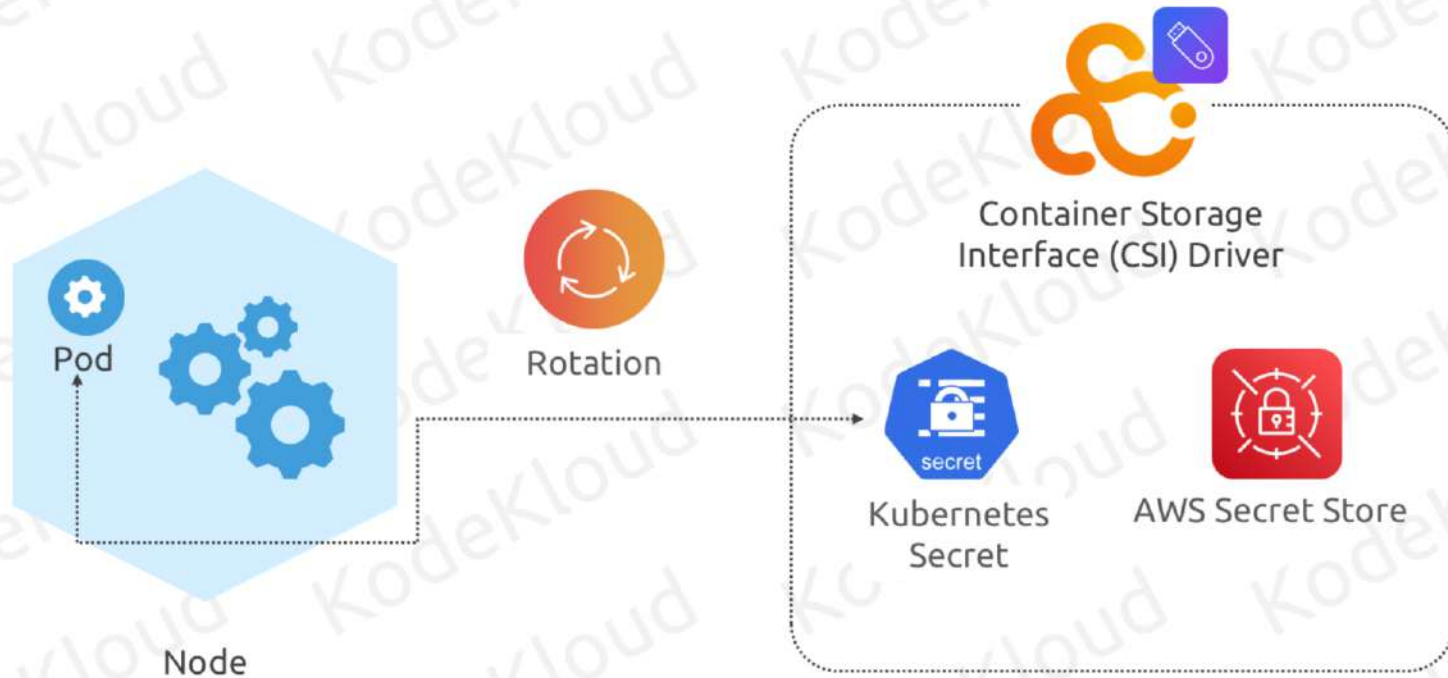
Secret Syncing



© Copyright KodeKloud

So the Kubernetes secret CSI driver can also manage secrets secret syncing for pods directly so that it's actually going to create a temporary AWS or it's going to create a temporary Kubernetes secret from the AWS secret store and it's going to mount that in however you want to consume it in your pod, whether that's environment variables or a volume. And then when that pod goes away, if it terminated or the workloads over or you don't need that secret for some reason, the driver, the secret store driver will actually delete the secret from the Kubernetes cluster. So those are short lived secrets inside the cluster. That's not something that you created once and it lives there forever.

Rotation



© Copyright KodeKloud

It also can handle things like rotations, so we can have our secrets rotated over time. And that's one of the really good benefits about having an external secret storage provider. Those secrets aren't static. It's not something that is always living in the same, you know, however you paste it in is how it's going to be always. So when a new workload comes in and says, oh, I need that secret again, it may have rotated, which is actually a good thing for you because you don't have to worry about the rotation, but the workload can get the most up to date secret that is available to be able to fetch those workloads or do whatever it's trying to do with that secret it.

Things to note



Secrets built into Kubernetes are not encrypted. They are base 64-encoded



To protect secrets, put a stronger RBAC and namespace separation and control those secrets inside as native secrets



Using something external is a more secure way of managing secrets

So just to recap on this, the **secrets that are built into kubernetes are not encrypted. They are base 64 encoded.** And that is just a way to obscure them and not actually protect them in any way. **If you are going to be protecting your secrets,** you have to put stronger RBAC and namespace separation and you have to control those secrets inside of Kubernetes as native secrets. **The more secure way of managing your secrets** in any Kubernetes cluster is to use something external, something outside of Kubernetes that allows for things like secret rotation and other access controls that maybe aren't tied directly to RBAC.

Things to note



AWS Account



AWS IAM



Secret Store Driver

And so you would have things like in an **AWS account** you have **IAM** and different policies based on tags on who can access which secrets. And then you're using your pod identity or your ura inside of a workload inside of an EKS cluster. And we will go over that. There's another section all about security and pod identity, but those are ways that a pod can identify itself inside of Kubernetes or inside of AWS to say, oh, I'm this workload, I have these permissions, please give me that secrets. And it could fetch that directly, but we don't usually want to put that directly in the workload. We don't want to put all that logic in the workload itself. So the third option is usually to use the a **secret store driver**, a CSI driver

Things to note



that is going to take those secrets from AWS secret store and then mount them into the workload for you as a file on disk or something that the workload can see and it doesn't have to worry about, oh, what is my iam permissions and where is this endpoint and all those other things that you might have to do to be able to fetch secrets inside of AWS? In this case, it's just reading a file and the secret store driver is the one that put that file there and is able to do that rotation and all those other things that you might need for getting access to the secrets without even knowing about where the secret comes from.

AWS Load Balancer

Kubernetes Controllers: Early Days vs. Now



© Copyright KodeKloud

Let's talk about how to get traffic in and out of a Kubernetes cluster and gather round, because we're going to talk about the old days when Kubernetes was started. Back in the early days, Kubernetes had these controllers and the controllers were built into the controller manager API server. It had a lot more integrations that weren't broken out, things like the cloud controller. This was a thing that was part of the main Kubernetes API server control plane. Now it's a separate controller. Now it's something that you run in a cluster separately. If you're running eks, eks runs that controller on your behalf. This is how Kubernetes interacts with whatever cloud environment it's working in.

Kubernetes Controllers: Early Days vs. Now



Main API Server



Cloud Controller

Let's talk about how to get traffic in and out of a Kubernetes cluster and gather round, because we're going to talk about the old days when Kubernetes was started. Back in the early days, Kubernetes had these controllers and the controllers were built into the controller manager API server. It had a lot more integrations that weren't broken out, things like the cloud controller. This was a thing that was part of the main Kubernetes API server control plane. Now it's a separate controller. Now it's something that you run in a cluster separately. If you're running eks, eks runs that controller on your behalf. This is how Kubernetes interacts with whatever cloud environment it's working in.

Elastic Load Balancer



Cloud Controller



Load Balancer

So when you say, I want a service in my cluster, the cloud controller is going to go out and give you a load balancer. And always used to do this with an Elb. The last classic load balancers that AWS had that you probably don't want anymore, these were just the default. The elastic load balancers were the default load balancers. And that's all they really had for a very long time.

Elastic Load Balancer



Cloud Controller



Elastic Load Balancer

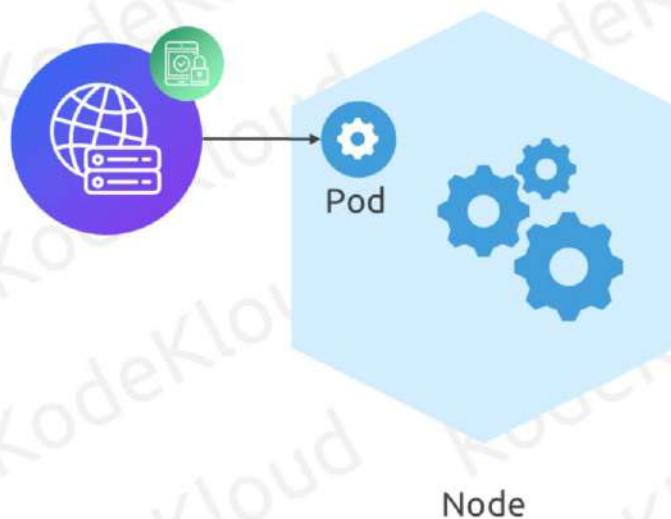
So when you say, I want a service in my cluster, the cloud controller is going to go out and give you a load balancer. And always used to do this with an Elb. The last classic load balancers that AWS had that you probably don't want anymore, these were just the default. The elastic load balancers were the default load balancers. And that's all they really had for a very long time.

Different Types of Load Balancer



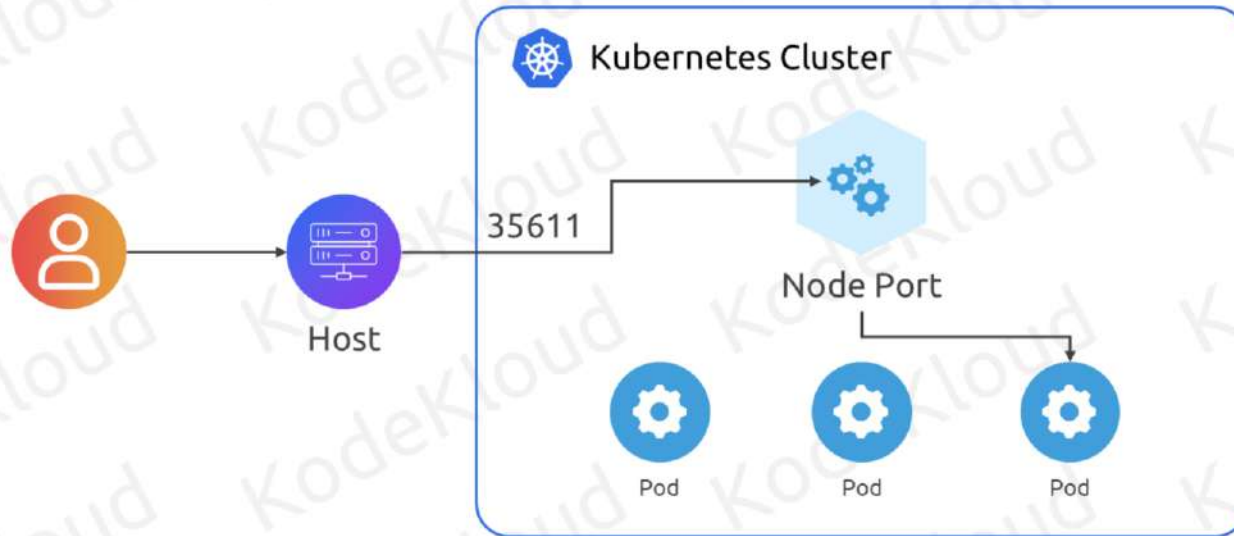
And Kubernetes started breaking these things out and eventually AWS had their own load balancer controller. And the load balancer controller is just for creating different types of load balancers because you don't always want an elb. Sometimes you actually want an application load balancer or a network load balancer, whatever it is you wanted. The load balancer controller was how you could get that.

Understanding Load Balancer in Kubernetes



And so let's just show a couple diagrams. If you know how load balancer works, this might be just catch up for you. And you've seen this before, but if you're not familiar with how it works inside AWS, let's go ahead and diagram it and show you what it does. So as always, we are going to start with a node, and this is a part of a Kubernetes cluster. And I'm going to put a pod somewhere on this node and I want to access this pod. It runs a web server or something that needs to be accessed from the outside.

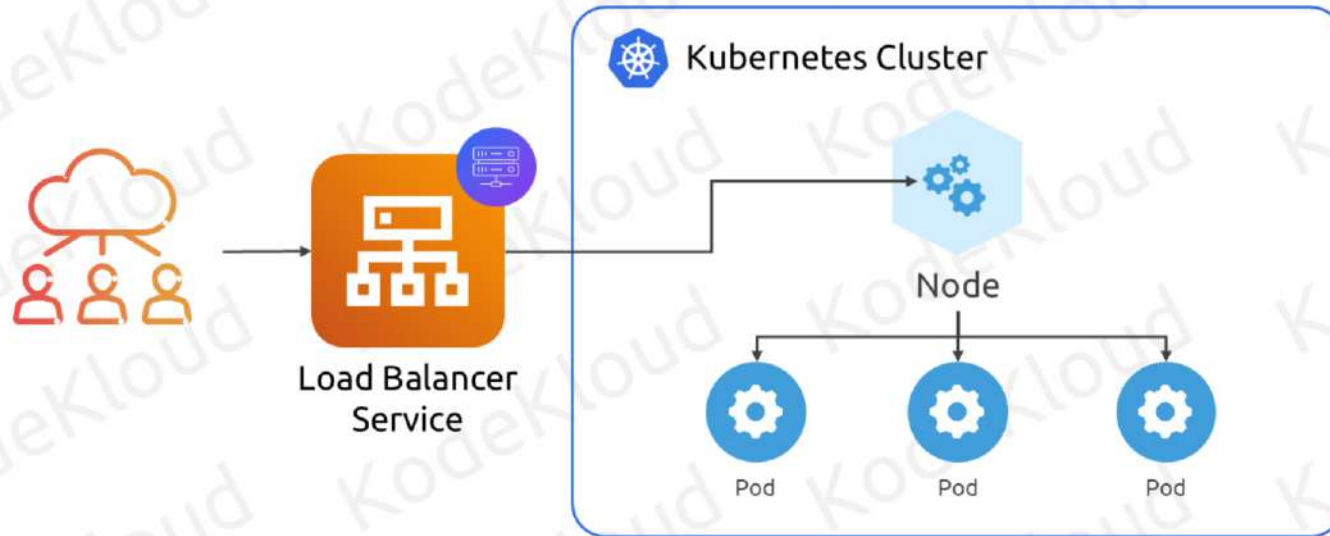
Node Port Service



© Copyright KodeKloud

There are different types of services inside of kubernetes and pretty much all of them boil down to a pod and a node port being exposed. Some port on the host gets exposed and there is a type of service called a node port that only does this. It only says, hey, I only want the ports. And then however you access this, let's say it's on port 35611, some high level, port number, whatever, you could just type in host colon and you will route right to that port. And that's how a node port works.

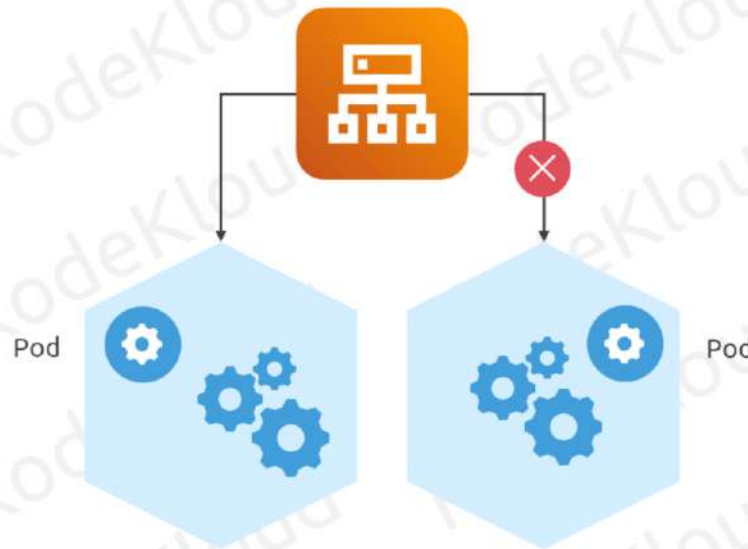
Load Balance Service



© Copyright KodeKloud

Well, a load balance service or a typical, the default for a service is a load balancer that still exposes a port. The host here still needs a port exposed to be able to route traffic. The only difference is the other side of what's calling it and doing health checks. The and routing traffic to it is, is some load balancer. And that load balancer is outside of the node typically, but it's actually not always depending on where you're running this. If you want to run something on prem or you want to have more control over those, you can actually run a load balancer any way you want.

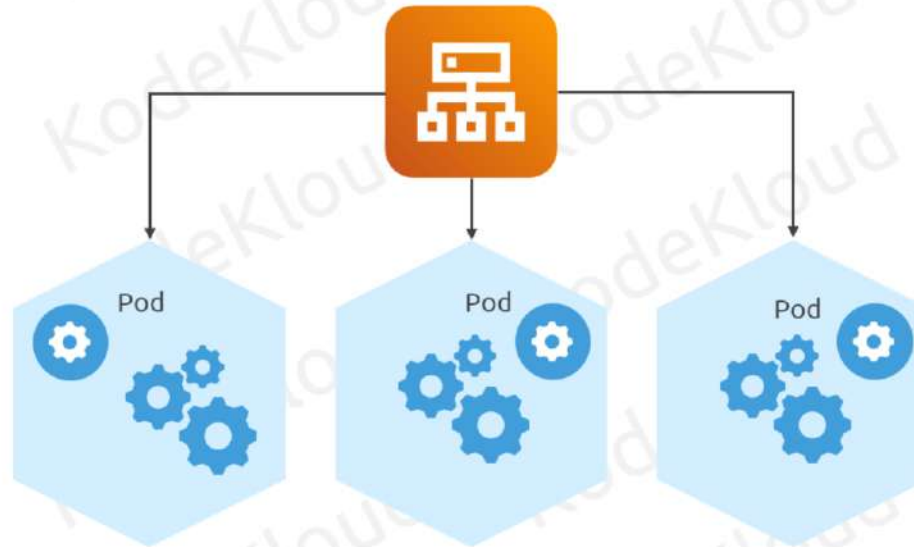
Kube Proxy in Kubernetes



© Copyright KodeKloud

One interesting thing about Kubernetes is one of the defaults they did is if I have another node over here and I'm not running that pod on it, it's just another node. I only needed two replicas, but I have three nodes. For whatever reasons, I still get that port exposed. By default there are settings that changes, but by default I'm going to get that port and the load balancer is still going to try to check if that port is healthy or if that port is available and the application is available. Typically, because there's no application running here, this check here on this port for the load balancer would fail. That's how it typically would work.

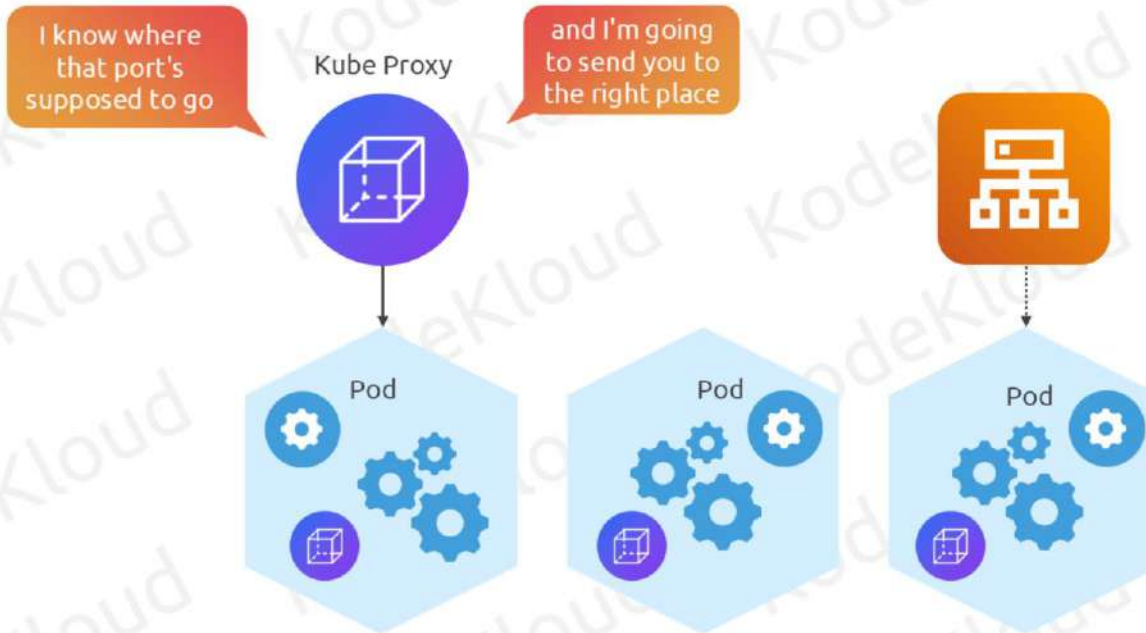
Kube Proxy in Kubernetes



© Copyright KodeKloud

One interesting thing about Kubernetes is one of the defaults they did is if I have another node over here and I'm not running that pod on it, it's just another node. I only needed two replicas, but I have three nodes. For whatever reasons, I still get that port exposed. By default there are settings that changes, but by default I'm going to get that port and the load balancer is still going to try to check if that port is healthy or if that port is available and the application is available. Typically, because there's no application running here, this check here on this port for the load balancer would fail. That's how it typically would work.

Kube Proxy in Kubernetes

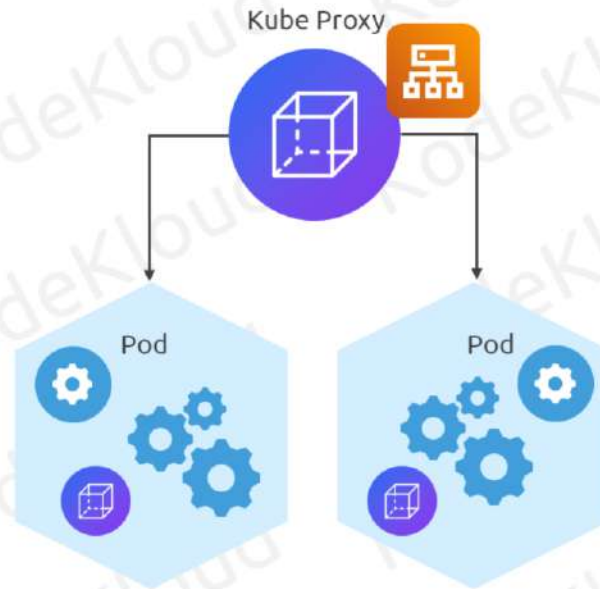


© Copyright KodeKloud

But we run a service on all of our nodes called the kube proxy. And the kube proxy says, oh, I know where that port's supposed to go, and I'm going to send you to the right place. And so it will literally just take that traffic in and bounce it over to the, the correct node with that same port and say, actually there, it'll route it to that same port and say, this is actually going to go over here and it'll just reroute it and say, oh, I accepted that traffic, I know where that port is. I know where that application is running. So I'm going to send you over that way so the load balancer doesn't have to know about it. And again, we can change settings here on the load balancer. And one of the things we usually recommend you change is this setting to say hey, actually only expose this, this port here, only expose that port on a node where the application is actually

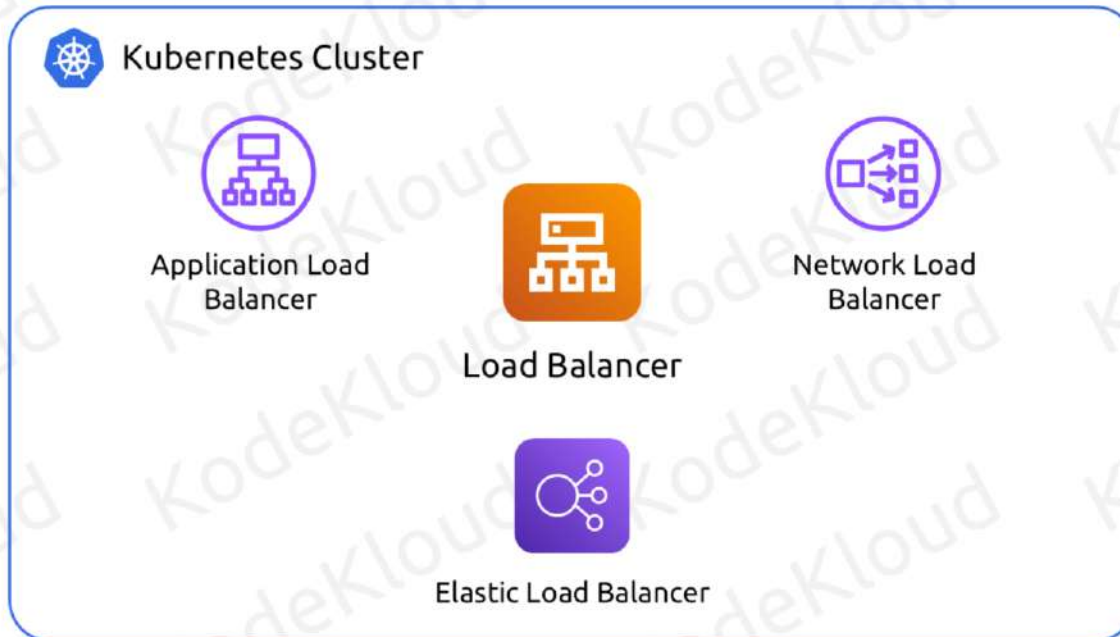
running because that'll save you this kubeproxy hop to go somewhere else, which sometimes will go across availability zones and cause more traffic, or sometimes it'll cause other issues as it passes through a node that it doesn't have to.

Kube Proxy in Kubernetes



So if we just disable this port on that service, then the load balancer will take it out of, out of the like rotation. It won't be balanced because it's not exposing the port. But that is all a load balance service does inside of kubernetes is it exposes a port and the load balancer will then health check that and route traffic to it.

More about Load Balancer



And so the load balancer controller inside of your cluster is going to run and it's going to look for services and it looks for special annotations on those services to know what type of load balancer that you're asking for. It will by default give you a load balancer. I don't know where it's default, I don't know which one's default. The load balancer controller will the load balancer controller in your cluster will look for services that are being requested and if they are requesting a load balancer, not a node, port or cluster IP, something that says hey, I need a load balancer for the service. It will look at the annotations on that service and create a load balancer however you requested it. So this might be an application load balancer, an ALB, it might be a network load balancer, an NLB, whatever it is that you requested, that's what it can create. Because that load

balancer controller is the thing that bridges your internal services to the external AWS resources. A lot of times it is very common to also run something like external DNS.

More about Load Balancer



Kubernetes Cluster

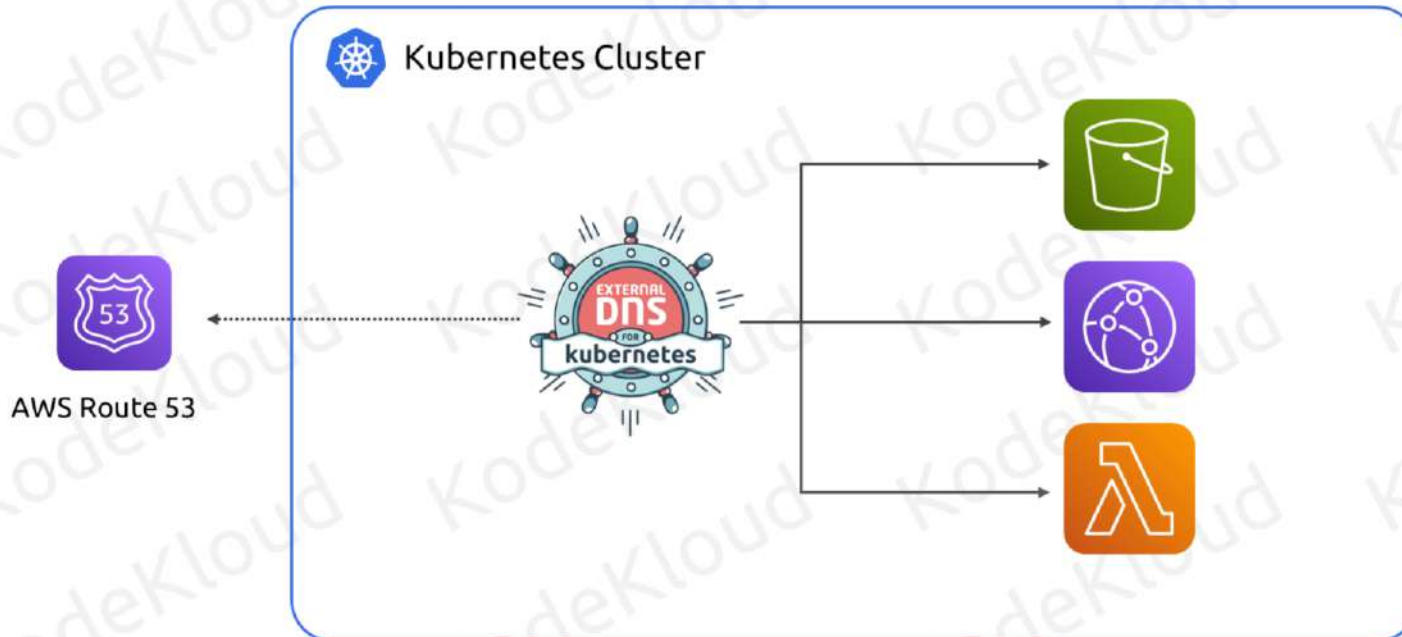


External DNS



Load Balancer

External DNS

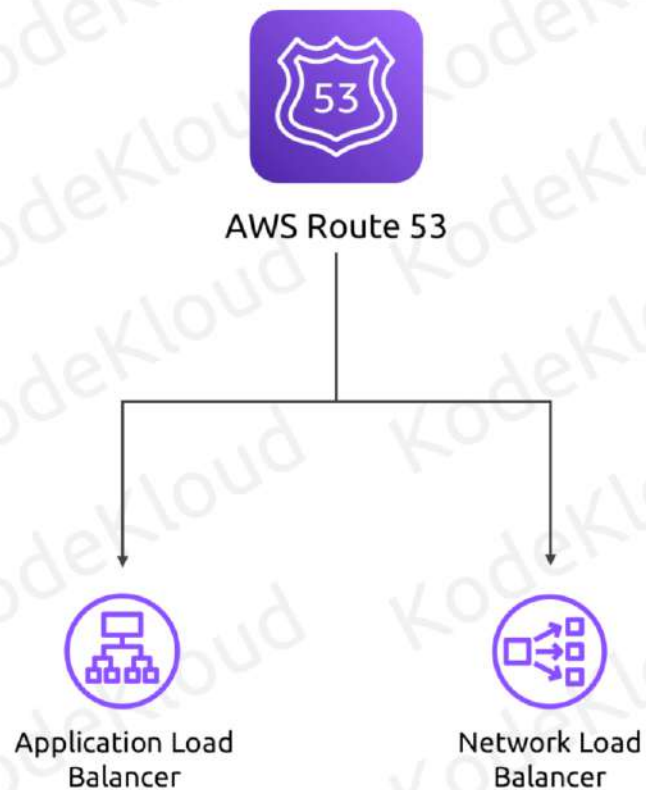


© Copyright KodeKloud

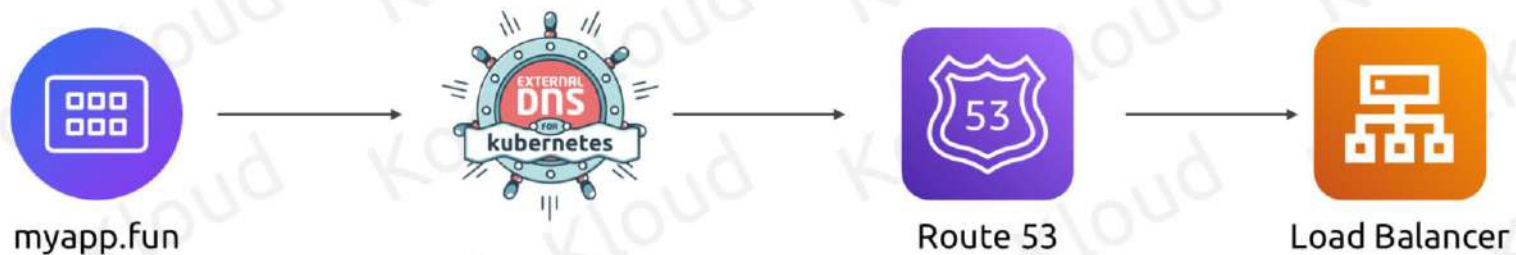
External DNS is an open source controller that will also look at services inside your cluster and then map those out to some external DNS provider in AWS. That would be route 53, which can also say which will take whatever the name of that service is and assign actual friendly domain to one of the ALBs or NLBs that are external from your cluster. So it can again look at a native Kubernetes resource, a service and then make things friendly for you. So if your internal service is called my app fun, and you assign that to a service, it can create an AOB. The external DNS controller can then assign that, can first create that entry inside a row 53 and then assign that to the load balancer so you don't have to go through and make your own terraform or try to automate this or go through the console and click those things. It just knows, oh, you want this service to

be exposed at this domain? Cool. We can make that happen for you.

External DNS



External DNS



External DNS



Terraform



Automate



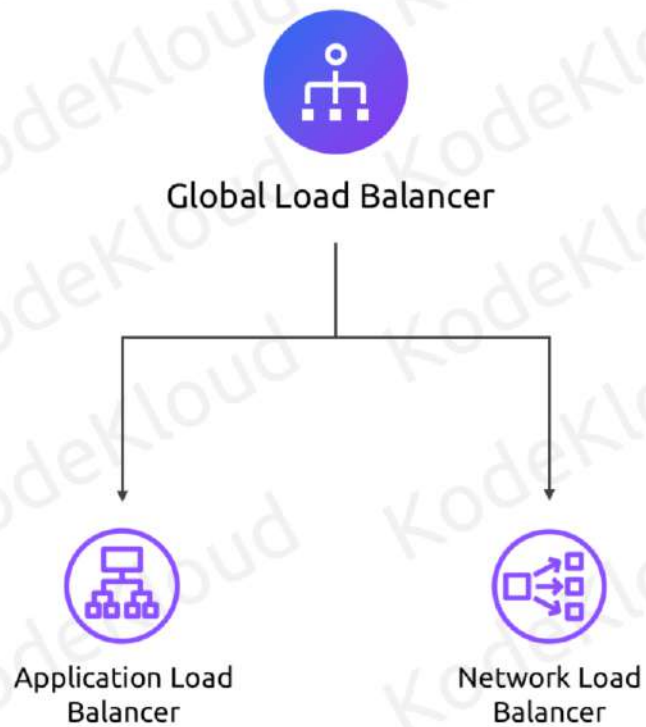
Click

Other Load Balancers



There are some other types of load balancing inside of AWS that the load balancer controller might or may not do. One is the global load balancer, and the global load balancer is a global resource. Not many resources in AWS are globally available, but this is one that is outside of a region. It exists globally inside of your account. And so when you say I want a global load balancer, you can assign, say, a route 53 domain to the global load balancer, and then that load balancer can route to different load balancers underneath it. So it could do EC two instances, but in the case of Kubernetes, you probably have load balancers that are staying around longer than a node might be. So you take this global load balancer and you point it to one of your load balancers that is pointing to one of your services.

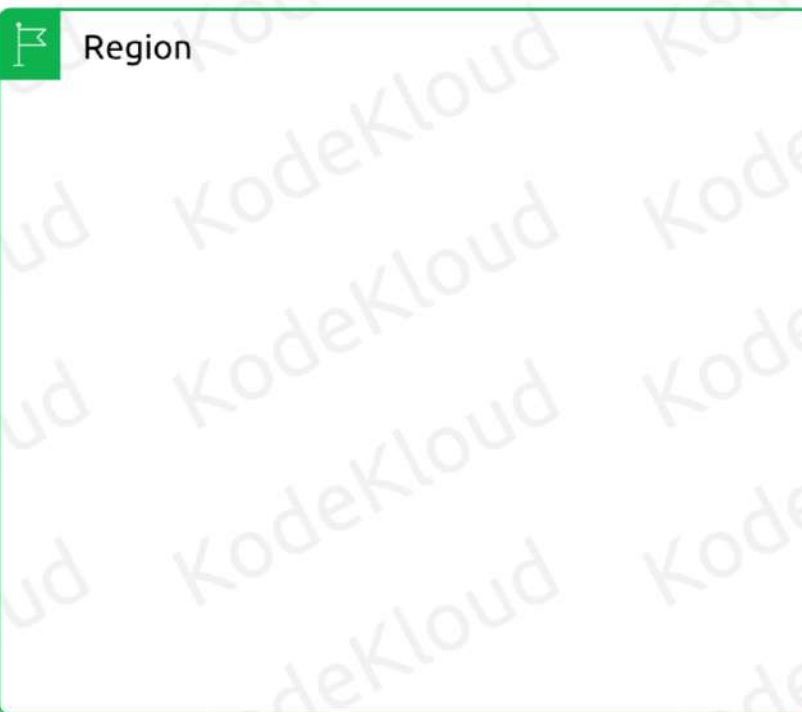
Global Load Balancers



Global Load Balancers



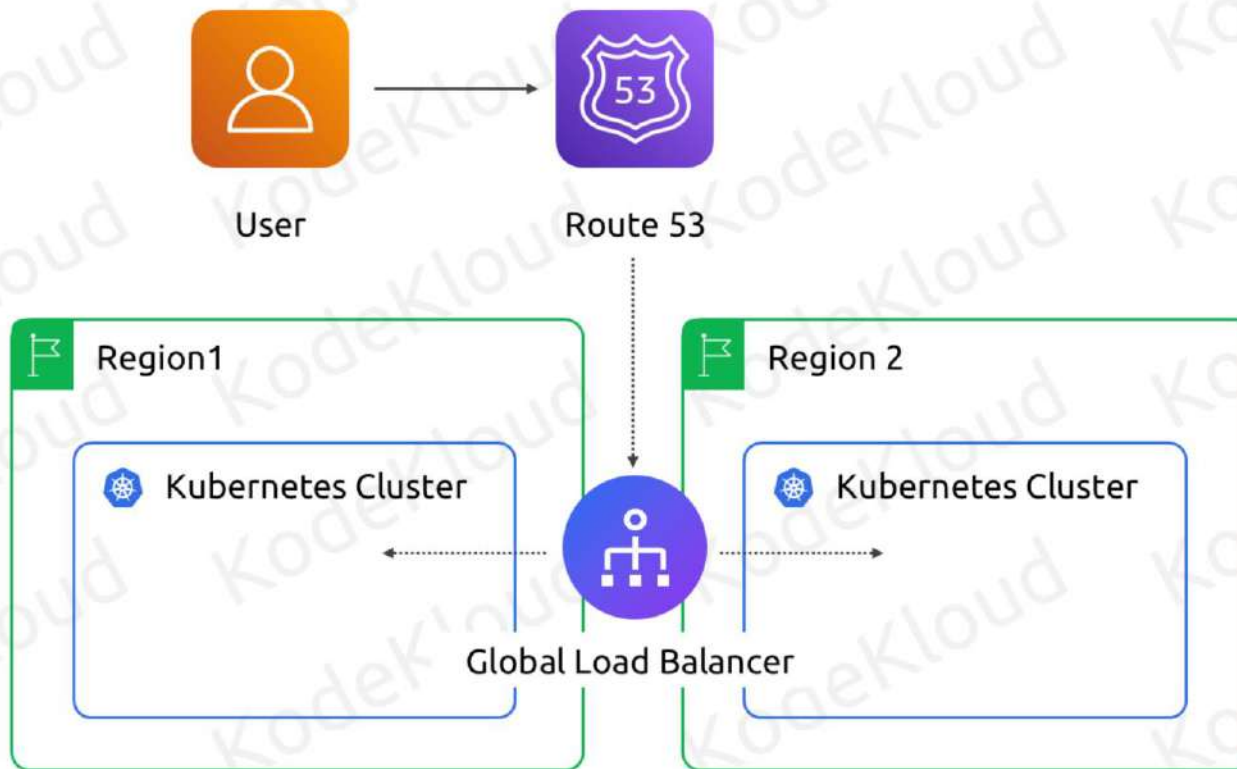
Global Load Balancer



And the nice thing about this is because the global load balancer exists outside of a region, you can actually point to load balancers in different regions. So if you have another cluster that is running in a different region completely, but is exposing the same services, you can have a global load balancer in front of both of them. And as users are requesting services from this route 53 domain on the global load balancer, they will be balanced between regions. And you can do that based on where their source ip is coming from for like geolocation. Or you can do it based on some weighted traffic split that you say, oh, I want 60% over here because we have more resources set there and 40% over here. Whatever it is, the global load balancer can do some of that. Right now, the global load balancer isn't controlled by the AWS load balancer controller

because it's not. Right now it's only managing services inside of a region.

Global Load Balancers



Global Load Balancers

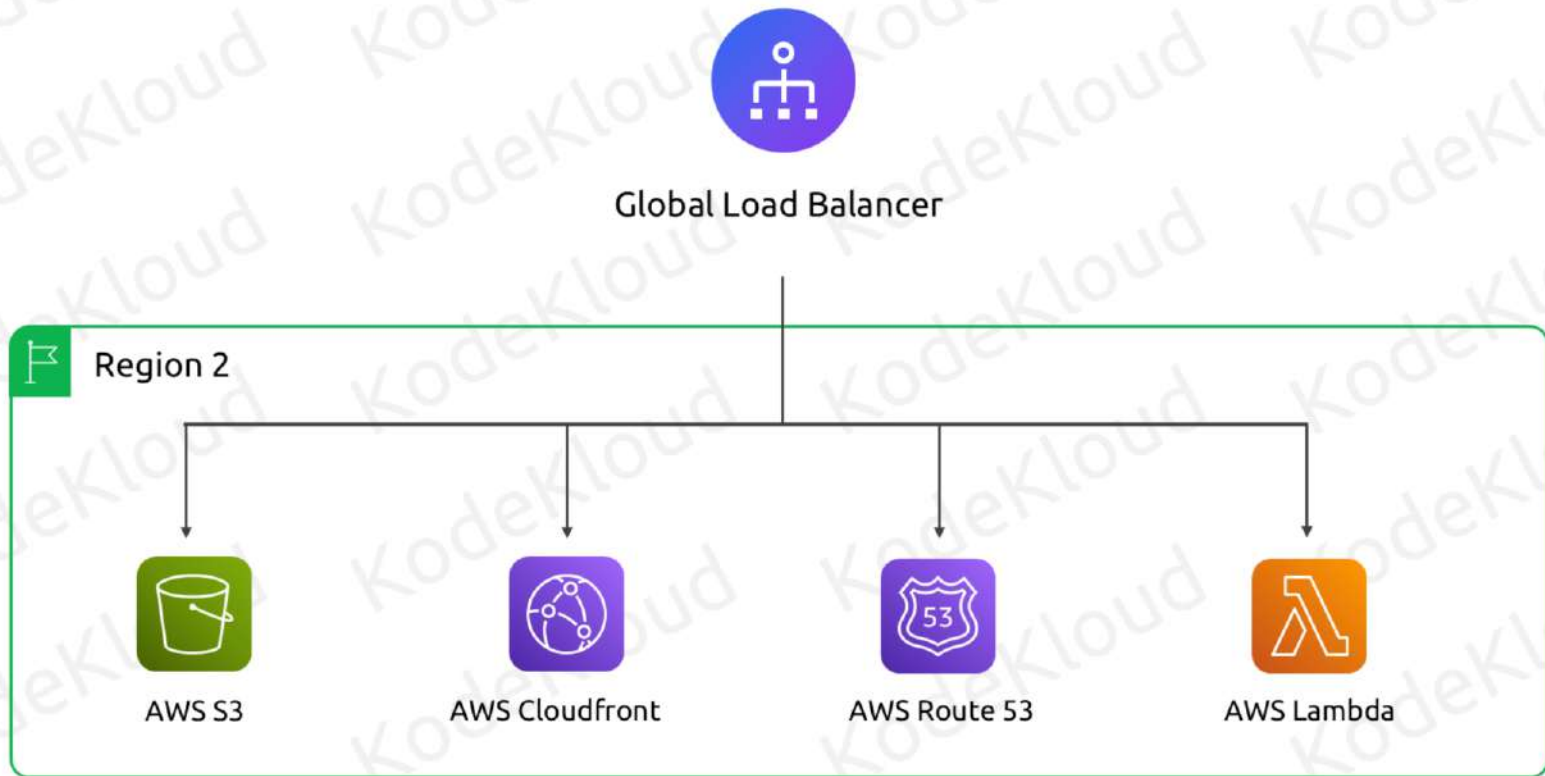


Global Load Balancer



AWS Load Balancer
Controller

Global Load Balancers



Summary

- 01 What a service is inside a Kubernetes Cluster
- 02 How services interacts with the infrastructure
- 03 How services interacts with Load Balancers in general
- 04 How the AWS Load Balancer Controller can create and manage Load Balancers that connect to services inside a Kubernetes Cluster

So just as a quick review, we were talking about what a service is inside of a Kubernetes cluster, how it interacts with the infrastructure around it and load balancers in general, and then how the AWS load balancer controller can assign how the AWS load balancer controller can create and manage load balancers that connect to your services inside of a Kubernetes cluster.

Gateway Ingress

Introduction



Load Balancer

Now that we understand a little bit about how a service with a load balancer works, let's talk about Ingress. Because Ingress and load balancers often get confused in how they're used. And first up, Ingress is a resource inside of Kubernetes that isn't graduating to stable. It is something that was there to give us a stopgap on how some of this should work, but it's not actually going to be moving forward into a stable Kubernetes API. The thing that has replaced Ingress is gateways, and gateways work similarly to Ingress, but are a lot more flexible and have some added benefits that we don't get with Ingress anymore. Back when we first were doing ingress controllers inside of not AWS, back when we first were doing Ingress controllers inside of Kubernetes, the thing we had was just Nginx. NginX was the main ingress controller that pretty much

everyone used. And what we did was we had this Nginx that we run inside of.

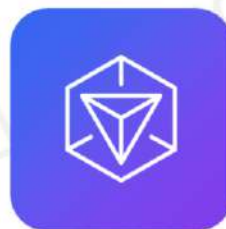
Ingress and Load Balancer



Load Balancer



Kubernetes Cluster

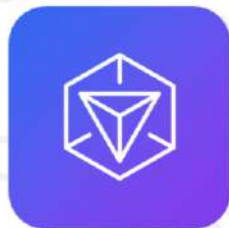


Ingress

Ingress Inside Kubernetes



Ingress Inside Kubernetes



Ingress



Gateway

Ingress Inside Kubernetes



Gateway

Now that we understand a little bit about how a service with a load balancer works, let's talk about Ingress. Because Ingress and load balancers often get confused in how they're used. And first up, Ingress is a resource inside of Kubernetes that isn't graduating to stable. It is something that was there to give us a stopgap on how some of this should work, but it's not actually going to be moving forward into a stable Kubernetes API. The thing that has replaced Ingress is gateways, and gateways work similarly to Ingress, but are a lot more flexible and have some added benefits that we don't get with Ingress anymore.



NGINX

Main ingress controller
that everyone used

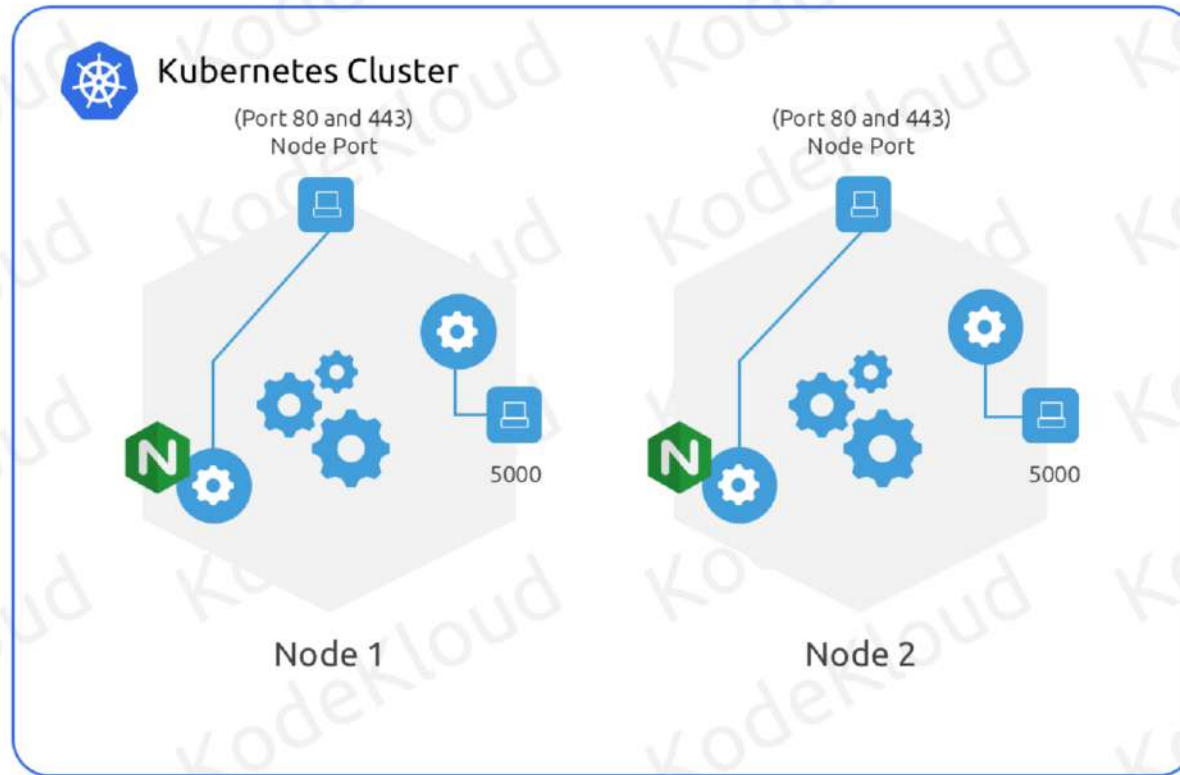


NGINX

© Copyright KodeKloud

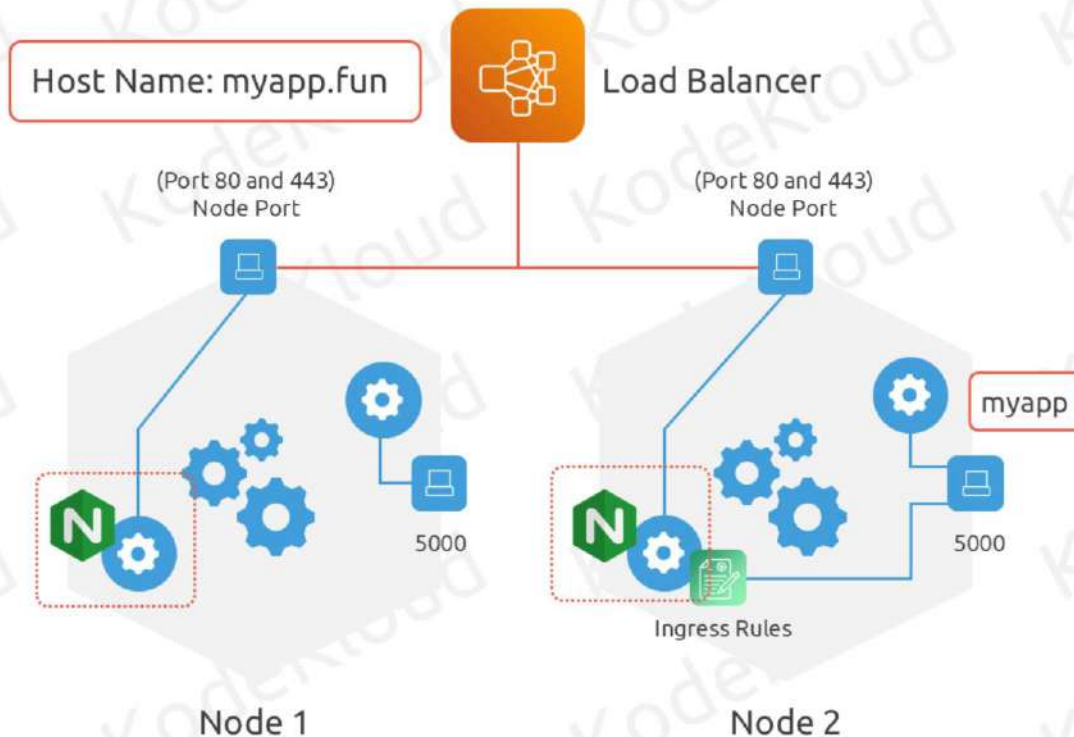
Back when we first were doing Ingress controllers inside of Kubernetes, the thing we had was just Nginx. NginX was the main ingress controller that pretty much everyone used. And what we did was we had this Nginx that we run inside of. Let me just diagram it. Let me show you real quick how an ingress controller worked inside of a cluster, and then we can talk about how gateway and how some of the implementations of ingress work. Come on, stop it. Okay, so let's say you have a Kubernetes cluster with two nodes, and we are going to run one pod on each one. And instead of installing or exposing those as a service that we want publicly routable, we are going to call it a cluster IP. And if you remember from the last section, every service is just a node port. And so each of these get the same node ports. And we'll say it's 5000, doesn't

matter.



Back when we first were doing Ingress controllers inside of Kubernetes, the thing we had was just Nginx. NginX was the main ingress controller that pretty much everyone used. And what we did was we had this Nginx that we run inside of. Let me just diagram it. Let me show you real quick how an ingress controller worked inside of a cluster, and then we can talk about how gateway and how some of the implementations of ingress work. Come on, stop it. Okay, so let's say you have a Kubernetes cluster with two nodes, and we are going to run one pod on each one. And instead of installing or exposing those as a service that we want publicly routable, we are going to call it a cluster IP. And if you remember from the last section, every service is just a node port. And so each of these get the same node ports. And we'll say it's 5000, doesn't

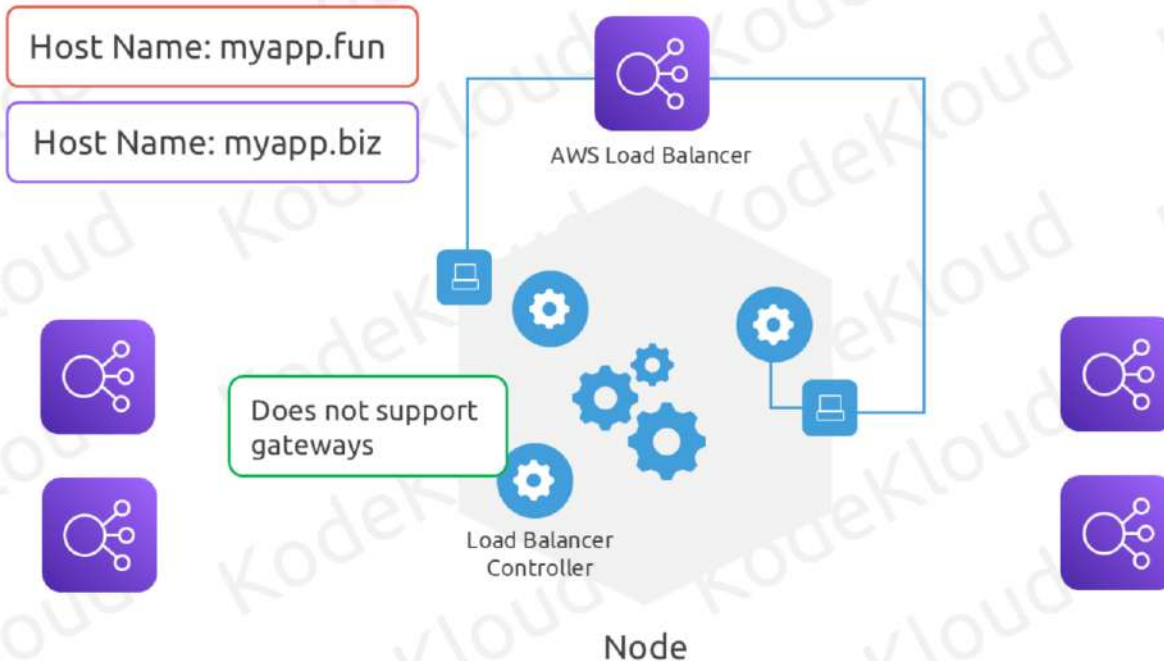
matter. The port that automatically gets assigned and that port gets routed to the pod itself. But in this case we're going to run an the ingress controller, typically Nginx, if you're doing it and that's going to be just another pod on the node and that's just it's running. We say we want to deploy this, especially if you're on prem or if you're managing load balancers yourself. This typically would run as a daemon set if you're not auto scaling, but it could run as a larger deployment. And that also is going to expose some ports with node port. And you could, if you were again managing all this yourself, you could say, I want this to expose port 80 and 443 and it would take port 80 and port 443 from every node in your cluster and that gets routed into your Nginx or your Nginx or your ingress controller, however you want to do it.



And then out front you put a load balancer, whatever it is, and you say, we're going to route that to every port. 80 and 443 go to that Nginx instance on the nodes. So anytime you request something from a website externally you request it from the load balancer. That then goes internally. The load balancer here should not be doing anything special. The load balancer is just sending traffic from the ports to the port and you could remap port whatever. But the real magic happens when I say I want a specific host name if I want my app fun. The load balancer doesn't have to know about that. What actually knows about that is these ingress rules. And we can create an ingress rule that then says, oh, my app fun is going to route to the service internally called my app. And so what it would do is it actually does a reverse proxy from NginX to port 5000. And in

both cases those Nginx reverse proxies just get wired up to say, okay, go to port 5000. The place that we actually care about the hostname is here on the ingress controller, and that is an L7 routing, so that we know that it's only HTTP traffic, it's HTTP coming in. We look at the host name and we reverse proxy it. That is how a reverse proxy has worked for forever. If you haproxy Apache, doesn't matter. All of them basically look work the same way. They look at what hostnames coming in, which one's being requested, and then send it to the port that the application is actually running on the app. The port transition is completely transparent to the user. You don't have to worry about that section. So the interesting thing here is this is how it has been run for a very long time with things like Nginx.

Managing Ingress With AWS Load Balancer



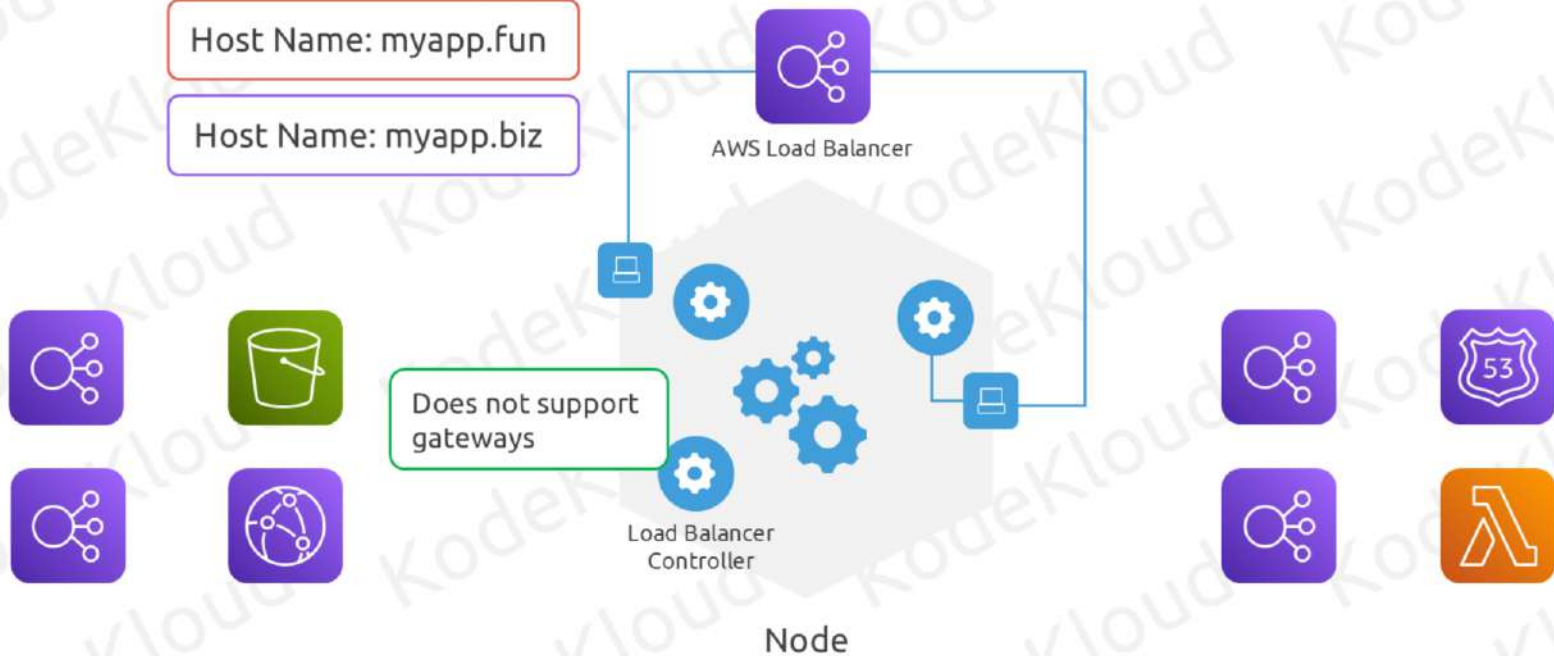
But another thing we can actually do is the AWS load balancer controller actually can manage ingress for us too. So let's get rid of all of this stuff. I'll just delete it because I can't get it to erase just part of it. So we're back with our node and we have this pod again. It's exposing, is exposing a port and we can make a cluster IP. It doesn't, it's not a load balanced port. We just say, hey, we want a cluster IP for this pod or this deployment and it's going to be a service if we are running the AWS load balancer controller. It also can look at ingress type resources and it knows what to do. So if an ingress is created inside the cluster, it knows that that's I seven routing and it can create a lb for you inside of AWS, and that LB will automatically connect to the port that's being exposed. So you're skipping a step of managing that within the cluster because the cluster

ingress controller could be routing that. And you can say, hey, I want that to go externally to an ALB controller. And one of the downsides that sometimes happen here is when you have a lot of services, you end up with a lot of load balancers, and all of those load balancers cost money and they take time to manage and do whatever it is you want to do with them. If you have hundreds of services, actually having one ingress controller and routing all of the hostnames was a little easier or cheaper or whatever. And so you can actually take multiple ingress controller or ingress resources inside the cluster and map them to the same Alb. So you can actually tell the load balancer controller hey, I need an ingress, but I also want it to use that same LB that I was using before. You don't need to provision another one, you don't need to do anything. All it's going to do is, is create new host rules. So that's my app fun and my app biz will both get to the correct service inside of your cluster using the same host routes from externally. One thing to note with that AWS load balancer controller is it currently does not support gateway rules. So if you're creating gateways and ingress classes inside of your cluster to try to route traffic, the current AWS load balancer, at least while recording this course, doesn't support gateway. So it would be something that you would have to be using the older beta APIs to get an ingress routes. But I'm sure that would be coming in the future as gateway is being used more and more throughout kubernetes.

Managing Ingress With AWS Load Balancer

Host Name: myapp.fun

Host Name: myapp.biz



Ingress Controllers



NGINX

© Copyright KodeKloud

And we've already talked about the NginX Ingress controller, which was very common and it had a lot of configuration options. It was one of the most configurable annotated software I ever try to use inside of kubernetes. And pretty much anything you could do inside of an NginX config you could do with annotations on an ingress rule with the NginX controller. There are other ingress controllers that are worth talking about conveyor is one. What's the other one giveaway. I don't want to mention other ones because it's just going to be picking favorites. There are plenty of other ingress controllers that you can now run inside of your cluster if you want to have that as part of the cluster and a single load balancer pointing to the ingress controller. So go look at your other options, because some of them might be based on Nginx, some of them are

based on envoy, on the reverse proxy. Various things will also route this traffic internally. So there's plenty of new ingress controllers that do support gateway API and are worth looking at if that's something that you want to or need to support.

Service Meshes



Istio

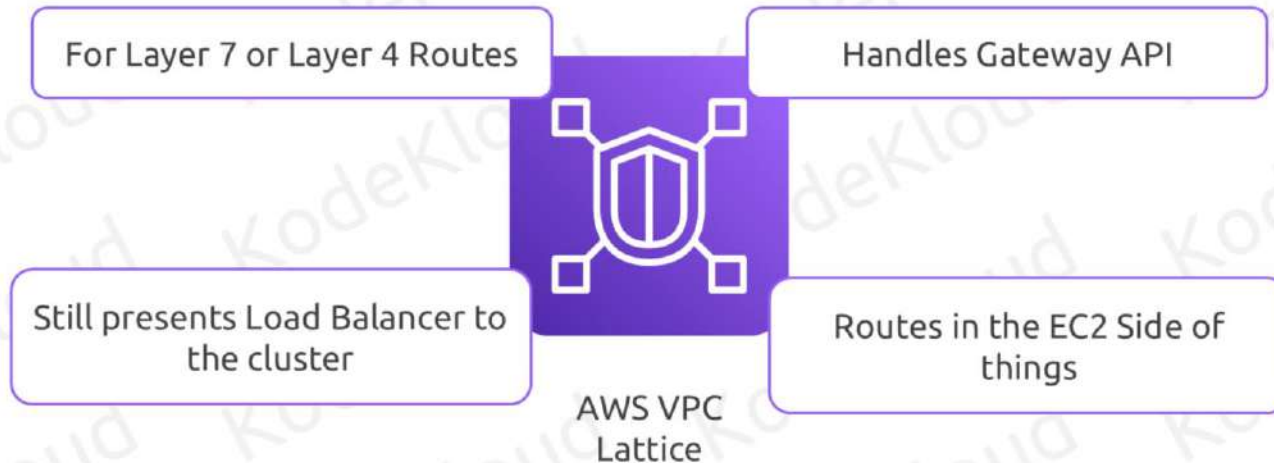


Linkerd

There are also some service meshes. Things like istio or Linkerd can handle north south traffic, things going in and out of your cluster. And if you're trying to route traffic internally, some of them might support ingress rules, some of them might only support services. Depending on what you're using, you have to look up how that's supported. So we're not going to go into all the depth on service mesh versus an ingress controller versus a gateway. They're all different ways of getting traffic in and outs, and if you're already using one of them, then you might be able to use it for that traffic as well. If you don't maybe want to use all of the AWS load balancer controller resources, you can use it. You can offset some of that other traffic routing and handling with some of these other things. I believe some CNIs also even support this natively to be able

to get traffic in and out.

AWS Lattice



There is one final way to get traffic in and out that is also supported inside of AWS, and that is with lattice. Lattice is a new AWS service which is kind of an invisible layer of network routing for your services. This is specifically for layer seven and maybe some layer four routing of traffic, but it does handle the gateway API. So this is the newer way of saying, hey, I want to use the gateway resources, but I maybe I can't do that with the AWS load balancer controller. In that case, you may want to look at lattice as a way to get that traffic in and out. So it's still going to present a load balancer external to the cluster, but then how it actually routes that traffic internally might be a little different because this is all kind of happening on the AWS or the EC two side of things where we it actually can route that traffic without you needing to set up all of the rules,

and lattice is supposed to be a little more intelligent about doing that, either across accounts or different cases where you actually need to route. Maybe traffic from within a Kubernetes cluster out to a lambda function or an EC2 instance so it can more seamlessly transition that traffic within your AWS accounts. But it's not as full featured as something like NginX ingress controller or maybe as completely flexible as the AWS load balancer controller. So it's one other thing you'll want to look at, but not necessarily something you have to go with. So in this section we just talked really quickly about the gateway API and ingress controllers and how they work and some options that you might want to look at as you're going through and trying to figure out how to get traffic in and out of your cluster. If you don't want to use a load balancer for every service and every resource needing its own dedicated load balancer, Ingress is a great option for that.

AWS Lattice

More full featured



NGINX

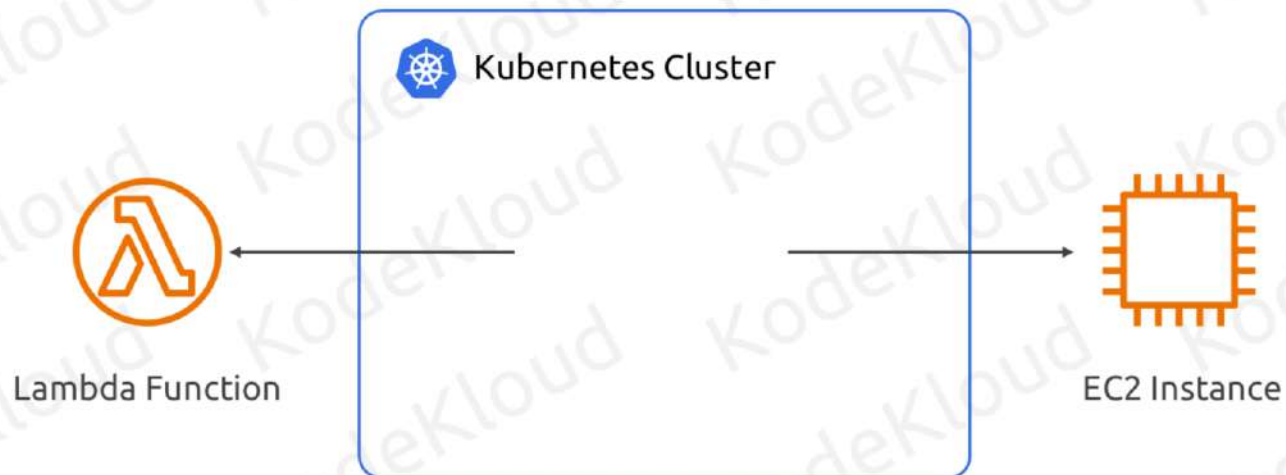


AWS Lattice



More flexible

AWS LB Controller



Summary



Gateway API and Ingress Controllers and how they work



Other options to get traffic in and out of the cluster



For those who don't want to use load balancer, Ingress is a great option

So in this section we just talked really quickly about the gateway API and ingress controllers and how they work and some options that you might want to look at as you're going through and trying to figure out how to get traffic in and out of your cluster. If you don't want to use a load balancer for every service and every resource needing its own dedicated load balancer, Ingress is a great option for that.

VPC – Lattice

Kubernetes Gateway API – Introduction

 Kubernetes



AWS
Load Balancer



Nginx
Ingress Controller

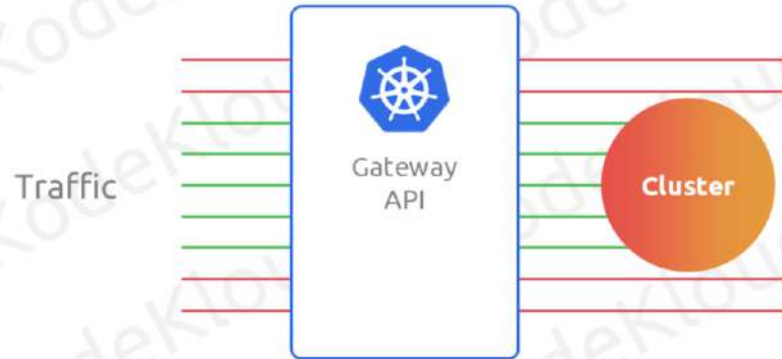
Alright, so we talked about load balancing and ingress controllers inside of kubernetes.

Kubernetes Gateway API – Introduction



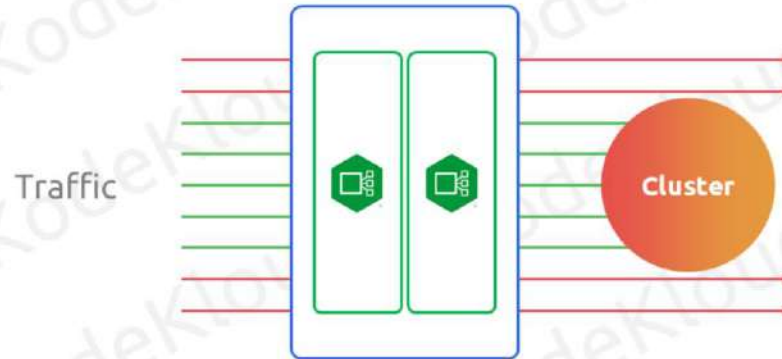
Let's talk about the other way we get traffic in and out of a cluster, and that's with the gateway API. The Kubernetes Gateway API is kind of the graduation or the v two of ingress controllers. Ingress controllers were all about getting that layer seven traffic into your clusters and maybe routing them based on paths or hostnames. And gateway controller takes that to another level with more flexibility and more supported protocols. So the Gateway API is the new way to do that sort of layer seven or high-level application-level traffic getting into your cluster.

Kubernetes Gateway API – Introduction



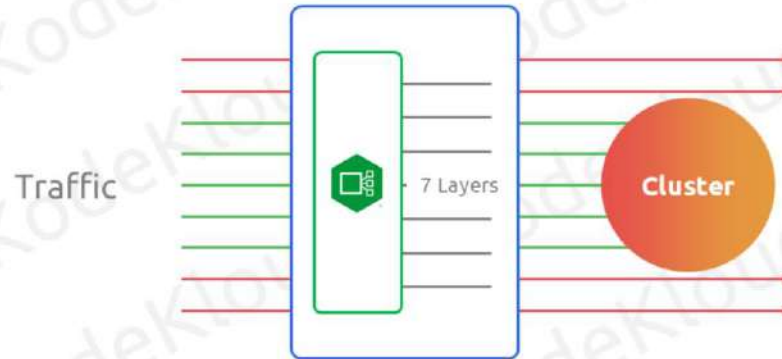
The Kubernetes Gateway API is kind of the graduation

Kubernetes Gateway API – Introduction



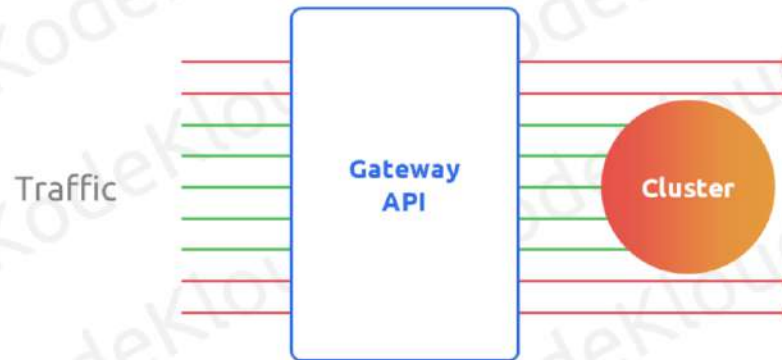
or the v two of ingress controllers.

Kubernetes Gateway API – Introduction



Ingress controllers were all about getting that layer seven traffic into. Ingress controllers were all about getting that ingress traffic into your clusters and maybe routing them based on paths or hostnames.

Kubernetes Gateway API – Introduction



And gateway controller takes that to another level with more flexibility and more supported protocols. So the Gateway API is the new way to do that sort of layer seven or high-level application-level traffic getting into your cluster.

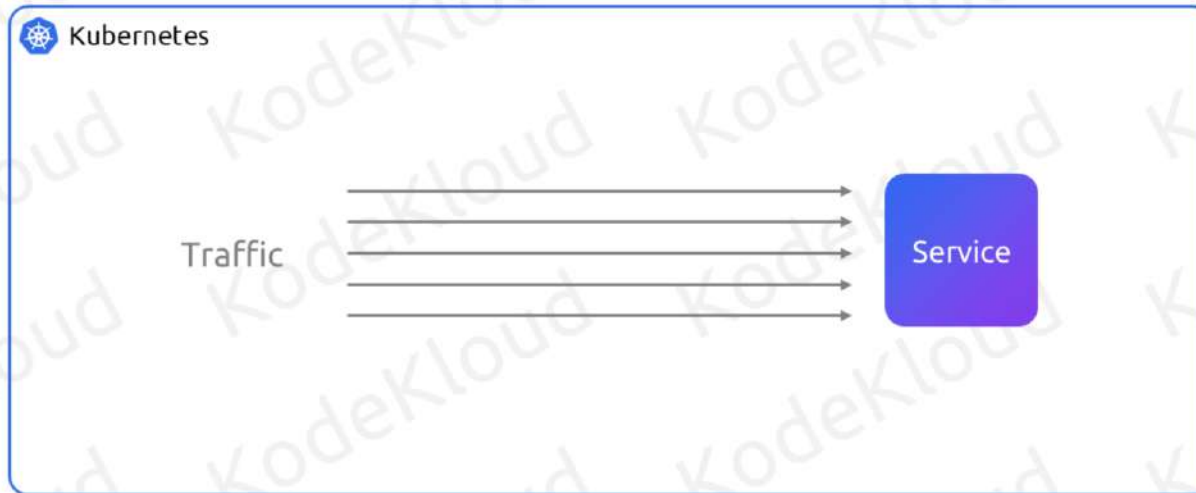
AWS EKS – Role of Lattice Controller



And inside of AWS, in eks there's what's called the lattice controller, which manages gateway resources.

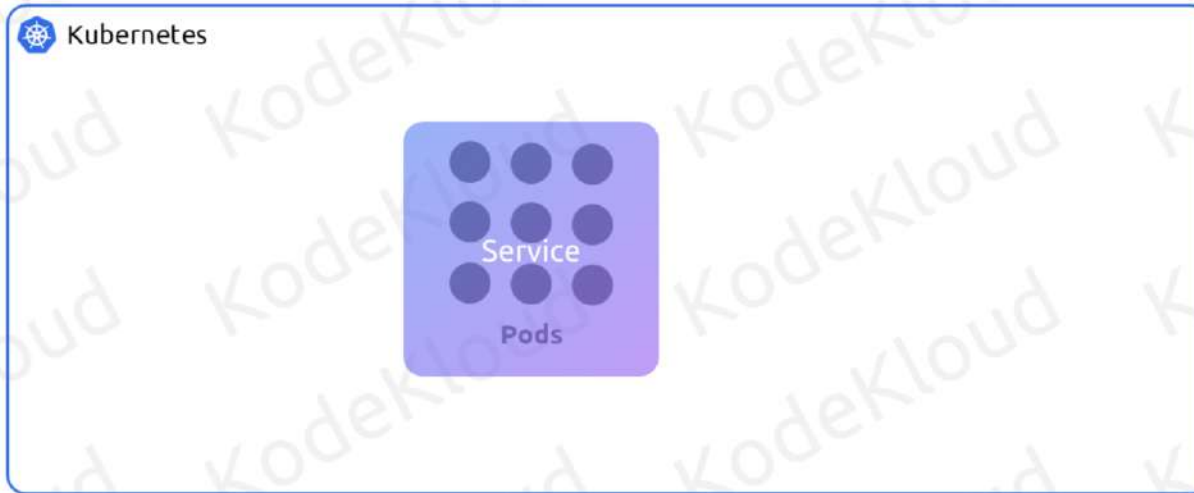
So let's talk about both real quick and just make sure you understand how they work.

Traditional Approach With Ingress Controllers



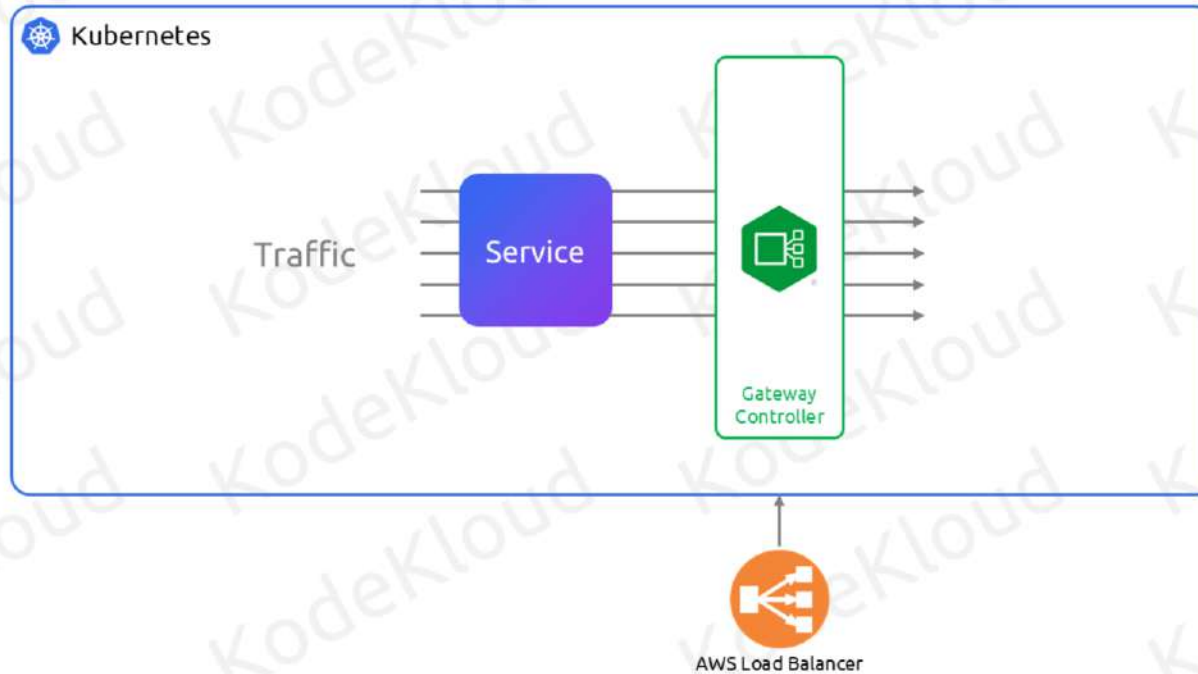
So, let's just start with our high-level view of a Kubernetes cluster here. And let's say we have a service here inside the cluster and we want to get some traffic to that service.

Traditional Approach With Ingress Controllers



And again, behind the service is pods or something like that. But we don't really care about those right now.

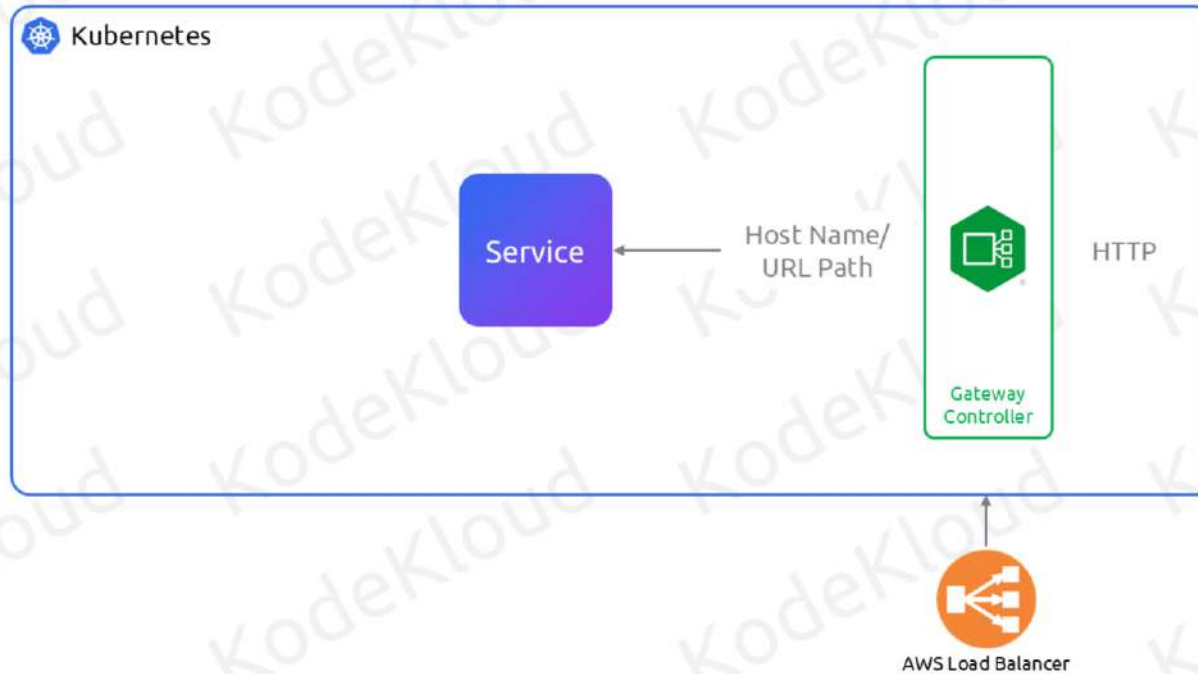
Traditional Approach With Ingress Controllers



© Copyright KodeKloud

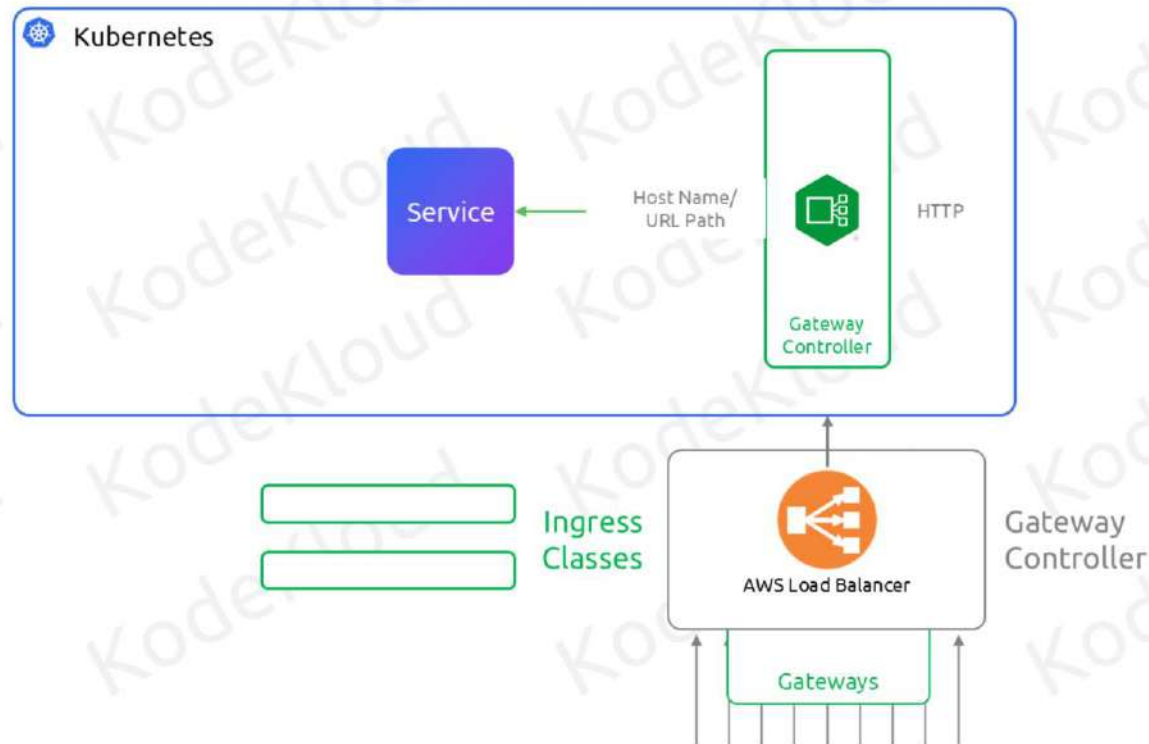
We only care about this service endpoint. How do we get traffic to that point? And so traditionally with Ingress we're going to do something like have an ingress controller inside the cluster and this is going to be behind a external load balancer.

Transition to Gateway API



And that ingress controller inside the cluster is going to look at HTTP requests as they're coming in, and then routes based on a host name or a path URL path or something and say, oh, that one goes to this service, great, we're gonna send that traffic with gateway, not a whole lot changes there.

Gateway Classes and Objects – Overview

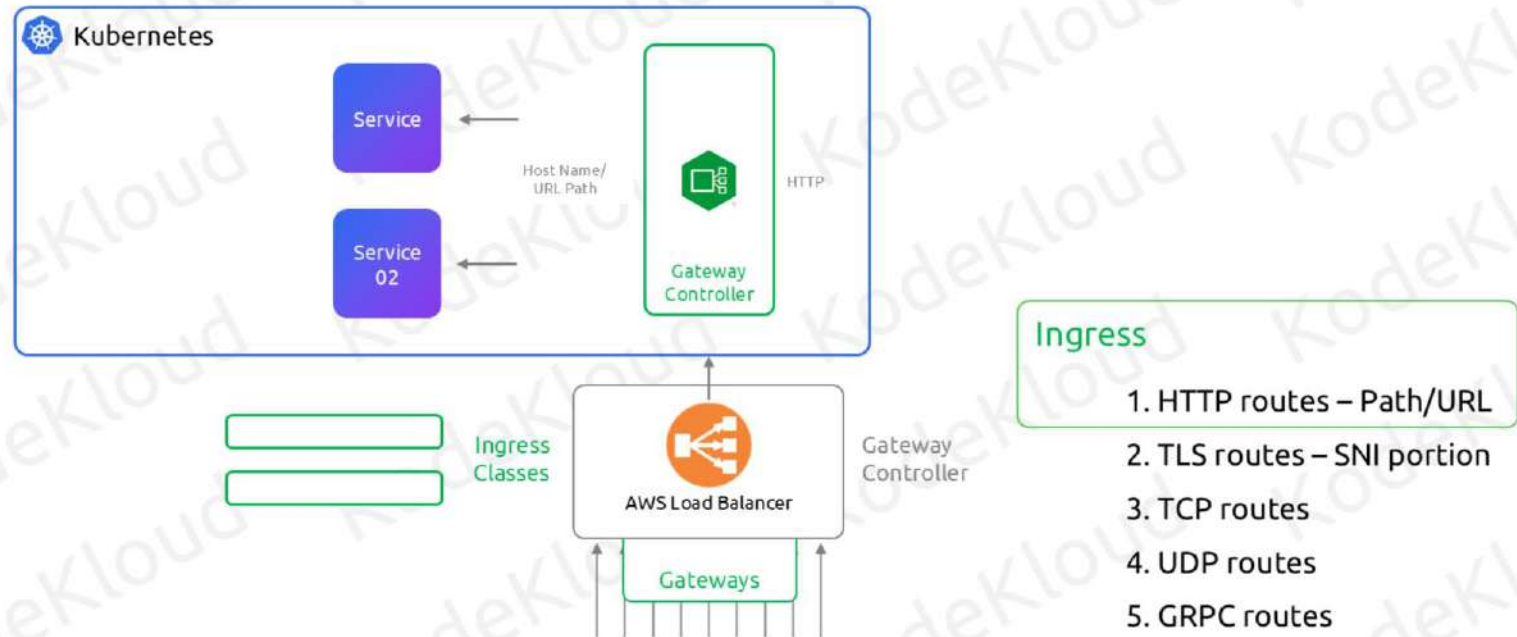


© Copyright KodeKloud

This still has an external load balancer here. This still has some sort of controller that looks at these gateway routes, what they call, yeah, it's gateway. It still has a gateway controller here that runs inside the cluster and that's going to point to different routes. And so, in this case, just like with ingress, we had ingress classes, we have gateway classes and that is just a grouping of how we want to manage gateways. And usually that's separated by like the gateway controller. So, if you have two of them running and you want one on one gateway controller and a different one controlling another one, you can split them up with classes. And that's a fairly common way of doing it.

But in general, you might just want to have a default. And the default is good enough for most of your services. Below the gateway classes will actually have a gateway. And this is going to define some information about how that traffic is going to come in. It's, wait, gateways. Okay, this, so this gateway object is going to translate some external source into a service inside of our cluster. And that just says, hey, we're going to point this gateway to that service that we want to route traffic to below those below the gateway objects and the gateway class objects. You're going to have a variety of different ways you actually route that traffic.

Routing Options With Gateway API



© Copyright KodeKloud

Things like an HTTP routes is going to look at things like a slash path on the URL and say, oh, if it goes to service one, it's going over to this service, if it goes to service two, it goes to another service that's like an HTTP, something based on the URL. We can also have something like a TL's routes and that's going to look at the host's name requested based on a TL's request. Because if you know about service name indexing, service name indexing indication, server name indication, there, the TL's route is going to look at SNI or service server name indicator, which is just saying, hey, I want to route to this traffic behind this TL's cert. And the gateway controller is going to handle a lot of different TL's certs. They're all going to be managed and probably terminated at the gateway controller. And the request is going to look at just that first SNI portion

of it to say, oh, that's this hostname. And that hostname maps to this service behind me inside of kubernetes, you can also get things like a TCP routes, a UDP routes, and various other ways to just say, hey, this is how that traffic is going to be determined, where it routes inside the cluster. All of these things work similarly to how Ingress worked, but Ingress had fewer options.

Ingress in general would only look at HTTP hostname or path and some, a few other things, but this separates out those indicators into very distinct ways of routing that traffic. So we could also have a service that accepts something like TCP traffic, and also GRPC is another route type. And so we could have both of those being separate if we wanted them to be, and route to services depending on what that traffic came in as.

AWS Lattice for Network Meshing



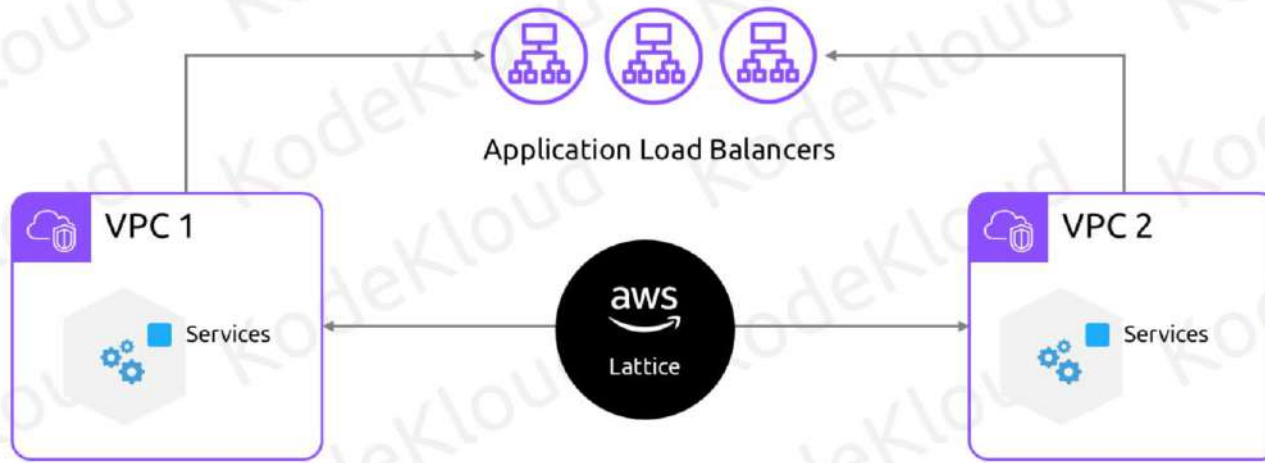
So how does all that relate to how Aws does it with lattice?

AWS Lattice for Network Meshing



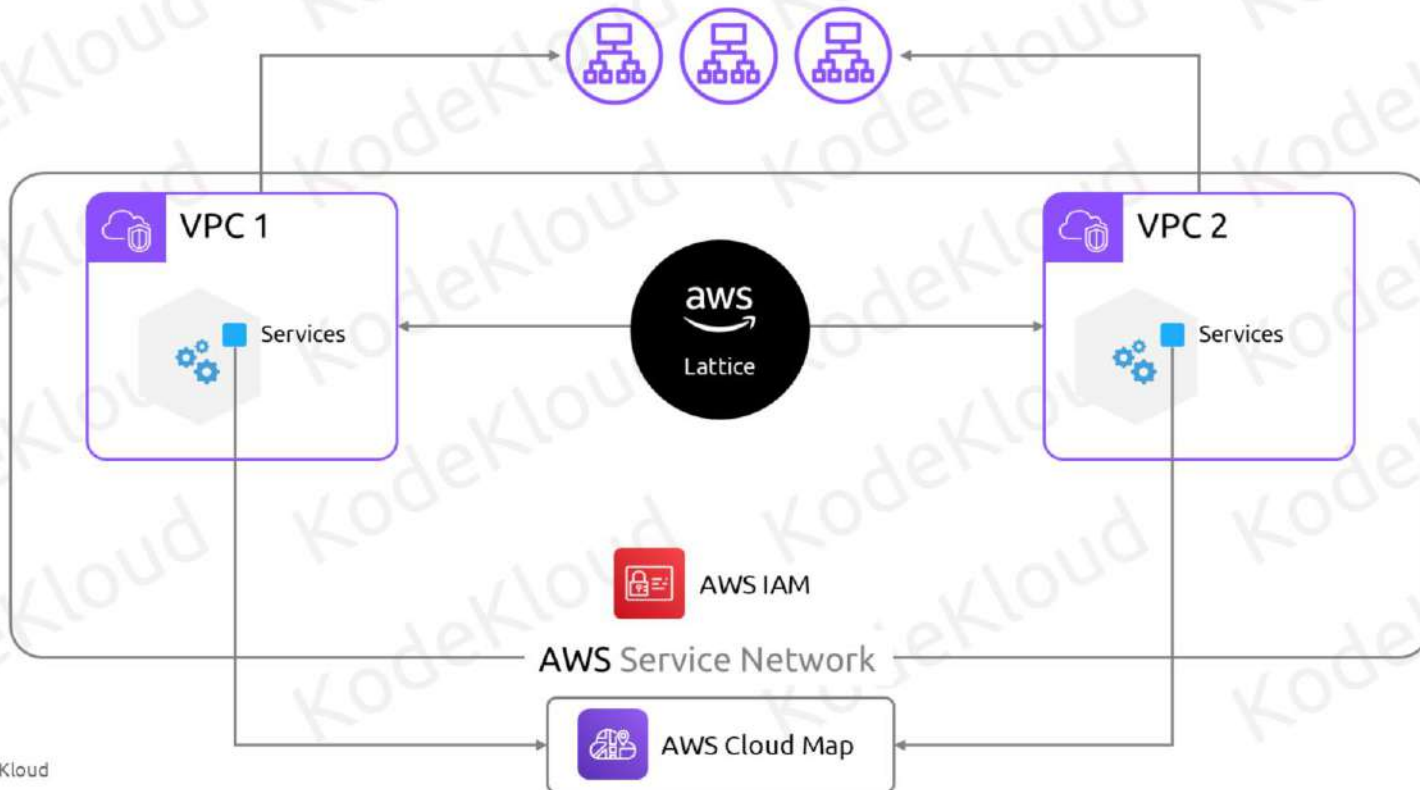
So AWS, lattice is a network meshing or network routing service that is offered that sits kind of below your VPC or at least it's transparent to your VPC, because the goal here is to allow a seamless integration between applications, and it doesn't matter where those applications exist. So, if I have a VPC one here and a VPC two over here, and I have two services that need to talk to each other, lattice will be able to route both of those things together. Doesn't matter how those vpcs are actually connected, because traditionally with a VPC, you're going to have to go up and have like some way that they actually can route traffic between the two. And lattice is an abstraction on top of the vpcs or under them, depending on how you want to describe it.

AWS Lattice for Network Meshing



So AWS, lattice is a network meshing or network routing service that is offered that sits kind of below your VPC or at least it's transparent to your VPC, because the goal here is to allow a seamless integration between applications, and it doesn't matter where those applications exist. So, if I have a VPC one here and a VPC two over here, and I have two services that need to talk to each other, lattice will be able to route both of those things together. Doesn't matter how those vpcs are actually connected, because traditionally with a VPC, you're going to have to go up and have like some way that they actually can route traffic between the two. And lattice is an abstraction on top of the vpcs or under them, depending on how you want to describe it.

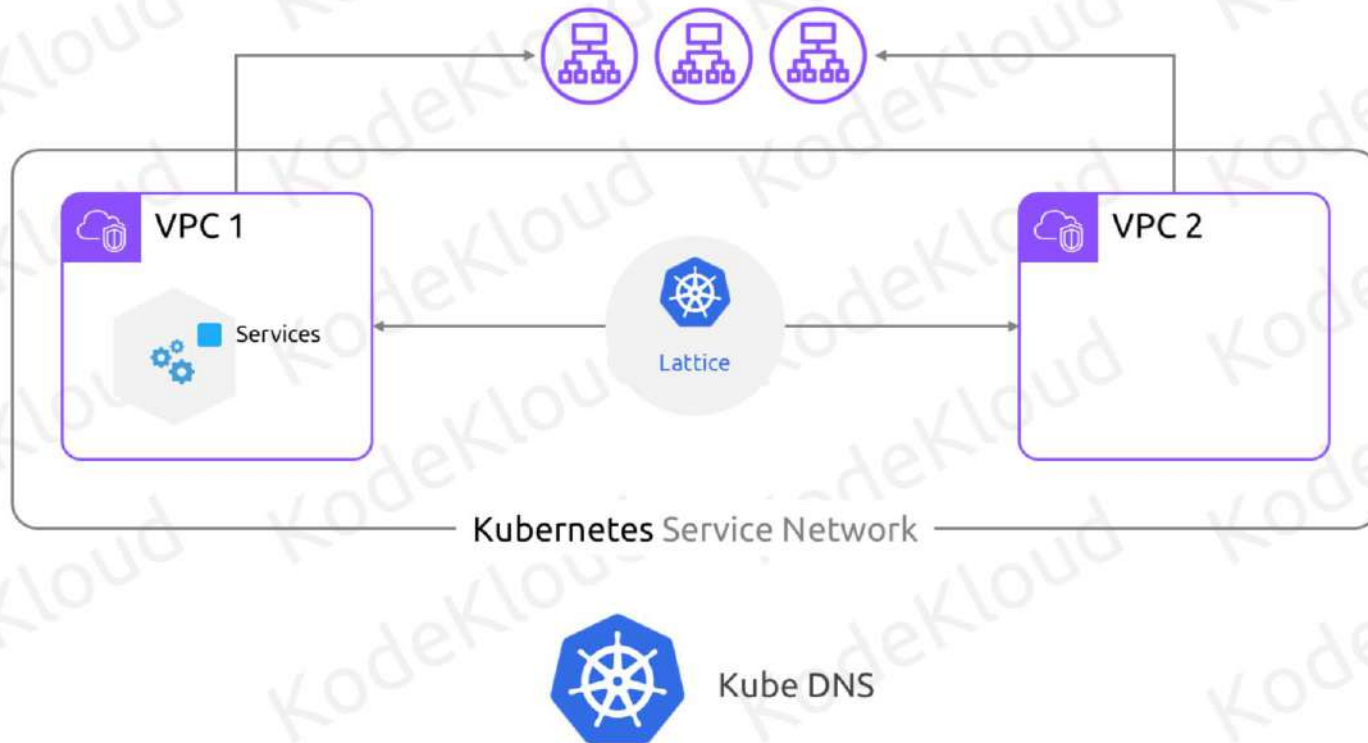
Service Networks in AWS Lattice



© Copyright KodeKloud

And the way it does this is with an abstraction called a service network. Not to be confused with a Kubernetes service. This is an AWS service at some level. And so, you're going to have this service network, which is that connectivity of not necessarily vpcs, but the services inside of this service network. And those two endpoints can talk to each other as long as IAM allows them to. It also will do a form of service discovery within the service network. And so, it uses AWS Cloudmap to register services based on some hostname that is routable inside the service network. And if you're again using kubernetes, you're going to be used to this inside of kubernetes.

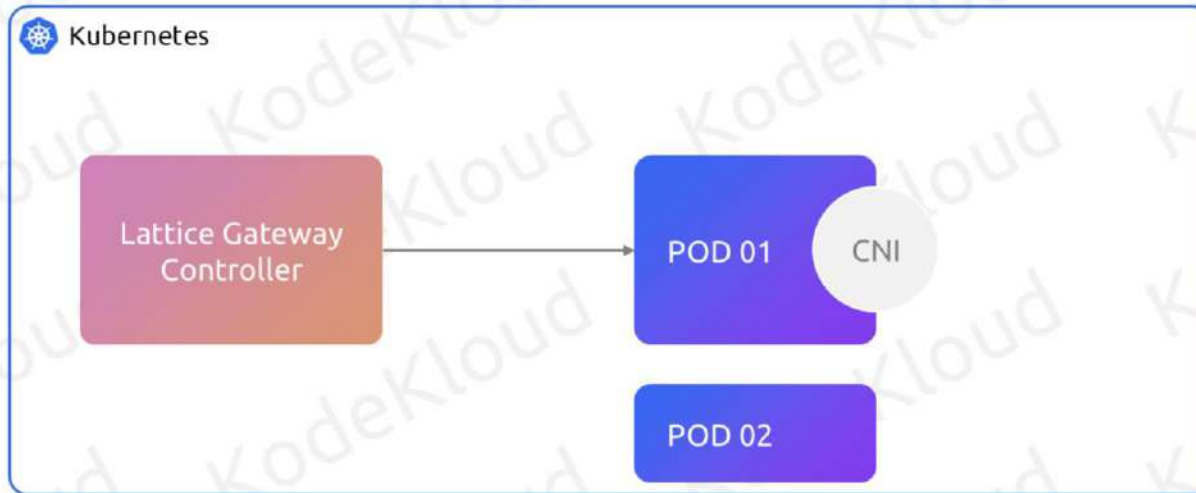
Service Networks in AWS Lattice



© Copyright KodeKloud

We do a similar thing with Kube DNS. Kube DNS would be similar to what cloud maps doing, where we have a hostname map that allows us to basically point to an IP address. And our CNI, our container network interface, or our network that we have inside the cluster is a way that we can flatten out a complex network. And so one of the rules of Kubernetes is that all of your pods should be able to directly route to each other based on this CNI. And that's usually why we run a CNI, is just so that we get a flat IP space for all of the pods. Lattice is doing something similar at an AWS layer. And so when we are running services inside of a Kubernetes cluster, if they are part of a service network and part of lattice, then you're going to have multiple layers of how they actually get routed. I'm just going to redraw some of this and try to show what it looks like.

Service Networks in AWS Lattice

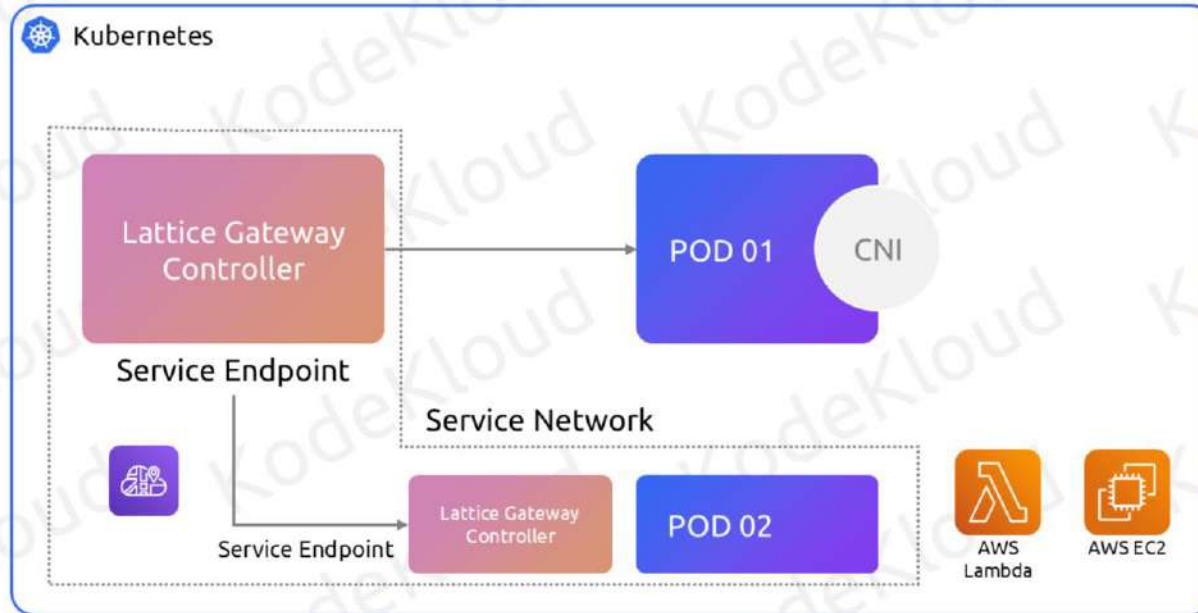


© Copyright KodeKloud

So let's say we have a pod, and this pod is running inside of Kubernetes, and that pod is going to have a CNI network attached to it. So it's gonna have some IP address that's part of the cluster, and that cluster allows it to route to other pods in the cluster. But let's also say that this pod is part of a service network, or gets routes to a service network as part of an AWS service network for lattice. The way that works is we actually have the lattice gateway controller that exists outside of that pod. It's within the cluster, so it can route things, but this gateway is going to be the portion that's part of lattice itself. So the gateway is going to point to the pod into this CNI network. But if you are routing from this pod to a pod that exists within the service network, but again, it can be a different VPC, it can be a different AWS accounts, it could be a lot of

different things, and the service network should transparently route those things together.

AWS VPC Lattice



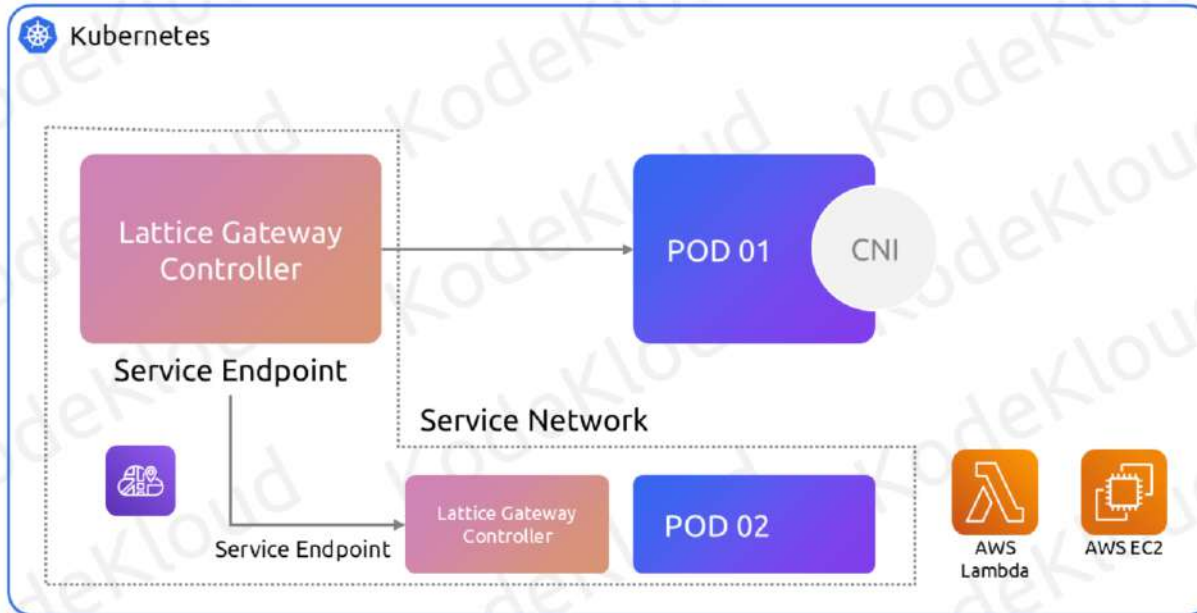
© Copyright KodeKloud

So, in that case, the gateway is going to create a service endpoint in our servicenet service network is going to encapsulate all of this over here, and we're going to have another gateway in front of this pod which routes the traffic in there.

And so this is another service endpoint inside of the service network. So when we are trying to, when this pod says I need to talk to pod two, or service to, because I guess this would be a service over here, that traffic is going to be routed automatically onto the service network through that service endpoint and into the other Kubernetes cluster. And the benefit here, or at least the supposed benefit, is this works with not just gateway services inside of a Kubernetes cluster,

but this also can work with Lambda, this can work with EC two, this can work with anything that's part of the VPC that has some idea of how to create an HTTP endpoints or a service endpoint in this service network. That's the goal here. That's really what lattice is trying to do. And again, these service endpoints gets registered in Cloudmap. It actually gets really confusing. This is an advanced feature to use lattice in this way.

AWS VPC Lattice



- Avoid Lattice for small-scale setups
- Better for complex enterprise environments
- Challenges arise with scaling

I would not recommend doing lattice. If you have one or two Kubernetes clusters, there are much easier ways to route traffic between two services and two pods and two networks. This is really for advanced sort of enterprise customers and in environments where they have a lot of different accounts and a lot of different vpcs, because once you start scaling up to a lot of vpcs, and I mean like dozens or hundreds of vpcs and clusters, all of those network connectivity just ends up being a very difficult problem.

AWS Lattice – Challenges

- Heavy reliance on IAM permissions
- Dependence on AWS services for provisioning
- Service network creation time delay



One of the downsides of lattice in general and these service networks is twofold. One of the downsides is that everything is IAM based. And so every time you have a pod or traffic going out of your cluster, you're routing to something else that needs more and more advanced IAM permissions and in service or in VPC lattice that also includes context about the call. So you're going to be relying on IAM to work much more often and not saying it doesn't scale and it doesn't work, but you're relying on yourself to get some of those rules, rights and the lattice controller to do that for you.

AWS Lattice – Challenges

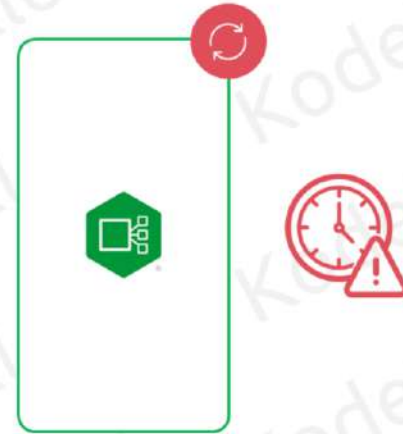
- Heavy reliance on IAM permissions
- Dependence on AWS services for provisioning
- Service network creation time delay



The second difficulty here is you are now dependent on a lot of AWS services to be provisioned and updated as quickly as your Kubernetes cluster may be.

AWS Lattice – Challenges

- Heavy reliance on IAM permissions
- Dependence on AWS services for provisioning
- Service network creation time delay



And things like a service network will take between five and ten minutes to create. And if you're used to how Kubernetes works, you don't want to wait around five or ten minutes. Usually when you have an ingress or a gateway, it's available pretty instantaneously. And maybe you're only creating service networks a few times, but as you try to sync more and more resources external to the cluster and things like Cloud Map and services endpoints, the latency that is involved of oh, is this broken or am I just waiting for it? Is going to be more and more frustrating.

AWS Lattice – Considerations

01

Designed for advanced
use cases

02

Suitable for large-scale
environments

03

Requires separation of
concerns between
teams

04

Integration across
different
environments and
organizations

05

Complexity hidden but
not simple to use

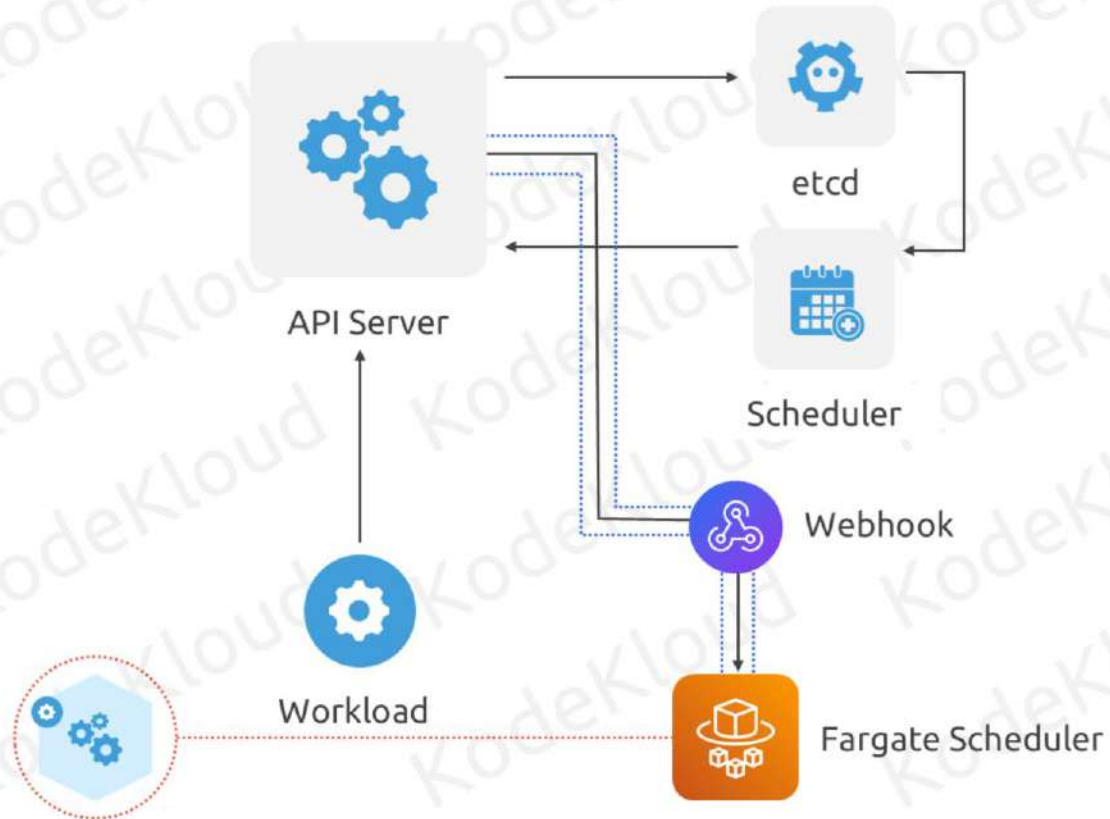
And so service the and so the VPC lattice is a really cool feature that hides some of the complexity of connectivity away, but it's really meant for those advanced use cases and for large, larger environments and large customers who have a lot of separation of concerns between multiple layers of platform teams and network teams and cloud teams, and even those teams that work in different environments or in different organizations. So this isn't a simple thing to use, but it is something that is available to you and can work and integrate a Kubernetes cluster into a VPC lattice.

Fargate

© Copyright KodeKloud

Let's talk about compute. It is a very important part of your cluster and something that is required for you to run any workloads in AWS. There are three main sections of compute. There are in an EKS cluster, there are three main ways that you can get compute attached to your cluster. And the first one is Fargates, which is quote unquote, a serverless option. There's also node groups which are managed and unmanaged. And then there's carpenter, and we're going to discuss, and we're going to discuss all three for you so you can understand how they work, how they're similar and how they're different and when you might want to use them. So let's dive into it. And we're going to start with Fargate because Fargate is the most unique to AWS, but also the most unique in when you might want to use it.

Fargate – Introduction



© Copyright KodeKloud

Okay I have my eks cluster and I have an API and I want to submit a new workload to this cluster and I'm going to send it to the API. Normally this API will create it and put it in ETCD as the database and say okay, we're going to store that and the scheduler is going to come along and figure out where it's going to run. Because the scheduler is constantly looking, why is this so buggy today? Because the scheduler is constantly looking at the API, fetching that data from ETCD and saying, hey, which pods need to be scheduled on a node. Now when you run a cluster with a Fargate profile, Fargate profiles creates a webhook and these web hooks and these webhooks will take this pod as it's coming in and modify it to use a different scheduler. Instead of having the standard Kubernetes scheduler, they're going to say, hey, we're going to use this for

Fargate scheduler to schedule that workload because we know how to manage Fargate. The normal scheduler in Kubernetes has no idea what Fargate is. It doesn't know what a Fargate profile is. It doesn't know what a Fargate node is. The Fargate scheduler on the other hand knows, hey, I'm going to tightly integrate how I schedule this workload to a node in the cluster because all of the places that pods have to run are nodes. You can't make a server a completely serverless or nodeless Kubernetes cluster because all of the semantics for a pod and a workload require there to be a node.

Fargate Scheduler Backend



Fargate Scheduler



Fargate Node



Kubelets

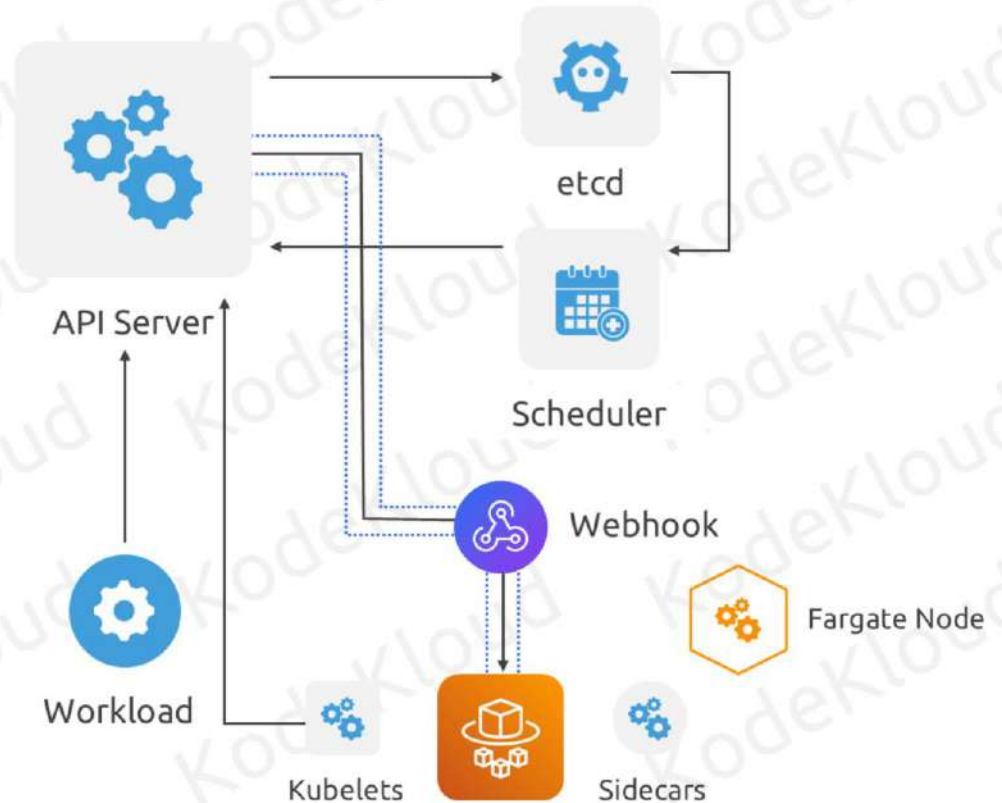


Sidecars

So when the pod comes in, it gets this webhook that puts the Fargate scheduler on it, and it's going to wait because the other scheduler isn't going to do anything with it. The other scheduler is like, hey, that's not me. I'm just going to let whoever else claims they're going to schedule that for me do the thing they're supposed to do. So the Fargate scheduler here on the back end is actually going to call out and get a Fargate node. It's going to do this based on the size of the requests coming in for that application. So if you requested one cpu and two gigs of ram, it's going to find the closest match of a profile it can for this Fargate node to come up. And it does this by basically taking this minimal amount of requests, one cpu, two gigs of ram, it adds a percentage of overhead for the, for other things that are going to run on the, on the Fargate

node itself. So we're going to add things for the kubelets and any other sidecars that you might need. And these sidecars, if you're familiar with Kubernetes, they're just other workloads that are going to run with your application. So all of these things get spun up inside of this Fargate node. And this also is going to add, I won't say that yet.

How Fargate Scheduler Works

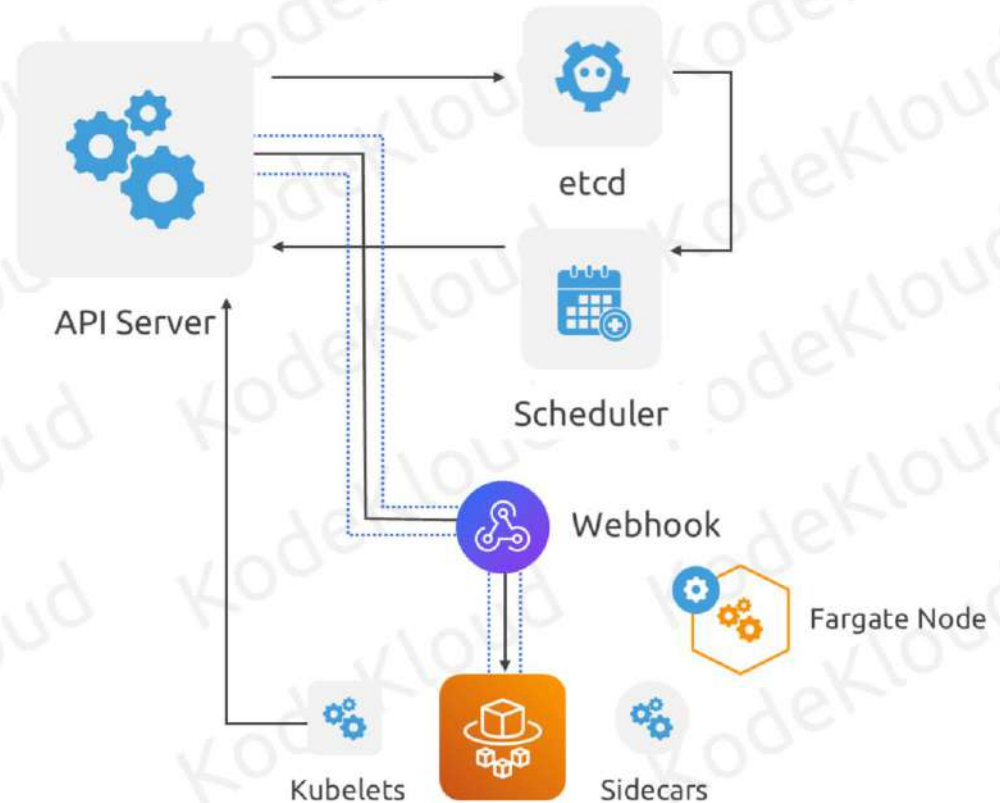


© Copyright KodeKloud

And so the Fargate schedule actually goes out, talks to the Kubernetes API. Oh no, no, no. The Fargate schedule actually goes out, talks to the AWS APIs, creates this node, waits for the kubelet to join the cluster, and it binds this pod to this new node because it knows, hey, I'm creating this node just for this pod. So as the pod comes up, it's going to get scheduled here because it's using this special purpose Fargate scheduler. If you've ever used Fargate before in the past, in things like ecs, you'll know that this node isn't an EC two instance. It's not showing up as an EC two instance for you. Inside of your accounts. You get a Fargate node that shows up in the eks cluster, but you don't have a new EC two instance. This is something that's, in that regard is serverless because you don't see how or where it runs. Fargate nodes do not, because

Fargate nodes do not run inside of your account. They have some limitations. There are some things that they can't do. And as we talked previously, things like EFS volumes or dynamic EFS volumes don't work in Fargates. Some other things that are just a little tricky because how it's actually mounting stuff is EBS volumes don't work. They don't get mounted from your accounts, crossing this boundary into an AWS account, and then getting mounted into the fargate profile. Other things like daemon sets inside of daemon sets inside of the Kubernetes cluster don't run on Fargate nodes because a daemon set is a per node pod essentially, and Fargate is handling all of those nodes dynamically.

How Fargate Scheduler Works



© Copyright KodeKloud

And so the Fargate schedule actually goes out, talks to the Kubernetes API. Oh no, no, no. The Fargate schedule actually goes out, talks to the AWS APIs, creates this node, waits for the kubelet to join the cluster, and it binds this pod to this new node because it knows, hey, I'm creating this node just for this pod. So as the pod comes up, it's going to get scheduled here because it's using this special purpose Fargate scheduler. If you've ever used Fargate before in the past, in things like ecs, you'll know that this node isn't an EC two instance. It's not showing up as an EC two instance for you. Inside of your accounts. You get a Fargate node that shows up in the eks cluster, but you don't have a new EC two instance. This is something that's, in that regard is serverless because you don't see how or where it runs. Fargate nodes do not, because

Fargate nodes do not run inside of your account. They have some limitations. There are some things that they can't do. And as we talked previously, things like EFS volumes or dynamic EFS volumes don't work in Fargates. Some other things that are just a little tricky because how it's actually mounting stuff is EBS volumes don't work. They don't get mounted from your accounts, crossing this boundary into an AWS account, and then getting mounted into the far gates profile. Other things like daemon sets inside of daemon sets inside of the Kubernetes cluster don't run on Fargate nodes because a daemon set is a per node pod essentially, and Fargate is handling all of those nodes dynamically.

Fargate Nodes – Limitations



Dynamic EFS Volumes
don't work in Fargate



EBS Volumes don't
work in Fargate



Daemon sets inside Kubernetes
don't work in Fargate

So if you have one deployment and it's scheduling say 100 pods, you actually need 100 individual nodes to be created in your cluster. And it would also need to do say 100 daemon sets along with those. Instead of using daemon sets with Fargate nodes, you have to convert all of your daemon sets to be sidecars. So if you have something that's like a monitoring application or some way to get logging, and you need that to run on the node level, you're going to have to put that as a daemon set, or you're gonna take that daemon set and make it into a sidecar. And so that's going to increase this overhead for your fargate pods because you no longer will take one big node. Why freaking pen. You will no longer take one big node with say three or four daemon sets and a bunch of workloads on top of it. You are now going to be splitting every one of

those workloads into their own nodes, and then you're going to have to replicate all of those daemon sets onto each one.

Fargate Nodes – Limitations



© Copyright KodeKloud

So if you have one deployment and it's scheduling say 100 pods, you actually need 100 individual nodes to be created in your cluster. And it would also need to do say 100 daemon sets along with those. Instead of using daemon sets with Fargate nodes, you have to convert all of your daemon sets to be sidecars. So if you have something that's like a monitoring application or some way to get logging, and you need that to run on the node level, you're going to have to put that as a daemon set, or you're gonna take that daemon set and make it into a sidecar. And so that's going to increase this overhead for your fargate pods because you no longer will take one big node. Why freaking pen. You will no longer take one big node with say three or four daemon sets and a bunch of workloads on top of it. You are now going to be splitting every one of

those workloads into their own nodes, and then you're going to have to replicate all of those daemon sets onto each one.

Fargate Nodes – Limitations



© Copyright KodeKloud

You will no longer take one big node with say three or four daemon sets and a bunch of workloads on top of it. You are now going to be splitting every one of those workloads into their own nodes, and then you're going to have to replicate all of those daemon sets onto each one. How do I say that? Because Fargate is isolating every pod to its own node, you have a different reason you should be using or where you should be using Fargate. This is not a general purpose. Use everything that you use in Kubernetes on Fargate.

Fargate Nodes – Limitations



You will no longer take one big node with say three or four daemon sets and a bunch of workloads on top of it. You are now going to be splitting every one of those workloads into their own nodes, and then you're going to have to replicate all of those daemon sets onto each one. This is going to use a lot more compute at scale. This is going to use a lot more resources at scale. This is something that you have to redo over and over again and get outside of some of the standard Kubernetes ways that it wants you to share nodes and isolates them into these individual nodes per workload or per pod. How do I say that? Because Fargate is isolating every pod to its own node, you have a different reason you should be using or where you should be using Fargate. This is not a general purpose. Use everything that you use in Kubernetes on Fargate.

Why Use Fargate?

01



Isolation for
Security

02



Isolation for
Compute
Resources

The main thing that makes sense for Fargate are things that you either want to have completely isolated and secure. So, if you have workloads that no other workload could be scheduled with it, or if you want things that are isolated for their compute resources, these are both great options for when you should be using Fargates. And not a lot of people have clusters that have strict security boundaries. And if you do have a strict security boundary, you probably need to isolate that into its own Kubernetes cluster because there are still things inside of the cluster that are kind of leaky in the way that they allow things to be managed across different namespaces or different RBAC. In a Kubernetes cluster, there are things that are global. And so if you need that secure isolation for a workload, you probably need to put this in its own Kubernetes

cluster because those global entities like ingress controllers or volumes, and some things just are not isolated enough, depending on how much security you need. So, Fargate isn't actually the best security boundary because it's still within the same Kubernetes cluster. This is only a boundary for the process and the memory and the file system. Maybe resources, however, is a very valid option because things that run in your cluster that don't need to be fully isolated for security, but always need to have dedicated resources available for compute, or always need to be available for the API, make perfect sense to run inside of Fargates.

Kubernetes Services



Metric Server



Karpenter

I like to call these things kubernetes services or internal services. And for me this is often things like the metric server, things like a node scheduler, like the cluster autoscaler or carpenter. Anything else that your workloads depend on inside of the cluster make great sense to put in Fargate, because they get isolated and they're outside of the cycle of normal upgrades and they are outside the cycle of normal disruptions that might be happening to your other workloads. If, say, certain things aren't working or you're upgrading a cluster, the Fargate profiles stay a little bit isolated and have different ways of managing them. So this might be a good way to, this might be a good idea to put some of these services inside of Fargate just so that they are, they are isolated from other workloads and you don't have to worry about outages that might

affect those other things. Of course, putting things inside of Fargate is not a complete solution. You still do need to make sure that you're handling things like high availability. If you need the metric server to always be available, don't just run one, it's only going to be in a single Az. You'll want to put three of them out there. And so again, you have to balance this availability and availability performance and cost all into kind of the same bucket to make sure that you're doing the right thing for what you need for your cluster.

Kubernetes Services



Kubernetes Services



Availability



Performance



Cost



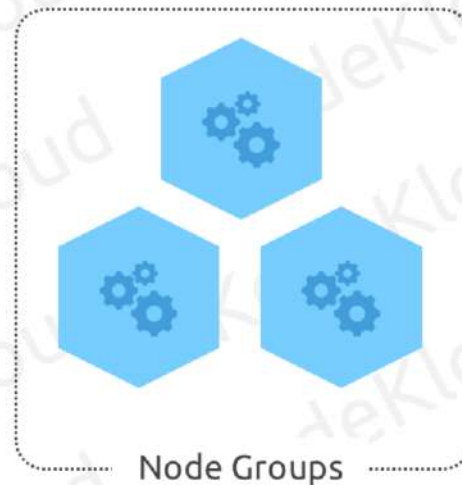
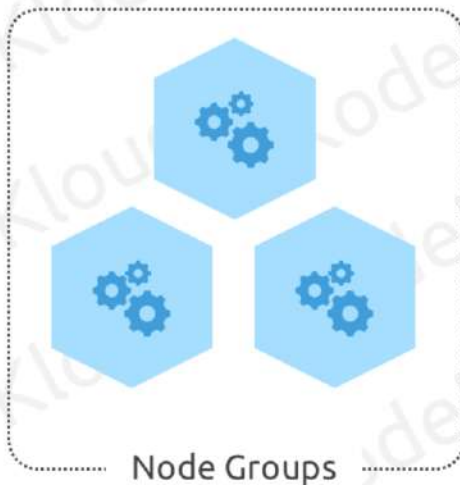
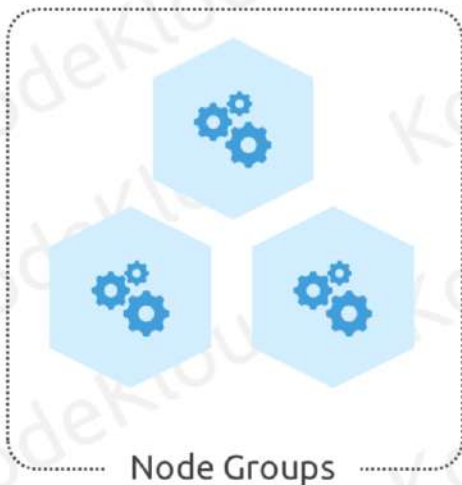
Metric Server



Karpenter

EKS Node Groups

Node Groups



Now let's talk about the most common use, the most common. Now let's talk about the most common compute that's used inside of eks and that's node groups. Node groups are just a way of grouping a type of node or compute available to an EKS cluster. And when eks first started, the only way you could get a compute or an EC two instance to attach to a EKS cluster was to bring your own node. And this was unmanaged node groups. Unmanaged node groups were just creates an auto scaling group, which is a construct inside of AWS that says, hey, I want this type of EC two instance this many times and it would scale up and down and it would add or remove those things. Similar to deployments with pods, you can scale them up and down. Auto Scaling group was kind of the original idea.

Node Groups



Now let's talk about the most common use, the most common. Now let's talk about the most common compute that's used inside of eks and that's node groups. Node groups are just a way of grouping a type of node or compute available to an EKS cluster. And when eks first started, the only way you could get a compute or an EC two instance to attach to a EKS cluster was to bring your own node. And this was unmanaged node groups. Unmanaged node groups were just creates an auto scaling group, which is a construct inside of AWS that says, hey, I want this type of EC two instance this many times and it would scale up and down and it would add or remove those things. Similar to deployments with pods, you can scale them up and down. Auto Scaling group was kind of the original idea.

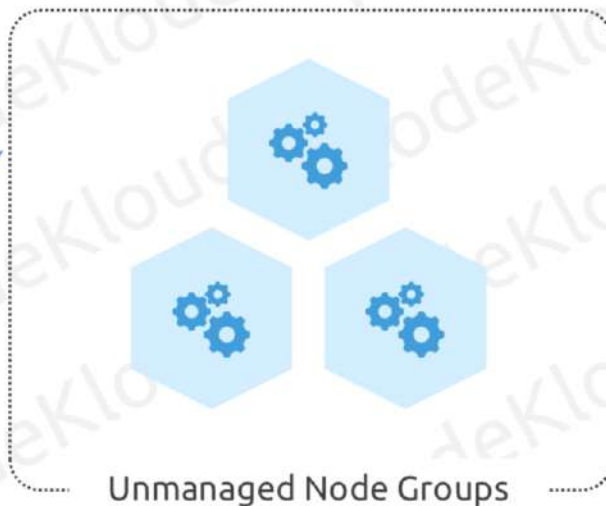
Node Groups



Now let's talk about the most common use, the most common. Now let's talk about the most common compute that's used inside of eks and that's node groups. Node groups are just a way of grouping a type of node or compute available to an EKS cluster. And when eks first started, the only way you could get a compute or an EC two instance to attach to a EKS cluster was to bring your own node. And this was unmanaged node groups. Unmanaged node groups were just creates an auto scaling group, which is a construct inside of AWS that says, hey, I want this type of EC two instance this many times and it would scale up and down and it would add or remove those things. Similar to deployments with pods, you can scale them up and down. Auto Scaling group was kind of the original idea.

Unmanaged Node Groups

Hey! I want this type of EC2 Instance these many times



Scale Up



Scale Down

Now let's talk about the most common use, the most common. Now let's talk about the most common compute that's used inside of eks and that's node groups. Node groups are just a way of grouping a type of node or compute available to an EKS cluster. And when eks first started, the only way you could get a compute or an EC two instance to attach to a EKS cluster was to bring your own node. And this was unmanaged node groups. Unmanaged node groups were just creates an auto scaling group, which is a construct inside of AWS that says, hey, I want this type of EC two instance this many times and it would scale up and down and it would add or remove those things. Similar to deployments with pods, you can scale them up and down. Auto Scaling group was kind of the original idea.

Unmanaged Node Groups

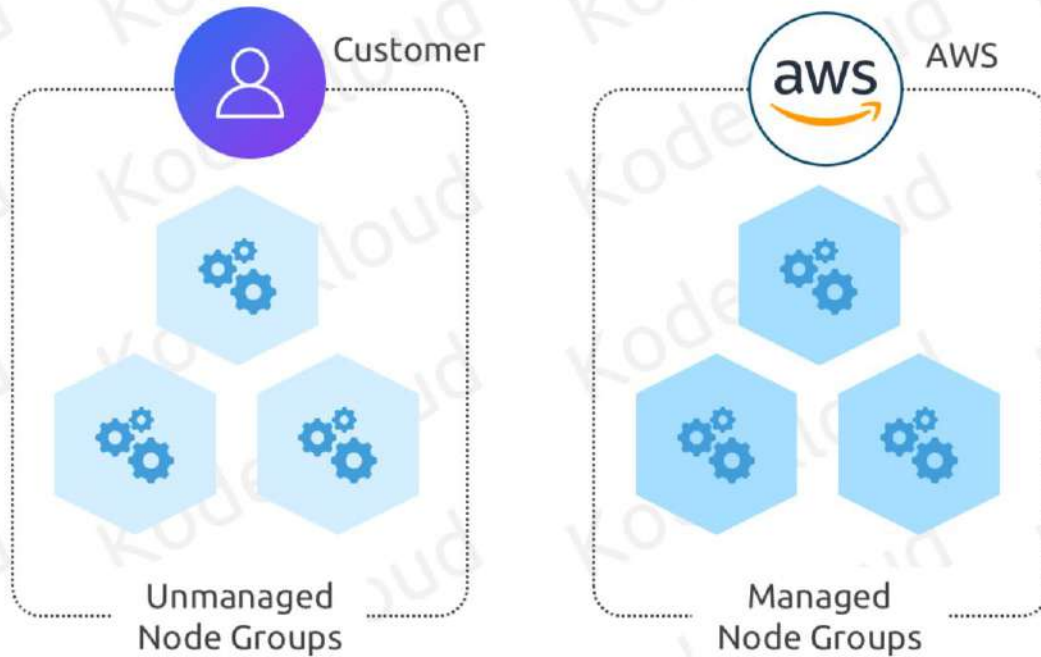


© Copyright KodeKloud

Now let's talk about the most common use, the most common. Now let's talk about the most common compute that's used inside of eks and that's node groups. Node groups are just a way of grouping a type of node or compute available to an EKS cluster. And when eks first started, the only way you could get a compute or an EC two instance to attach to a EKS cluster was to bring your own node. And this was unmanaged node groups. Unmanaged node groups were just creates an auto scaling group, which is a construct inside of AWS that says, hey, I want this type of EC two instance this many times and it would scale up and down and it would add or remove those things. Similar to deployments with pods, you can scale them up and down. Auto Scaling group was kind of the original idea. Idea of that with vms and unmanaged node groups are I just

have this auto scaling group and I want it to talk to that eks cluster and it would scale up and down and that's how you would manage your auto scaling for any sort of node. But all of the work to create the auto scaling group, to put the node in place to make sure the Kubelet could join the rights cluster, the IAM credentials, to be able to do that, all of that stuff was on YouTube. You had to manage it. And that was the only way to get nodes into an eks cluster. That was a lot of work. It was something that not a lot of people like doing, and it was never the end goal.

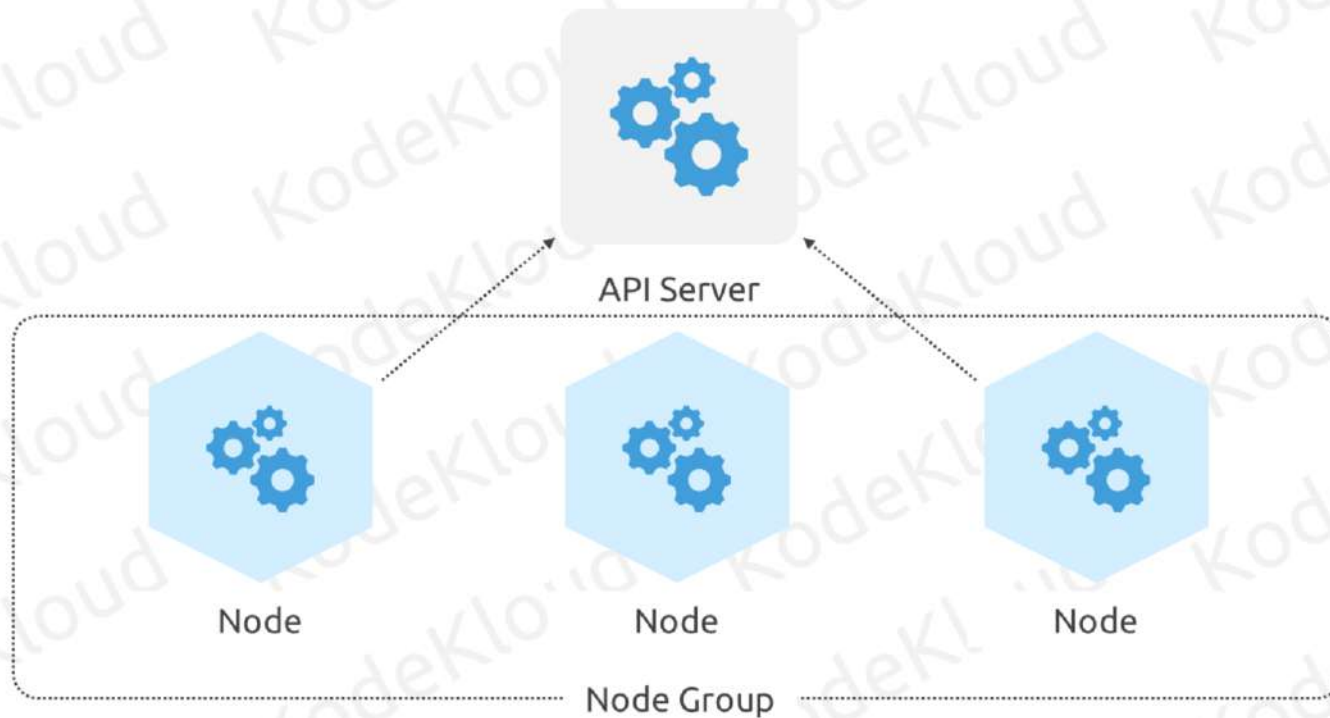
Node Groups



© Copyright KodeKloud

And so within eks, there are also managed node groups. We had to differentiate how those things work, what they were called, just so you would know, oh, one of them is me. As a customer, I have to do the management and the other one is managed by AWS.

How Node Group Works

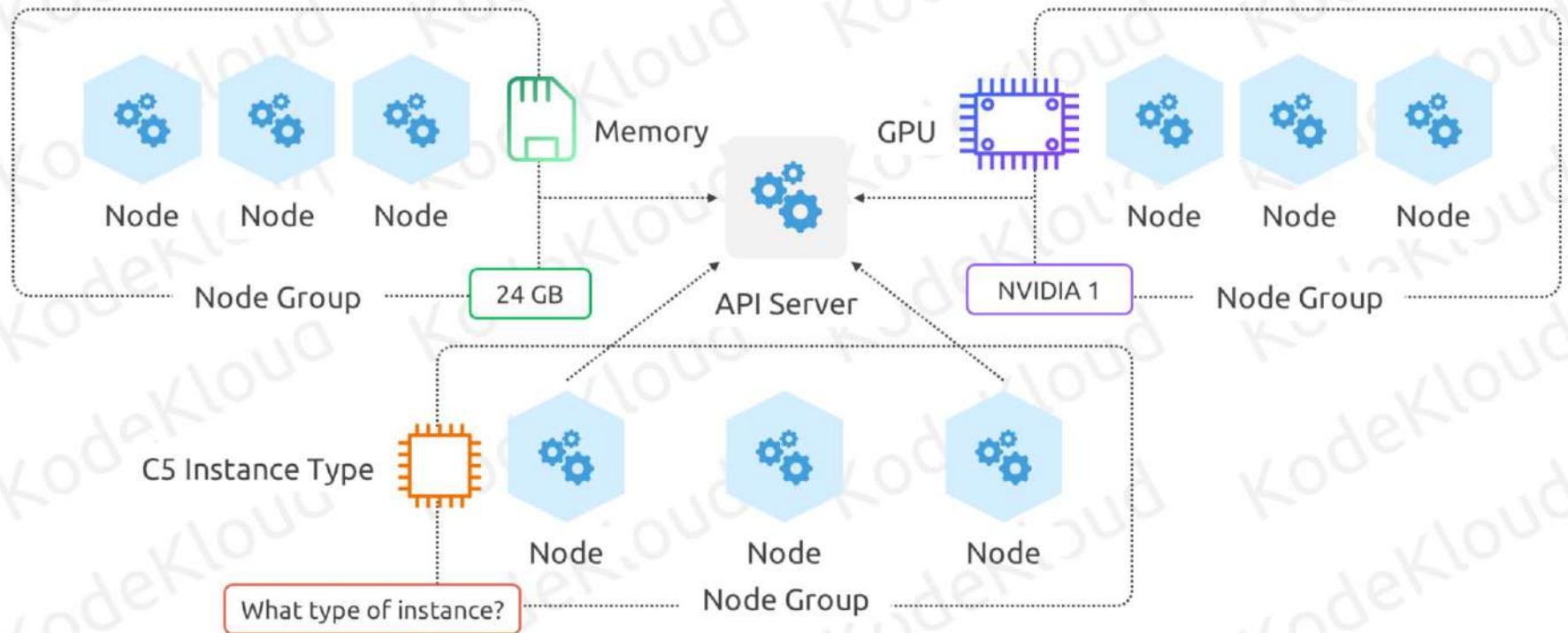


© Copyright KodeKloud

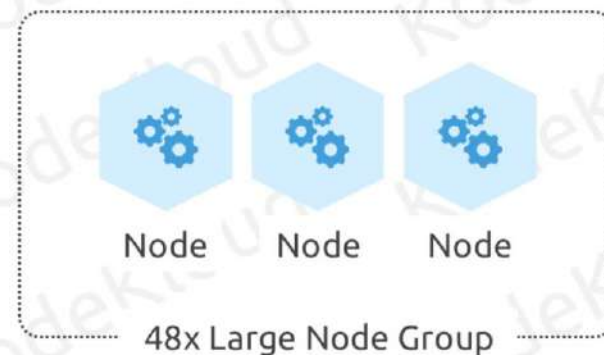
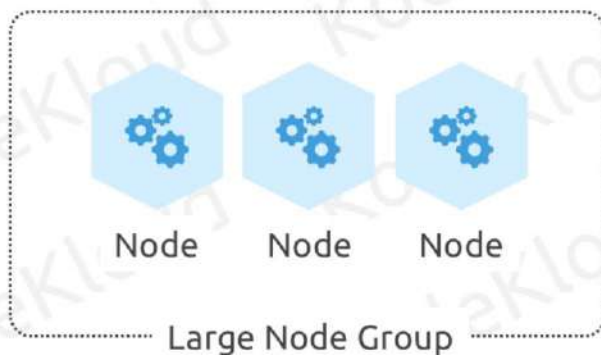
Let's show you just in general how a node group works, whether it's managed or unmanaged, so you can understand just how it's working under the hood. So here's my API server, and I'm going to create a node inside of AWS, right? Like this is just any node that's going to say, hey, I want to use or be part of this cluster. And so I'm going to join that cluster and then I probably want two of them, right? So I have high availability or I want to really, this is super annoying. If I need high availability or for whatever reason, I have two nodes that are joining this cluster and in my workloads can schedule onto them as you would expect in any sort of Kubernetes cluster. And a node group is just a way to take this idea and say, actually, these are kind of the same thing. Both of these things will be managed as a group, and if we are going to create

more of them, they will fit inside of this, this node group as what we're calling it. And you can have multiple of these node groups. These are not, you can't, isn't a requirement that you have just one per cluster. You could say have one over here and we want another one down here. And these can all be different things. They can all have different types of node groups. So if this first node group that we created is like a general compute node group, maybe like a c five instance type, this other one over here could be something with a GPU. So we could have a GPU node group for workloads that require GPU.

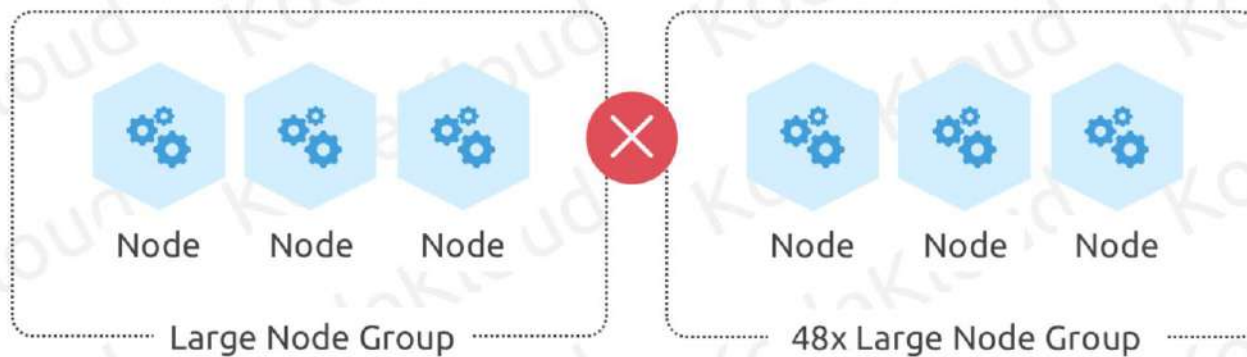
How Node Group Works



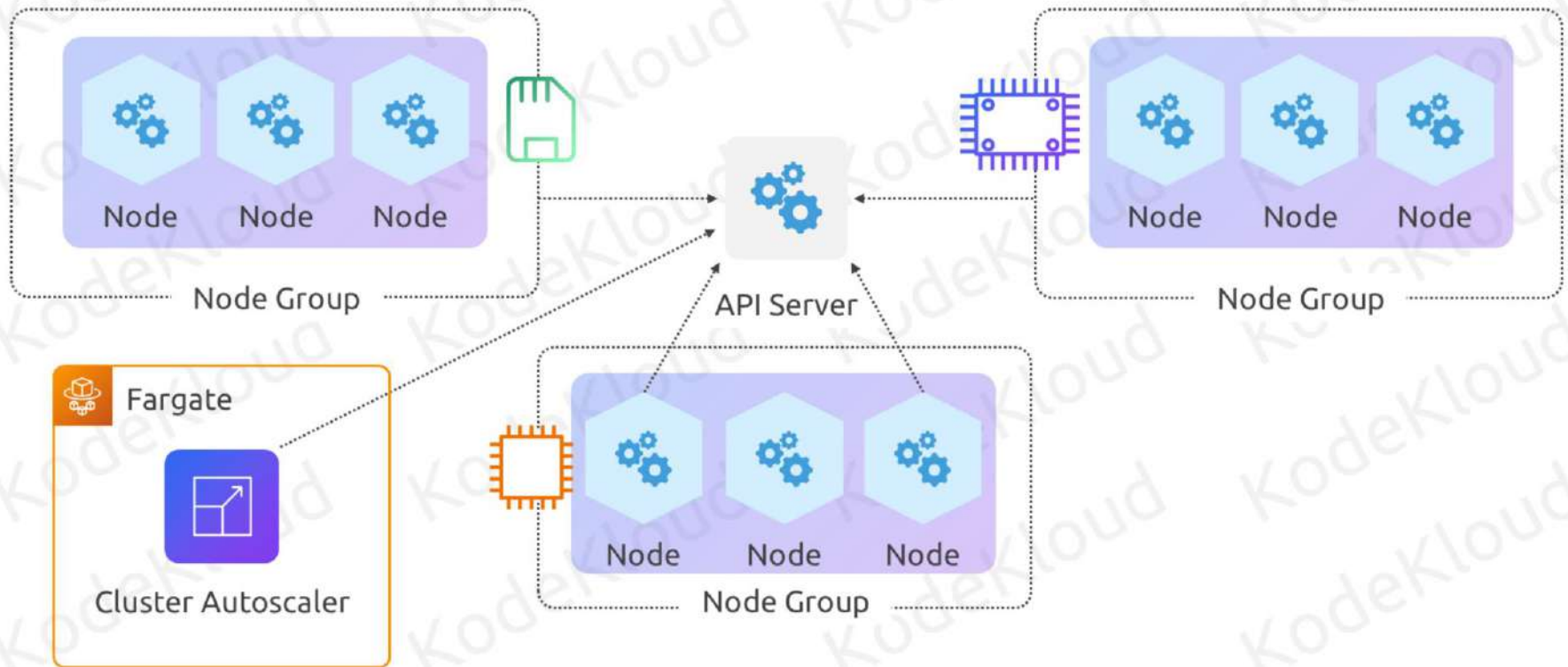
How Node Group Works



How Node Group Works



How Node Group Works

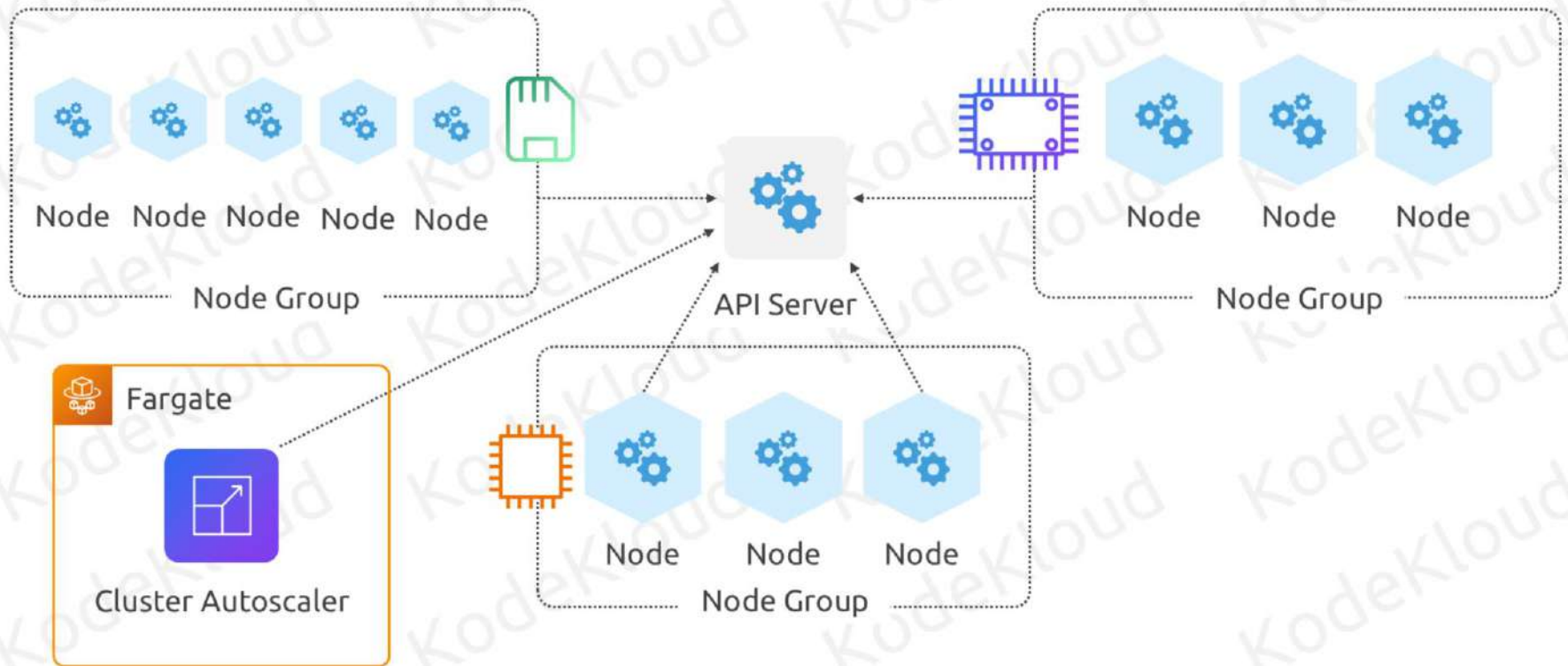


© Copyright KodeKloud

Like we said in the last, like we said in the last section, things like the cluster autoscaler might be best to run on something like Fargate because you don't want the cluster autoscaler jumping between all of these node groups over and over again as maybe they scale up and down and saying, oh, I need to scale over here, I need to go over there. Something like an isolated environment where this is only running on Fargate make a lot of sense because it's not going to be scaling up and down dynamically like you might be in some of these other node groups, but that cluster autoscaler needs to know about all of these node groups because it's the one that's usually responsible for adding new nodes into the group as a workload comes in. We'll talk about how to simplify that with carpenter in just a little bit. But right now let's keep focusing on how these

node groups work.

How Node Group Works



Unmanaged Node Groups – Benefits



Custom Amazon
Managed Image
(AMI)

A big benefit of unmanaged node groups or bring your own nodes was the fact that you could use any image you wanted so you could have custom Amis for that node group. And this was often the case because people already had an AMI that, that they were using inside of AWS and they vetted it for whatever their workloads were. They pre installed some tools. Whatever the case may be, they wanted a custom AMI to be able to use inside of AWS. This was the biggest reason that people stuck with unmanaged node groups and is actually no longer the case

Managed Node Groups – Benefits

01



Custom Amazon
Managed Image
(AMI)

02

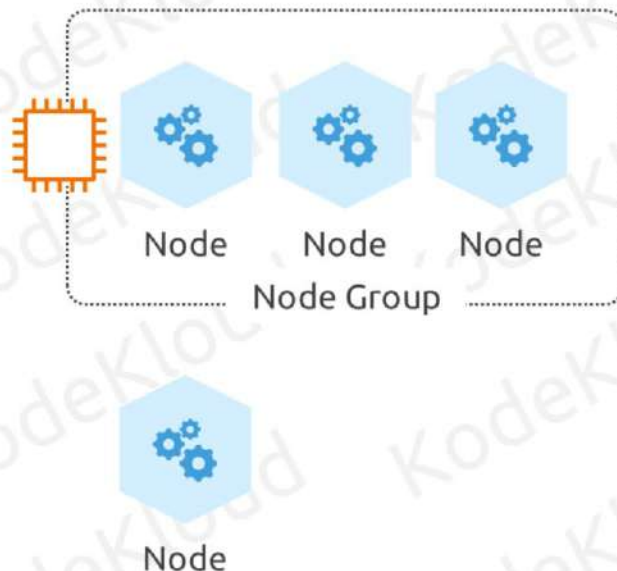


Managed
upgrades

because managed node groups can also use custom AMIs as long as you're using things like launch templates to customize them and declare which AMI you want to use. And so if you need a custom image that is still available inside of managed node groups. And the biggest benefit, the absolute biggest benefit you get with managed node groups is you get managed upgrades. And so when you are going to upgrade your Kubernetes cluster and you say, I'm going to update this from one version to another, EKS handles control plane. All of the control plane is going to be upgraded for you without you even thinking about it. They will roll the vms from one version to another and make sure that as that happens, you know, tests are done and the API is available, all of that stuff. But once you're in this new API, all of your node groups also need to be

updated to that new version with managed node groups. You can trigger that and you can say, hey, I want you to go ahead and manage this node group, this GPU node group, say, and put that into the new version and it will start rolling these vms for you. And it does that by creating the new, the launch template that's attached to this, the launch template that's attached to this auto scaling group. It changes the version of the AMI that's being used and, and then allowing the auto scaling group to replace them. That's what auto scaling groups are great at because it can delete one instance and create a new one. And so you get these new instances being created as old ones are being removed, and the workload should automatically be drained and carry over to each instance as they go through. Depending on how big your node group is, this could cause a lot of churn for your workloads.

Churning Workloads

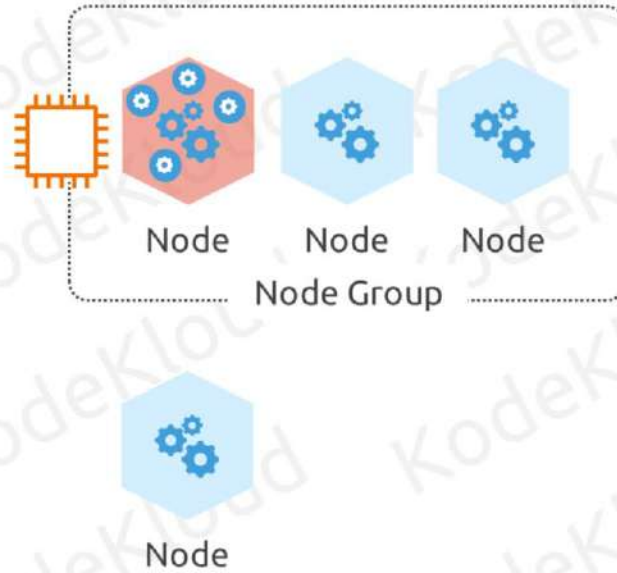


© Copyright KodeKloud

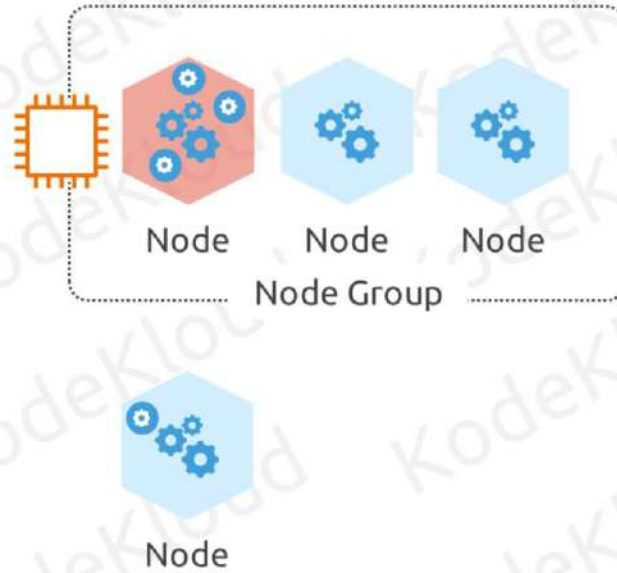
Because let's say, come on. Because let's say I create a new node for this cluster, let's say I create a new node for this cluster and then I delete the old one by deleting this one or marking it for deletion. All of the workloads that we're running on here will need to find a new place to run, and some of them might end up on this brand new instance that we created, but more than likely other ones are going to end up on older instances that also will eventually be deleted. So if we create another node and then we do that cycle again, now these need to be migrated again and so auto. So while managed node groups will handle that automatic rolling out of a new AMI and new versions for you, you do want to be aware that your workloads are probably going to be scheduled onto old workloads before they end up finally making it onto the they'll be scheduled

on old instances with an old version before they're actually scheduled on the newest instance once it's fully replaced. So while you're going through that upgrade process, you will likely get pods that are scheduled three, four, five times, and they keep getting deleted and added again. So upgrades are still disruptive inside of your cluster, depending on how large they are and depending on your tolerance for how many of these workloads are running at a time or not. And while you're doing those upgrades, the cluster auto scaler is also still over here looking at each node group and saying, oh hey, I need to scale that up or down, which also might be fighting things like the upgrade process.

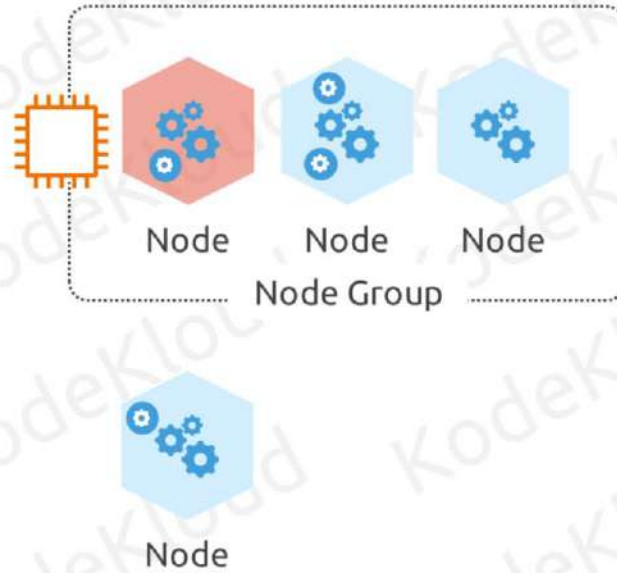
Churning Workloads



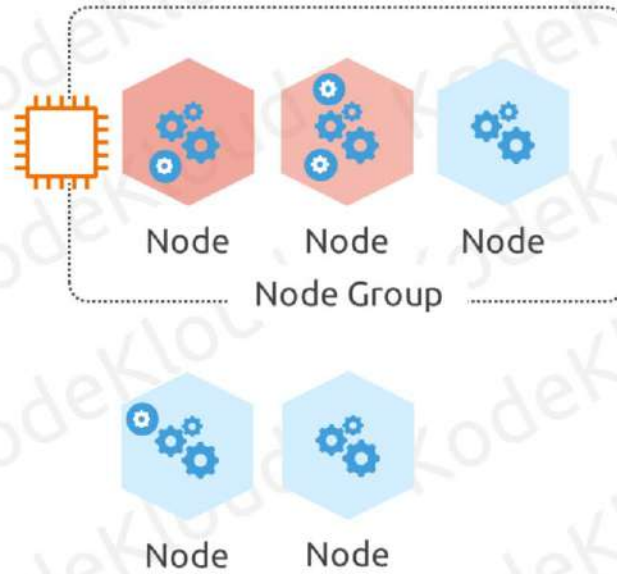
Churning Workloads



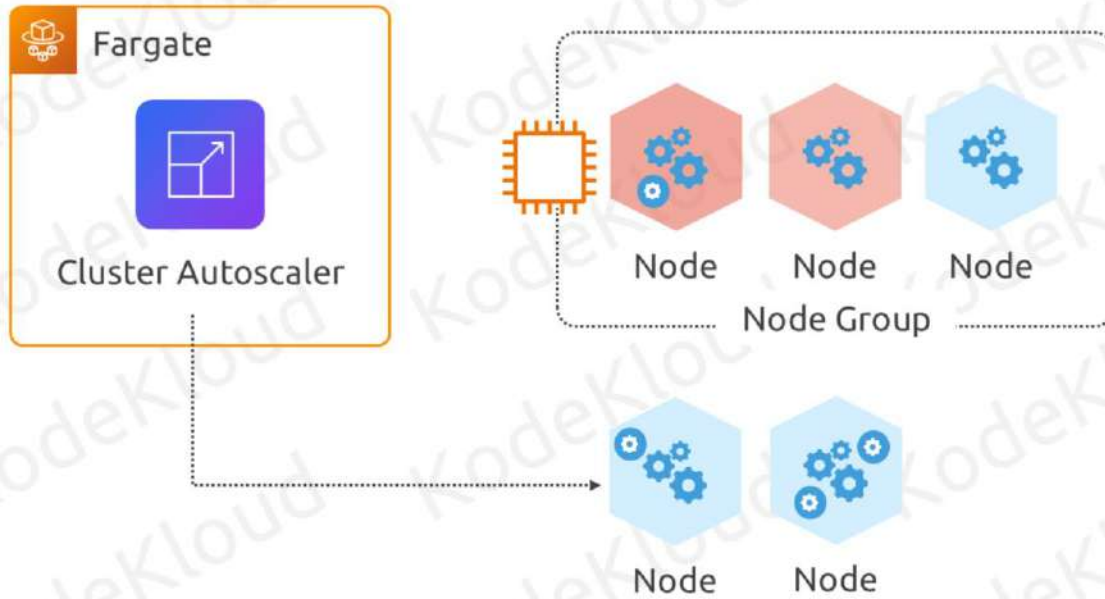
Churning Workloads



Churning Workloads



Churning Workloads



Karpenter

© Copyright KodeKloud

So now let's talk about Karpenter

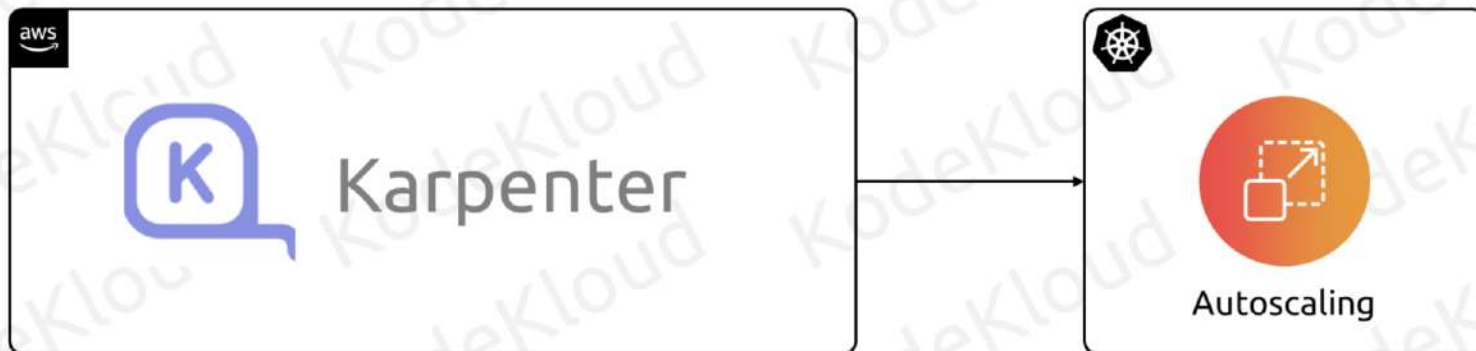
Karpenter – Introduction



© Copyright KodeKloud

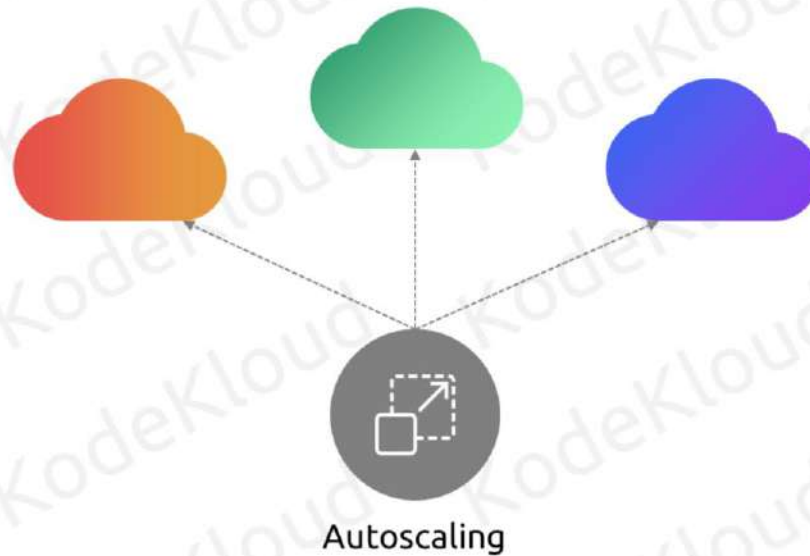
So now let's talk about Karpenter. Karpenter is the newest way to get compute into a cluster, and it was developed by AWS

Karpenter – Introduction



and donated to the Autoscaling sig for Kubernetes.

Karpenter – Introduction



So it is a generic autoscaler that can work in multiple different clouds, and I know some other clouds already have implementations of that.

Karpenter – Introduction



EKS



AWS

But it was designed for EKS, and it was designed for AWS for two very specific reasons.

Karpenter vs Traditional Autoscaling

01

Complex Configuration

Declaring node groups in advance requires detailed planning to specify instance types and capabilities.

02

Scalability Challenges

Scaling node groups efficiently is difficult, leading to wastage and administrative burden as administrators navigate configuration adjustments.

Managed node groups are difficult, or node groups in general are difficult to manage. Not only do you have to declare all of the node groups ahead of time to know what type of instances you want or what capabilities those instances should have, but then scaling them and configuring them to scale up only as you needed one versus another ended up causing a lot of waste and a lot of things that administrators had to do to figure out like, oh, which one do I want to tweak and which flag?

Karpenter's Approach



Karpenter



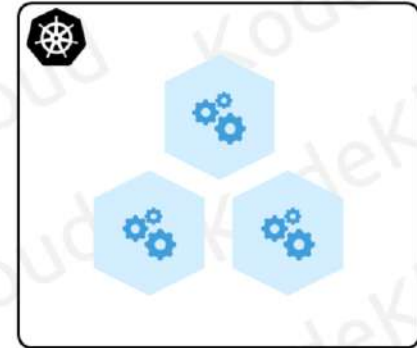
Autoscaling

So carpenter changes some of that. Carpenter is an autoscaler and it will provision nodes into your cluster, but it doesn't do it with an autoscaling group.

Karpenter's Approach



Karpenter



Instead, carpenter does it in a Kubernetes native way with machine pools or node pools. Node pools are a grouping of types, again, that are like ASGs, but we don't have to mix and match how we're doing this, and we don't need all of the abstraction that AWS made and make it work inside of Kubernetes.

Karpenter's Flexibility

01

Scaling Nodes in Different AZs

- When adding a node to a node group, it may end up in a different Availability Zone (AZ) based on your cluster setup.
- Unlike autoscaling groups that rotate through AZs, node groups distribute nodes across various AZs.

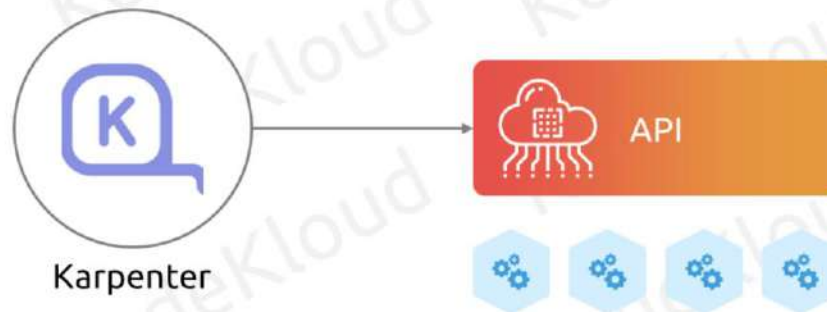
02

Considerations for EBS Volumes

- If you have an EBS volume located in a single AZ, it may pose challenges to autoscaling groups.
- Autoscaling groups may not place nodes in the correct AZ, affecting the accessibility of EBS volumes.

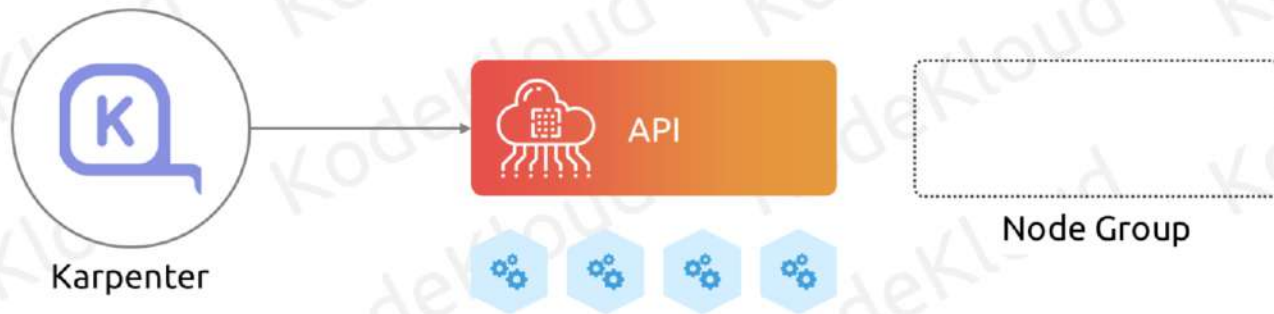
Because things like scaling up a node, if you have a node group and you want to add one to it, where does it go? It goes in different AZs depending on what you have in your cluster. For an autoscaling group, it adds one and it basically just rotates through the AZs. This might be fine for some workloads, but let's say you have an EBS volume and that EBS volume is located in a single AZ. How does it get there in an autoscaling group? Autoscaling group may not put it in the right place. Let's just diagram it and show you some of the differences here.

Workload-Native Node Scheduling



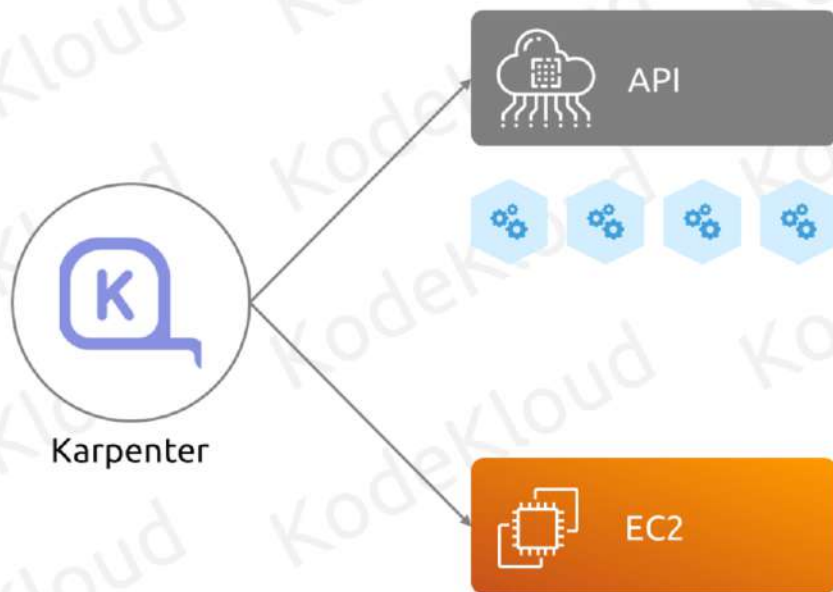
Karpenter is a standard autoscaler that runs inside of the cluster, and it works similarly to cluster API, where if a pod comes in, a workload comes in, and it is unscheduled, meaning it won't fit on an existing node, carpenater will see, oh, hey, I got some pods that need nodes cluster Autoscaler does the exact same thing.

Karpenter's Integration With EC



The difference with cluster autoscaler is cluster autoscaler finds a group of nodes, a node group that's one of these workloads would fit in and it adds one or more than one to that node group.

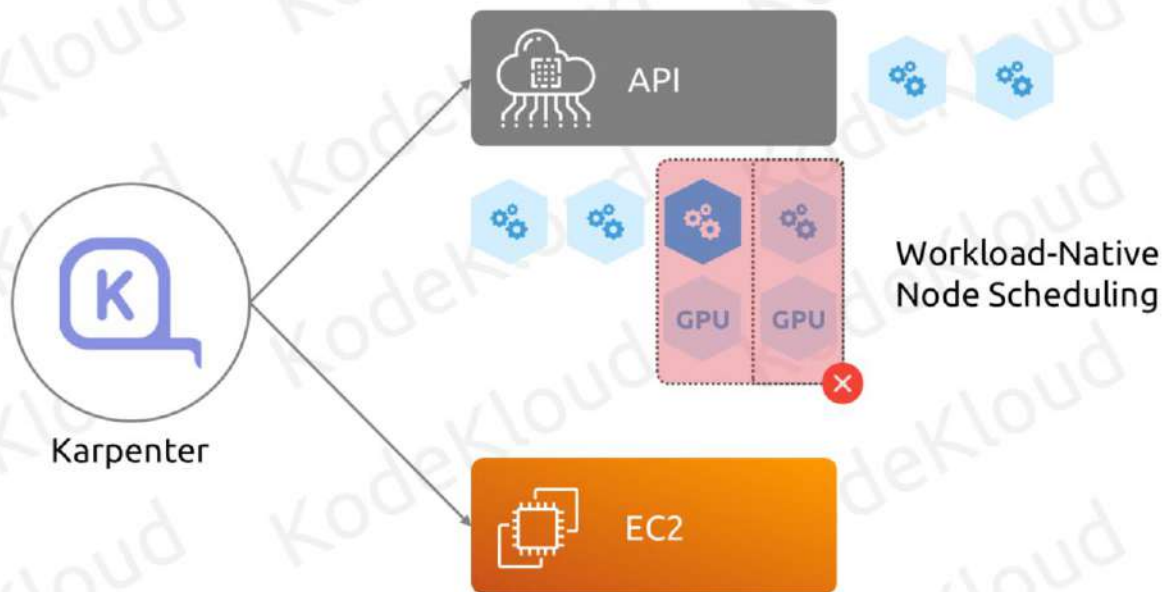
Karpenter's Integration With EC



© Copyright KodeKloud

And with node groups, they're defined externally from Kubernetes. This is something that is, uh, attached into Kubernetes as nodes, but Kubernetes doesn't know anything about what node groups are. It doesn't know what an autoscaling group is. It doesn't have any idea what that's like. And the bridge there is cluster autoscaler. That's the thing that actually says, oh, I can translate. I need a node or a certain amount of nodes and I'm going to add multiple, uh, nodes to a node group. Karpenter is the bridge to EC. Instead of looking at nodes, it's going to know, Hey, I have all these EC instances and we can look at all of them. This isn't just what's defined in groups already. This is what's available in your region because not all EC instances are available in all regions.

On-Demand Node Deployment

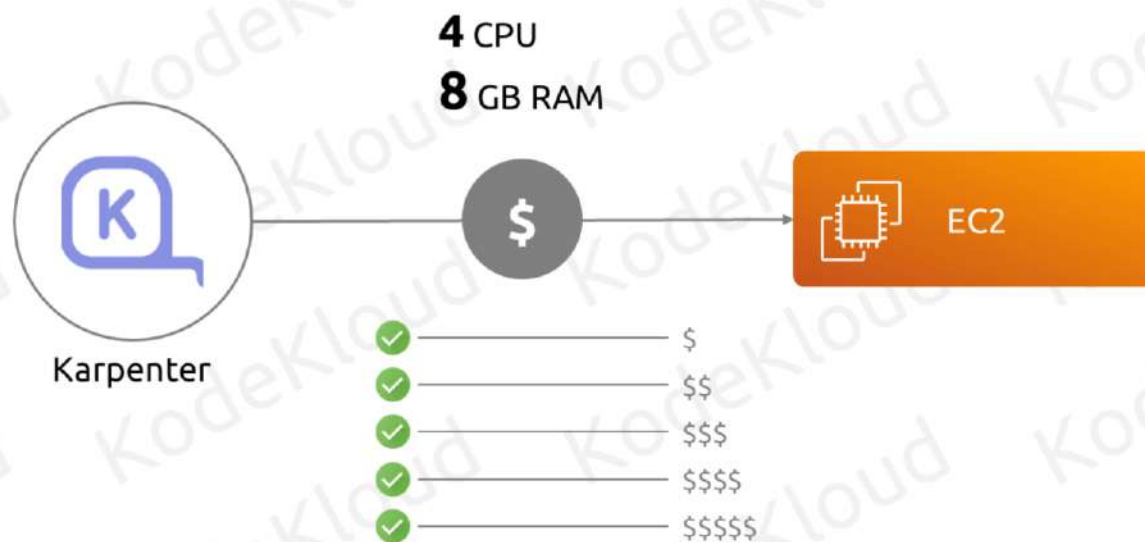


© Copyright KodeKloud

So if we have one workload that's coming in one pod that's unscheduled and it says it needs A GPU, but I don't have any GPU node groups attached to my cluster, carpenter's gonna say, oh, don't worry I know exactly what GPU instances are available in this region and I'm gonna make a node just for you. Or maybe if two of them needed GPUs, carpenter will say, oh, I'll make a node group that satisfies all of the requirements of as many of these workloads as possible and they will fit on this new node. So it's doing on demand node deployment based on what the workloads require I like to call this workload native node scheduling because depending on what the node, uh, because depending on what the workload comes in and says it needs carpenter's gonna find the right instance type for that workload This eliminates some work that you need as a

administrator of a cluster because you don't have to say, I need five different, I know this cluster's gonna have five different types of workloads, and I'm gonna make those node groups for those five types of workloads. As more and more types of workloads come in with different requirements, maybe even high memory like lots of memory or lots of CPU usage, you may not have a node group that fits that. You do still have to create a node pool to say, Hey, this is roughly what I want to allow in my cluster 'cause if you don't wanna allow any GPU instance types, you could say in your node pools, no one gets a GPU that's either too expensive or for whatever reason you can't have a GPU. Uh, so you can disallow certain things in the, in the cluster, uh, maybe you don't want, uh, really expensive or metal type instances. All these different things that, uh, end up being really hard to track because EC, uh, turns out has a lot of instance types. Uh, if you combine the types with sizes, you're looking at hundreds and hundreds of different combinations that any workload can land on and Carpenter knows about them.

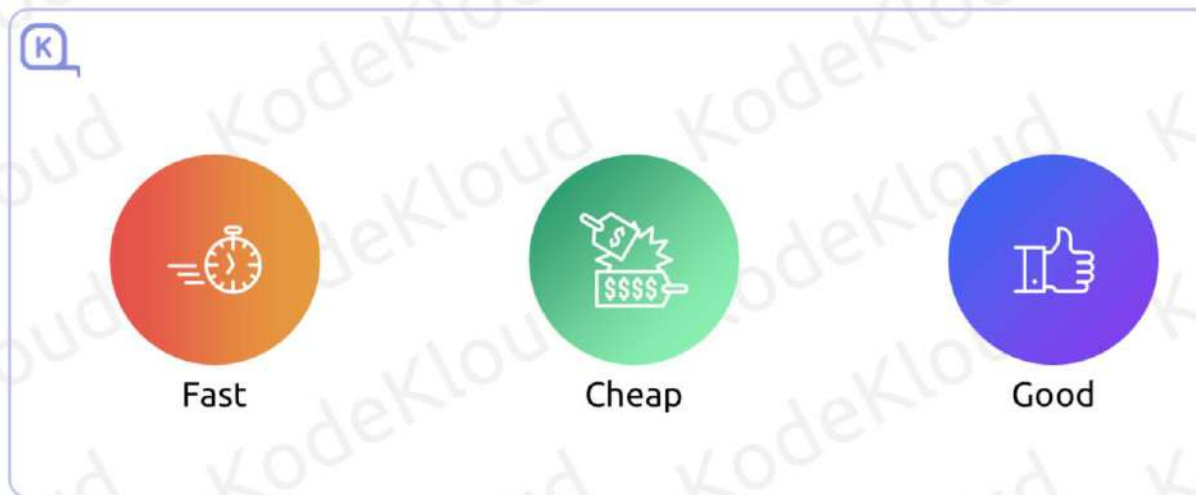
Automated Cost and Resource Optimization



© Copyright KodeKloud

Another thing that Carpenter does that's really nice is it actually knows the price of EC instances. So when your workload comes in and says, Hey, I need to run on, uh, let's say four CPU and eight gigs of ram, that can actually fit on a lot of different instance types. This isn't a, oh, we found one node or one instance type and you're ready to go. Um, this probably matches at least a couple dozen different instances. So what carpenter does is it looks at that list of instances that all come back and say, Hey, these all fit your workload. Which ones the cheapest? We're gonna give you the one that costs the least amount of money so that you can schedule your workload on the least expensive thing.

Workload Cost Optimization



© Copyright KodeKloud

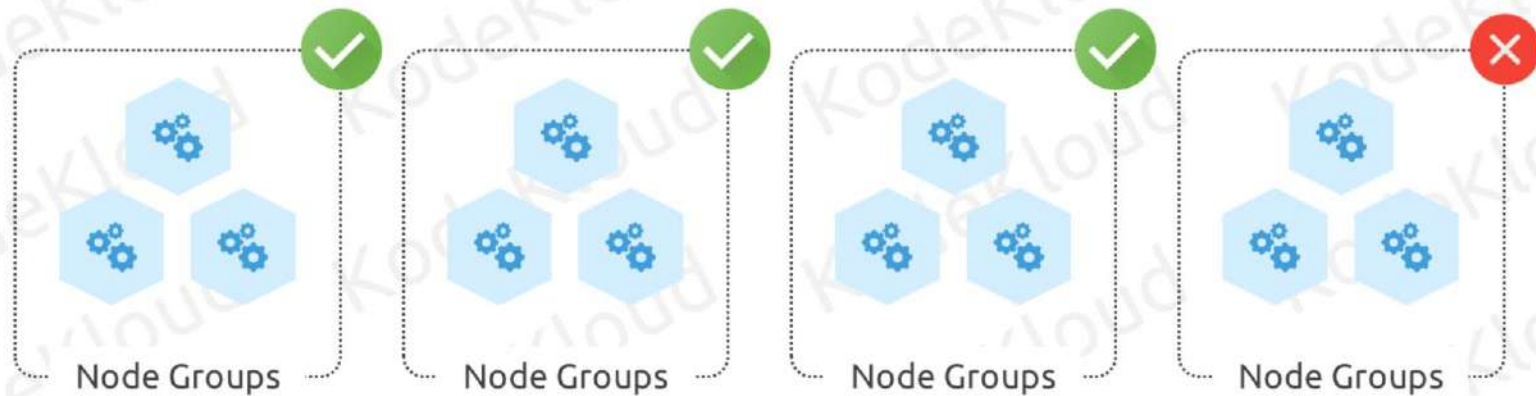
If you've ever heard about, uh, in any sort of engineering, you can do something like, uh, fast, cheap, or good. Uh, carpenter actually manages to get all three of those things because it has more information about the infrastructure where it's running, and it's not requiring you to spin up architecture or groups beforehand. Um, these groups can't, again, be really broad and you can have catchall groups that say, Hey, anything that doesn't fit these, uh, throw it out there as as available. Carpenter's going to do that again in a group list way. So in your cluster, you're not gonna see a node group, you're not gonna see a way to maybe upgrade the cluster as, uh, you normally would with cluster autoscaler or with managed node groups. Instead of being in these groups and rolling out new versions, one at a time per group, carpenter's gonna have to

manage that upgrade process. So as you're going through and upgrading an instance, carpenter will look at what's running in the cluster and then say, okay, well, we're gonna go ahead and delete this node. We're gonna drain it. If those workloads can land on existing nodes, then we don't even have to replace it. We don't have to do anything. It's actually great because they're just gonna fit where we had overhead. Uh, and so Carpenter is a little more aware of what the clusters actually needs. The other really cool thing about cluster, or Another really cool thing about Carpenter is what is called consolidation. And consolidation is going to look at all of the instances as a whole and say, Hey, uh, like Utilization Carpenter's gonna look at utilization across the cluster, and let's say that's on this node over here, we're using maybe percents. Uh, this one over here, we're using %, and this one's about percents. And normally with the cluster autoscaler, it can also look at this wasted space, this unused capacity of this instance and say, Hey, I don't need that. We're gonna go ahead and scale this down. And then these workloads will see where they fit in the cluster. And so in this case, it would say, oh, well I, it can probably fit over here and we're just gonna go ahead and delete that node and they're gonna end up running on that other node that has spare capacity. Well, carpenter will do the same thing. It will look at what capacities here, and it can do the same. Hey, we're gonna delete that node and make sure that it goes away. But it also, again, because it knows about cost in the cluster, it knows, Hey, maybe this instance is really expensive and or maybe this instance is actually the most expensive in the cluster. And so we say, Hey, you know what? Instead of deleting, uh, this node on this side that's cheaper, let's get rid of this other node and shift the costs or the workloads over there. Or let's replace both of these nodes and fill up a new node to % that is the cheapest possible one that we could run. Carpenter can take this into consideration and it continually looks at what's running in the cluster, what nodes are provisioned, how much they cost, and then it does this calculation with what's available and says, Hey, can we make this cluster cheaper and more efficient? And so it will try to fill up more of the nodes at maybe a higher utilization than cluster autoscaler at the benefit of, Hey, we're gonna use more of what we provision, and we're also going to try to find the cheapest available instance types available. And you can set this consolidation to run at various times. You can set it to run continually. You can change all these things in a Kubernetes Dave way. These are all part of the CRDs that are in the node groups. And you say, Hey, I want this to, uh, provision new nodes as soon as possible or, or only on the weekends. Those sorts of things are all values that Carpenter is gonna look at and make sure, Hey, how does this cluster upgrade? How much does it cost and how much is it being utilized? These are all benefits of Carpenter inside of an EKS cluster. And to

this day, if you're not using Carpenter, uh, you're probably doing more work, uh, than you need to because you're gonna do more configuration. You're gonna try to fine tune some of those node groups, or you're gonna be doing some manual upfront negotiations with what workloads are running that with Carpenter you may not need to do. The biggest downside or warning about Carpenter is your workloads need to be mature. And by that I mean you need to have some things built into your workloads that allow it to be reprovision and moved around a little more frequently than you might need to on servers. So things like, uh, pod disruption budgets are a big, are a big thing that you will need inside a carpenter where you say, Hey, I want to make sure I always have some percentage of this workload running because Carpenter is going to move things around maybe a little more than you were used to. Uh, things like, uh, topology spreads inside your workloads are things, uh, to allow multiple workloads to schedule across different AZs in different nodes. By default, if carpenter's looking for the cheapest way to run your workload and you don't specify any topology spread, it'll run all of your workloads on one instance because one instance is the absolute cheapest way that you can run any workload and it'll say, Hey, you get one large instance and everything goes there. But if you want high availability across AZs or across nodes, you have to define that in your workload to make sure that you have the high availability and trade off between cost and availability. And the last thing is, uh, resource requests. Resource requests for how much CPU, how much memory those things need to be defined in your workloads. If you're coming in with a workload that has no pod disruption budgets and no topology spreads and no resource requests, carpenter will schedule that super fast and I guarantee you'll have an outage because at some points, carpenter will remove the node or it will, uh, try to schedule other things on top of that node because you haven't requested any resources to the node. And so it's going to just fit whatever you're saying I need. So you have to move some of this stuff that maybe you got by defaults by using the cluster autoscaler because it would by default spread across AZs, or maybe you got it by default with the Kubernetes scheduler because it would try to spread workloads across nodes. But with Carpenter being the one responsible to saying what's available in the cluster and what nodes are available, those things have to be pulled into your application and your application needs to be a little more mature about what it requires to run effectively. So all, all of these different compute options are available in EKS, and there are options that you can decide how you want to manage the cluster. Carpenter's usually the easiest to get started and the one that can scale and react really quickly to changes inside the cluster. Uh, but also Carpenter is a cluster service, and so you probably want cluster running in Far Gate or managed, uh,

as, as a managed service so that you don't have to worry about carpenter being moved around between these instances. So let's dive into showing a little differences on how any of these workloads might get scheduled to these types of nodes with all three different types. That wasn't a good way of saying it. So let's show a quick demo on how these difference, uh, compute resources. So let's dive into showing how some of these compute resources differ in an actual cluster, and we'll show you a workload that can run across different, uh, compute resources. Sure.

Streamlined Cluster Management

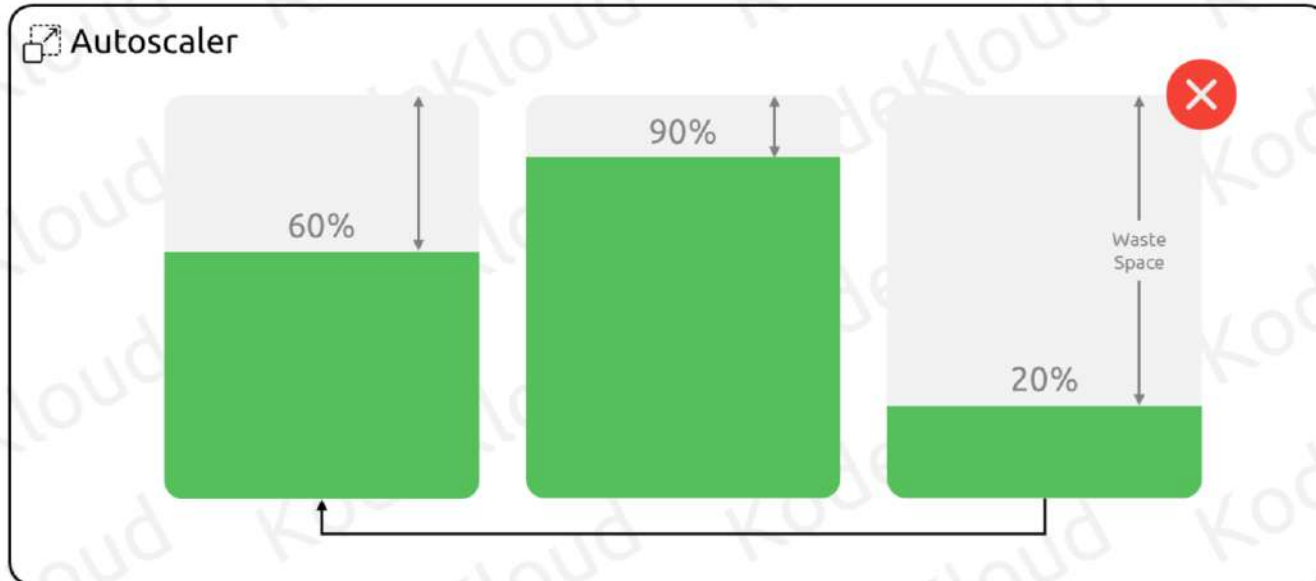


© Copyright KodeKloud

Carpenter's going to do that again in a group list way. So in your cluster, you're not gonna see a node group, you're not gonna see a way to maybe upgrade the cluster as, uh, you normally would with cluster autoscaler or with managed node groups. Instead of being in these groups and rolling out new versions, one at a time per group, carpenter's gonna have to manage that upgrade process. So as you're going through and upgrading an instance, carpenter will look at what's running in the cluster and then say, okay, well, we're gonna go ahead and delete this node. We're gonna drain it. If those workloads can land on existing nodes, then we don't even have to replace it. We don't have to do anything. It's actually great because they're just gonna fit where we had overhead. Uh, and so Carpenter is a little more aware of what the clusters actually

needs.

Consolidation



© Copyright KodeKloud

Another really cool thing about Carpenter is what is called consolidation. And consolidation is going to look at all of the instances as a whole and say, Hey, uh, like Utilization. Carpenter's gonna look at utilization across the cluster, and let's say that's on this node over here, we're using maybe percents. Uh, this one over here, we're using %, and this one's about percents. And normally with the cluster autoscaler, it can also look at this wasted space, this unused capacity of this instance and say, Hey, I don't need that. We're gonna go ahead and scale this down. And then these workloads will see where they fit in the cluster. And so in this case, it would say, oh, well I, it can probably fit over here and we're just gonna go ahead and delete that node and they're gonna end up running on that other node that has spare capacity.

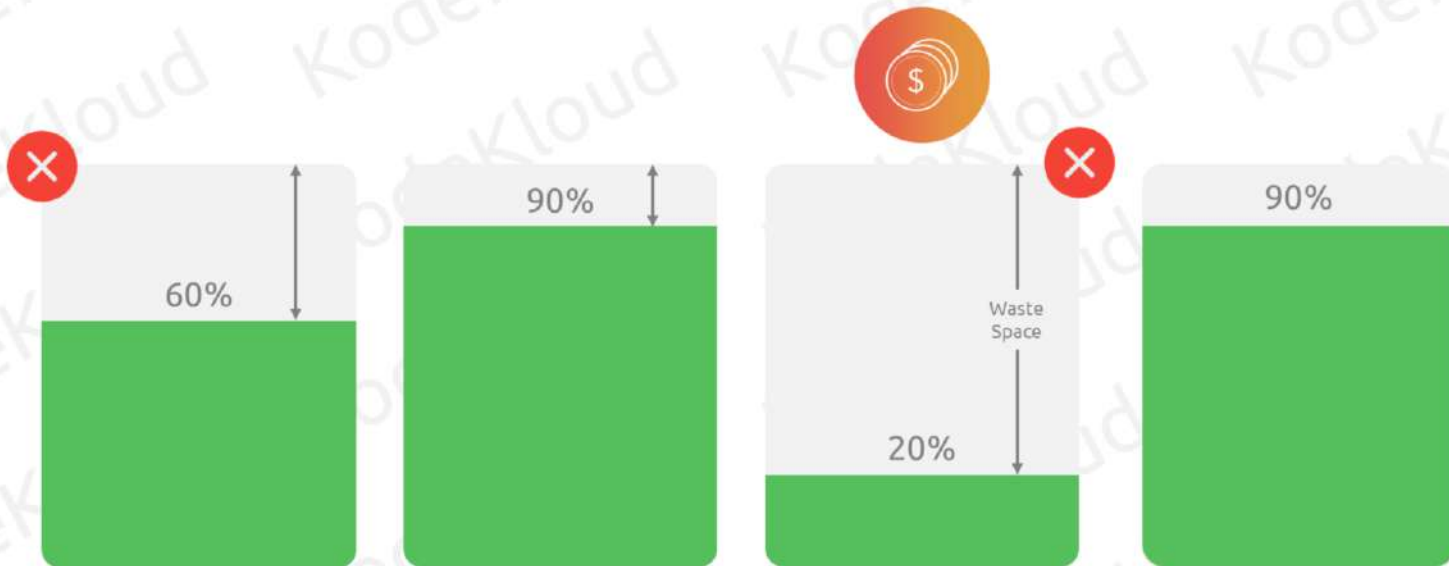
Consolidation



© Copyright KodeKloud

Well, carpenter will do the same thing. It will look at what capacities here, and it can do the same. Hey, we're gonna delete that node and make sure that it goes away. But it also, again, because it knows about cost in the cluster, it knows, Hey, maybe this instance is really expensive and or maybe this instance is actually the most expensive in the cluster. And so we say, Hey, you know what? Instead of deleting, uh, this node on this side that's cheaper, let's get rid of this other node and shift the costs or the workloads over there. Or let's replace both of these nodes.

Consolidation



© Copyright KodeKloud

and fill up a new node to % that is the cheapest possible one that we could run. Carpenter can take this into consideration and it continually looks at what's running in the cluster, what nodes are provisioned, how much they cost, and then it does this calculation with what's available and says, Hey, can we make this cluster cheaper and more efficient? And so it will try to fill up more of the nodes at maybe a higher utilization than cluster autoscaler at the benefit of, Hey, we're gonna use more of what we provision, and we're also going to try to find the cheapest available instance types available. And you can set this consolidation to run at various times. You can set it to run continually. You can change all these things in a Kubernetes Dashboard way. These are all part of the CRDs that are in the node groups. And you say, Hey, I want this to, uh,

provision new nodes as soon as possible or, or only on the weekends Those sorts of things are all values that Carpenter is gonna look at and make sure,



Consolidation

How does this **cluster upgrade**?

How much does it **cost**?

How much is it being **utilized**?

Hey, how does this cluster upgrade? How much does it cost and how much is it being utilized?

Considerations



EKS

These are all benefits of Carpenter inside of an EKS cluster. And to this day, if you're not using Carpenter, uh, you're probably doing more work, uh, than you need to because you're gonna do more configuration. You're gonna try to fine tune some of those node groups, or you're gonna be doing some manual upfronts negotiations with what workloads are running that with Carpenter you may not need to do.

Biggest Downside

Pod Disruption Budget

- Users need to specify a desired percentage of workload stability within Karpenter.
- Users must adapt to Karpenter's dynamic workload movements.

Topology Spreads

- Users need to define topology spreads to enable workload distribution across various Availability Zones (AZs) and nodes.
- Users must balance cost and availability by specifying workload distribution across AZs and nodes.

Resource Requests

- Specify CPU and memory requirements for your workloads to ensure proper resource allocation.
- Implement pod disruption budgets, topology spreads, and resource requests to enhance workload stability.

The biggest downside or warning about Carpenter is your workloads need to be mature. And by that I mean you need to have some things built into your workloads that allow it to be reprovision and moved around a little more frequently than you might need to on servers. So things like, uh, pod disruption budgets are a big, are a big thing that you will need inside a carpenter where you say, Hey, I want to make sure I always have some percentage of this workload running because Carpenter is going to move things around maybe a little more than you were used to. Uh, things like, uh, topology spreads inside your workloads are things, uh, to allow multiple workloads to schedule across different AZs in different nodes. By default, if carpenter's looking for the cheapest way to run your workload and you don't specify any topology spread, it'll

run all of your workloads on one instance because one instance is the absolute cheapest way that you can run any workload and it'll say, Hey, you get one large instance and everything goes there. But if you want high availability across AZs or across nodes, you have to define that in your workload to make sure that you have the high availability and trade off between cost and availability. And the last thing is, uh, resource requests. Resource requests for how much CPU, how much memory those things need to be defined in your workloads. If you're coming in with a workload that has no pod disruption budgets and no topology spreads and no resource requests, carpenter will schedule that super fast and I guarantee you'll have an outage because at some points, carpenter will remove the node or it will, uh, try to schedule other things on top of that node because you haven't requested any resources to the node. And so it's going to just fit whatever you're saying I need. So you have to move some of this stuff that maybe you got by default by using the cluster autoscaler because it would by default spread across AZs, or maybe you got it by default with the Kubernetes scheduler because it would try to spread workloads across nodes. But with Carpenter being the one responsible to saying what's available in the cluster and what nodes are available, those things have to be pulled into your application and your application needs to be a little more mature about what it requires to run effectively.

Best Practices

01

Abundance of Compute Options

- EKS offers various compute options for managing clusters, providing flexibility in cluster management.

02

Ease of Deployment

- Karpenter is often the simplest option to begin with, offering quick deployment and scalability.

03

Responsiveness to Changes

- Karpenter swiftly adapts to cluster modifications, ensuring responsiveness to evolving requirements.

04

Cluster Service Integration

- As a cluster service, Karpenter's seamless integration with managed services like Fargate ensures stability and convenience in operation.

So all, all of these different compute options are available in EKS, and there are options that you can decide how you want to manage the cluster. Karpenter's usually the easiest to get started and the one that can scale and react really quickly to changes inside the cluster. Uh, but also Karpenter is a cluster service, and so you probably want cluster running in Fargate or managed, uh, as, as a managed service so that you don't have to worry about Karpenter being moved around between these instances.

Conclusion

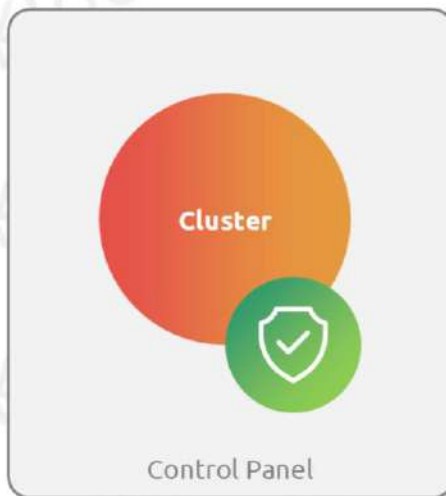


Compute Resources

So let's dive into showing a little differences on how any of these workloads might get scheduled to these types of nodes with all three different types. That wasn't a good way of saying it. So let's show a quick demo on how these difference, uh, compute resources. So let's dive into showing how some of these compute resources differ in an actual cluster, and we'll show you a workload that can run across different, uh, compute resources. Sure.

Cluster Access

EKS Cluster Security – Introduction



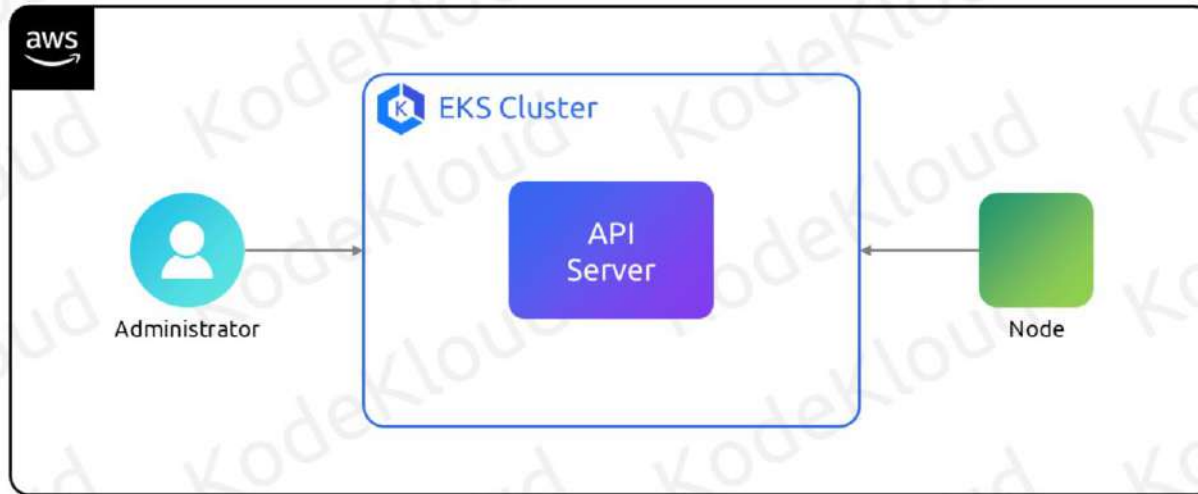
Okay, we need to talk about the security of your cluster, your control plane specifically.

EKS Cluster Security – Introduction



And that has evolved a little bit over time with eks. And in this section, we're going to show you how that's changed over time and how you're going to go about managing that. But first, let's just show you what is in, or what we're talking about when we say an EKS cluster and securing that.

EKS Cluster – Components

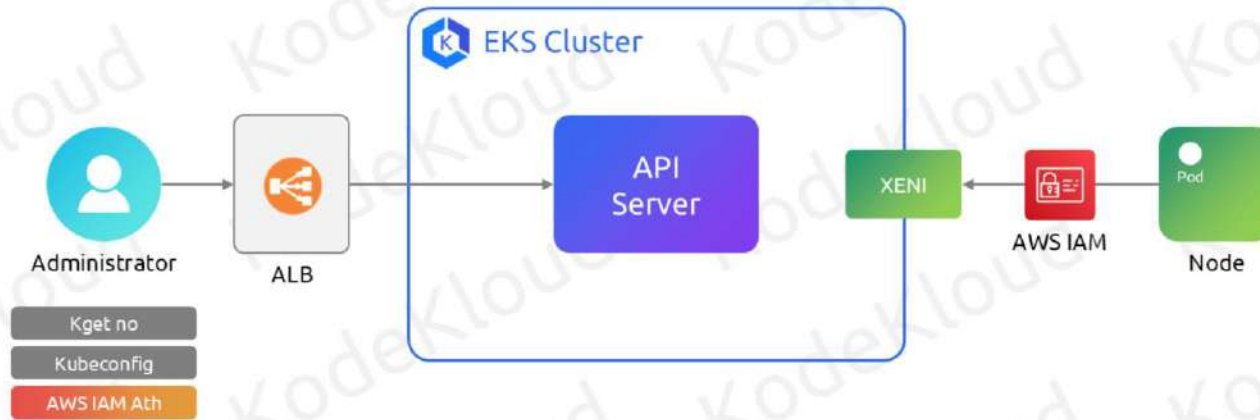


© Copyright KodeKloud

Okay, so here is our eks cluster. And when we create this, it is living inside of an Amazon account. You don't have access to the control plane portion of it, which includes things like the API server. And that API server is actually kind of important.

You need to be able to talk to that. So do your nodes, your nodes are live in your accounts and they need to call out to that. And you as an administrator also need to make calls to that with things like Kube control when you're, when you're querying things in the cluster.

EKS Cluster – Traffic Management

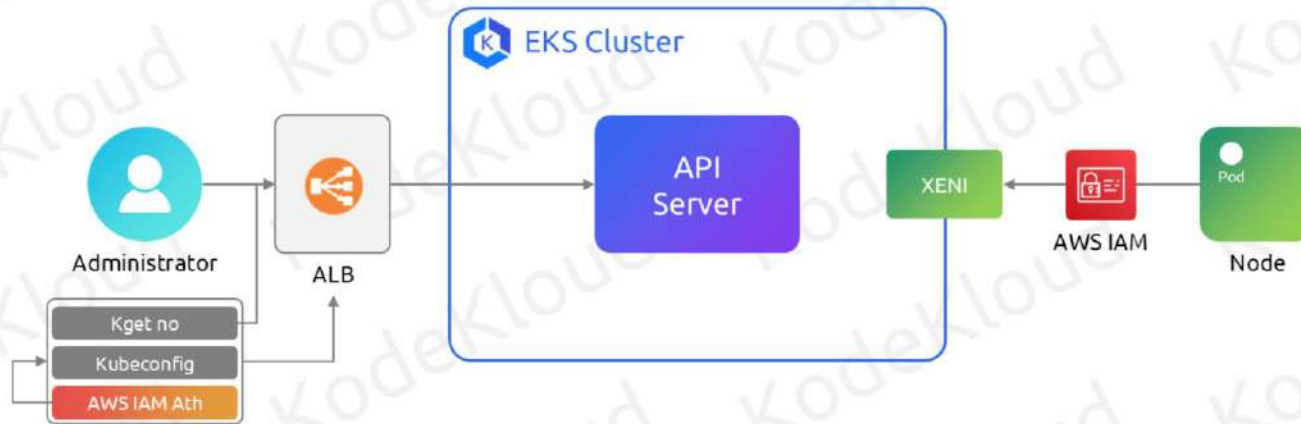


© Copyright KodeKloud

But how does traffic get in and out from that point? Well, we've already talked about in the intro how a node in the cluster and pods and workloads and all that stuff that runs inside your accounts, they go through an Xeni which is a network attached to your VPC. It's a network port basically that they all talk to this API service and they have IAM credentials that allow them to do that. Well, when you create an EKS cluster, what actually ends up happening is you get an ALB, an application load balancer in the front of your EKS control plane that Amazon manages. This is managed for you with permissions that allow things to talk to the API server. And so when you call out, if you do like a Kubectl get nodes, let's say, and when you actually are calling to the API that is defined inside of your kube config, there's a setting in there for how

you're being authenticated. And there is a plugin called the AWS iam authenticator.

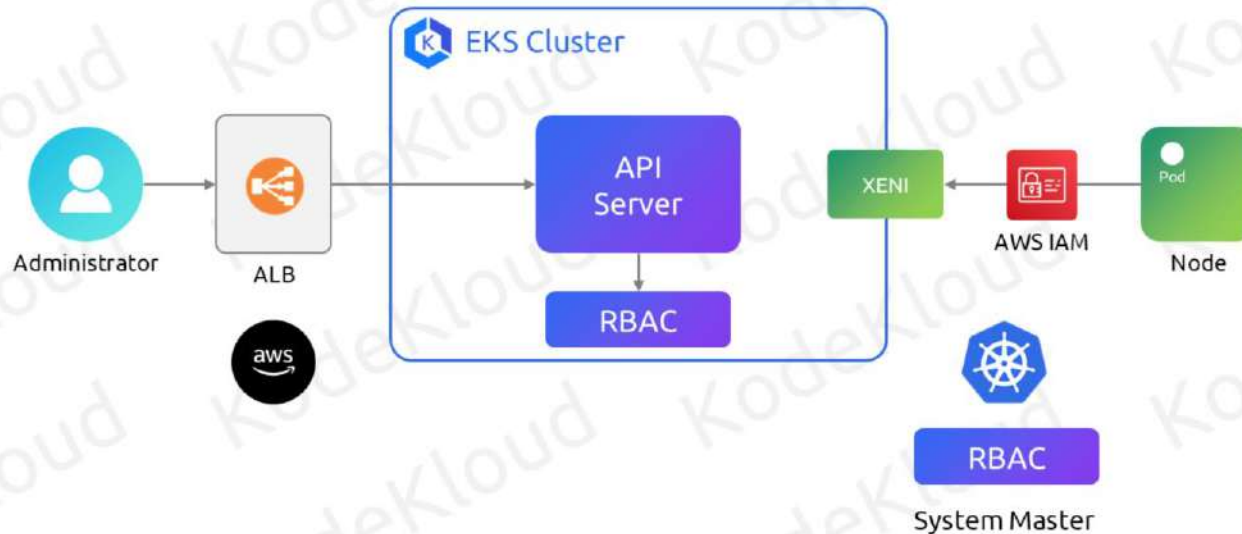
Authentication Process



© Copyright KodeKloud

And so when you make that call, this gets executed and will return credentials for you to call through that ALB so that you will present your token or whatever it is. However you're authenticating to AWS. This could be single sign on, whatever that may be. AWS IAM authenticator returns those credentials up to Kubectl and then you send that over to the ALB. And then that allows you or doesn't allow you to get into the API. So if someone's externally, because this is ALBs are public on the Internet, this endpoint is available publicly for the world to see. And the thing that protects it is AWS authentication. Just like every other AWS API, it needs authentication to be able to access it.

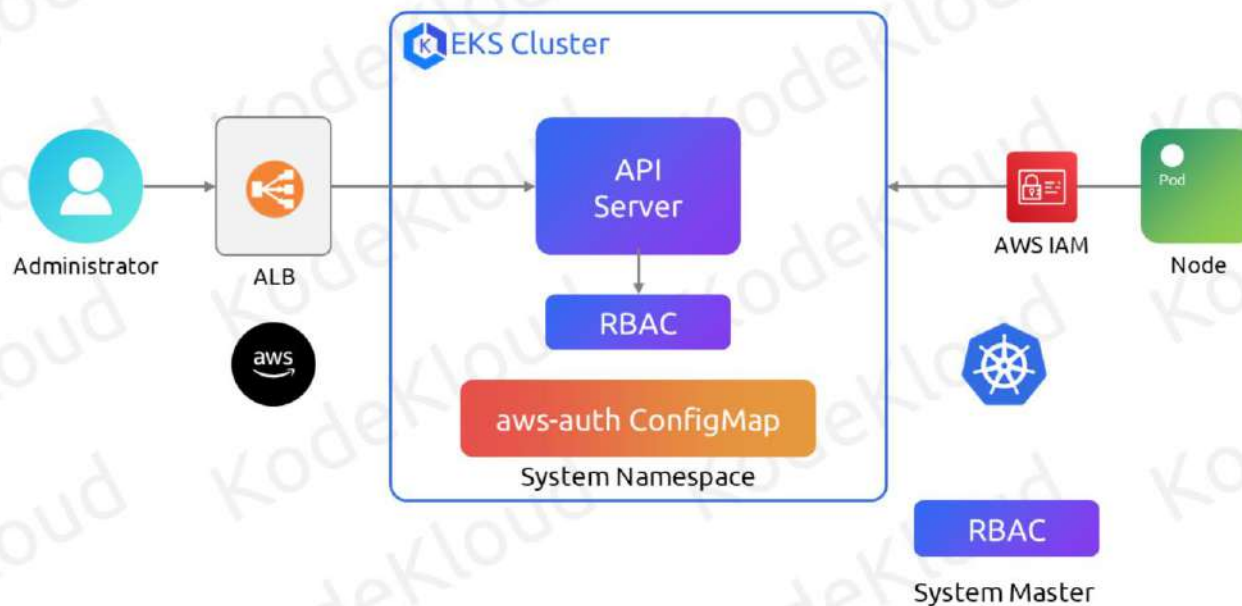
A Bridge Between AWS and Kubernetes Authentication



© Copyright KodeKloud

But just like a lot of things inside of any Kubernetes cluster and living inside of AWS, we have to bridge this gap between AWS authentication and Kubernetes authentication. So inside the cluster we're dealing with things like RBAC. And that has its own mappings of roles and permissions for things like say system master is a role inside of your cluster that can do pretty much anything. It's like a root account inside of your cluster, but who has access to that? Who should have access to that? When I call the AWS alb and say, add a node, it needs to know what I'm mapping my user account to internally to RBAC and how that actually works. So there needs to be something there that does that binding that remaps an AWS arn or identity of some sort, an image account to an internal RBAC rule and permissions.

AWS Auth Configuration



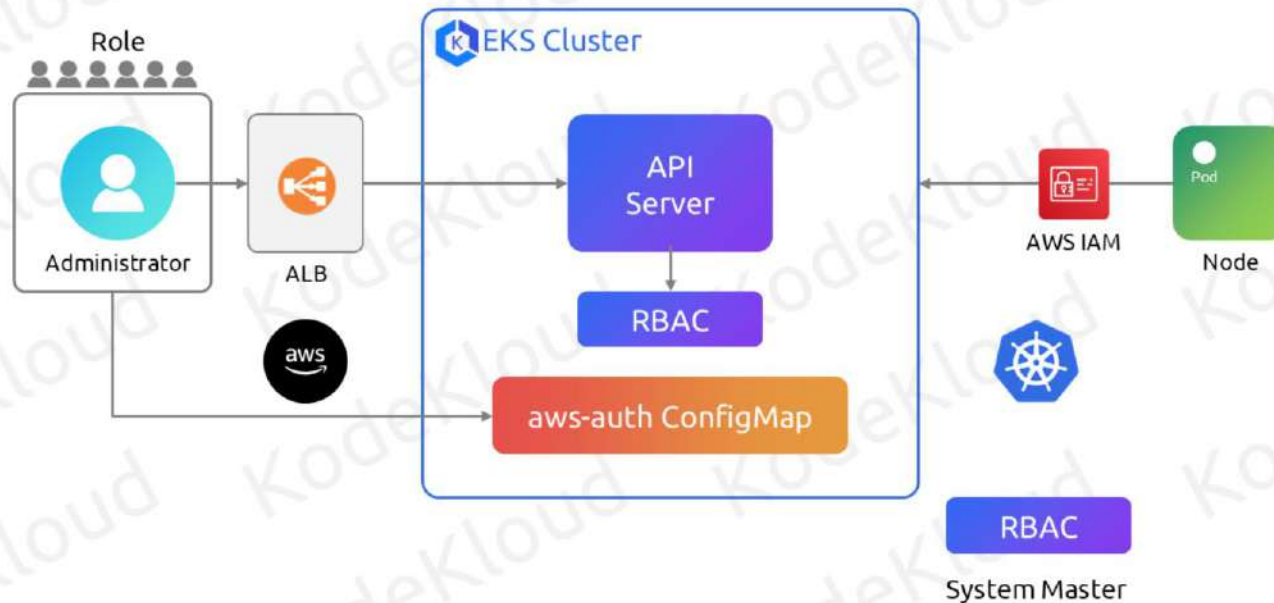
© Copyright KodeKloud

And so the first thing that AWS did was they actually created this thing called AWS Auth. And AWS Auth was a config map that lived inside your cluster, in the system namespace. And it contained all of those mappings. Anything that you wanted to map from an IM role or a user or whatever it was, to internal RBAC would be defined here in AWS Auth. And this was okay because it gave us kind of a Kubernetes way of dealing with who could do things.

But it was a little bit awkward because IAM is outside of the cluster. And if you're on a big team and you don't manage IAM, or maybe you don't even manage creation of IAM or clusters, you had to go figure out, okay, what Arn is, do I need, and

what identity and what role, and then you could manage the things internal in Kubernetes.

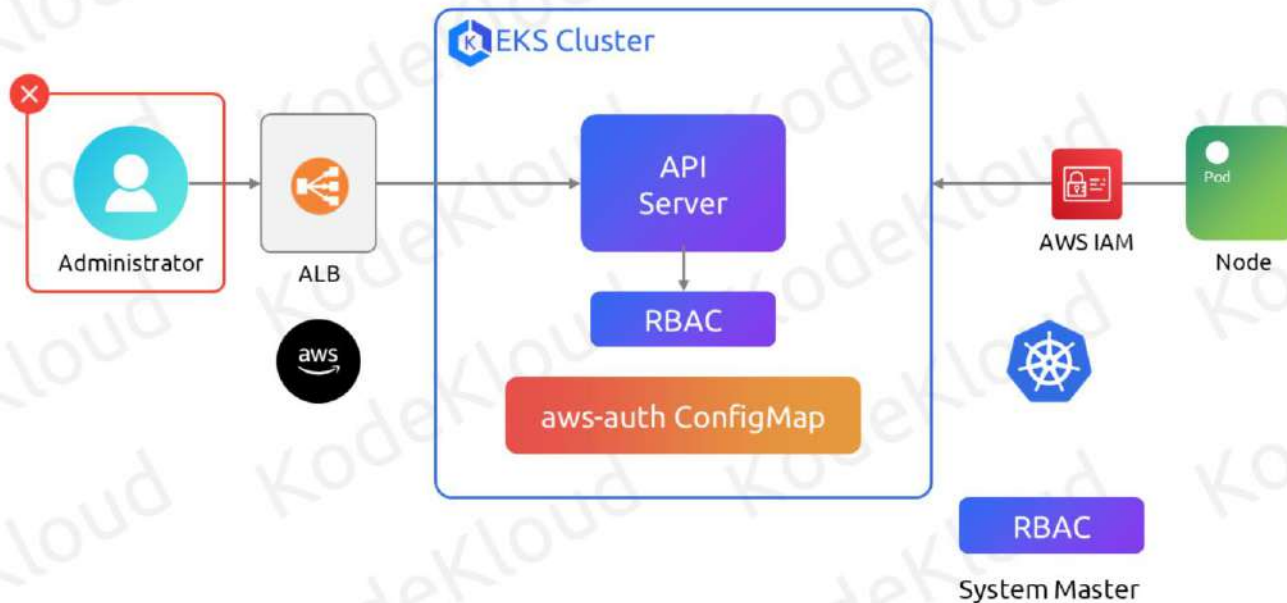
AWS Auth – Drawbacks



© Copyright KodeKloud

There also was some other oddities with this, because whoever created the cluster, this user out here was always given cluster admin privileges. This was like you could do anything to the cluster as long as you were the one that created it. And there was no way to map that into this AWS auth file or remove it. And so oftentimes it was always recommended that you would create clusters with a role which multiple users could be mapped into. And then if someone left the team or doesn't, shouldn't have access anymore, they should be removed from that role. Because if someone did leave or switch teams, they would always have access if they were the ones that created it.

Transition to Cluster Access APIs

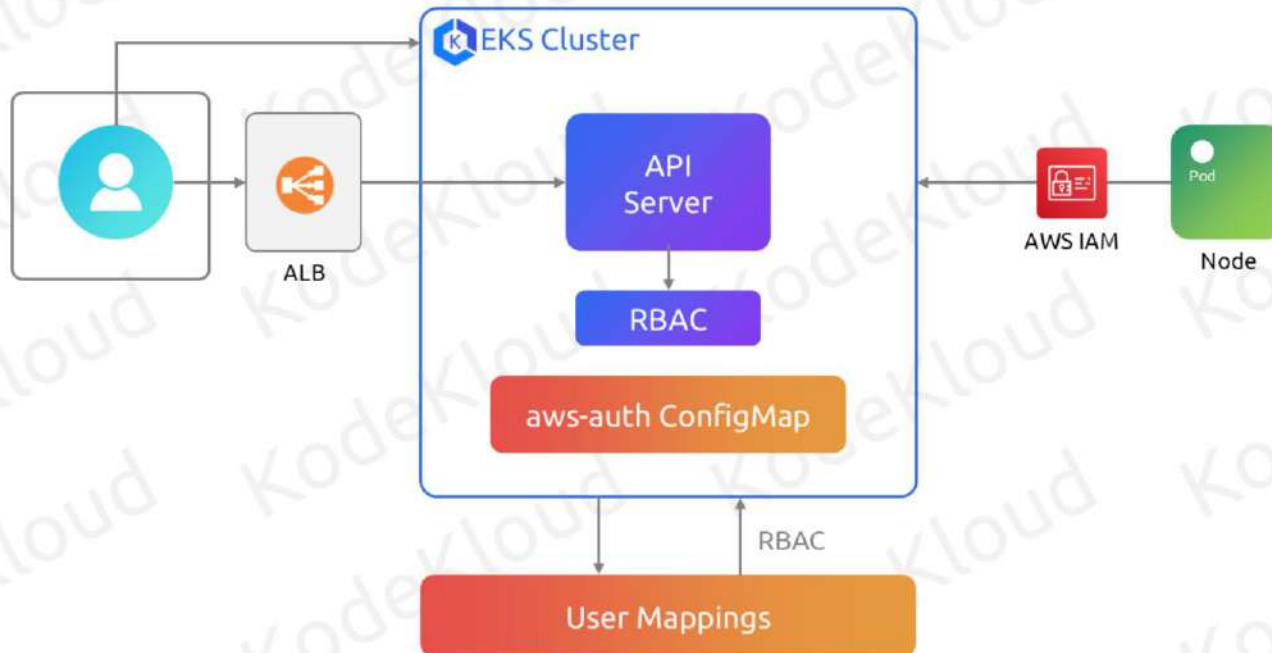


© Copyright KodeKloud

And also if this person, if that, and also if that IAM account or role was deleted, then you don't have admin rights to your cluster anymore. There's no way to get that back.

There was no way to actually restore access without opening a ticket and asking someone from AWS, hey, could you change this? And so things were changed from AWS Auth into the new cluster access APIs.

Cluster Access APIs



© Copyright KodeKloud

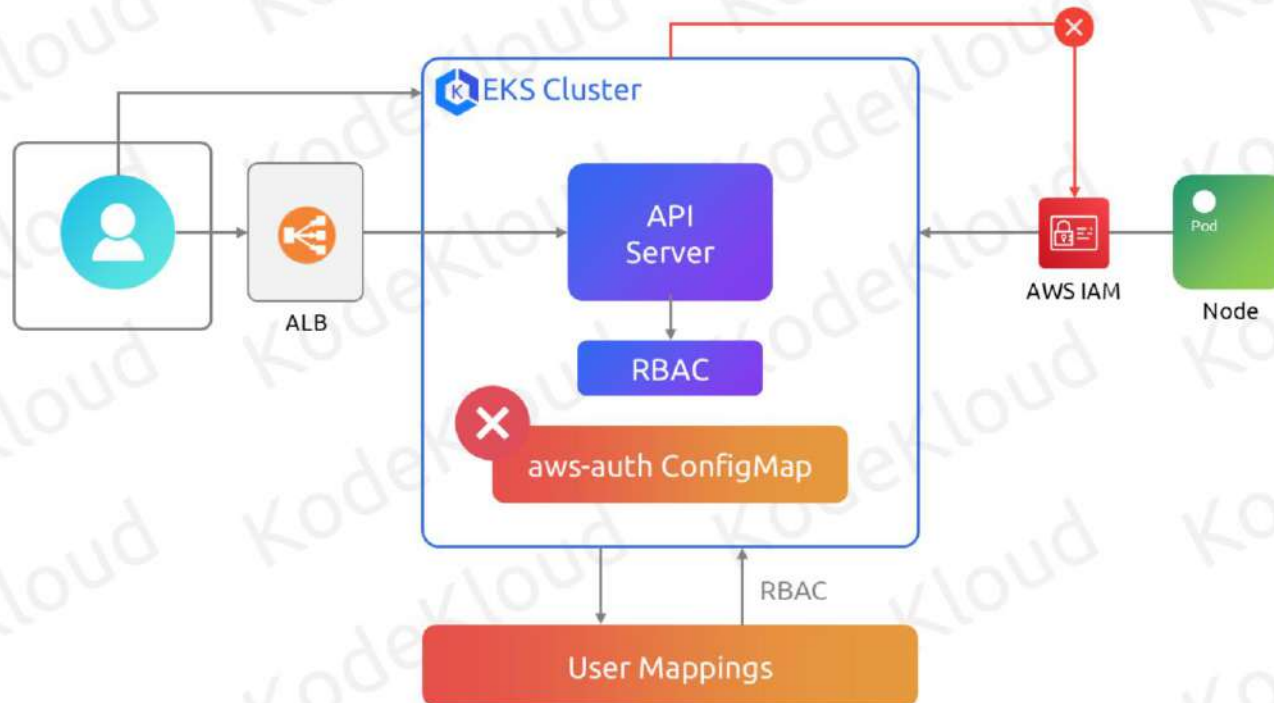
And what cluster access APIs does is it actually takes away this aws-auth file. It actually still exists and it still can be used, but it's not the way you should be using it. So AWS Auth is there, but it's not really what we'd recommend anymore. Instead, this AWS auth is part of the EKS API. So we have a way in the EKS API to add user mappings. So that when I call out to the ALB, this is so when I call to the ALB looking for my cluster, it actually can talk to the EKS API, make sure that I map to the correct internal RBAC role, and then I can do things in the cluster.

And now all of my mappings can be handled declaratively on the EKS mapping on the now all of my mappings of who should

be able to do what in the cluster is handled declaratively on the EKS cluster access APIs instead of handling it through this AWS auth bridge that would kind of get synced to the backend to allow things to happen. And config maps work okay, but if you messed up the the other nice thing is config maps were just standard text, and if you messed up the formatting of that file, it could also break things. And so this AWS auth wasn't always the safest thing if you were trying to automate it or script it. And so people ended up writing wrappers to try to make sure that it was linted properly and test it. But one time that you mistyped something or changed the wrong value, you could lock yourself out with the EKS API.

Now, if I'm the one that creates the cluster, I don't necessarily have cluster admin. We can actually delegate that to a different user or set of users or roles to have cluster admin access. So a team could be responsible for creating clusters, another team for administering clusters.

Native IAM Integration – Advantages



© Copyright KodeKloud

All of this moves that authentication layer, which is mostly handled in IAM, into a native interface, into eks itself. And that is really helpful because having it inside of a or inside of eks or inside the Kubernetes cluster and then trying to map that back out to IAM was a little more difficult than having that be a native IAM and then map that into Kubernetes resources or have that map into Kubernetes users.

Conclusion

01

Manages cluster access
for resource control.

02

Nodes and IAM need
AWS API access.

03

Authentication via load
balancer links IAM with
Kubernetes roles.

04

New cluster access APIs
replace old methods.

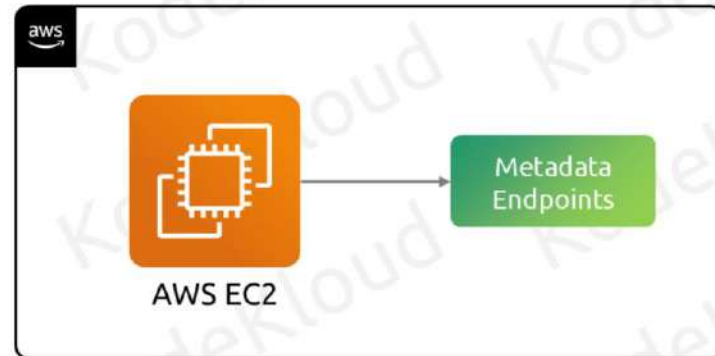
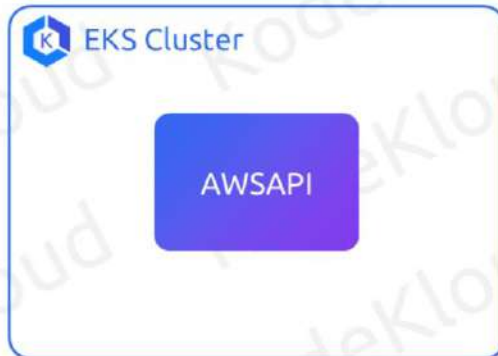
So just to recap, recap, so just to recap real quick, what we've talked about so far is just how a cluster access works. And this is for a user and a cluster administrator to actually manage resources and who can do what. There are things like nodes that'll need access in RBAC, but also an IAMb to be able to call the AWS API and also add nodes inside of eks. And so those nodes need a way to map things. And we usually get that through roles on the EC, two instances. How does that map internally?

I'm just thinking. And we use, and we usually get that from roles and IAM profiles on the EC, two instances that allow it to

make those calls. But to recap, the authentication is all handled through this application load balancer and mapping from AWS IAM into Kubernetes roles. And that mapping can happen either through the AWS auth config map, which isn't really recommended anymore, although it still exists, or through the cluster access API as of recording. This cluster access is fairly new, so you need to make sure that you have a new version of EKS. And you might need to change some tooling like terraform or however you're creating it, but it is the way you should be managing your access for IAM users, being able to do Kubernetes things in an EKS cluster going forward.

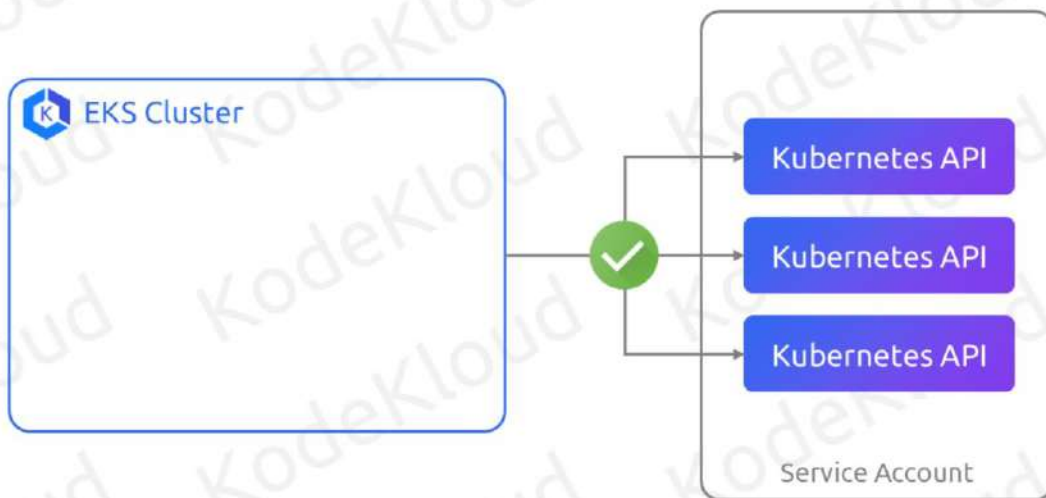
IAM Roles for Service Accounts (IRSA)

AWSAPI Access in Kubernetes – Introduction



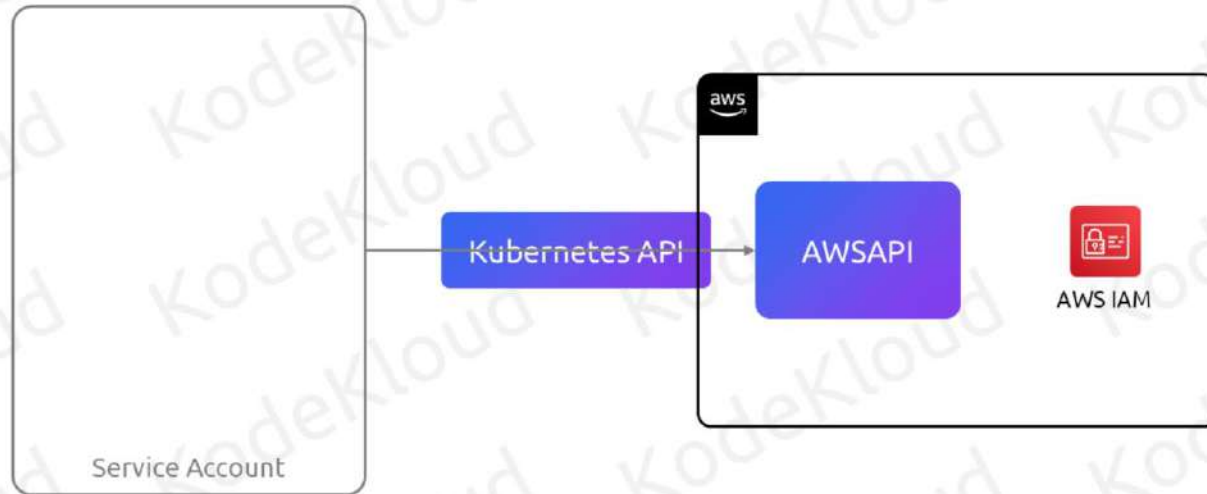
We just talked about the first option of how a pod or a workload inside of Kubernetes cluster can access an AWSAPI with something like Cube to im And that has really, that has really close similarities with how EC instances traditionally have access to metadata endpoints to get credentials to go talk to something else within AWS

Downsides of Direct Annotations



Now, the downside of that is adding annotations to directly to workloads for what permissions they need isn't the typical way that Kubernetes workloads go about accessing other Kubernetes APIs. In many cases, they want to use what's called a service account. And a service account is what gets mounted into the workload to give it those credentials. And it says, Hey, I'm gonna give you a file on disc, typically, or it could be an environment variable or whatever that says, here's your service account credentials, and you use this to call the Kubernetes API.

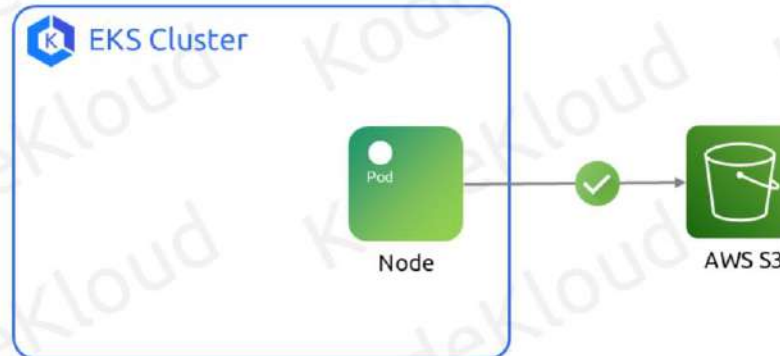
IAM Roles for Service Accounts (IRSA) – Introduction



So now it had this weird mix of, oh, if I wanna talk, talk to a Kubernetes API, I put that in a service account. But if I want to talk to an AWSAPI, I have to put that as an annotation on the workload. So AWS created, So AWS created IAM roles for service accounts. Now let's explain how that one works.

IAM Roles for Service Accounts (IRSA) – Introduction

IRSA



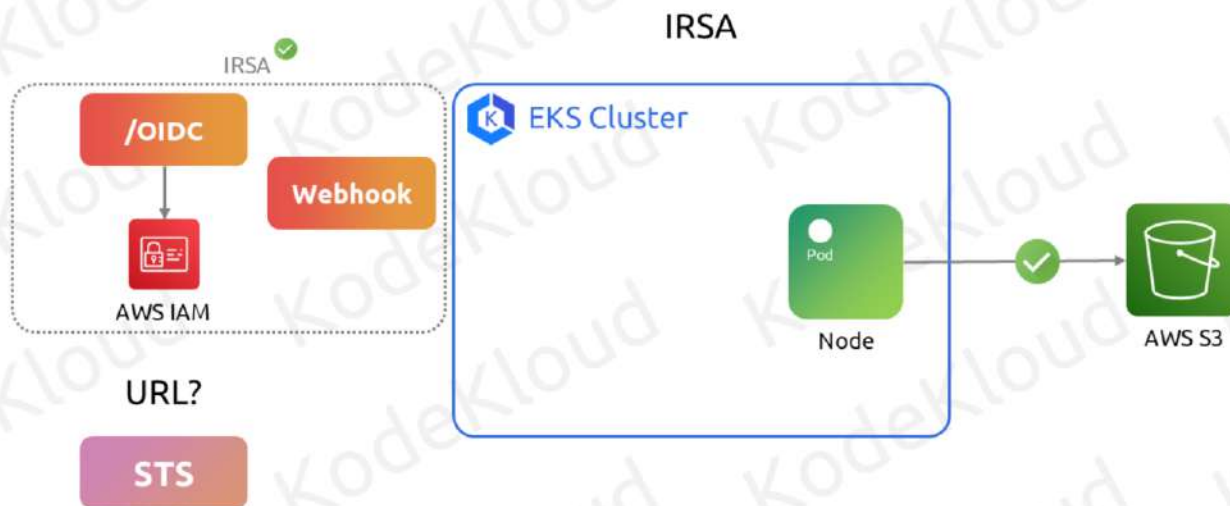
Okay, so let's talk about, I, I am roles for service accounts. We're just gonna call it Ursa because that's what most people call it. And we're gonna go through the same scenario of, I have a pod on a node, and it needs to access an S buckets, and it needs to get permissions to do so because it's calling an AWSAPI from within a Kubernetes cluster.

IRSA – Setup Process



And so a couple things we need to do here that are gonna be different than, uh, the cube to IM is, first of all, we need to have an external OIDC endpoints. And this comes by default with an EKS cluster. Uh, one one thing to note is Ursa can work with non EKS clusters, uh, but you have to run a few extra things yourself. Um, so we're gonna talk about it with just EKS, which gives you an OIDC endpoints, uh, to, to be able to get, uh, tokens, jot tokens, uh, for when you're requesting these things. Um, so that's already gonna be there as part of an EKS cluster, and you don't need, really need to worry about it.

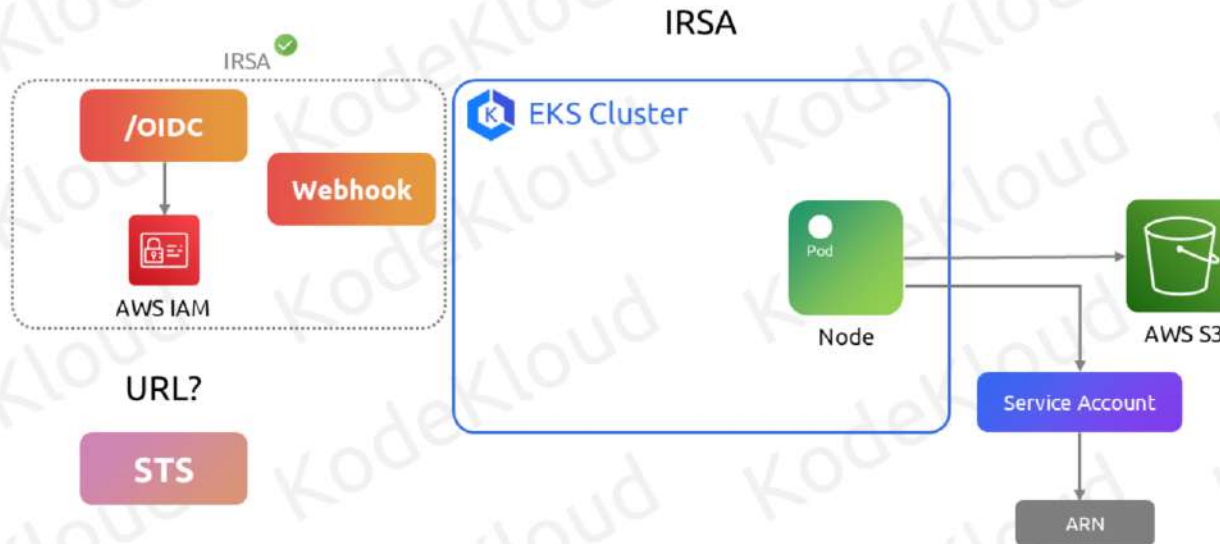
Webhook Setup and Service Account Annotation



Uh, but you do need to know, um, where it is. You need to know the URL for it. Uh, because when after your clusters set up, you need to take this OIDC endpoint, which is gonna be AAURL, and you're going to put it as part of your IAM, whichever role that, uh, this pod here needs to call. So it's gonna be a trusted relationship, uh, on the IAM credentials. That's, is, is going to be allowed to call STS to assume role, um, based on the JWT token. The OIDC endpoints can give me it again for some reason. Uh, this is more confusing than just standard IAM. So, uh, we need to really be careful about how this, how we walk through some of these steps. But this is just setting it up. We need the OIDC endpoints and we need the IAM role to have a, a trusted relationship with this OIDC endpoint 'cause

OIDC is gonna give us tokens and, and STS the other AWS service, um, STS is going to have to validate that that is, is true. That what OIDC, the JWT token it gave us, is valid for us to be able to call an API. So those are two things that are coming, uh, with the cluster that we need to set up beforehand. This IAM account, we need to have that trusted relationship. The other thing we need to do is we're gonna have a, a pod, a web hook, a mutating web hook. And this is gonna run inside the EKS control plane. Uh, so this is outside of your cluster. If you're running a non EKS cluster, this would run inside your cluster. But, uh, for this, when we have IRSA enabled, our EKS cluster is gonna get this webhook for us. We don't need to worry about it. Uh, but it's how we are going to get some, uh, extra credentials in our workload to make sure that we are calling the, the right endpoint.

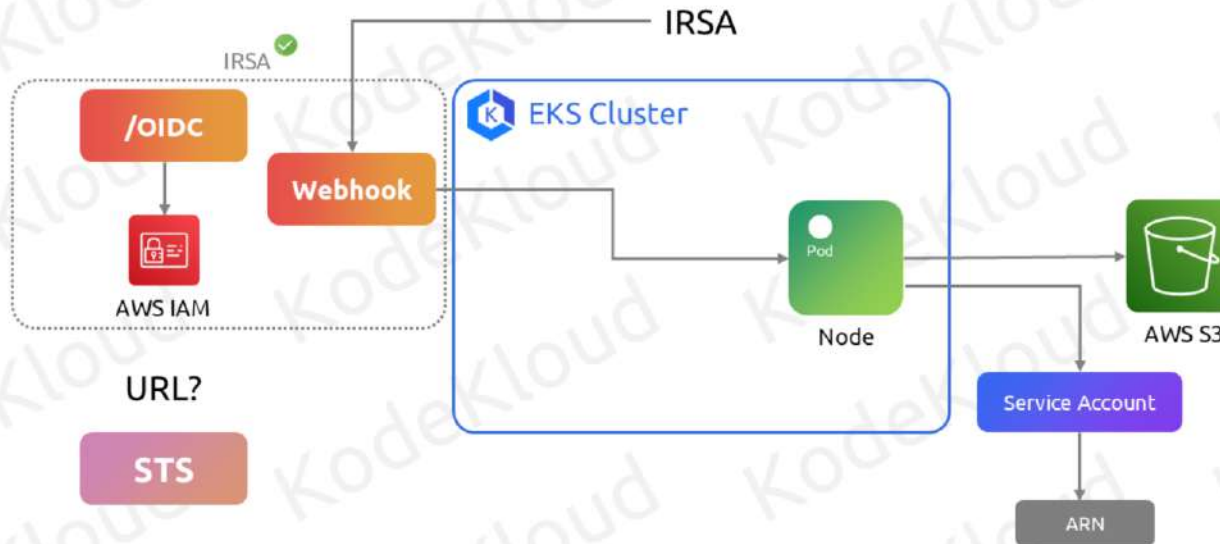
IRSA – Challenges and Limitations



And the last thing we need to do is we actually need to set up the service account and that service account is going to have an annotation. Remember with kubed, IM, we put this annotation on the workload, uh, which is awkward because inside of Kubernetes we use service accounts. Service accounts are how we call Kubernetes APIs, and that's how IRSA works. Uh, but then it became awkward to have like these, these annotations split with different permissions in different places. So this service account is going to have an annotation for the ARN of the IAM that we want to assume. And, and that's gonna be put in the service account. And then your workload is going to reference a service account, say, Hey, I need that service account. That service account references the IAM and that IAM is how I'm going to actually eventually call this S3.

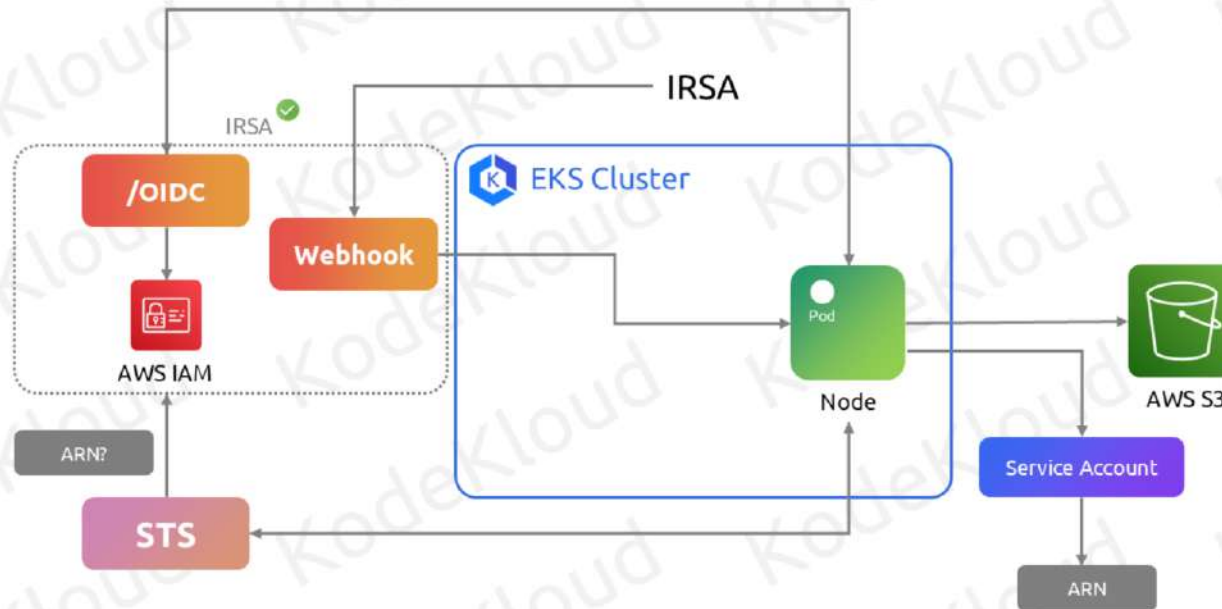
bucket

Scalability Issues and Alternative Solutions



So we submit our workload to the Kubernetes API, uh, that gets mutated by this mutating web hook and gets some environment variables shoved into it that say, uh, how the SDK that's running in our workload should call out and get some credentials to call our bucket

Complexity and Recommendation

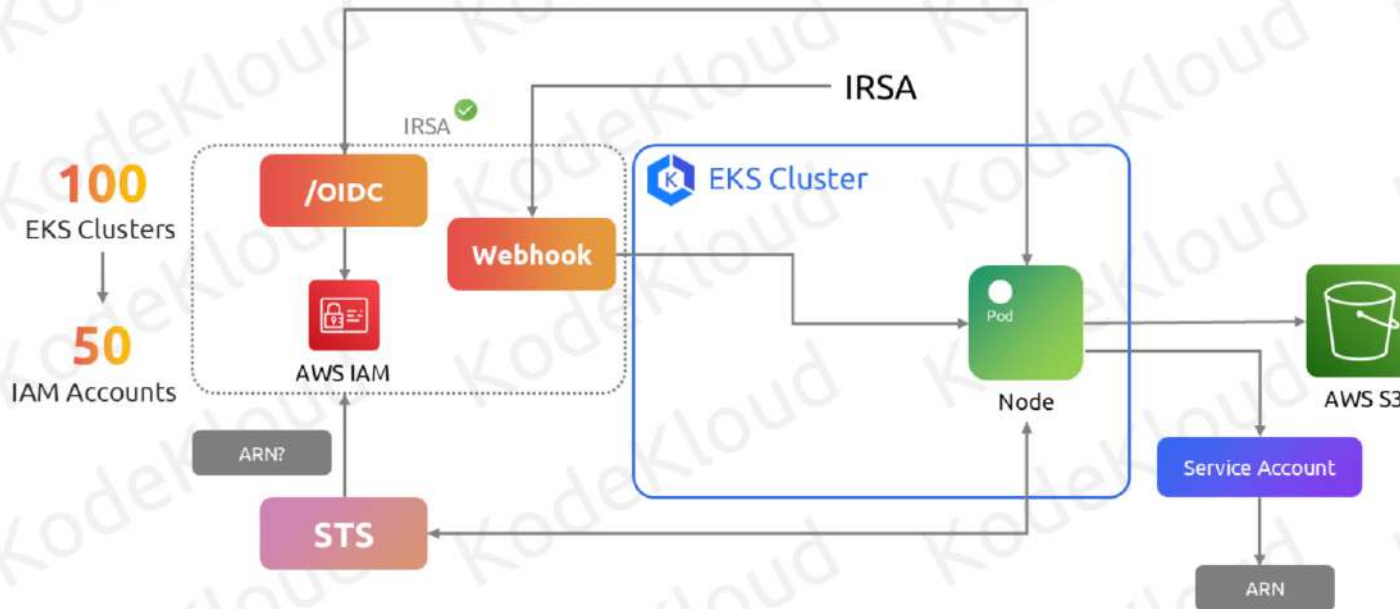


One of those things that it's gonna get is the endpoint for the OIDC, for getting a Jot token. So we're gonna take some credentials from this workload, and we are going to call out to OIDC and say, Hey, I need a token because I actually need permissions. It's gonna return a token to us. And then we're gonna take that token and, and our other information that we have. Um, this, by the way, this I am has other things like what namespace, what cluster, uh, where I'm calling it from. So that's all part of the IAM spec. So I need to know ahead of time where this workload's gonna come from. That workload gets that job token and then it calls STS and says, Hey, I need to assume this role. Um, this is the role that I need. But it doesn't really know. It just says, here's my job token. Um, STS validates with Iam, Hey, which cred, which I am, which I am

arn, do I actually need to return to this workload? And it also verifies that it's trusted with the OIDC 'cause it's getting that jot token, that jaw token sign and says, Hey, this is my endpoints, uh, this is who I am Uh, and then the workload goes back and says, Hey, I got this from this endpoint over here Uh, can you gimme the actual credentials? Um, so SD SDS validates that and returns back the credentials for I am for the actual assume role So now this workload has a jot token that it's doesn't need anymore 'cause it uses SSTs for that It will use it for, uh, revalidating and getting new permissions as they expire But the jot token is, is for the workload now, and it uses this assume role to call the S bucket

This was a lot of things calling a lot of other things Uh, this is not a simple process This is not something that was, uh, oh, let's make EC metadata endpoints And uh, EC I am easy, uh, because all of these things work on standard EKS clusters, but like I said, also non EKS clusters So if you're running your own control plane, you can run these components, uh, and integrate it yourself OIDC is something that commonly large companies have their own IIDC endpoints and they trust those in their I AM accounts And so they can say, Hey, uh, I have an OIDC endpoints and I wanna trust that one And you fetch your credentials from here

Scalability Concerns and Limitations



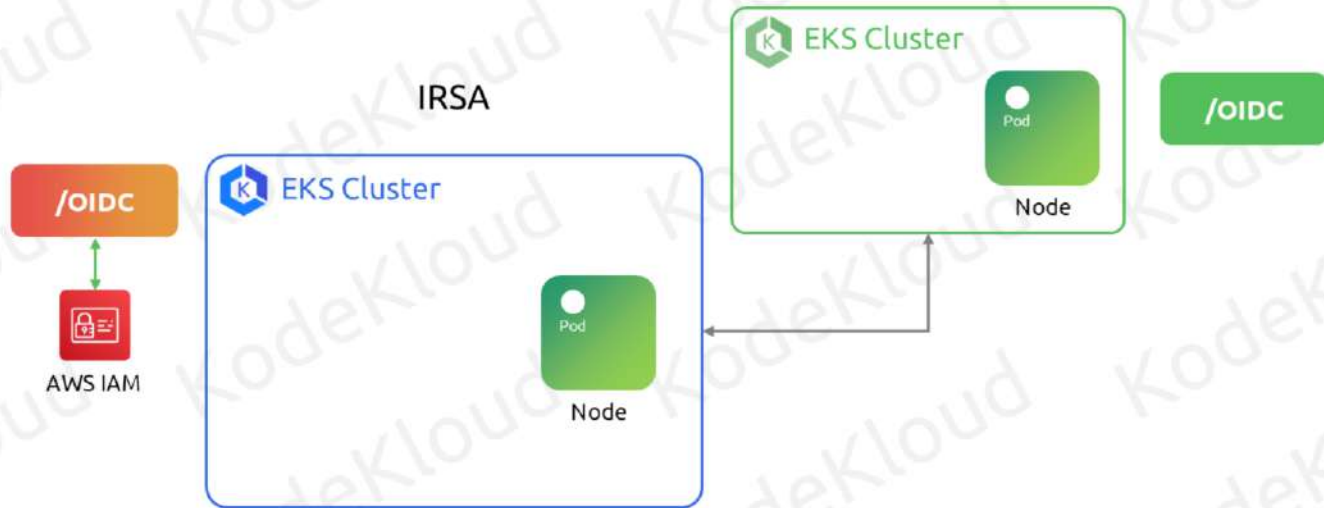
© Copyright KodeKloud

Now, some of the downsides of using SA inside of uh, accounts is things like the OIDC endpoints. Um, there's actually a limit on your AWS account, how many of those you can have, and that limit is. And so if you have more than say, EKS clusters, you're gonna hit OIDC limits and you're gonna have a, a problem of how do I make sure that, uh, I'm not, I can have more web hooks and endpoints for my pods to access them.

Uh, another problem is these, I am, because there's this trusted relationship between I am and OIDC, I don't actually necessarily know this OIDC endpoint until my cluster is ready. And so I can't precreate or pret trusts a bunch of I am roles.

until my cluster is available. So a workflow of say, I need to provision a cluster and I know I need these IM accounts, uh, I might need to wait and I need to know what the OIDC endpoint is.

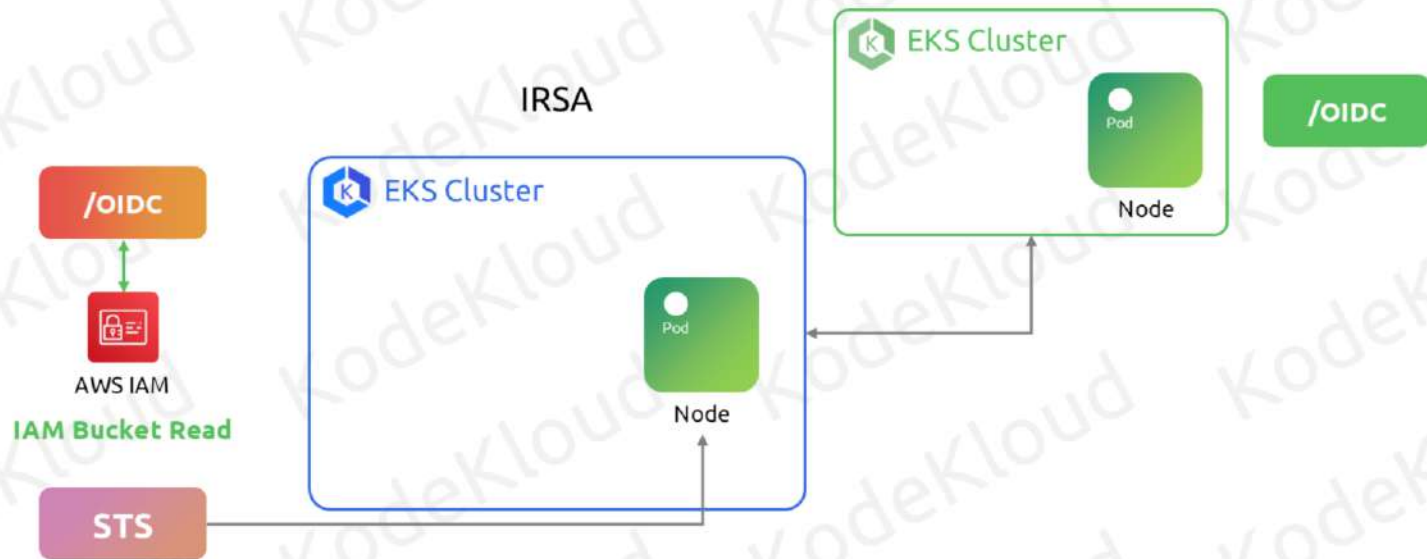
Management and Maintenance Challenges



© Copyright KodeKloud

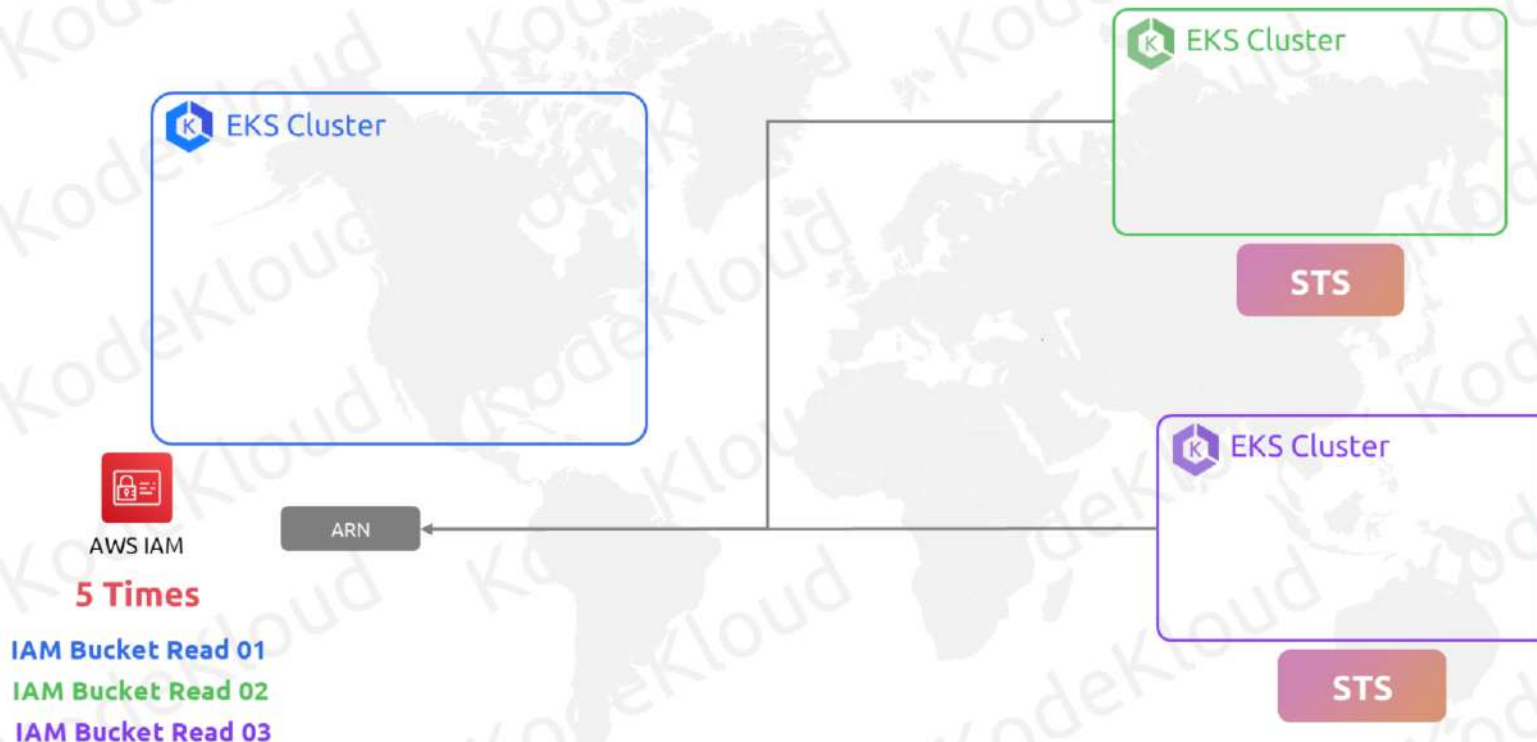
Also, if you do something like, uh, blue green cluster upgrades where you, uh, take an old cluster and then you, you build a new cluster and you run workloads in that, and then you migrate all of your like DNS and load balancers, uh, from one cluster to the other one, um, you now need to know the OIDC on this other cluster and that trust relationship for that second cluster also needs to be updated on all of your I am, uh, ARNs. And so you, you now need to trust two OIDC endpoints. Uh, and, and maybe that's okay, but like you'll need to know when and how to roll those things and how to change them.

Management and Maintenance Challenges



Uh, but let's say I have like a generic, I am a bucket read and I say, I just want that one to work for any, any workload on any cluster that needs to read, needs to read a buckets And maybe that's okay for, you know, these two clusters you have

Management and Maintenance Challenges



© Copyright KodeKloud

Uh, but IMS Global IM is not a regional service. So if you have more clusters in more regions, and they also are going to use that same ARN to do their bucket reads, um, there's, you start to get a limitation on how many of these, uh, trusted relationships and JWT tokens, you can actually send STS. And it turns out that limit is five. And so for every five times you want to reuse this IAM credential, uh, you need to create a new one. And so you're gonna have like a bucket read one and a bucket read two, and a bucket read three if you have multiple, uh, clusters and workloads that you want to keep reusing this. And also remember, um, those IAM credentials should be scoped to a namespace and a cluster. Um, they don't have to be, you can, uh, allow them to be broader, but again, that, that then broadens your permissions for these workloads. Uh, so

SA in general is, is something that was integrated and starts using surface accounts, but it was, it was very complex and it is not something I, uh, often would recommend. It's like if you have to do it this way, uh, and you don't have large scale or a lot of clusters or a lot of workloads and a lot of I am, um, maybe you don't want to do this. And, and so I often would recommend people go back to c to I am because they understood a little bit how that worked. It was EC endpoint or EC metadata, uh, which might have too many permissions on the node, uh, but the, the proxy was still able to kind of mix and match some of those things. The thing that came after Ursa is pod identity and pod identity is much more integrated in EKS. And we're gonna show how that works in the next section because it's a much cleaner solution.

Alternative Solutions and Recommendations



The thing that came after Ursa is pod identity and pod identity is much more integrated in EKS. And we're gonna show how that works in the next section because it's a much cleaner solution.

Pod Identity

Introduction

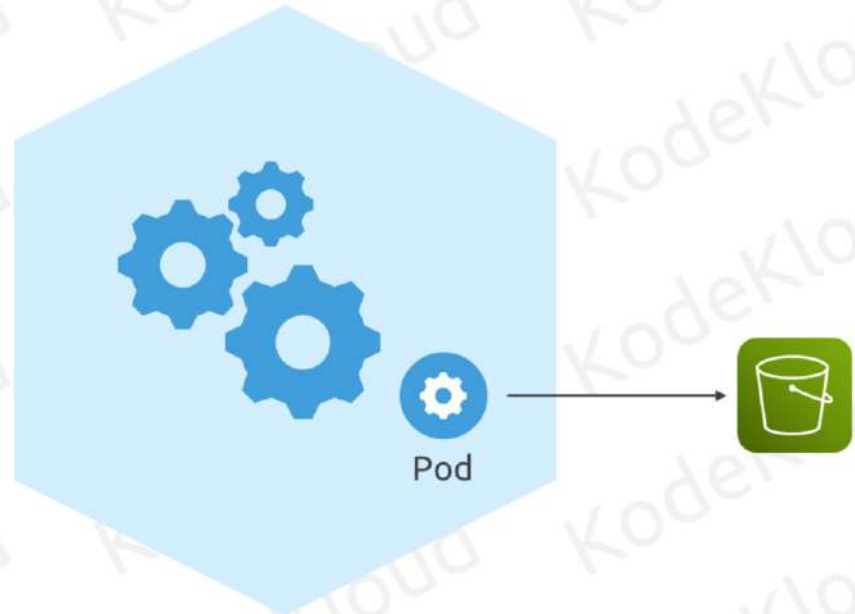
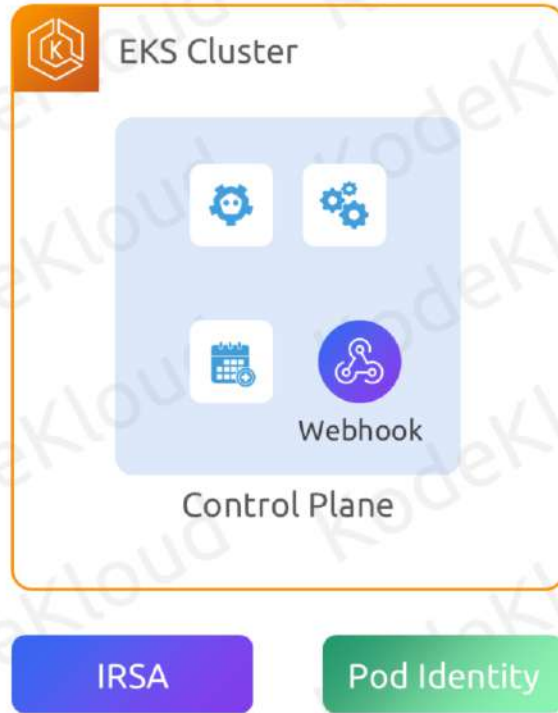
Kube2iam

IRSA

IRSA V2

Okay, so we've already talked about Cube to IAM for getting permissions or credentials for a pod to go access something within aws. We've also talked about Ursa, which is another way of doing that, which is kind of the first attempt at integrating some of that into kubernetes or into eks specifically. And finally, the newest one is called pod identity, and this is commonly known as like Ursa v two. But it's like a new evolution of what IAM roles for service accounts did, and it takes a little more ownership and removes some of those limitations that were there before.

Introduction



© Copyright KodeKloud

So let's look at how pod identity works. So we have the exact same setup here where we have a node and we have a workload that wants to call out to this s three buckets, and we need to get some credentials to be able to do that. So in this case we are still going to have, just like Ursa had the web hook that is going to run on the EKS control plane. So that's when we say if pot identity is enabled and we say, hey, I'm going to submit this workload, it's going to inject a couple environment variables here for us in our workload that inform the AWS SDK how it's supposed to fetch those credentials. And this webhook is knows about Ursa versus pod identity if they're doing different things.

Pod Identity



IRSA

169....

Pod Identity

In one case it might be giving the OIDC endpoints, and in pod identity it's actually going to give a local 169 address endpoints.

Pod Identity

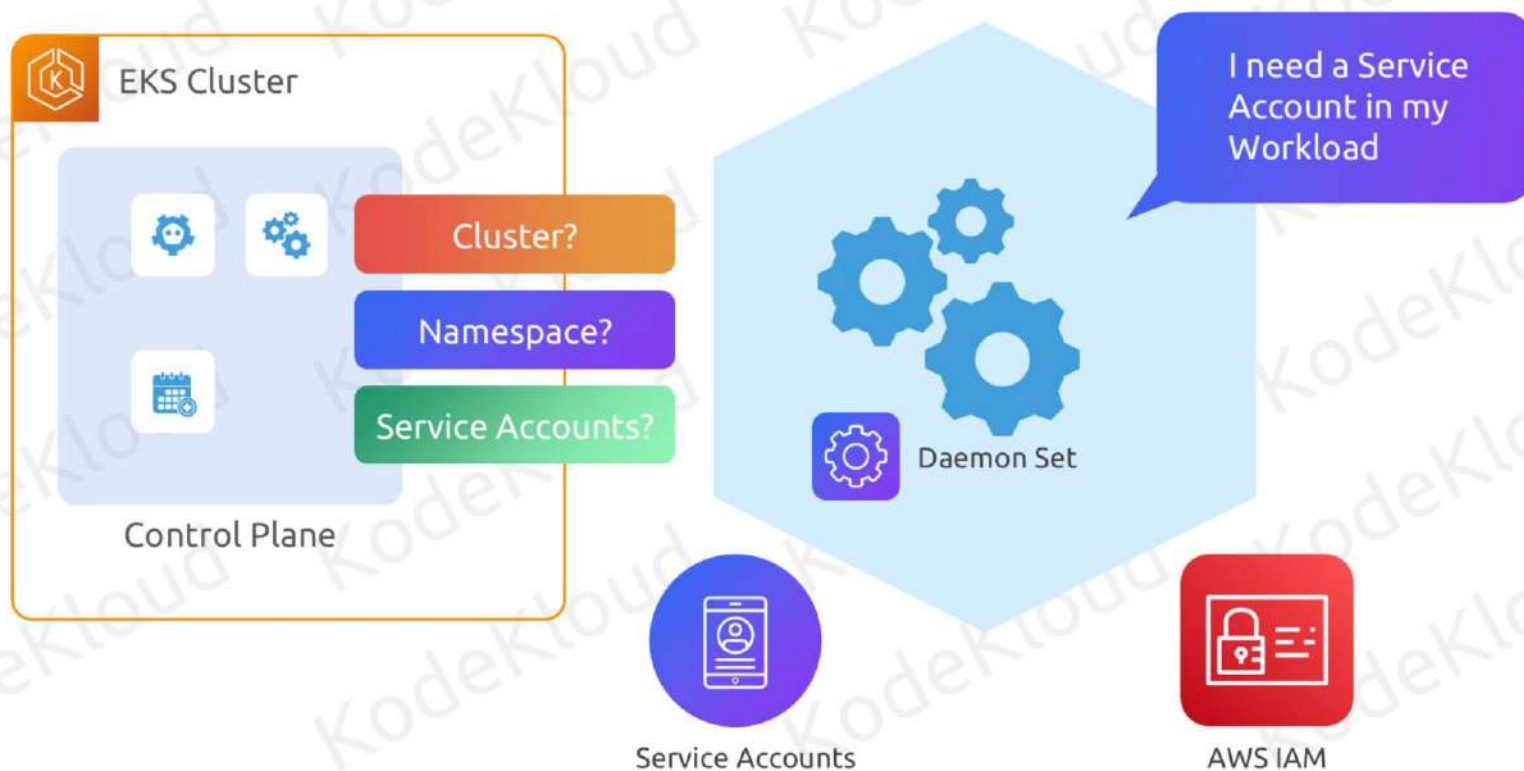


© Copyright KodeKloud

So with pod identity you need to run a daemon set on, on all of your nodes that creates a new sort of proxied IAM credentials, just like we had with Cube to IM. It also is a privileged pod with host networking. So it appears to be coming from the EC two instance, and it does reuse the role from the EC two instance to be able to call the Kubernetes API, because all of your nodes have to be able to call the Kubernetes API or the EKs API to be able to do certain things like join a cluster. So this daemonset in this pod is going to say, okay, I know how to get credentials, but I need some association between a service accounts that is going to be mapped into your workload and an IM that I'm going to fetch credentials for. And so with pod identity, the EKs API's added a couple different, a couple different commands to be able to make this

association. So instead of putting an annotation on the SA with the, the ARN for the IAM credentials we want to use, we do that directly in the EKS API.

Pod Identity



© Copyright KodeKloud

And so we are going to say in the EKS API what cluster, what namespace and what service accounts is going to be associated with which I am Arn. And so that's all something that gets stored in the EKS APIs. I don't need to put that in my workloads. I'm not doing anything else. I just say, hey, I need a surface account on my workload and the EKS APIs is going to figure out which IAM credentials that is. The nice thing about integrating this directly into the EKS API is it makes things work a little more like native AWS services, where typically in

Pod Identity



© Copyright KodeKloud

an IAM role you might have something like a principal service and then whatever the service you're using, if it's like RDS or S three, in this case it's going to be pods dot eks dot Amazon aws.com. And you're going to put that as part of your I am role whatever it is you're setting up and say, hey, I need this principle to be able to access and give out credentials for this I am role model. So now you can reuse that.

Pod Identity

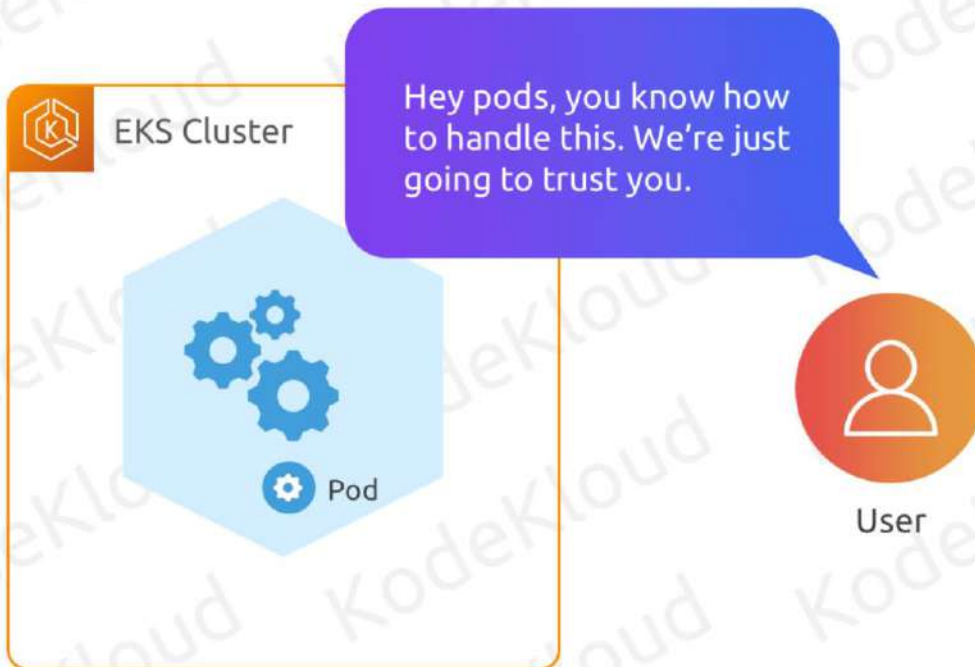


Because with Ursa, as we remembered, we had to set OIDC endpoints because we had to know where the JWT token was coming from and we needed to know ahead of time like what that was going to be. And if the cluster wasn't created yet, we didn't know the OIDC endpoint. All those things were kind of problems. And in this case, it's always the same,

Pod Identity

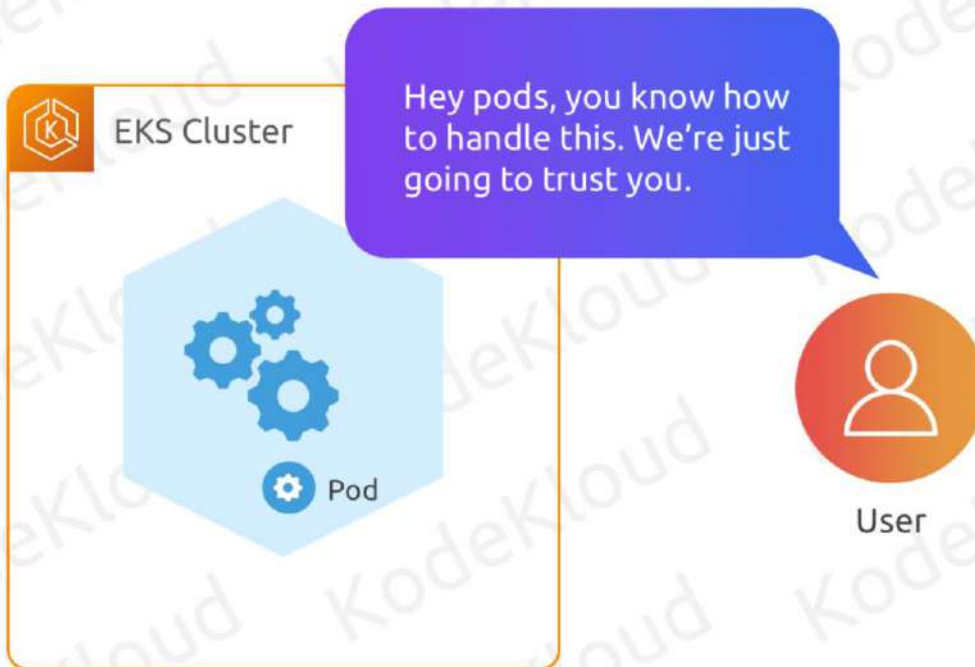


getbuckets



it's always the same service principle that we say, hey, pods, at eks, you know how to handle this. We're just going to trust you. And I can now reuse something like my get buckets over and over again without needing to know all of the OIDC endpoints.

Pod Identity



it's always the same service principle that we say, hey, pods, at eks, you know how to handle this. We're just going to trust you. And I can now reuse something like my get buckets over and over again without needing to know all of the OIDC endpoints. (just move to this slide to add an effect)

Tag-Based Credentials



ABAC

Attribute-Based Access Control

The other benefit this has is tag based credentials and so if you're used to anything inside of AWS, giving out credentials is usually based on tags or, or what are called, or attributes, and you often hear it called ABAC, a back attribute based access control.

Tag-Based Credentials



© Copyright KodeKloud

Whereas in kubernetes usually we have RBAC. These are two different ways of providing credentials to something. One is looking at who you are and the role you are saying you are presenting. The other one is looking at some attribute about the resource you are accessing.

Tag-Based Credentials



Bucket



Attributes

You can write to this S3 bucket if it has the right tags



User

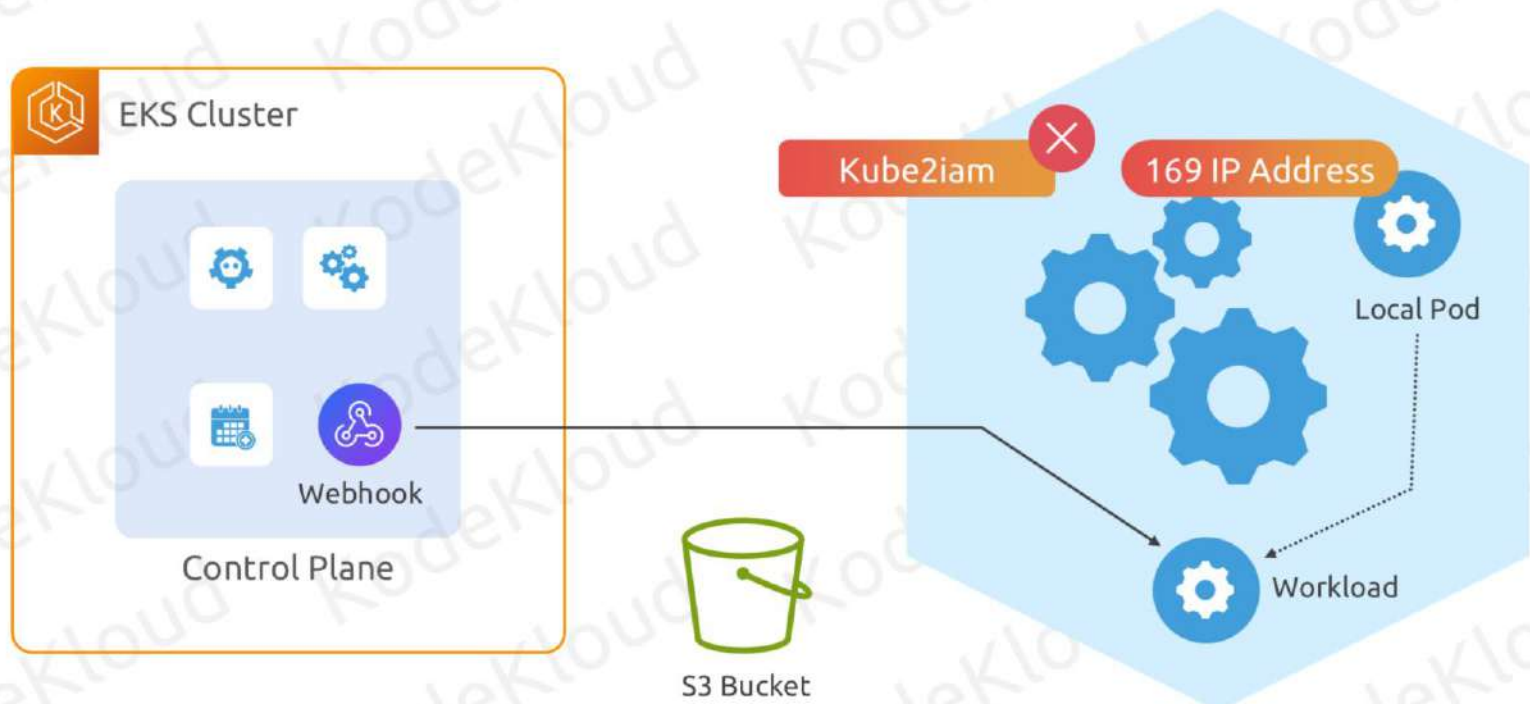
Pod Identity

Namespace

Cluster Name

In AWS, that's usually tags. Tags is the most common attributes that you say, hey, you can write to this s three bucket if it has the right tags, or you can read this secrets if it has the correct tags. Sometimes it's even so much as like you can write to this bucket if it's this path and the path is part of an attribute. And so there's all these different ways of getting the that a back to be recognized within the AWS accounts. And now that the EKS service, the API itself, is giving us back credentials, we're sending it some information about where we sent from, such as our pod identity or our namespace or our cluster name. All of those things can then be used as where I can access something inside AWS.

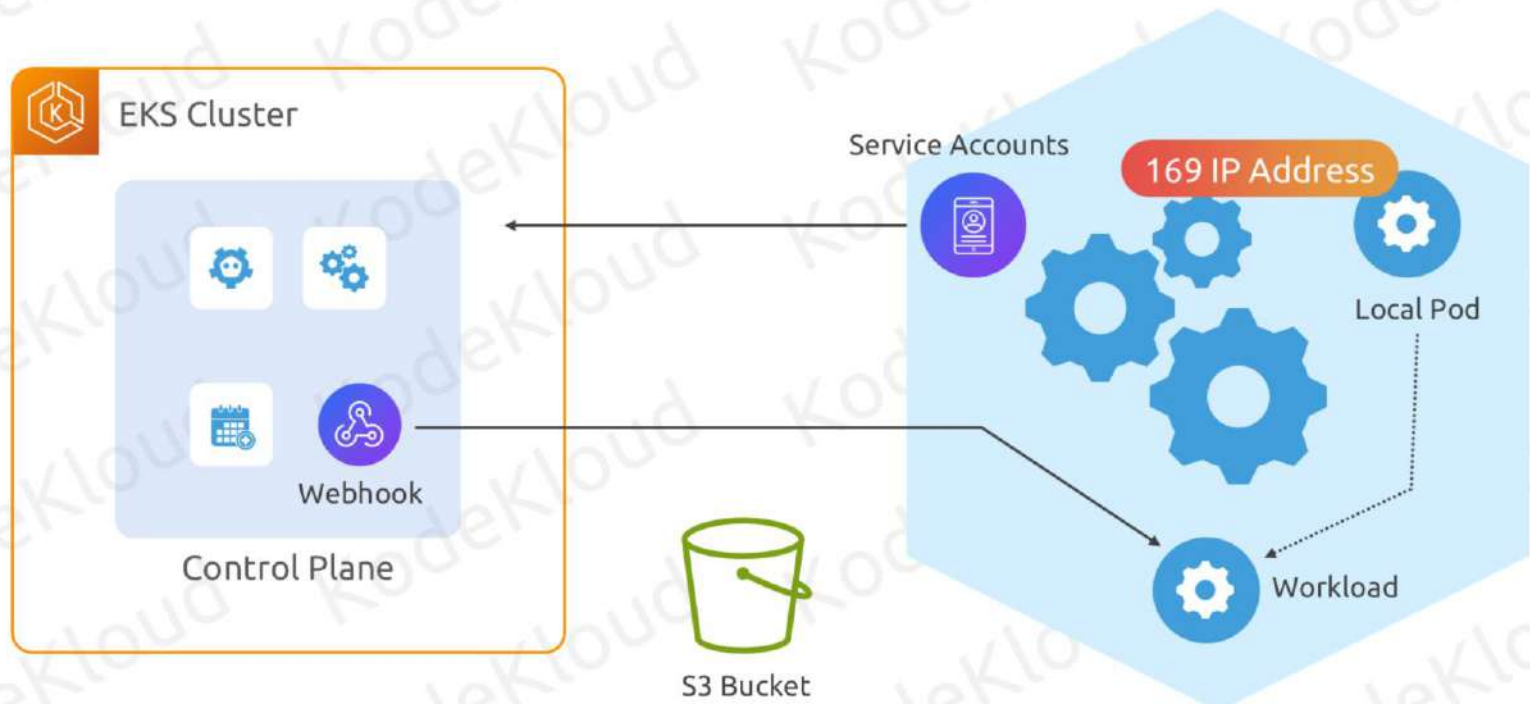
How It Works



© Copyright KodeKloud

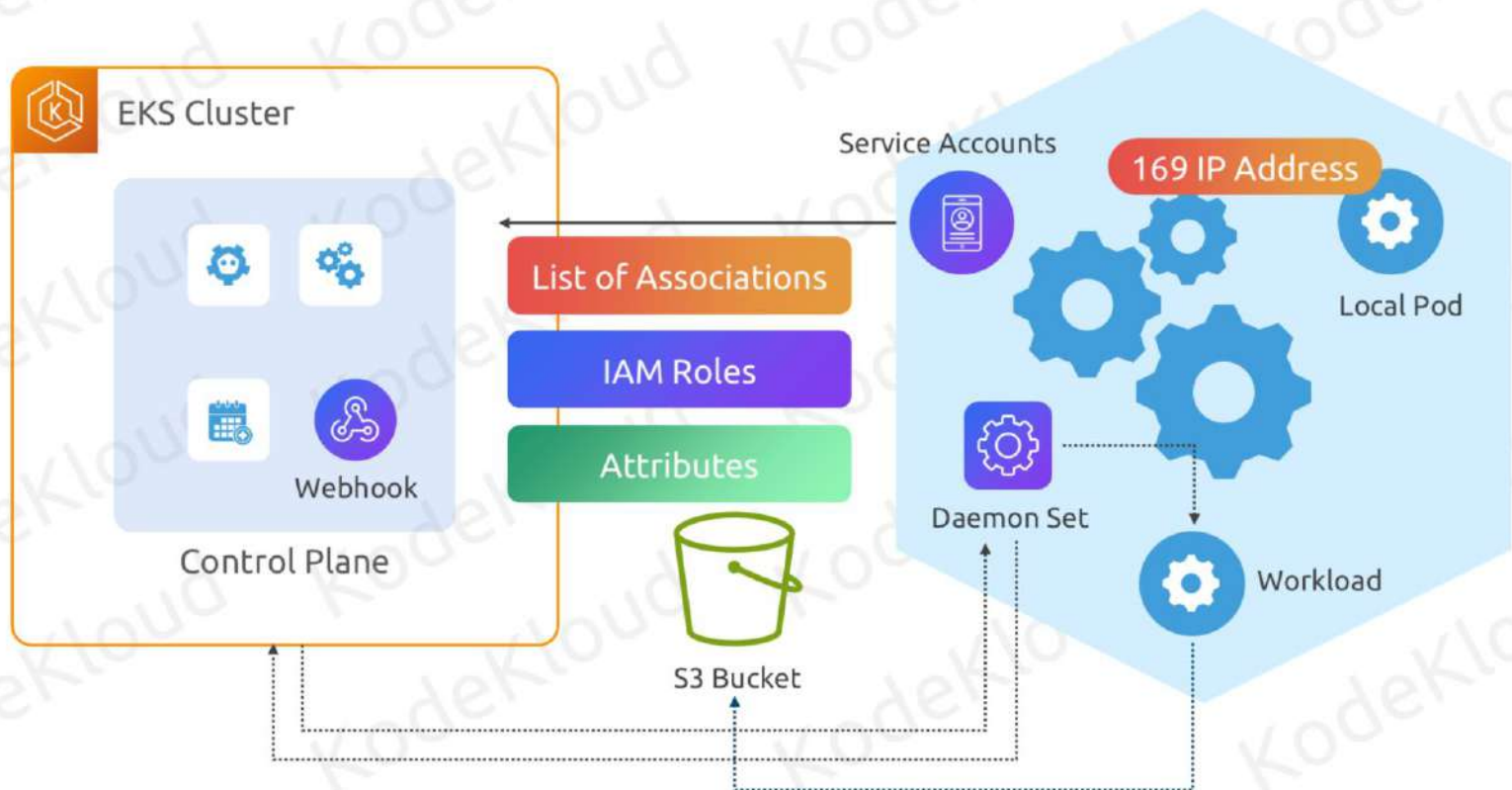
So let's say I submit this workload and I say I need a new workload to run. It's going to access that S3 bucket. First thing it's going to get is the mutating webhook is going to inject, is going to inject those environment variables that say, hey, I have pod identity turned on. So you're going to have to fetch all of your credentials from this 169, 169 address. That's a local pod that's running on your node. So anytime you're going to call out, you say, hey, get this other endpoint. And interesting enough, this is actually a different IP address than what we used to have with kubed images. Cube to IAM was reusing the EC2 metadata endpoint from the node with iptable rules to redirect it transparently. In this case we have a webhook that says, nope, you get a new endpoint, that is this new pod. You fetch your credentials from there.

How It Works



So the workload calls out gets those, asks the pod hey, I need these credentials. And that pod is going to take the service account token that was sent, that was mounted into the pod and it's going to send it directly to eks and say, hey eks, you know, the associations, I don't know which iam I even need for this. There's no association inside Kubernetes that says where that association should be handled.

How It Works



© Copyright KodeKloud

So that pod or that daemon set calls directly to eks. EKS looks at the list of associations that it has for service accounts and, and what other IM roles it's supposed to give back, as well as which attributes it should be able to access, and it's going to send back the correct role for that pod to access. And this daemon set is going to send back that, those credentials and then you can access the bucket. So compared this to UrSA, this is a much more simple solution as far as the amount of hops and things I had to work through. I don't need a JWT token to be able to access this. I don't even need to annotate my service accounts inside the cluster. That is all handled from the EKs APIs

Removal of Limitations

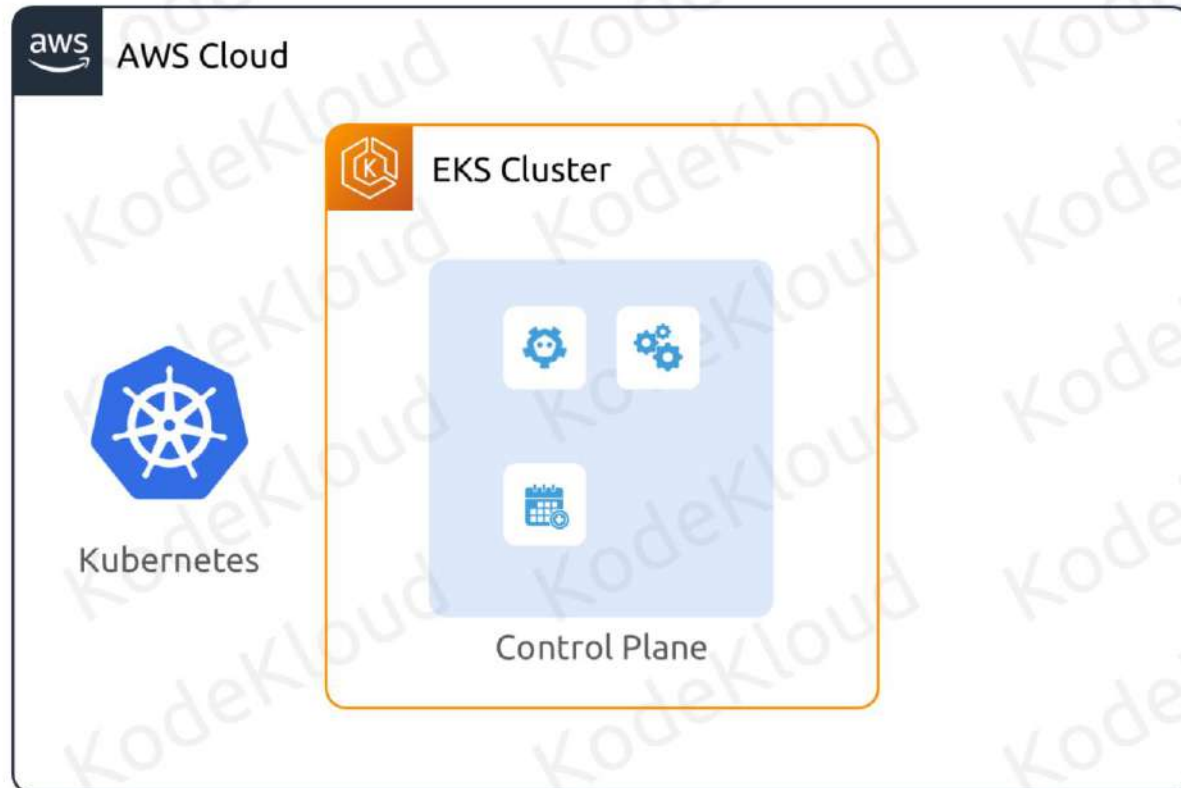


© Copyright KodeKloud

And the other benefits of limitations we had with Irsa like 100 OIDC endpoints and reusing the IAM credentials over and over again. Having a limit on that is completely gone because we have attribute-based access control on those labels. I can just reuse the same IAM for things like a bucket reader and I can rely on those tags on the resources I'm looking for to give me the scoped permissions for my cluster or for even my pod. All of those things kind of collapse down to say hey, I kind of know where this came from. EKS is giving me the right things back to say hey, you get the permissions for these tags that are available throughout AWS.

Pod Identity – Downside

Kube2iam



© Copyright KodeKloud

The one thing that is kind of a downside with pod identity is it only works for eks if you're running a kubernetes cluster inside of AWS and you want to use this if you're running the control plane, you can't because all of those associations are done through the EKS APIs. EKS on the backend is able to figure out who you are, what you're calling for, and then say oh, this IAM role trusts me as the EKS service, which it should. And so I'm going to give you the credentials that you need. You don't need the JWT token and proving yourself and all those other things that you had to go through with Ursa. So overall, if you're trying to get workloads to access anything inside the cluster, you have those main three options, which are something like kube to IM.

Comparison



© Copyright KodeKloud

There's a couple of different ones out there, but Kube IAM was one of the most popular. IRSA or pod identity. And in those three choices, if you're using clusters that are not only EKS clusters, you'll want to look at something like kube2iam. I don't even know that I would bother recommending UrSa at this point. Or if you're using EKS, then you should be using pod identity because that's the most native way for both inside of AWS and within Kubernetes to use service accounts and still get all of the benefits of ABAC inside of the AWS accounts. I will also note that IRSA and pod identity work at the same time. So if you have clusters that have IRSA enabled, you can also enable pod identity and make those mappings as associations when they're both enabled. EKS the webhook will prefer pod identity, so it'll try to inject pod identity in the

nodes itself or in the in the workloads to say where to fetch the endpoints. But if you have, you know, one or the other, and maybe some workloads work with the with one and some work with the other, you can have both of them run in parallel. So it's not an all or nothing, but you definitely want to limit how many different ways you're getting access to various resources and how you're getting credentials from AWS just to help with debugging purposes.

Conclusion



So that wraps up how we're getting IAM credentials and access within the pod. And let's look at how pods actually call out to other things with security groups, with network access, because that's another layer that is separate from IAM, but also pretty well integrated into how VPCs and EC two works.

Security Groups for Pods

Security Groups



Node

Let's talk about another way that pods can access internal AWS resources, and that's typically with security groups. And security groups are kind of like a firewall in your VPC. It allows two endpoints to talk to each other or not talk to each other. And if you're used to building EC2 infrastructure, you've probably been tripped up by security groups at least once in your lifetime. I know it happens to me regularly when traffic just doesn't flow between two endpoints. And I realized that I didn't allow a certain portion or I didn't have two way traffic between the two endpoints. And security groups can also be applied to pods. Now I'm just going to go ahead and tell you that you probably shouldn't be using this.

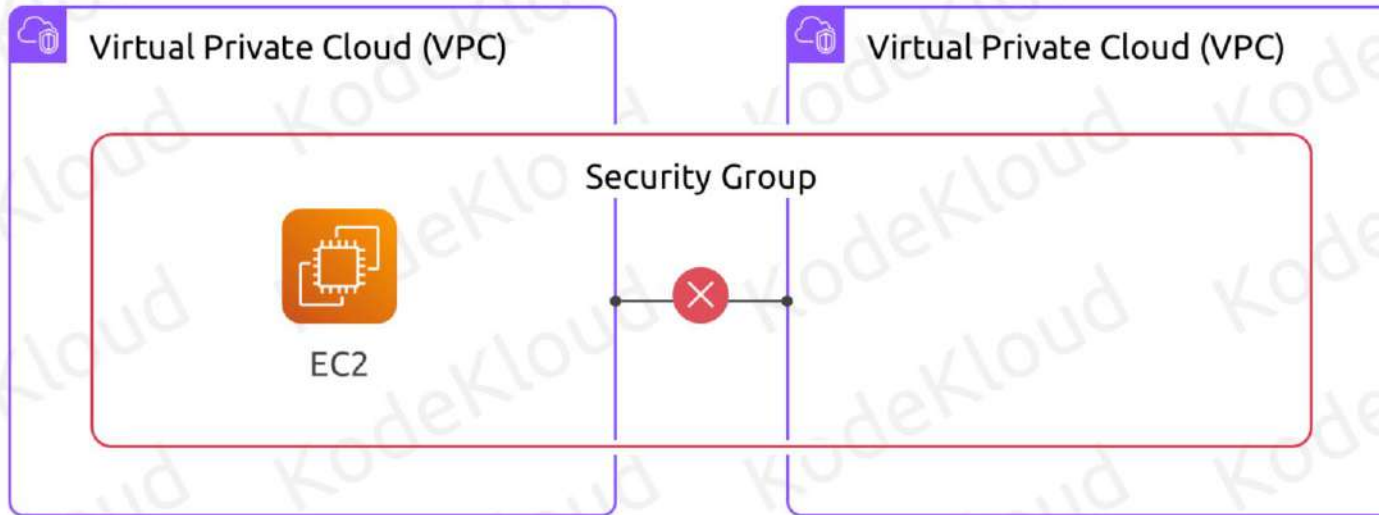
Security Groups



© Copyright KodeKloud

Let's talk about another way that pods can access internal AWS resources, and that's typically with security groups. And security groups are kind of like a firewall in your VPC. It allows two endpoints to talk to each other or not talk to each other. And if you're used to building EC two infrastructure, you've probably been tripped up by security groups at least once in your lifetime. I know it happens to me regularly when traffic just doesn't flow between two endpoints. And I realized that I didn't allow a certain portion or I didn't have two way traffic between the two endpoints. And security groups can also be applied to pods. Now I'm just going to go ahead and tell you that you probably shouldn't be using this.

Security Groups



© Copyright KodeKloud

Let's talk about another way that pods can access internal AWS resources, and that's typically with security groups. And security groups are kind of like a firewall in your VPC. It allows two endpoints to talk to each other or not talk to each other. And if you're used to building EC2 infrastructure, you've probably been tripped up by security groups at least once in your lifetime. I know it happens to me regularly when traffic just doesn't flow between two endpoints. And I realized that I didn't allow a certain portion or I didn't have two way traffic between the two endpoints. And security groups can also be applied to pods. Now I'm just going to go ahead and tell you that you probably shouldn't be using this.

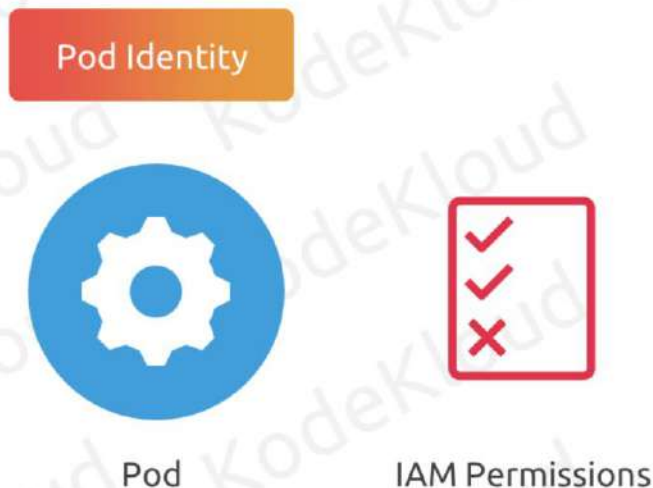
Security Groups



© Copyright KodeKloud

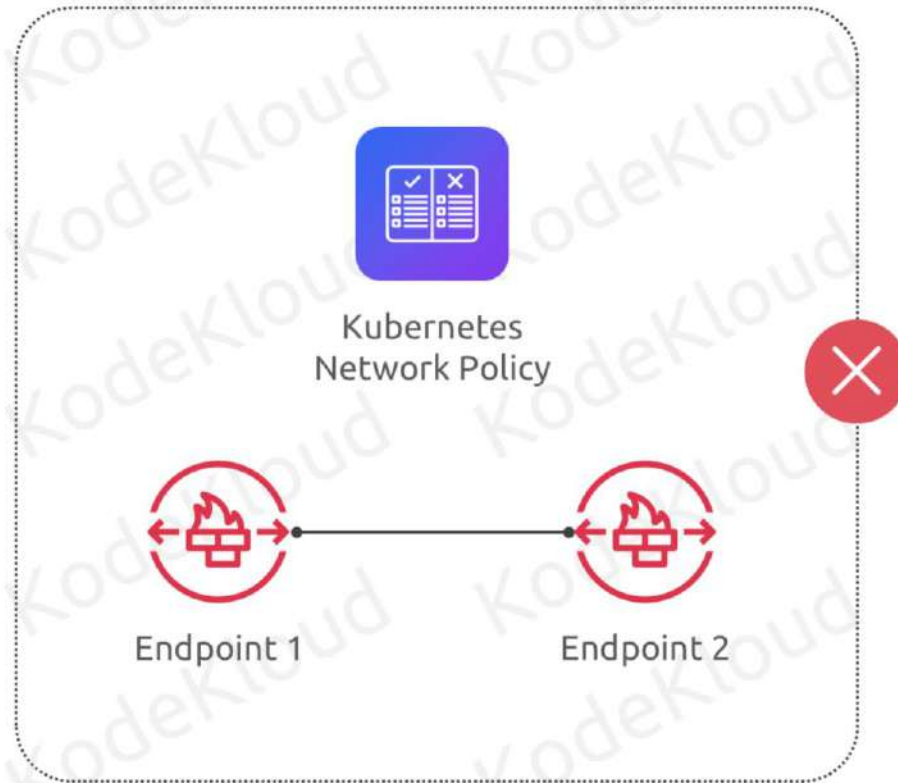
Let's talk about another way that pods can access internal AWS resources, and that's typically with security groups. And security groups are kind of like a firewall in your VPC. It allows two endpoints to talk to each other or not talk to each other. And if you're used to building EC two infrastructure, you've probably been tripped up by security groups at least once in your lifetime. I know it happens to me regularly when traffic just doesn't flow between two endpoints. And I realized that I didn't allow a certain portion or I didn't have two way traffic between the two endpoints. And security groups can also be applied to pods. Now I'm just going to go ahead and tell you that you probably shouldn't be using this. This isn't something that you should be doing from the start as even like, oh, this is a nice thing to have. This is a very complex topic.

Pod Identity



And if you're not invested in security groups and tooling around managing security groups, you should be using pod identity, that's called. You should be using pod identity for adding IAM permissions to your pods to be able to talk to endpoints and also network policies inside of the cluster. Now, network policies don't go all the way into allowing certain endpoints to talk to AWS resources because a Kubernetes network policy doesn't really know about AWS resources the way AWS does. But security groups for pods just gets a little too complex and in my opinion, is maybe not the best solution to be handling this. Let's diagram it out and I'll show you. Now, let's say we have a node here and it wants to talk to this RDS instance.

Pod Identity



© Copyright KodeKloud

And if you're not invested in security groups and tooling around managing security groups, you should be using pod identity, that's called. You should be using pod identity for adding IAM permissions to your pods to be able to talk to endpoints and also network policies inside of the cluster. Now, network policies don't go all the way into allowing certain endpoints to talk to AWS resources because a Kubernetes network policy doesn't really know about AWS resources the way AWS does. But security groups for pods just gets a little too complex and in my opinion, is maybe not the best solution to be handling this. Let's diagram it out and I'll show you. Now, let's say we have a node here and it wants to talk to this RDS instance.

Pod Identity



Endpoint 1



Endpoint 2



Kubernetes
Network Policy

And if you're not invested in security groups and tooling around managing security groups, you should be using pod identity, that's called. You should be using pod identity for adding IAM permissions to your pods to be able to talk to endpoints and also network policies inside of the cluster. Now, network policies don't go all the way into allowing certain endpoints to talk to AWS resources because a Kubernetes network policy doesn't really know about AWS resources the way AWS does. But security groups for pods just gets a little too complex and in my opinion, is maybe not the best solution to be handling this. Let's diagram it out and I'll show you.

Pod Identity

AWS IAM



Pod



Node

Security Group 1

ENI (Primary)



Secondary ENI



AWS RDS Instance

Now, let's say we have a node here and it wants to talk to this RDS instance. So by default, if you're running the VPCC CNi and you have a pod here, remember this will get an Eni for traffic to flow through it, and there's one that is the primary, and pods can be assigned to that, and there also can be secondary ones. Now, all of these ENis on the node will actually share the same security group. So if you say that any of the workloads on this node need to talk to RDS, then all of them potentially could talk to RDS. Now that's still limited by IAM being able to be applied tightly around one pod at a time or one workload so that it can authenticate or not, but the security group is applied to the node. And this also applies typically to all of the nodes in your cluster when you think about it, because all of them are in an auto scaling group. You're not

assigning different security groups per node, you're just saying, hey, all of my nodes should be able to do this. So if any workload in your cluster needs to call an API, then all of them will have the network ability to do so. But again, you should be securing that at the IAM level around a workload and not necessarily layering that with networking if you don't have to.

Pod Identity

AWS IAM



Pod

Node

Security Group 1

ENI (Primary)



Secondary ENI



Kubernetes Cluster



etcd

api server



scheduler

Control Plane

Security Group 1



Data Plane

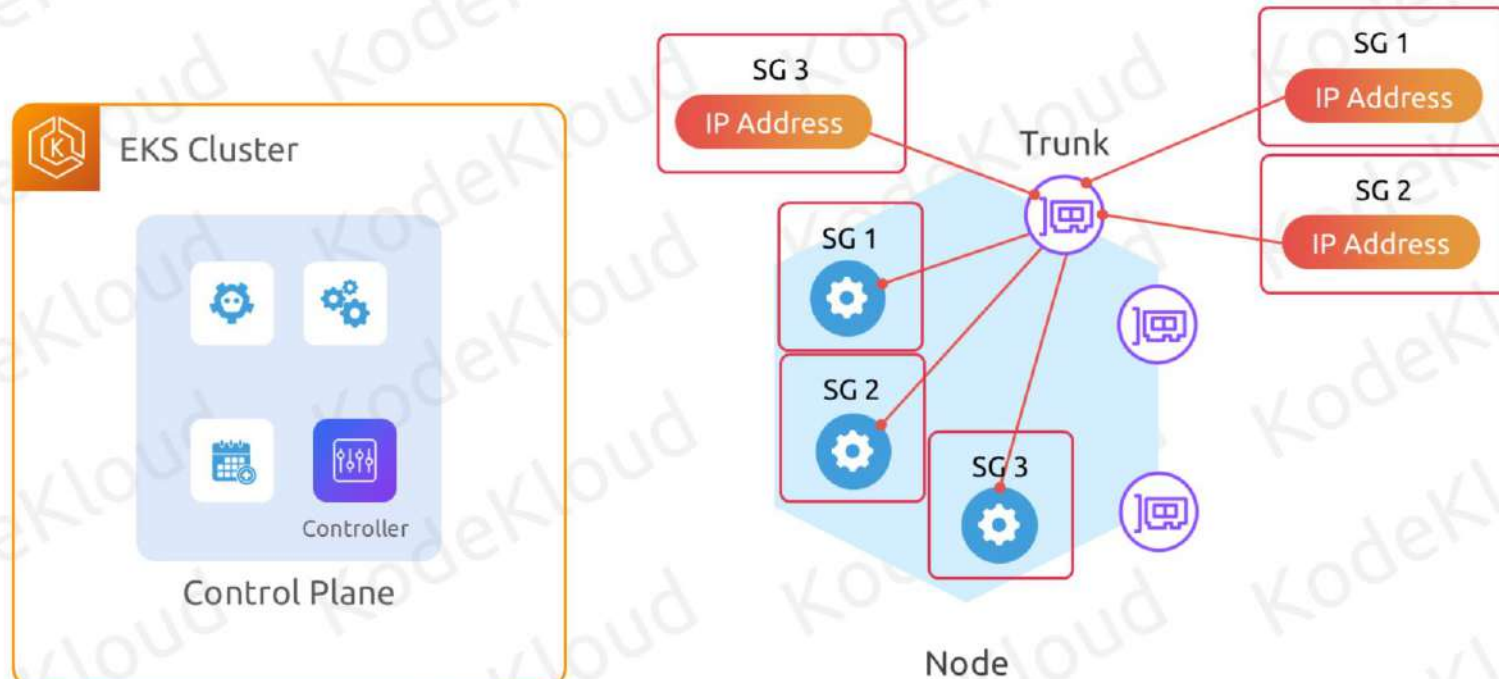
Pod Security



Open-source VPC
controller that uses
private API

So let's look at how, if we want to enable security groups for a pod works. And again, this is a little bit complicated. So first up, even just to make this work at all, there is a controller that runs in the EKS control plane. This is a VPC controller that is open source, but it actually uses private APIs. It uses calls that you are not allowed to make as a user. So this is completely opaque to you, even though it's open source and it's available in the SDK. What it does is it adds a new, a brand new node or a brand new ENI to your cluster for a trunk. And this trunk interface is something that you can't do.

Pod Security

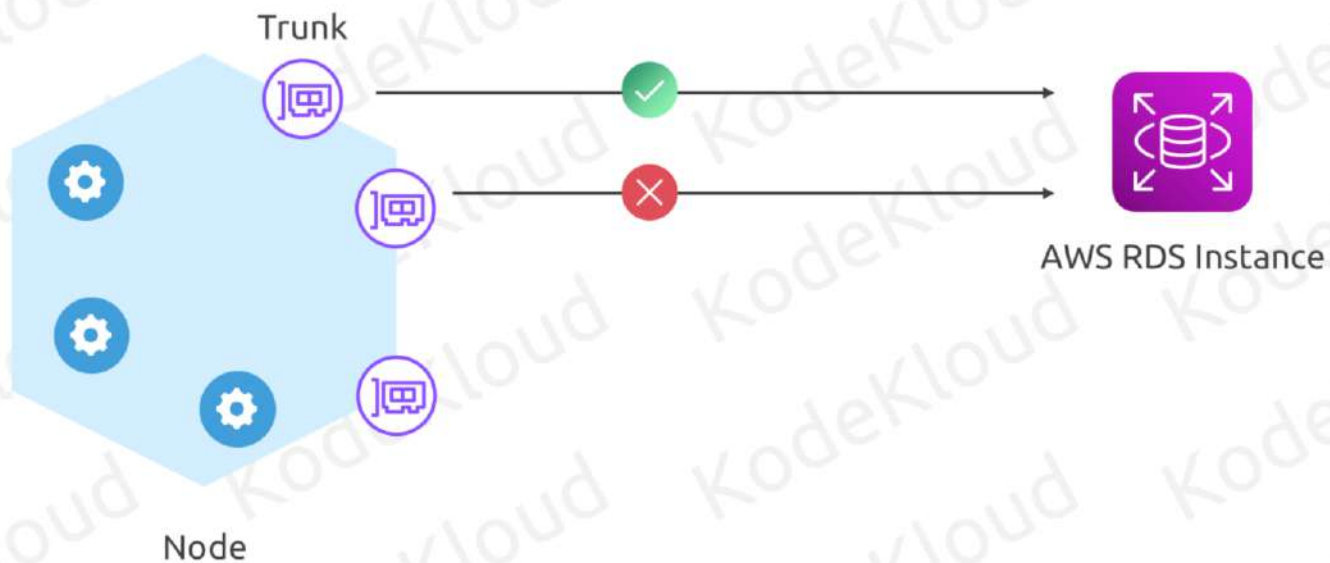


© Copyright KodeKloud

So let's look at how, if we want to enable security groups for a pod works. And again, this is a little bit complicated. So first up, even just to make this work at all, there is a controller that runs in the EKS control plane. This is a VPC controller that is open source, but it actually uses private APIs. It uses calls that you are not allowed to make as a user. So this is completely opaque to you, even though it's open source and it's available in the SDK. What it does is it adds a new, a brand new node or a brand new ENI to your cluster for a trunk. And this trunk interface is something that you can't do. Only AWS can do this. So right there, if you need to troubleshoot something or you need to debug something, this might get complicated. And this trunk interface will allow, just like these other interfaces allow different IP addresses on the node and those get mapped

into your workloads. Now the difference here on this trunk interface is these can actually have separate security groups applied to them. So as they map into each workload, you will get different security groups for different pods and workloads based on what you're applying externally or on the security groups for the trunk ports. And if this is allowed, then obviously this workload could go through and say, hey, I can do that, but maybe this other generic Eni would not be able to. And this doesn't seem so bad yet. I mean, we are using some private APIs, we are adding extra network interfaces.

Pod Security



Reminder

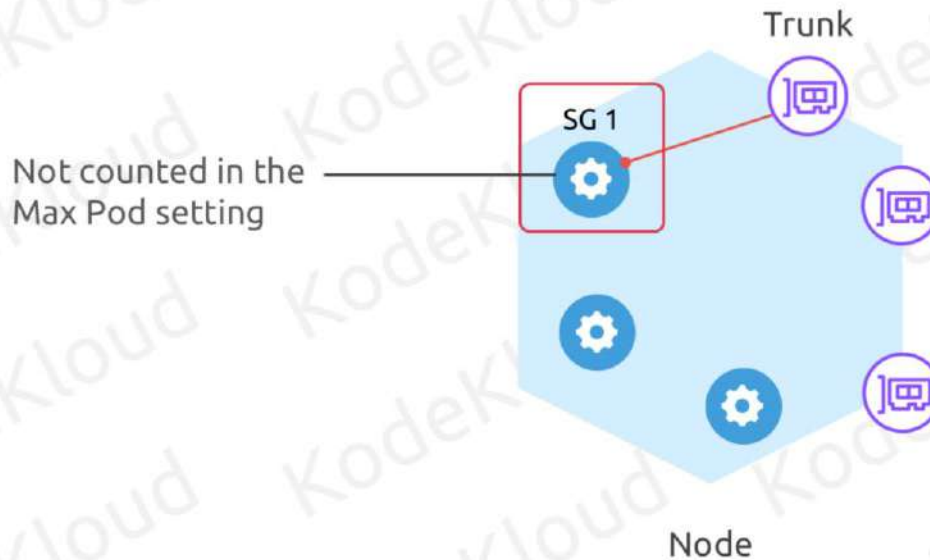


For this trunk port to be added, you need to limit which instances you're going to use, and you need to look them up.



But the other thing that kind of gets a little confusing here is not all AWS instance types support trunk ports. And so if you want to even allow for this trunk port to be added, you need to limit which instances you're going to use and you need to look them up. This isn't something you can call an API for. There is a large map in a, in a go templates that you need to look through and say, oh, we can use this one, we can't use that one. And usually it's older generations. A lot of the nitro ones can, but not all of them. So you still have to make sure you're doing the right thing here.

Max Pods

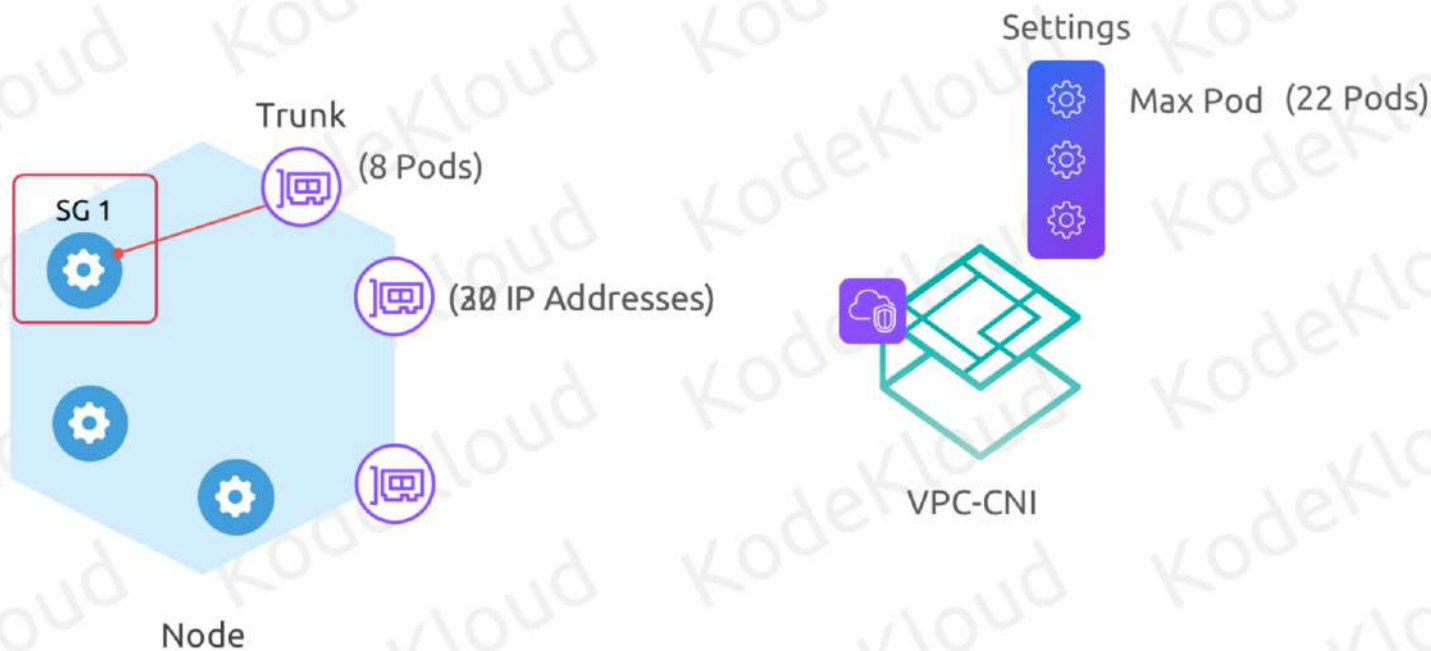


© Copyright KodeKloud

The second thing that's a little weird with security groups is this workload. Let's say it's using a security group, it's going through that trunk ports. It's not actually counted in the Max pods setting. Now Max pods is a setting on the VPC CNI that, that says hey this is the total number of pods that this instance can run and it usually is doing that because it's counting these Enis and ports. And so we say hey, this instance type can only have say 30 IP addresses based on those NI's. But if you are using security groups and you want to have say eight pods here, then you have to manually go through and find out which nodes would need to be reduced. And so this one might actually end up being 22 and you need to set that max pods. But remember, you can't set max pods on a per node basis. That has to be done on the node group or in, in carpenter on the

provisioning, on the provisioner or node classes it's called now.

Max Pods



© Copyright KodeKloud

The second thing that's a little weird with security groups is this workload. Let's say it's using a security group, it's going through that trunk ports. It's not actually counted in the Max pods setting. Now Max pods is a setting on the VPC CNI that, that says hey this is the total number of pods that this instance can run and it usually is doing that because it's counting these Enis and ports. And so we say hey, this instance type can only have say 30 IP addresses based on those NI's. But if you are using security groups and you want to have say eight pods here, then you have to manually go through and find out which nodes would need to be reduced. And so this one might actually end up being 22 and you need to set that max pods.

Reminder



You can't set Max Pods on a per node basis. It can only be done on the node group and Karpenter on the provisioning or Node Classes.



But remember, you can't set max pods on a per node basis. That has to be done on the node group or in, in carpenter on the provisioning, on the provisioner or node classes it's called now. And so you need to then break up which nodes in which workloads are going to land where so that you can manage some of them with security groups.

Additional Reminder



Calico



VPC-CNI



Cilium

But the complexity doesn't actually stop there unfortunately, because let's say you don't want to use the VPC CNI and you want to use something else like calico or psyllium. These are both pretty common replacements for the VPC CNI. And while you could still have trunk ports, the complexity of managing a self managed CNI on top of getting security groups is something that I still don't even understand. And this is getting really complex. I'm not saying that I should understand all this, but I would often probably have outages because some pod somewhere can't talk to an endpoint inside of AWS.

Additional Reminder



Node



AWS RDS Instance

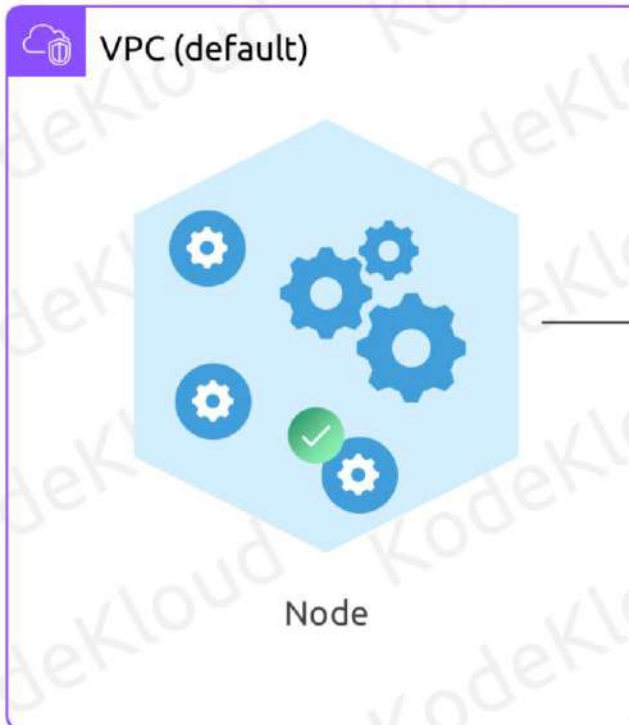
So unless you are already very invested and you have automatic tooling for security groups, I would pretty much avoid this with kubernetes. And the replacements would be first of all, I am where you want to have pod identity on each workload inside the node so that you can tightly constrain how they can authenticate. And yes, inside the VPC they might be able to talk to an endpoint or reach an endpoint. They will still fail authentication and so you won't be able to actually do anything with that except for the workloads that are allowed to. The other thing I would look at is network the other thing I would look at is network policies, and these aren't fine grained. Like I said, you can't do a network policy to a specific RDS ARN, which is how AWS identifies it. But you can do it for IP ranges. And so you might be able to stop the egress coming out of a

workload to some IP cidr to block that egress traffic. And so you could identify, oh, only these workloads actually need to talk to these RDS instances. They're on the cidrs or part of this subnets, because you would want to be able to isolate some of those things.

AWS IAM



AWS IAM



AWS RDS Instance

So unless you are already very invested and you have automatic tooling for security groups, I would pretty much avoid this with kubernetes. And the replacements would be first of all, I am where you want to have pod identity on each workload inside the node so that you can tightly constrain how they can authenticate. And yes, inside the VPC they might be able to talk to an endpoint or reach an endpoint. They will still fail authentication and so you won't be able to actually do anything with that except for the workloads that are allowed to.

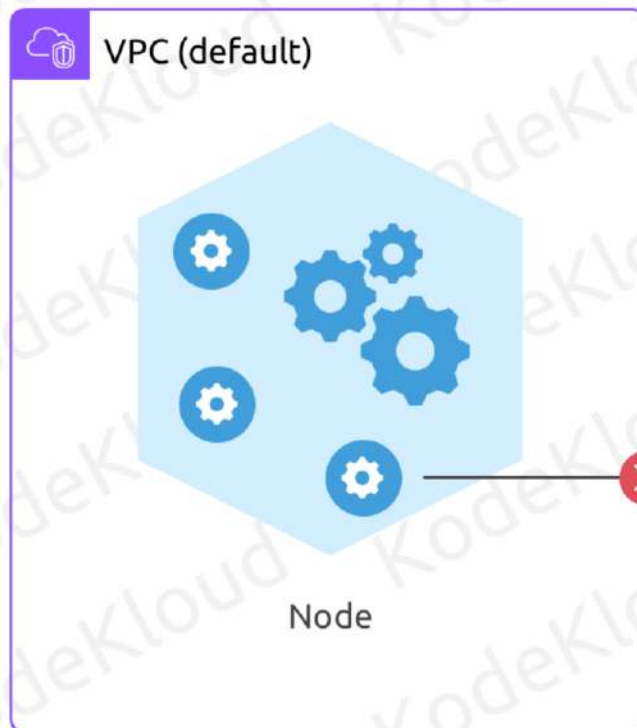
Network Policy



AWS IAM



Network Policy



AWS RDS Instance

IP CIDR

The other thing I would look at is network the other thing I would look at is network policies, and these aren't fine grained. Like I said, you can't do a network policy to a specific RDS ARN, which is how AWS identifies it. But you can do it for IP ranges. And so you might be able to stop the egress coming out of a workload to some IP cidr to block that egress traffic. And so you could identify, oh, only these workloads actually need to talk to these RDS instances. They're on the cidrs or part of this subnets, because you would want to be able to isolate some of those things. And you could talk to that through a network policy, or at least limit that traffic through a network policy rather than trying to do it through security groups.

Node Groups



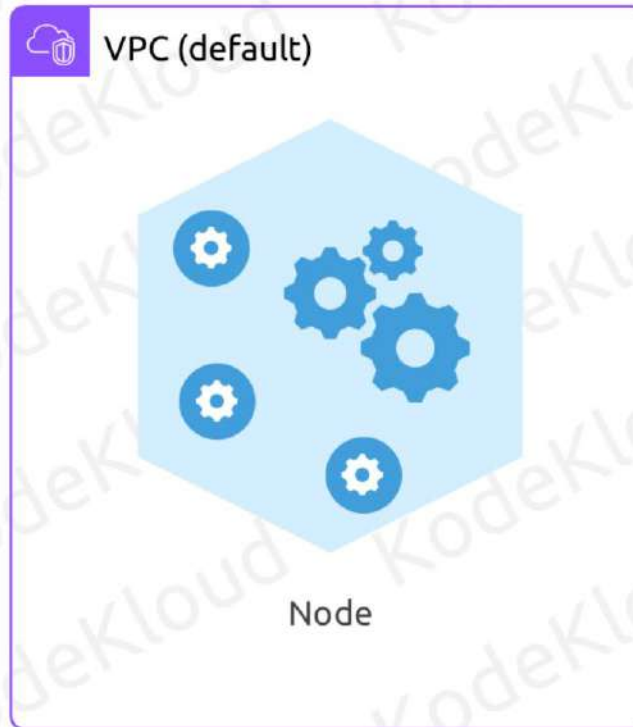
AWS IAM



Network Policy



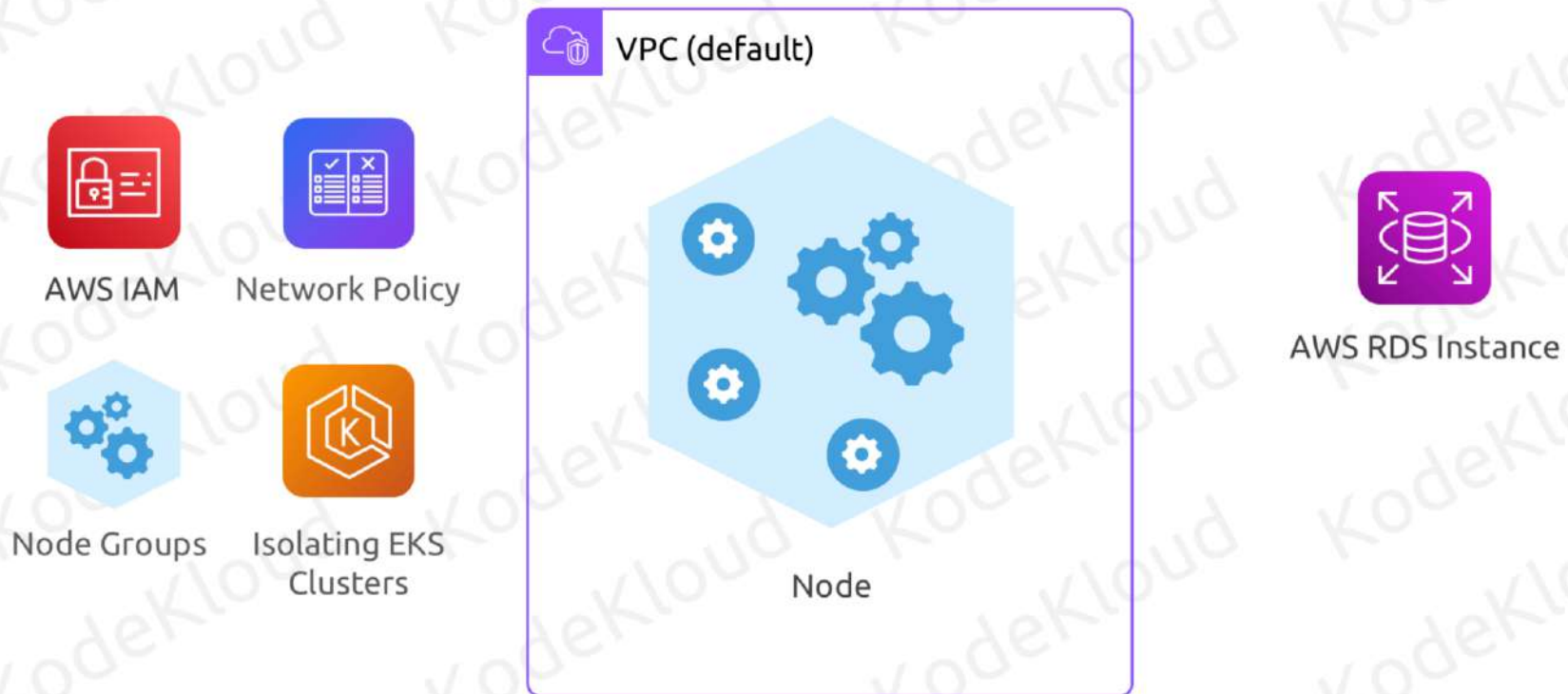
Node Groups



AWS RDS Instance

The other option, like I said, is to create different node groups or carpenter node classes, so that the nodes that need access to specific databases are isolated and have their own security group policies applied to the node itself. And so you could still apply a broader node group policy to some instances and allow only specific node groups to talk to endpoints internally. And then you have to manage where those workloads land. And so that you just know, okay, these instances have some different access. And that's a fairly common strategy because you can still scale up and scale down. You don't need to worry about special instance types. And people kind of understand a little more what a node firewall is better than, say, a pod specific firewall.

Isolating EKS Clusters



© Copyright KodeKloud

And the last one I would suggest is actually just looking at completely isolating EKS clusters. You could do this by creating different clusters that need access to different things. And while EKS clusters are more to manage, and they do have overhead and costs associated with them, if there is some endpoint that is actually secure, if there's some endpoint or some data that is actually secure inside of your network, either an s3 bucket, rds, whatever the case may be, there's some data that not everything should be, should have access to, that should already be separate, that should be in its own VPC, have its own security group, and have its own access controls, mainly for IAM and isolation, because anything inside of an EKS cluster kind of is assumed to be shared to some degree, whether that's resources on a per node basis or IAM

credentials or some other boundary. That may not always be a hard boundary because this is kind of a multi tenancy environment. And when you're doing multi tenancy, you have to just trust and assume that the other workloads and people using the cluster are allowed to do some things. So just to recap real fast, that security group for pods is something that's available. If you already have a lot of tooling and expertise in security groups, you can do it. But there are definitely some caveats.

Isolating EKS Clusters



AWS S3



EKS Clusters



AWS RDS

Isolating EKS Clusters



AWS IAM



VPC (default)

Security Group



AWS S3



AWS RDS

Summary

01

Security Groups for pods are available

02

Some things aren't available to do and to test

03

You have to work closer with AWS APIs and know how it controls those things

04

You have to manage downsides to figure out if a node is running too many pods or you ran out of IP addresses because pods were not counted in the VPC-CNI

So just to recap real fast, that security group for pods is something that's available. If you already have a lot of tooling and expertise in security groups, you can do it. But there are definitely some caveats. There's some things that aren't available to you to do and to test on your own. You're going to have to work a little closer with the AWS APIs and how it controls some of those things. And then you're going to have to manage any of those negative downsides to be able to figure out if a node is running too many pods or you run a run out of IP addresses on a node because those pods are not counted with the rest of them with the VPCC CNI. So, make sure you just know what you're getting into. And I wouldn't look to security groups or pods for isolation as much as I would try one of these other options.

Monitoring

Monitoring EKS Clusters



Amazon CloudWatch



Monitor



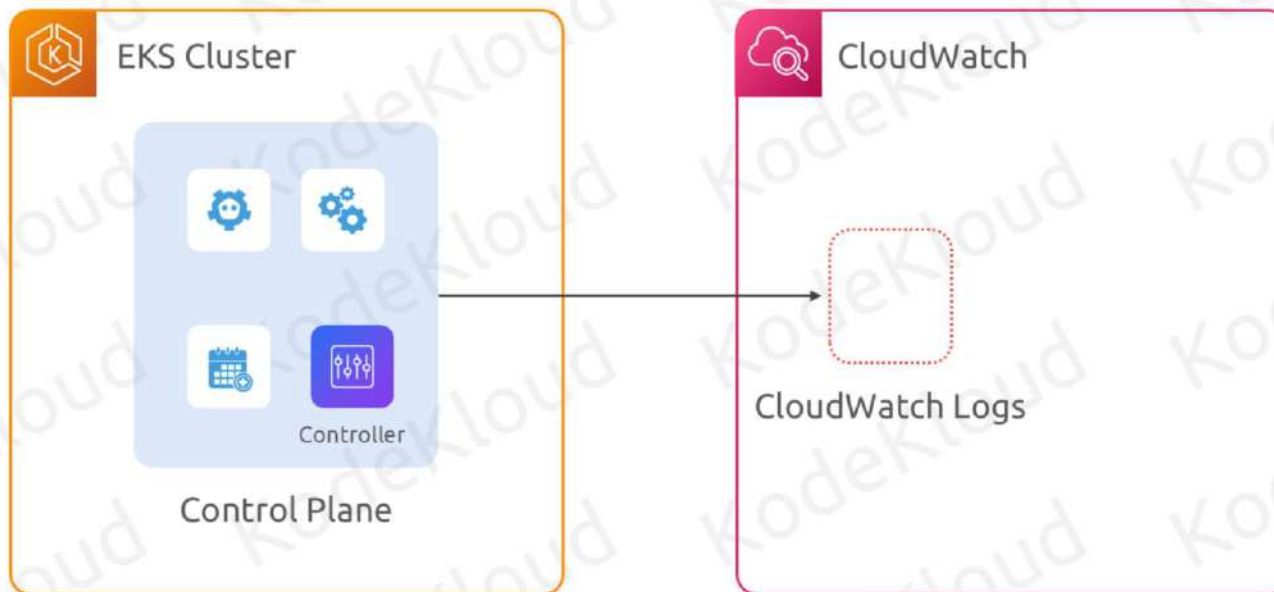
Log



Add Dashboard

Alright, so let's talk about the observability or monitoring of your EKS clusters, because that's a big part of day two operations. And pretty much everything in AWS is going to point you towards Cloudwatch. And Cloudwatch is the default way you monitor, you log, you add dashboards and metrics. Anything you're doing in AWS usually ends up in some form of Cloudwatch. At minimum it's going to end up there temporarily.

CloudWatch



© Copyright KodeKloud

Let's show you how at least some of that integrates with an EKS cluster. So every cluster that I set up has a couple default options as I create the cluster. And one of those things is adding control plane logging during cluster creation. And so that's just a setting on your cluster in the API that says hey, I want to monitor or I want to log these components of the cluster over in Cloudwatch. And you don't get to say where that goes, it just goes to Cloudwatch and it automatically will make a group in Cloudwatch that will be representative of your cluster so that you can come find it under some, you know, known state or known endpoint. So that's just default. If you want that, you're not responsible for any of that logging, you just say I want to turn this on and I want to see my logs in Cloudwatch. And then from those logs you can obviously query them and

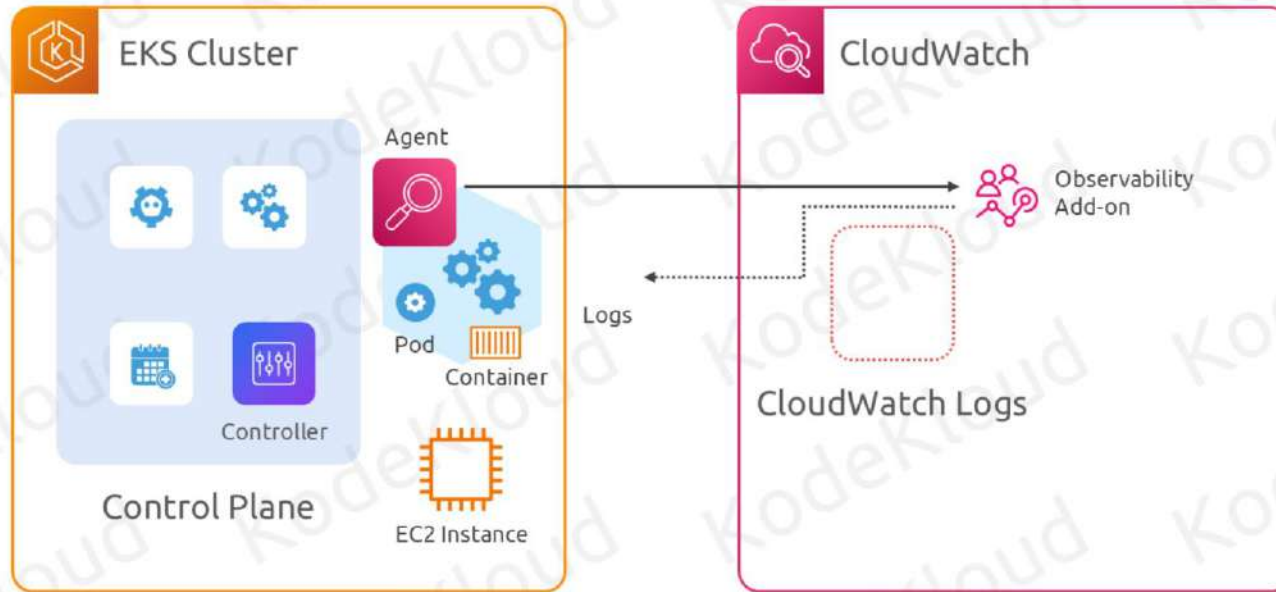
get some other things that you can do that you normally could do in Cloudwatch with logs.

Other Components



The other components is usually the Cloudwatch agents, and this usually gets installed in your cluster on each of your nodes. And so this is typically the logging agent of a node. And this is part of kubernetes or eks and not part of eks. If you just have an EC two instance and you want to log something happening on that node, you're going to install the Cloudwatch agent.

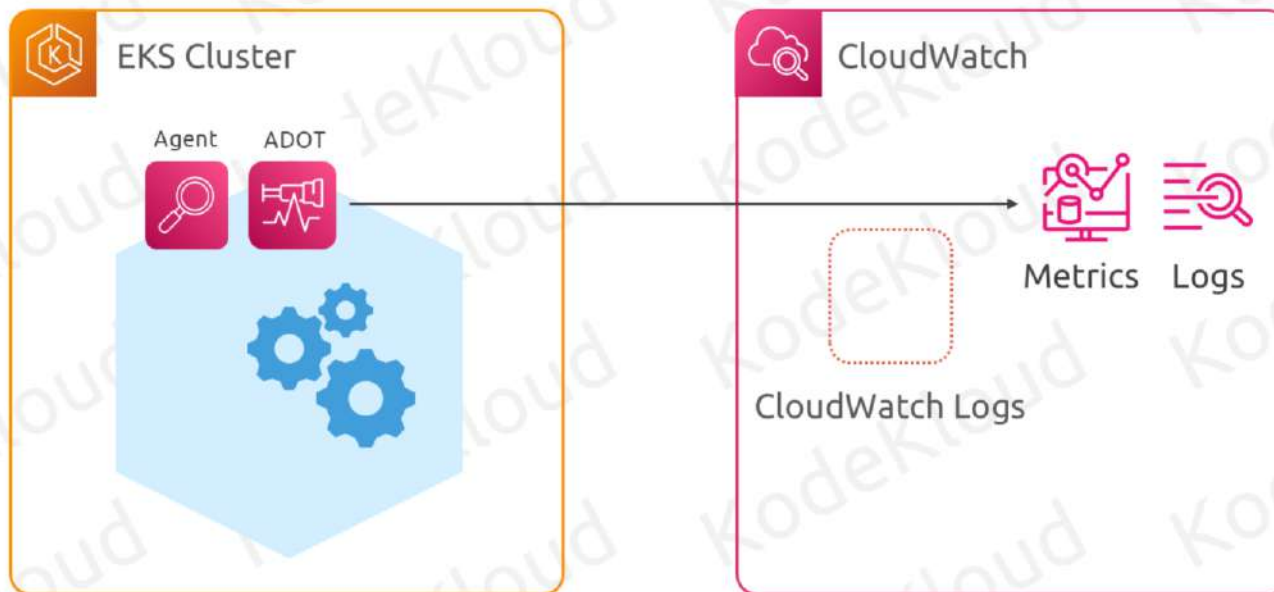
CloudWatch Log Agents



© Copyright KodeKloud

Cloud watch log agents and EKS has a couple different ways of doing this. There is an add on called the observability add on that'll install this agent for you. And while it is just installing the normal agents, it does hook up a couple things to get extra metrics to enrich some of those logs as they're coming out of your out of your nodes. And this includes not only the node logs but also the containers, the actual workloads running on those nodes. So there's a couple, again, there's layers of how this is working. So at the fundamental layer we just have that Cloudwatch log agents,

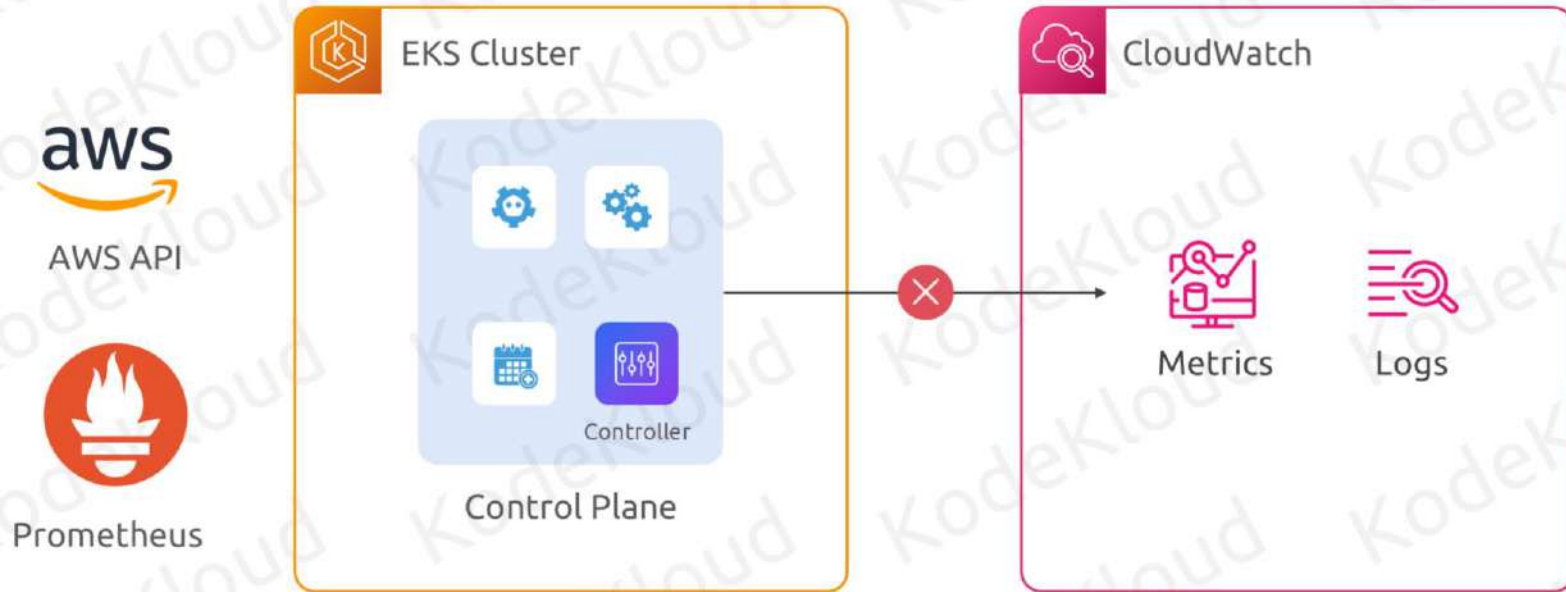
AWS Distro for Open Telemetry (ADOT)



© Copyright KodeKloud

We also can install the ADOT add on, which is AWS distro for open telemetry. And this builds on top of the Cloudwatch log agents to be able to send your metrics and logs not only to Cloudwatch, but it also can do things for any endpoint that supports open telemetry. So this might be third party monitoring tools, but mostly that's focused on your workloads and how to get workload information out, because that's what a lot of people, that's the important part of a cluster is typically are the workloads healthy or not? But it does also get metrics and some logs for the nodes themselves. So you can see a little more depth on how your cluster is running.

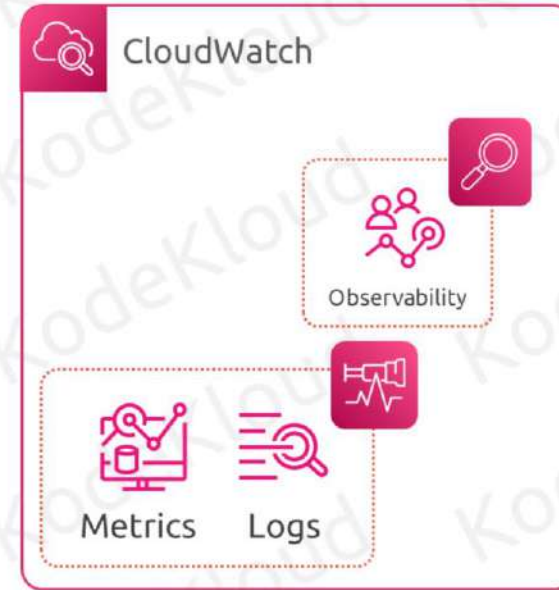
AWS Distro for Open Telemetry (ADOT)



© Copyright KodeKloud

But this isn't pulling stuff from the control plane, because the control plane is again, it's a fully managed endpoint. And if you want to get those sorts of logs, you have to query the AWS APIs and the Prometheus endpoints to get that information.

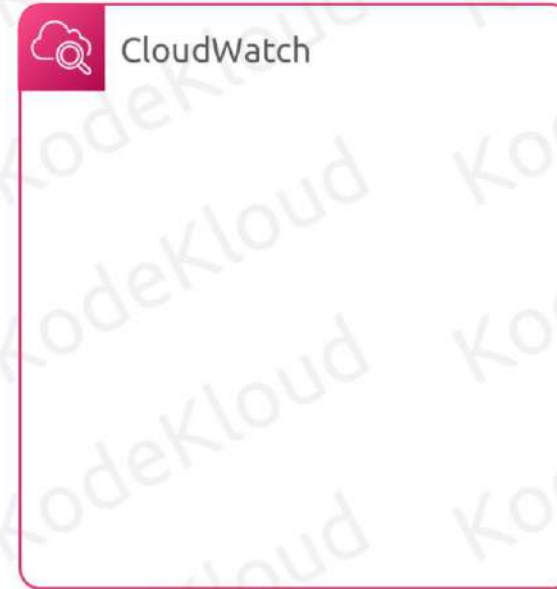
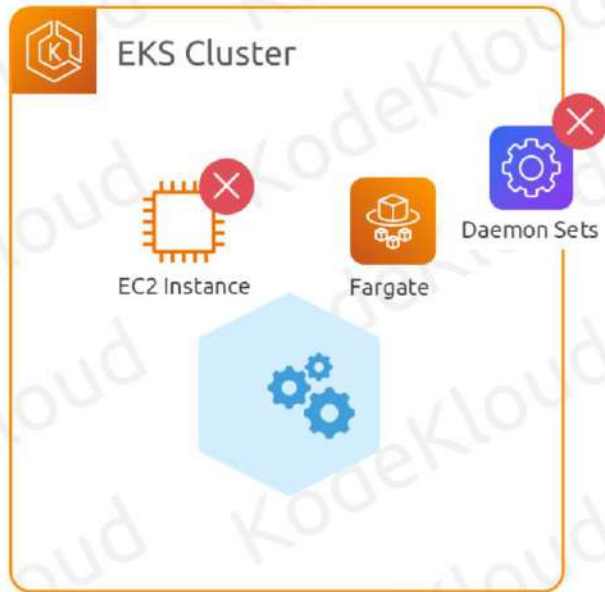
AWS X-Ray



© Copyright KodeKloud

There's also an AWS X ray service and add on that you can add and run in the cluster. And this again, it builds on top of ADOT. So you're going to have your Cloudwatch log agents plus AdoT plus Adobe x ray to be able to get traces from your workloads into AWS X ray. And maybe you need that specifically, maybe you don't. There are a lot of different ways to monitor workloads inside of a cluster. These are just the default ones that AWS documents and supports

Fargate Nodes



© Copyright KodeKloud

If you are running nodes that are not EC, two instances, but maybe Fargate nodes that would actually need a separate way of getting this login information. Because the Cloudwatch log agents, the adots agent, these are run as daemon sets on the node. And as we mentioned before, Fargate doesn't have daemon sets.

Fargate Nodes



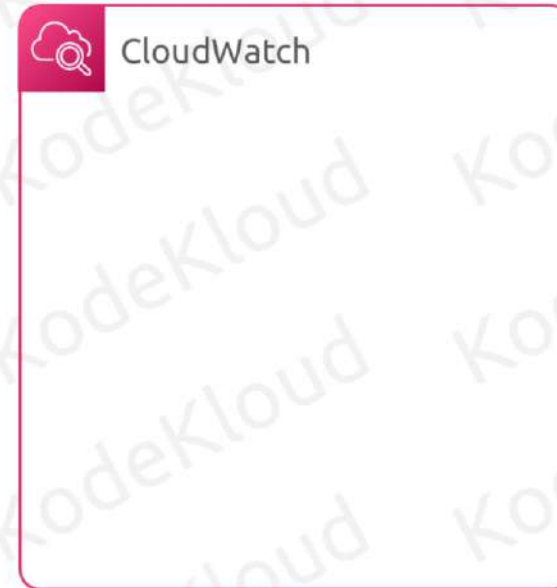
Fluent bit (Firelens for ECS)



Fargate

So in Fargate they run a managed version of fluent bits, sometimes called firelens for ecs, but also supporting eks. And so this fire lens or fluent bits add on runs without you doing anything to start it.

Fargate Nodes



But the thing you actually need to do is create a special AWS observability namespace and an AWS logging configuration, a config map in the cluster,

Fargate Nodes



© Copyright KodeKloud

and that will automatically trigger the and that will automatically trigger the fluent bits agent to start collecting logs from your workload and sending them over to Cloudwatch. This isn't getting information about the about the Fargate node itself. This is only for workload logging. So fargates nodes don't really have that rich metadata, and you're not going to get all the information that you normally would out of something like an ADOT or Cloudwatch logs agent

Fargate Nodes



© Copyright KodeKloud

because that's just you have more access to those EC two instances.

Amazon Managed Prometheus and Grafana



AWS Managed Prometheus
(AMP)



AWS Managed Grafana (AMG)



Workload



Cluster Health

The final two I do want to talk about as well is AMP AWS managed Prometheus and AMG AWS managed Grafana, and these are both normal monitoring stacks for people outside of eks. If you're running a Kubernetes cluster somewhere else, you probably have Prometheus and Grafana running to be able to create dashboards and monitor your workloads as well as your cluster health.

Amazon Managed Prometheus and Grafana



And AMP and AMG both are available as managed services in AWS. So if you want to maybe not look at Cloudwatch or not build all your dashboards in Cloudwatch and keep something that's a little more open source friendly or build on from what the community has, AMG and AMP will probably be the better ways to go there. And you can still scrape with both of these. Like AMP, Prometheus will be able to scrape your cluster just like it can scrape any normal cluster outside of AWS if you're running it locally or on Prem, whatever it is. Prometheus is a pretty known stack for being able to manage not only the nodes, the node metrics, but also the workloads. The thing that these gets inside of an AWS environment is really deep integration with things like IAM.

Amazon Managed Prometheus and Grafana



AWS Managed Prometheus
(AMP)



AWS Managed Grafana (AMG)

And AMP and AMG both are available as managed services in AWS. So if you want to maybe not look at Cloudwatch or not build all your dashboards in Cloudwatch and keep something that's a little more open source friendly or build on from what the community has, AMG and AMP will probably be the better ways to go there. And you can still scrape with both of these.

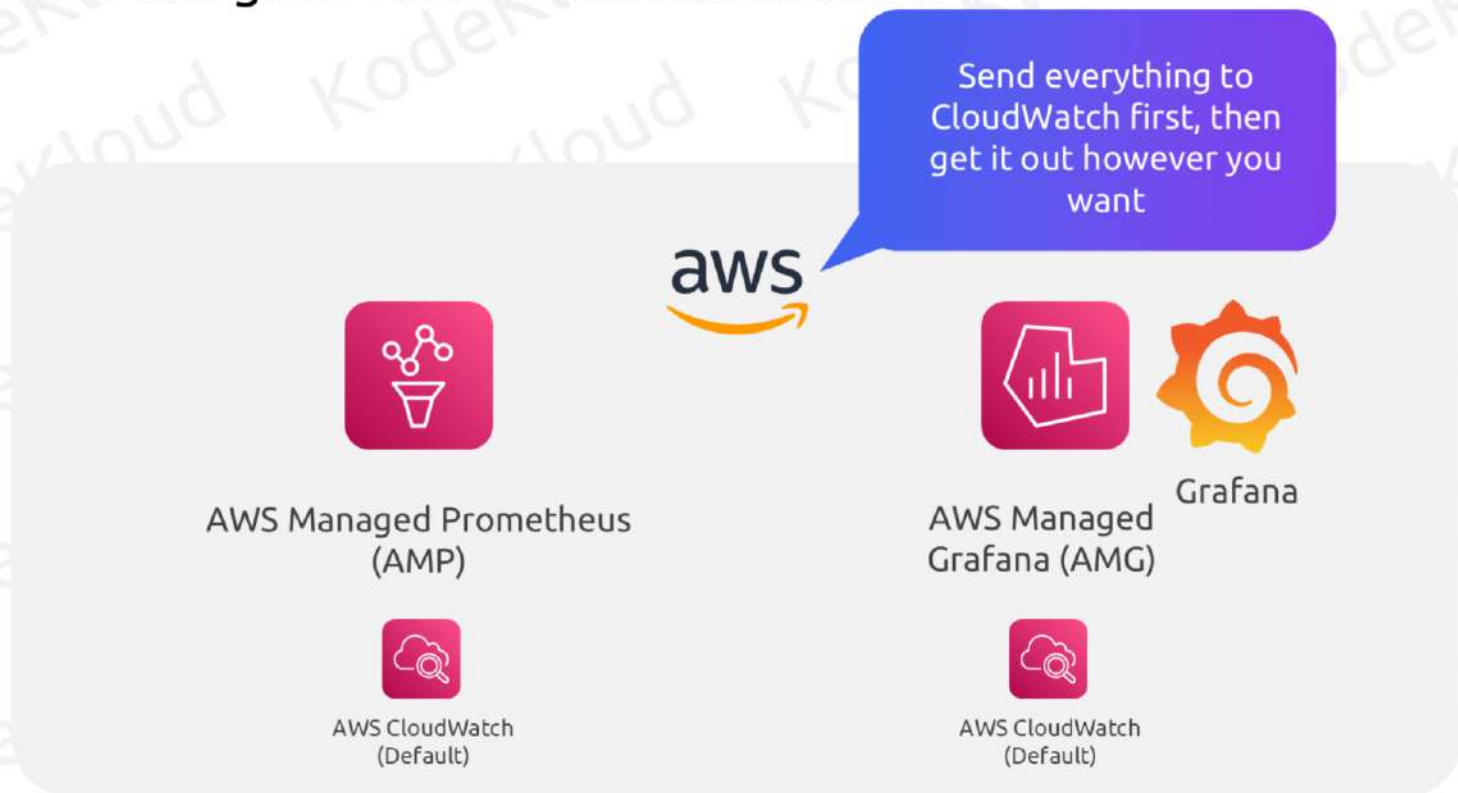
Amazon Managed Prometheus and Grafana



© Copyright KodeKloud

Like AMP, Prometheus will be able to scrape your cluster just like it can scrape any normal cluster outside of AWS if you're running it locally or on Prem, whatever it is. Prometheus is a pretty known stack for being able to manage not only the nodes, the node metrics, but also the workloads. The thing that these gets inside of an AWS environment is really deep integration with things like IAM. So all of your permissions are going to go through IAM. Your user management is going to go through AWS IAM.

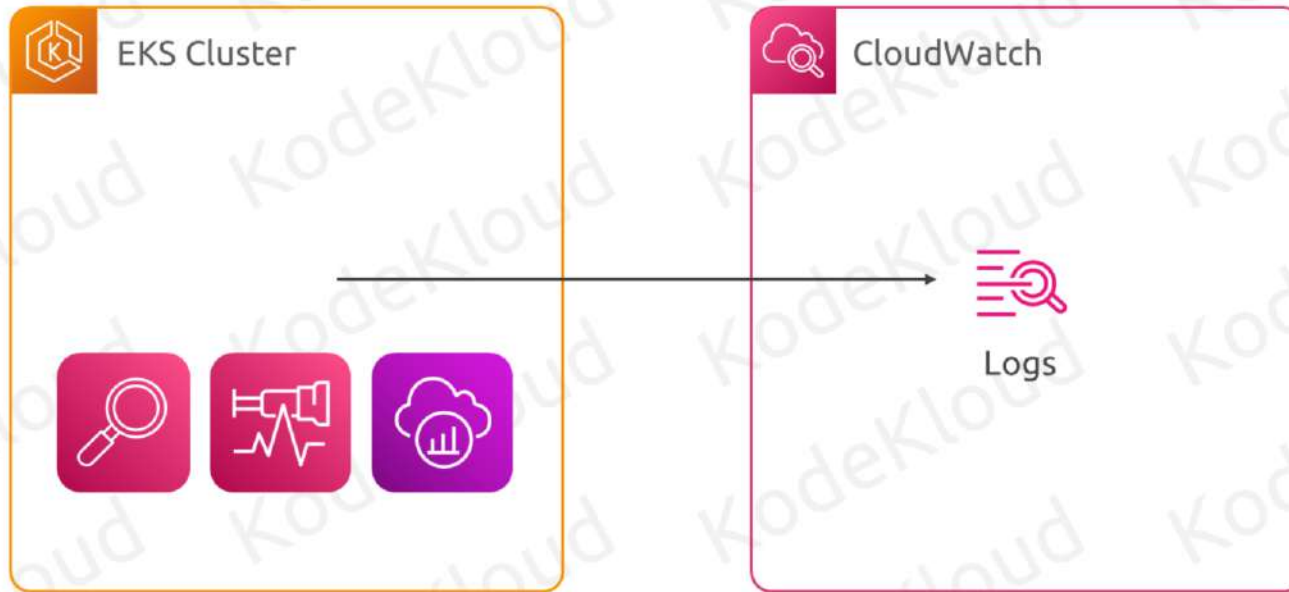
Amazon Managed Prometheus and Grafana



© Copyright KodeKloud

You can also get the enterprise versions of those services. You can subscribe to them and be able to get more features and support directly from Grafana if you want to. But there are some limitations because you can't pick and choose maybe any version of Grafana that's available, and there will be some limitations on the plugins. In most cases, something like an AMG at the Grafana instance is going to want to point to Cloudwatch by default, and that's one of the plugins that are there by default. And usually Amazon's going to encourage you, hey, send everything you have to Cloudwatch first, and then from there get it out however you want. If you want to pull that into a Grafana instance. If you want to send that to Firelens or a third party's three whatever it is, most monitoring is going to happen in Cloudwatch by default.

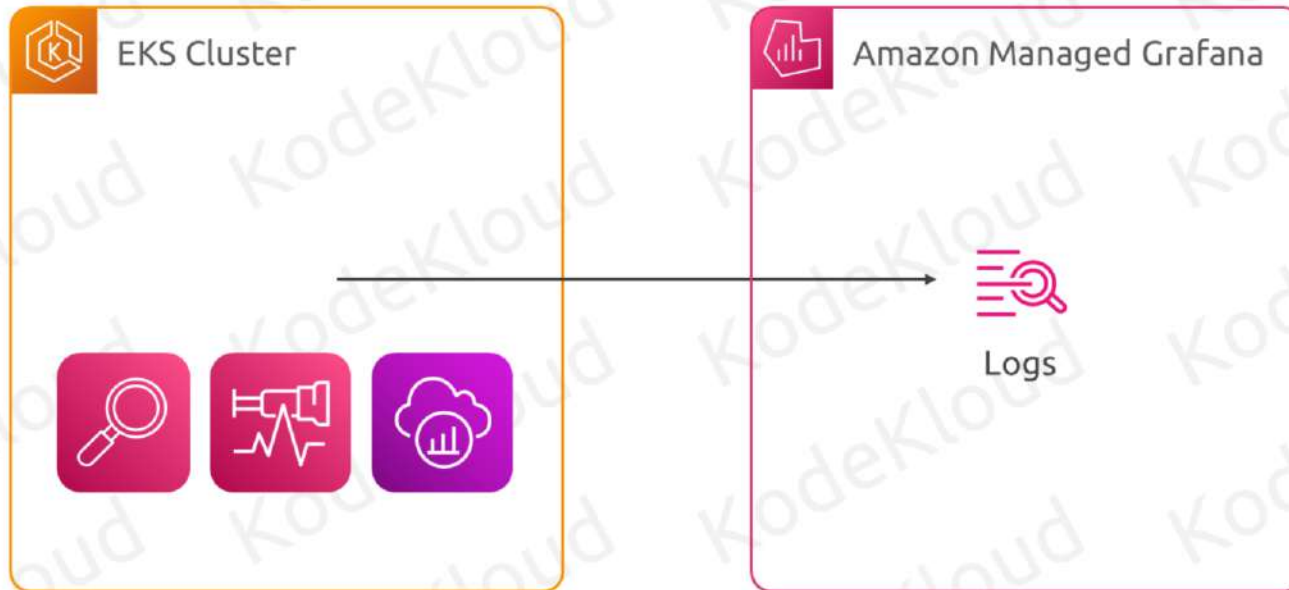
Conclusion



© Copyright KodeKloud

There are a lot of ways to monitor an EKS cluster. In any Kubernetes cluster there's a ton of third party options, but these are the overview of the Amazon supported ones and most of that just comes down to Cloudwatch and Cloudwatch logs and being able to send information into Cloudwatch. However you get that there and then other things will build on top of that. If you need tracing, if you want open telemetry, all of those other agents and services that you might get are going to build on top of that and either send that data into Cloudwatch

Conclusion



© Copyright KodeKloud

or maybe into Amazon's managed Grafana. And so both those are kind of the observability options that you're going to get inside of an EKS cluster and inside of AWS that is at least somewhat managed. Everything else is going to be completely run it yourself or a third party.

Upgrade Cycles

Upgrading EKS Clusters

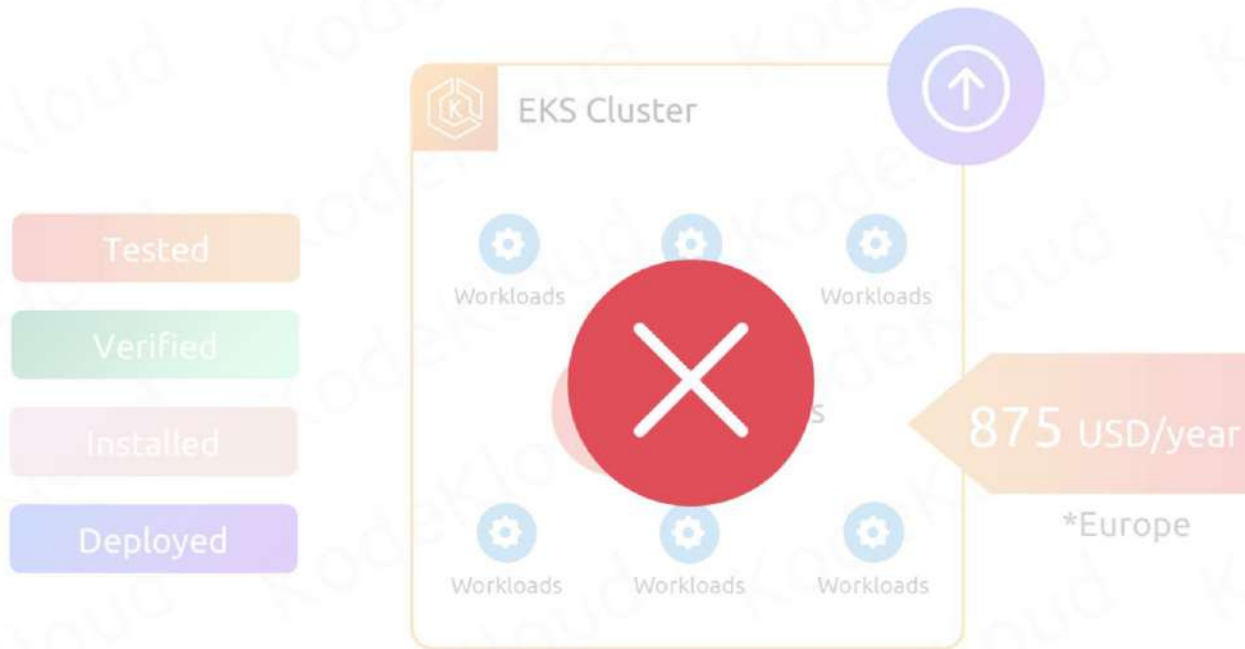


© Copyright KodeKloud

Let's talk about upgrading your EKS clusters. Not necessarily how you're upgrading them, but how often you're going to upgrade them. Because a normal EKS cluster version is supported for 14 months. And if you are deploying an EKS cluster, if you are getting workloads onto it, if you're trying to grow some adoption by the time you have it set up, ready to go, tested, verified, installed your plugins and deployed some workloads, you don't have a lot of time to use it. And so in 14 months you're going to have to upgrade that version or move into extended support pricing. And so a default EKS cluster is going to cost ten cents per hour, and that's roughly \$175 a year, and that's how much it costs for 14 months. So if you want to stay at that price, you need to stay upgraded or stay on a current version and basically perform an upgrade every three to four

months.

Upgrading EKS Clusters



And that's pretty disruptive. That's something that is going to constantly cause you to require testing and validation and make sure that things like deprecated APIs or removed APIs aren't being used. And this is sometimes difficult. So



Upgrade Insights



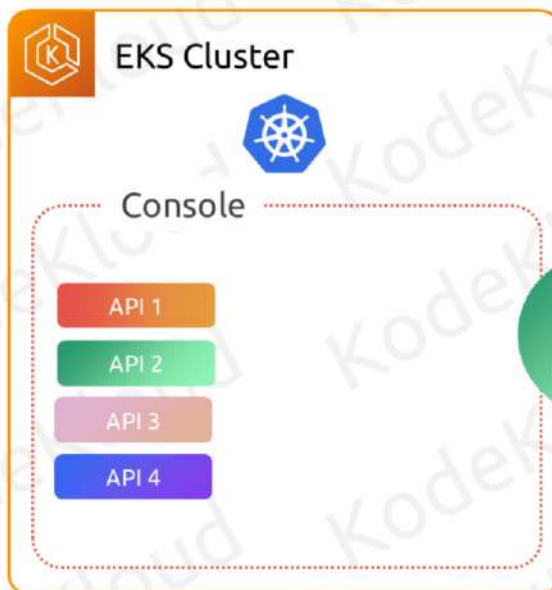
Upgrade Insights

So AWS introduced upgrade insights.

Upgrade Insights



Upgrade Insights



Am I going to lose access to some APIs in the cluster?

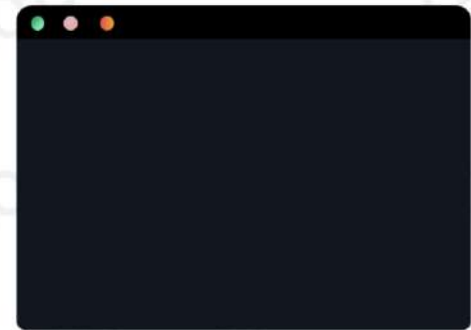
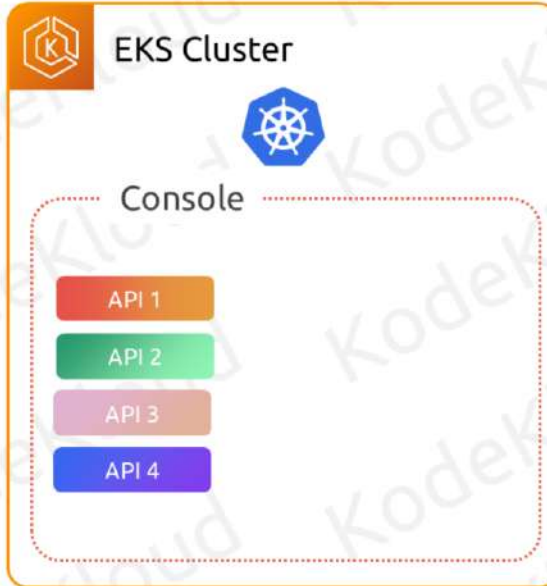
And upgrade insights on your eks cluster have a tab in the console that'll show you which APIs are being called, if they're being used by workloads, or if something's actively using a API that's going to be removed in the next version. So that's when you are going to upgrade that cluster. You can verify, hey, am I going to lose access to some API in the cluster? Because not only do you need to test all of your add ons, you need to make sure that your APIs aren't being, you need to make sure that removed APIs aren't being used in the cluster. And usually that's really hard to find because the Kubernetes APIs don't make that easy to find. You just have to know, oh, these versions are being removed and here's my workloads that are using them. But most of the time the person or the team that's running the clusters are not responsible for the

workloads on top of the clusters. And so it's really difficult to discover that. And that's what upgrade insights are for.

Upgrade Insights



Upgrade Insights



You can query those from the command line, and you can also query that or look at them in the console themselves.

Extended Support



But let's say you can't upgrade your eks cluster in time and you have to move past that 14 months. Well, you can also get the extended support, which is going to extend the life of the cluster for another twelve months.

Extended Support



© Copyright KodeKloud

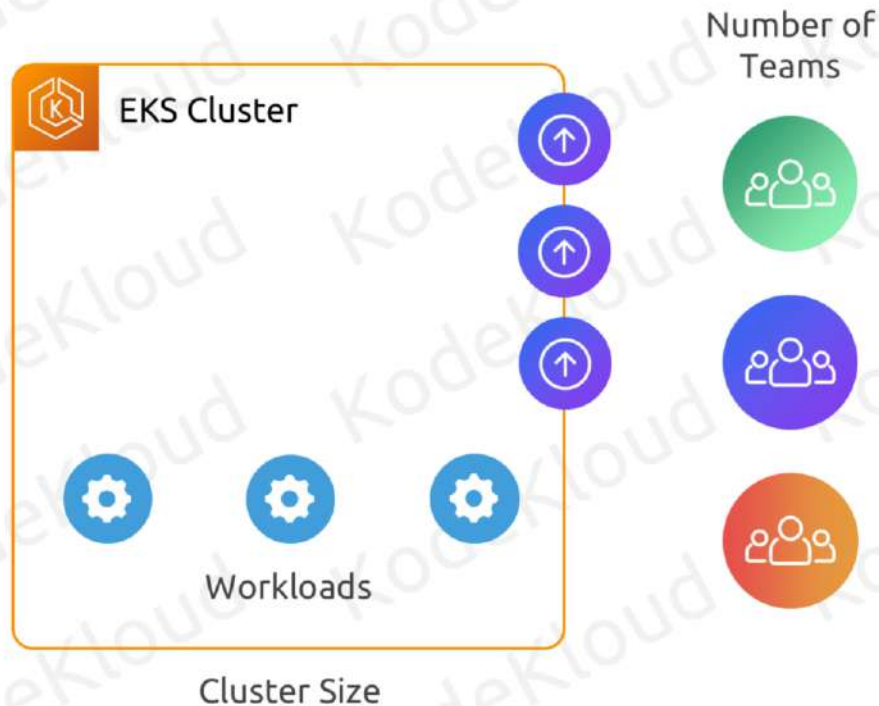
So instead of having 14 months to upgrade, you have a total of 26 months. But during that last twelve months, the cluster cost goes up considerably. Instead of being \$875, it is now \$4,380 for the last year. So it's up to you to decide if that's something that is manageable, if that's something, if that cost is okay for your environment and your bills. Because if you have one cluster, maybe that's okay. If you have ten clusters or 100 clusters, that gets really expensive really quick and you need to make sure, how do I keep this upgraded on time so I don't move into that extended pricing range? We can talk about, we'll talk about strategies for upgrading clusters in a different section. But this is just so you know that if you want to stay on the actual, the normal pricing, for any case cluster, you have to upgrade it at least every eight to ten months,

because if you want to stay current, you have to stay up to date with that.

Extended Support



Releases 3 times a year



So you're likely doing two, maybe three upgrades every year just to stay on the current version and not fall behind. And that's going to be a lot of work, especially depending on how large your clusters are, how many workloads they're running, how many teams you're supporting. It's something you constantly have to know about, so you can make sure that you stay ahead of that upgrade cycle. And this isn't something that's EKS specific, this is something because Kubernetes releases three times a year, and Kubernetes, the upstream project, only supports the versions for so long. So if EKS wants to stay up to date and current with the upstream Kubernetes, they have to stay in lockstep with what's being released. So just know that this is something that the upstream project has decided. That's how fast they want to make their releases, that's how

long they can reasonably support them. And almost every other distribution or hosted service of Kubernetes falls in line with that to some degree.

EKS Upgrades

Cluster Upgrades



Upgrades

Let's talk about cluster upgrades. In all of my time at Amazon on eks, the thing that I wrote and talked about the most was how to handle upgrades. If you're running an EKS cluster, you're going to have to upgrade it and you're going to have to do it a lot.

Cluster Upgrades



Upgrade 1



Upgrade 2



Upgrade 3

There are, at the current year, there are three releases every year, and if you start to fall behind, you are going to get force up.

Cluster Upgrades



Upgrade 1



Even with long term supports, which extends how, how long you can wait till you upgrade.

Cluster Upgrades



© Copyright KodeKloud

That's going to charge you a lot more money for those instances and for those clusters. And you're still going to have to catch up. You can't just stay on an old version forever. You're going to have to stay up to date at some point.

Cluster Upgrade



In-Place Cluster Upgrade



Blue-Green Cluster Upgrade

So when you're work, so when you're managing clusters, there's two basic strategies that are available to you, and that's an in-place cluster upgrade and there's a blue green cluster upgrade. And in all of my time working there and managing clusters, I'm just going to say you should try to do the in place cluster upgrade as much as possible, because as much as someone might want to tell you that a cluster is treated as disposable and you should be able to migrate workloads, that's simply not the case

Cluster Upgrade



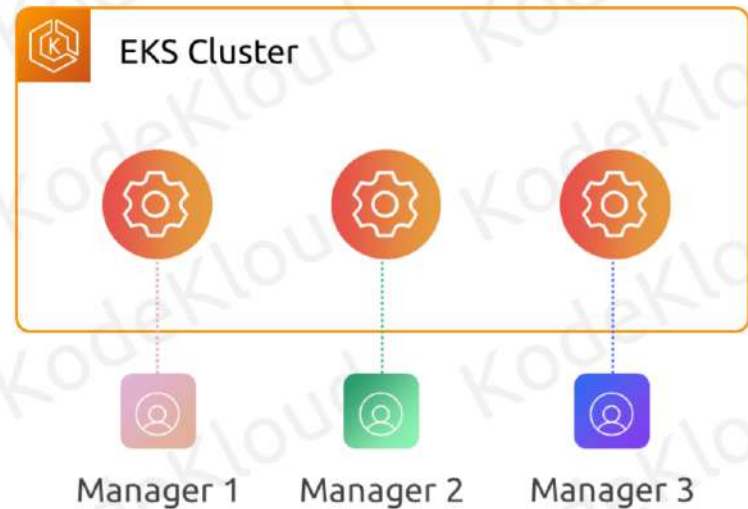
because there are things that people rely on like load balancers and DNS, and those things are hard to move because DNS cache exists and timeouts will happen in all of these situations where you say, hey, I just want to delete that old one and create a new one from scratch, that might be the best way to go. But just know that the migration process is going to take a lot longer than you expect

Cluster Upgrade



because depending on how many workloads are running in that cluster

Cluster Upgrade



you're going to have to work with everyone that manages a workload and make sure that everything they depend on is, is available and works the same way in the new cluster.

Cluster Upgrade



Duration (Months)

So generally what I've seen is if you only have a handful of workloads inside a cluster or you're the one that manages the cluster and the workloads, then you can probably move it okay in okay amount of time. But these things will extend for months and more months. And not only are you going to be spending time on long term support for the cluster itself, but you're now spending time,

Cluster Upgrade



Duration (Months)



More Expensive

you're spending money on two clusters. You've just doubled your spend to run two clusters with workloads and migrate between them.

In-Place Cluster Upgrade



In-Place Cluster
Upgrade

So stick with in place clusters as much as you can. And I'm going to diagram it out and just show you what that kind of looks like.

In-Place Cluster Upgrade

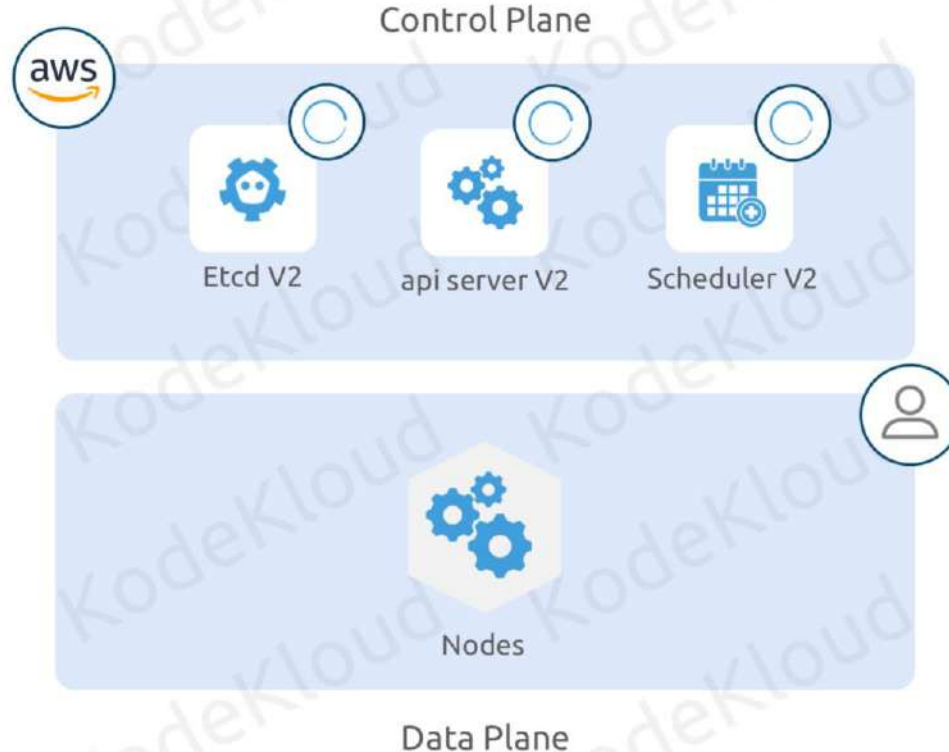


© Copyright KodeKloud

And I'm going to simplify how this looks by just saying that we have a control plane up here and we have data plane down here. And this is again, it's the control plane where your API server is and the data plane where your workloads and nodes are. Specifically, this is like the split between the stuff in the AWS account and the stuff in your account. So when you call the hey, upgrade my cluster please to the EKS API, Amazon will start by replacing the things in the control plane, the API server, the controller manager, the scheduler and any of the other components like we've talked about in the past, like webhooks and different controllers that may be running in this control plane to make sure that they're all up to date. And this process takes a little time. And what it does is it will replace each of those components in a rolling fashion. And most of

them are behind, if you're actually calling them like the API server are going to be behind a load balancer.

In-Place Cluster Upgrade



And so at some point you're going to end up with a v one and a v two of say the API server behind the load balancer and it's going to try to start routing you from the old one to the new one as soon as the new one is up and healthy. And again, this is just the first step of how a control plane gets upgraded. All of this is following Kubernetes best practice for how to upgrade a cluster,

In-Place Cluster Upgrade



© Copyright KodeKloud

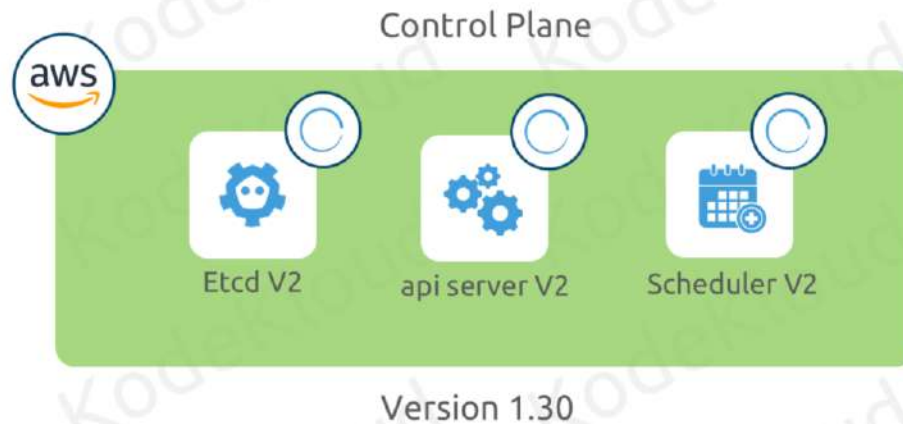
but this side of it on EKs is managed for you.

In-Place Cluster Upgrade



This also may involve things like eTCD being upgraded, because if ETCD requires changes or has a new version available that will also be upgraded and your data will be migrated with it.

In-Place Cluster Upgrade



© Copyright KodeKloud

So all that's happening behind the scenes and at some point this cluster or this control plane turns green as far as the new version is running and available. And so now we're say we are on version 1.0 30 on our control plane and that's great because that's what we wanted and that was managed and that's why manage and service managed services are great, but now we have to manage the data plane and that's all the stuff in our account and it depends on how you're using that

In-Place Cluster Upgrade



If you have node groups that are all in pools, or if you have carpenter which is groupless but it has node classes, or maybe you're using fargates, these all have different ways of upgrading them.

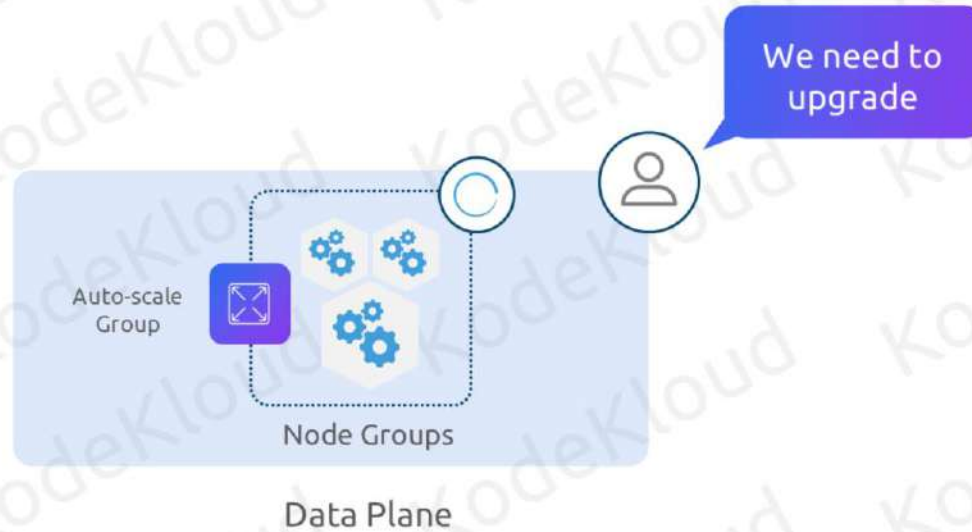
In-Place Cluster Upgrade



© Copyright KodeKloud

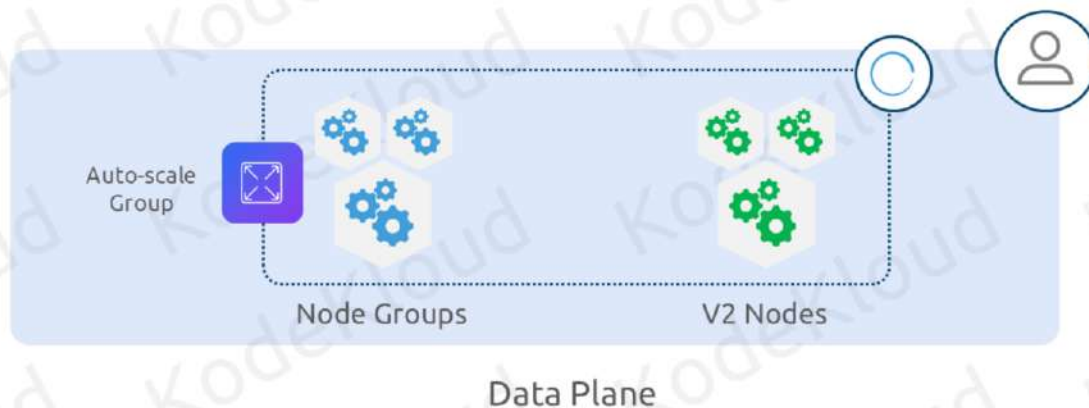
Not only that, but you have your own controllers running inside the cluster that you have to manage. Some of them might be add-ons, some of them might be generic helm charts that you deployed. You're responsible for knowing the compatibility of the helm charts or whatever workloads that are calling the AWS APIs. Add-ons are version mapped by AWS but still need to be upgraded individually by you.

Node Groups



So, let's start with a node group and say, how do we upgrade a node group? Because this might have three or four nodes in it. And remember, these are all inside of an auto scaling group, and for now they're all blue. The old version is going to be blue and we want to upgrade them in place. So what's going to happen is we're going to call that node group and say, hey, node group, we need you to upgrade, and we're going to upgrade you to a specific AMI or a new version

Node Groups



© Copyright KodeKloud

However it is you're managing that node group, you're calling the API, and it's going to go ahead and create a new node group. I'm sorry, it's going to create a new node in that node group with the new version. And so this is going to be a green version. As soon as that node is up and healthy, it's going to say, okay, now one of the old nodes can go away and it's going to replace that with another green node. And as soon as that one's up in healthy, it's just going to roll through all of them and replace them all until they're all green. This is fine. And that's how replacement should work. It doesn't do a node group like blue green swap to create fully new ones. It just rolls them out in the node group or in the auto scaling group that is existing.

Karpenter



For Karpenter, it's going to look at the node classes, and Karpenter actually manages some of this for you automatically.

Karpenter



© Copyright KodeKloud

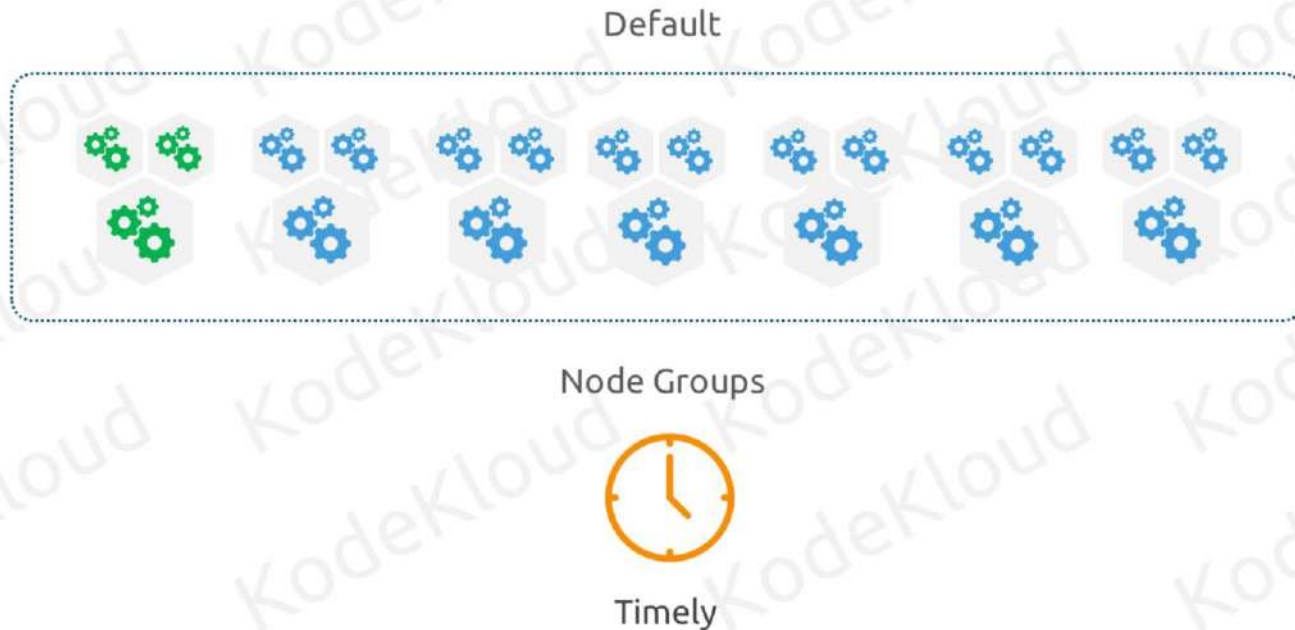
So carpenter, because it has a little more intelligence about how the control plane is versioned and working, it will look at, oh, I see that we are on version 130 now,

Karpenter



and all of my nodes in this node class are say, 129, it will go ahead and use some of the information that it knows about how you want to do upgrades when you have downtimes to start replacing those nodes.

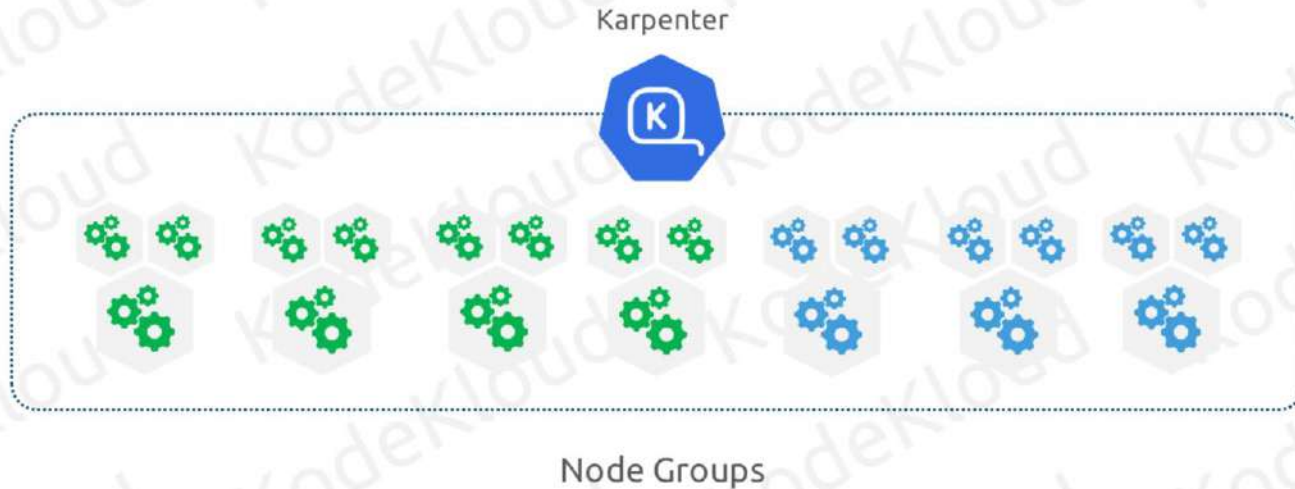
Karpenter



© Copyright KodeKloud

If you have things like, if you have things like consolidation turned on, this may be happening at a regular schedule. So whenever a new node is already scheduled to be created inside your node class, it's going to go ahead and just delete the old one and create a new one running the new version and it will automatically roll those out through your node groups. By default it's only going to replace one at a time, but you can change that setting inside a carpenter. That's also something I should point out with the node groups is by default it also will cycle through one node at a time.

Karpenter



But if you have really large node groups like say 50 or 100, that might just take a little while to cycle through them. So you can change the node group to be a certain number of nodes at a time. Say you want four nodes to be replaced at a time, or you can do it as a percentage and say 2% and this will, you know, change out as many nodes as you have on that setting to make it faster just to be able to get the new version out.



For Fargate



AWS Fargate

For Fargate,

For Fargate



Pod

Node

remember we have one node per pod

For Fargate



and so we have a bunch of these pods and we have a bunch of these nodes and so you have a lot of them. And in order to rotate those out, you will need to upgrade the control plane and then you need to create a new deployment version or somehow force a deployment to roll out which will replace all of these nodes. And this works if you have a small number of pods pretty easily.

For Fargate



50 Pods = 50 Nodes



Timely

But if you have, if you have say 50 or 100 pods, you are going to be replacing 50 or 100 nodes and that's just going to be slow

For Fargate



50 Pods = 50 Nodes



Deployment



Fargate cannot tune

Fargate doesn't have a way to tune how many should be deleted and deployed except for whatever the deployment says. And so the deployment is the one that actually controls how fast you're going to roll through these.



For Fargate



Replace pods with
new version

So make sure that that is how you're going to be able to call that API and replace those Fargate pods with the new version as you need to the managed add ons.

For Fargate



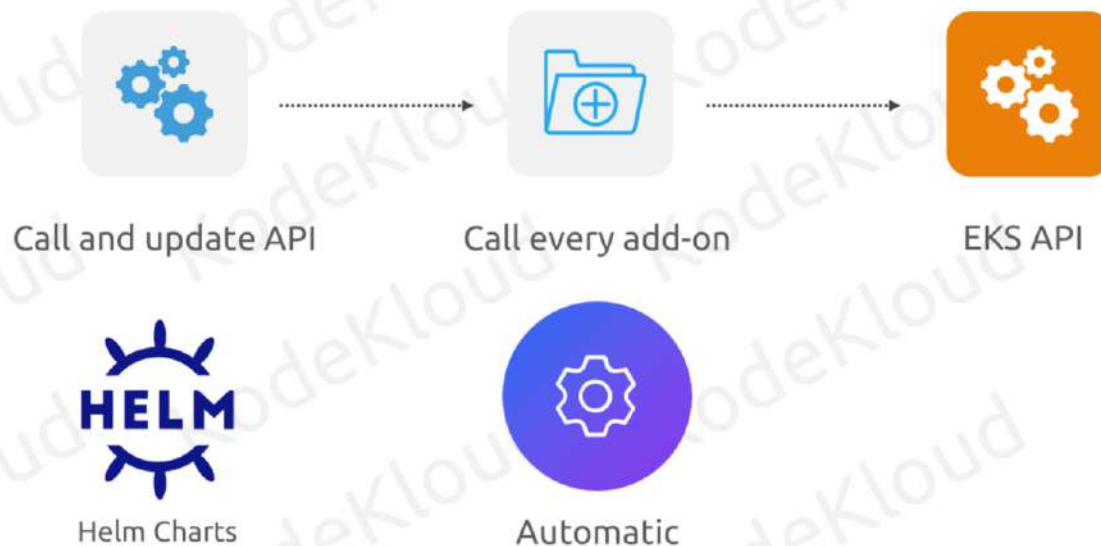
Replace pods with
new version



Replace nodes

Again, this is something that you should be doing before you start replacing nodes because some add ons need changes and they need configuration.

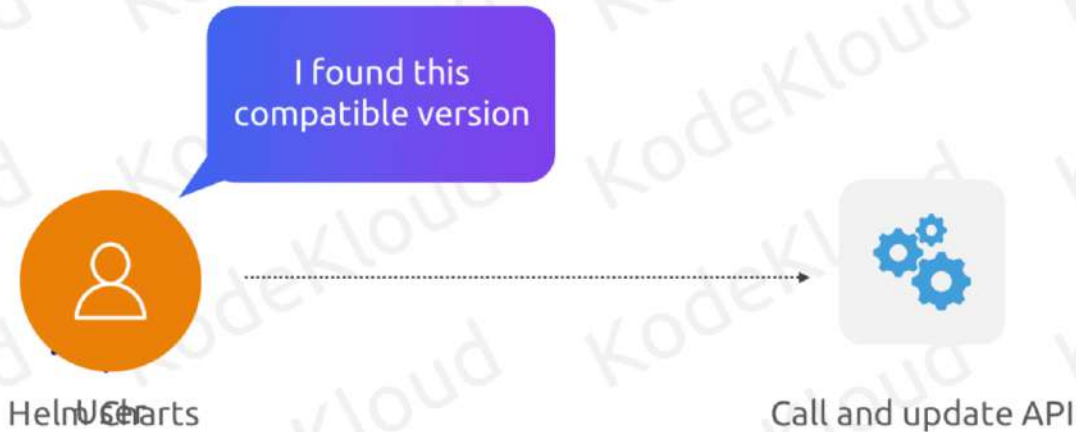
Managed Add-ons



© Copyright KodeKloud

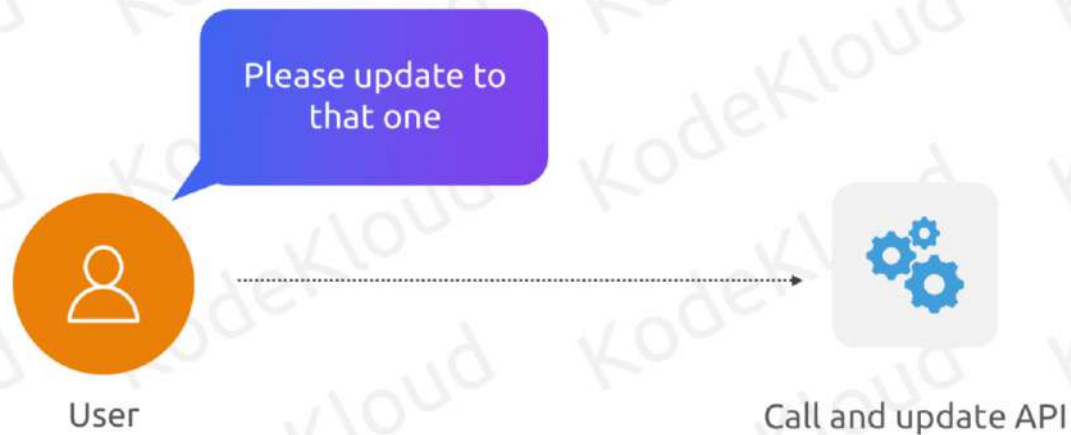
So, managed add-ons are something you will want to make sure that you're calling that API and updating them first. And add-ons need to be called per add-on that's running in your cluster and you need to just make a call and say, hey, go to this version. And so you're doing another API call for each add-on and that goes through and calls the EKS API, but then it goes into your cluster and changes the deployments. But it is up to you to trigger that this isn't going to happen automatically.

Managed Add-ons



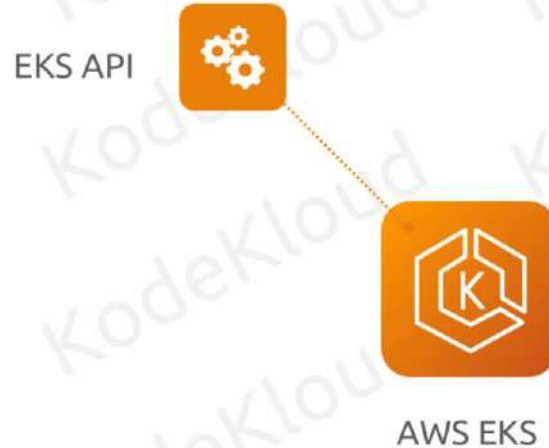
As much as it's an add on, it's not automatic. And so you'll need to make sure that you are looking at those, whether you're doing them through again through a helm chart that you manage, or calling the API and saying hey, I found this compatible version,

Managed Add-ons



please update to that one.

AWS EKS



Before you do any of this, it is important to also call out that there is an API in EKS for container or for cluster insights.



AWS EKS



EKS Cluster

And this is going to look at your cluster, any resources in the cluster, and tell you if anything is going to be deprecated or removed in the next version.

AWS EKS



So let's say when we moved from Kubernetes 124 and we went to Kubernetes 125,

AWS EKS



Kubernetes 124



Pod Security
Policy



Kubernetes 125



something called pod security policies, psps was removed. And so if you had psps in your 124 cluster and then you tried to make a 125 cluster, you would fail because these psps just don't exist in the API anymore. And so you need to be able to know, oh, I need to clean that up beforehand.

AWS EKS



User

Is there anything
that is going to be
removed?

And so you can call this cluster insights API and it will check and say, hey, is there anything in your cluster that is going to be removed from resources? We need to make sure that you take care of that. It's an okay API, it's an EKS API, but it doesn't have all of the details about every namespace and things that are it's happening in.



Kubent



Kubent

© Copyright KodeKloud

Another tool that I would usually recommend is called cube no trouble. It's cube nt.

Kubent



Kubent

I'm going from this version to the next



Kubernetes API

Another tool that I would usually recommend is called cube no trouble. It's cube nt. And that is something that will call the Kubernetes API and say, hey, I'm going from this version to the next.

Kubent



Kubent

I know which
versions are going
to be removed



Kubernetes API

It manages all of those mappings and says oh, I know which versions are going to be removed. And it gives you a little more detail about which things you need to clean up or remove and which workloads are using them because it can understand the Kubernetes API from your point of view.



Kubent

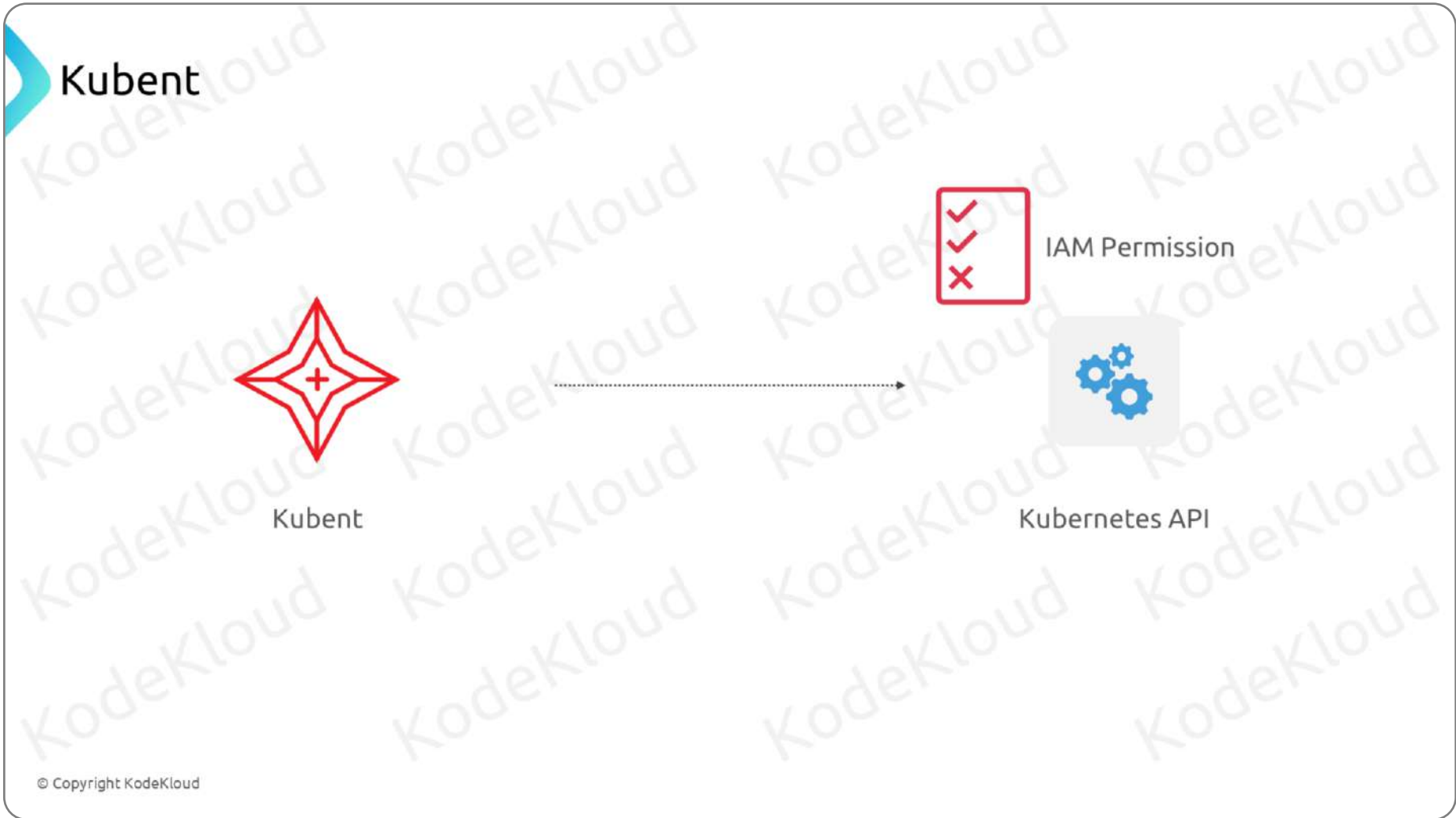


Kubent



Role-Based Access
Control (RBAC)

The thing you need to do with Kubent is you actually need to have some elevated permissions or RBAC that allows you to look into all the namespaces and those resources. For the EKs cluster insights.



You need to have IAM permissions to be able to call that API.

Kubent



As I briefly mentioned in the beginning, there is one other strategy for upgrading a cluster, and that's to just deploy two clusters and then migrate every workload from one to the other. And I don't usually recommend this unless you have two things that you need to do. If you need to make fundamental changes in your cluster

Kubent



© Copyright KodeKloud

like say switch out the CNI provider, the network, say you're moving from the VPC CNI to psyllium. That would be something that you may not want to do in place. You may want to be able to test that a little more thoroughly and test that with specific workloads that you are worried that changes are going to happen, especially if you're doing things like network policies in the new version. You'll want to test those things in a cluster that's a little bit isolated. The other reason you may want to do blue green is if you're jumping from major versions or large version gaps at a time.

Kubent



Kubernetes



Cilium



Kubernetes



Network
Policy

That would be something that you may not want to do in place. You may want to be able to test that a little more thoroughly and test that with specific workloads that you are worried that changes are going to happen, especially if you're doing things like network policies in the new version. You'll want to test those things in a cluster that's a little bit isolated.

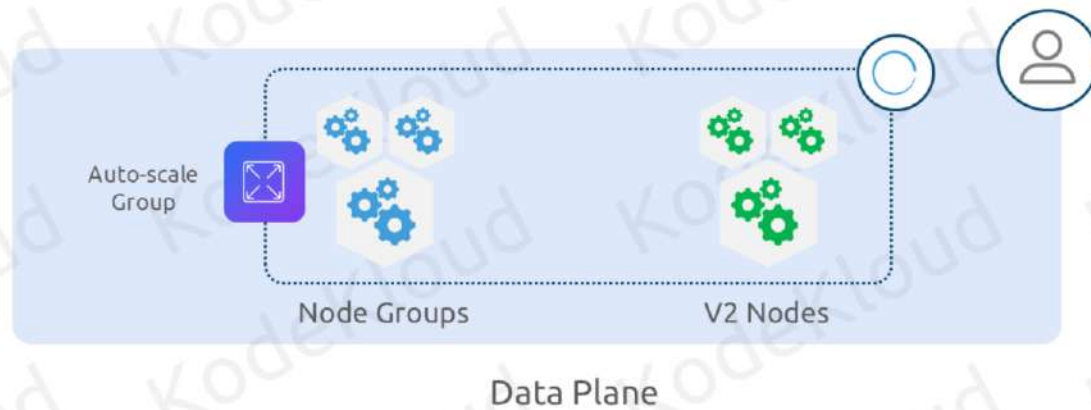
Blue-Green Cluster Upgrades



© Copyright KodeKloud

The other reason you may want to do blue green is if you're jumping from major versions or large version gaps at a time. So say we're going from 1.25 to 1.30, that to do in place you would have to go 1.25, 1.26, 1.27, 1.28, 1.29, and then you can get to 1.30. And that would just take a very long time.

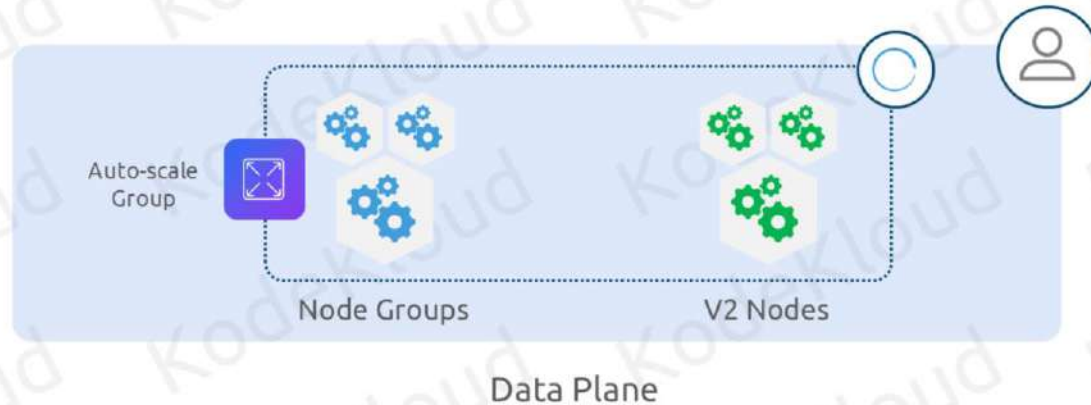
Blue-Green Cluster Upgrades



© Copyright KodeKloud

It may be faster for you just to do blue green upgrades and migrate your workloads. So those are the main times I would say do blue green, but if you have a policy or a specific use case that you do blue green for other things in your environments, it's an option.

Blue-Green Cluster Upgrades



Duration (Months)



More Expensive

But again, it usually takes a long time and it's a lot more expensive. And so it's something that I usually recommend against unless you know for sure that you need it.

Summary

- 01 Node upgrades
- 02 How control plane gets upgraded
- 03 Upgrading different types of compute in EKS cluster
- 04 Add-ons and calling the API
- 05 Cluster insights and Kubent
- 06 Blue-Green cluster upgrades

© Copyright KodeKloud

So just to recap, in this section, we talked about node upgrades with how the control plane gets upgraded and how to upgrade different types of compute that you get in an EKS cluster, whether that's a node group, carpenter, or fargates. We also talked about the add ons and calling the API to make sure those get upgraded. About cluster insights for and Kube, no trouble to make sure that your workloads and crds are ready for the next Kubernetes version. And we briefly touched on blue green network or, and we briefly touched on blue green cluster upgrades. And when you may want to use those.

EKS Add-ons

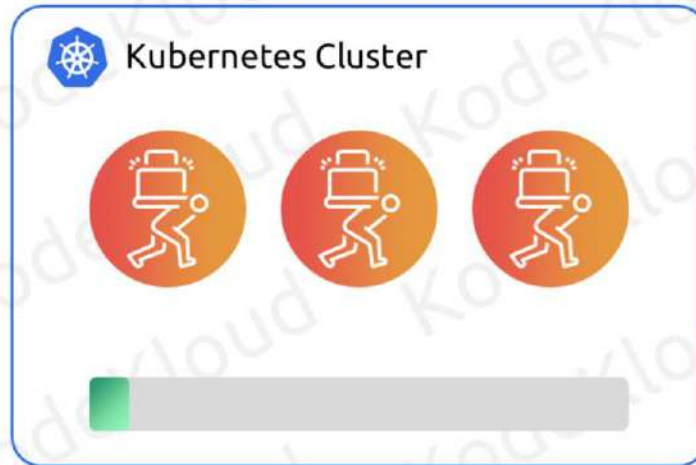
EKS Add-ons



© Copyright KodeKloud

Let's talk about EKS add ons, because add ons is kind of a weird thing that isn't really common. It's not a Kubernetes thing, it's definitely an EKS thing. And add ons are a way to add some kind of defaults cluster services inside of your Kubernetes cluster, inside of eks cluster that you might need for different workloads. And these are managed during their installation. But have some gotchas that you may want to be aware of when you're actually trying to roll it out. So I'm going to show you the GUI for this, at least the AWS console for this, because it shows off how they're kind of intended to be used.

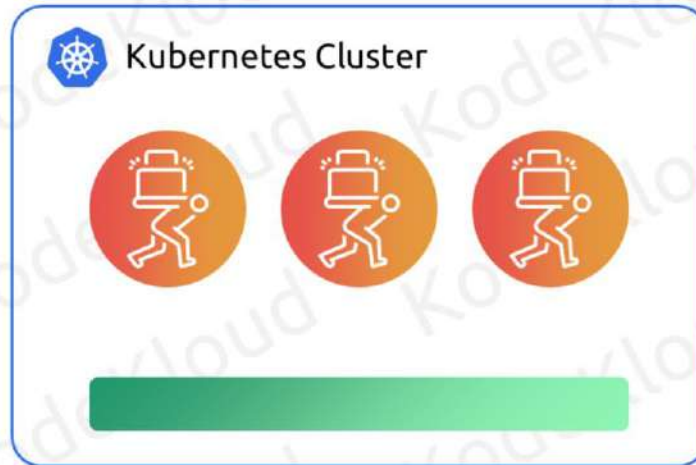
EKS Add-ons



© Copyright KodeKloud

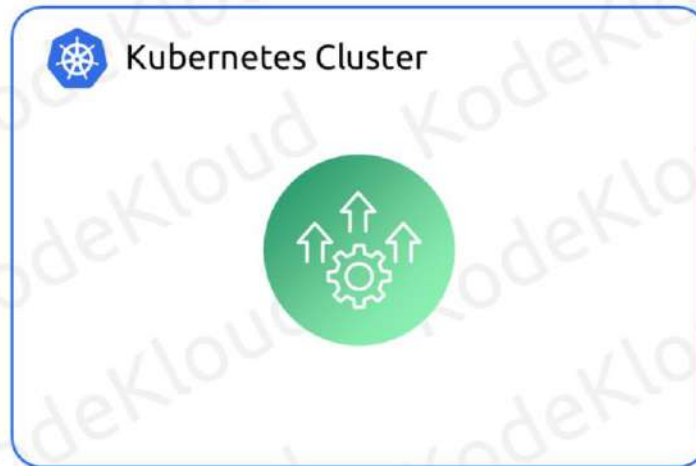
Let's talk about EKS add ons, because add ons is kind of a weird thing that isn't really common. It's not a Kubernetes thing, it's definitely an EKS thing. And add ons are a way to add some kind of defaults cluster services inside of your Kubernetes cluster, inside of eks cluster that you might need for different workloads. And these are managed during their installation. But have some gotchas that you may want to be aware of when you're actually trying to roll it out. So I'm going to show you the GUI for this, at least the AWS console for this, because it shows off how they're kind of intended to be used.

EKS Add-ons



Let's talk about EKS add ons, because add ons is kind of a weird thing that isn't really common. It's not a Kubernetes thing, it's definitely an EKS thing. And add ons are a way to add some kind of defaults cluster services inside of your Kubernetes cluster, inside of eks cluster that you might need for different workloads. And these are managed during their installation. But have some gotchas that you may want to be aware of when you're actually trying to roll it out. So I'm going to show you the GUI for this, at least the AWS console for this, because it shows off how they're kind of intended to be used.

EKS Cluster Upgrades



© Copyright KodeKloud

The more things that stand in the way of you upgrading your kubernetes clusters or your eks clusters, the harder it's going to be to upgrade your EKS clusters. And when you have to do it three times a year, take some ownership of what you can do to get out of the path of waiting for another company to validate the version or it to be pushed to a marketplace or for configuration to be available and inside of an add on marketplace.

Upgrades With Helm Charts or YAML



Add ons in eks is something that I have never recommended people do for a variety of things and instead move to something more like bootstrapping a cluster with helm charts or even just a standard set of manifest files or something like Gitops. Do something else to be able to bootstrap and own all of the yaml and base services that need to run in your cluster, rather than putting the things inside of your workload in the hands of EKs or a marketplace vendor. Add ons have come a long way since I first was working on them at EKS and now as a user. So they are adding more and more things to make some of that easier.

Upgrades With Helm Charts or YAML



© Copyright KodeKloud

Add ons in eks is something that I have never recommended people do for a variety of things and instead move to something more like bootstrapping a cluster with helm charts or even just a standard set of manifest files or something like Gitops. Do something else to be able to bootstrap and own all of the yaml and base services that need to run in your cluster, rather than putting the things inside of your workload in the hands of EKs or a marketplace vendor. Add ons have come a long way since I first was working on them at EKS and now as a user. So they are adding more and more things to make some of that easier.

Avoid Add-ons for Smoother Flow



© Copyright KodeKloud

But at this state in 2024, it's still kind of rough. If you are upgrading a cluster and you say have managed node groups, you would have to go upgrade your API server, you would have to go upgrade your node groups, and you would also have to upgrade each and every one of your add-ons. That's not a good experience. You want to make sure that you can test this holistically and just again own some of that, some of that yaml, some of the services that are based that are required for your workloads to run, because converting and migrating your workloads is going to take a lot longer than just validating that the cluster spins up and runs and the base stuff works. So, make sure you balance how much ownership you want to have inside the cluster and then figure out for yourself if that's something that you want to push off to someone else and

allow them to maybe stop you from doing what you need to do to upgrade a cluster.



KodeKloud

© Copyright KodeKloud

Follow us on <https://kodekloud.com/> to learn more about us.